A BUILD-ALONG COURSE

Building an ORM on tokio-postgres

Derive macros, a model layer and a typed query builder on tokio-postgres, built from scratch as shelf — for readers of Async PostgreSQL in Rust.

10 of 16 chapters written, the others sketched · every runnable example was compiled and run against PostgreSQL as written

python3 tools/book.py book --book orm

Contents

0 Introduction and setup . . . **1**
Contents . . . 1
0.1 What an ORM does, and what shelf will not do . . . 2
0.2 Where shelf sits . . . 3
0.3 The finished API in ten lines . . . 4
0.4 Three crates, and why . . . 6
0.5 How this book follows the build . . . 6
0.6 Setup . . . 7
0.7 The first derive, end to end . . . 8
Summary . . . 10
Exercises . . . 11

1 Mapping rows by hand . . . **13**
1.1 A trait of shelf's own . . . 13
1.2 By name or by position . . . 15
1.3 Three ways to fail . . . 18
1.4 Nested structs . . . 20
1.5 Fetch helpers . . . 21
1.6 Counting the boilerplate . . . 23
In stockroom . . . 25
When not to use a mapping trait . . . 26
Summary . . . 26
Exercises . . . 26

2 Declarative macros . . . **29**
2.1 Matching tokens . . . 29
2.2 A `model!` macro . . . 30
2.3 Tuples, one line per arity . . . 32
2.4 Hygiene, and `$crate` . . . 33
2.5 Where declarative macros stop . . . 35
In shelf . . . 38
When not to use `macro_rules!` . . . 38
Summary . . . 39
Exercises . . . 39

3 The first derive macro . . . **43**
3.1 Proc-macro crates . . . 43
3.2 Three crates . . . 45
3.3 Parsing with syn 3 . . . 45
3.4 Generating with quote . . . 47
3.5 Hygiene . . . 49
3.6 Seeing the expansion . . . 51
3.7 Errors as values . . . 52
In shelf . . . 54

When not to write a derive 56
Summary 56
Exercises 57

4 Attributes, errors and generics 60
4.1 Helper attributes 60
4.2 Parsing with `parse_nested_meta` 61
4.3 The six attributes 66
4.4 Every error at once 71
4.5 UI tests 74
4.6 Generics 75
4.7 What darling would save 79
In shelf 84
When not to use an attribute 84
Summary 85
Exercises 85

5 Checking a statement once, not every row 88
5.1 The answer belongs to the statement 88
5.2 A statement knows its columns 89
5.3 Types checked once 92
5.4 What the derive generates 95
5.5 Measured 97
5.6 Where shelf resolves 99
In shelf 101
When not to resolve once 102
Summary 102
Exercises 103

6 Tables: the model derive 106
6.1 What a table adds 106
6.2 `#[derive(Table)]` 108
6.3 SQL at compile time 110
6.4 Reading 114
6.5 Constants, and where they stop 117
In shelf 121
When not to use `Table` 122
Summary 122
Exercises 122

7 The executor and shelf's errors 125
7.1 Why not `GenericClient` 125
7.2 The `Executor` trait 126
7.3 Three ways to send a statement 131
7.4 Transactions 133
7.5 `shelf::Error` 136
In shelf 140
When not to use the executor 141
Summary 141
Exercises 142

8 Writing rows: insert, update, delete, upsert 147
8.1 `#[derive(Insert)]` 147
8.2 Database defaults 151

8.3 Update and delete 154
8.4 Upsert 156
8.5 Constraint errors 158
In shelf 162
When not to use shelf's writes 163
Summary 163
Exercises 163

9 Partial updates and optimistic locking 167
9.1 Absent or null 167
9.2 Three designs 168
9.3 `#[derive(Patch)]` 172
9.4 Version columns 176
9.5 `updated_at` 179
9.6 ETags and `If-Match` in axum 180
In shelf 185
When not to use `Patch` 186
Summary 186
Exercises 186

10 A typed query builder 190
10.1 What to type 190
10.2 Column constants 190
10.3 Expressions 191
10.4 Owned parameters 191
10.5 Order, limit and pages 191
10.6 What fails to compile 191
10.7 The escape hatch 192
The programs 192
Side by side 192
In stockroom 192
Claims 192
When not to use the builder 192
Exercises 192

11 Relations without N+1 194
11.1 Declaring relations 194
11.2 N+1, measured 195
11.3 Batched loading 195
11.4 Pipelining independent loads 195
11.5 Joins 195
11.6 Why no lazy loading 196
The programs 196
Side by side 196
The data 196
Claims 196
When not to use relation loaders 196
Exercises 196

12 Bulk writes and streams 197
12.1 `insert_many` 197
12.2 Binary COPY from the derive 198
12.3 Measured 198

12.4 Streams of models 198
12.5 A COPY rejected at its start 199
The programs 199
Side by side 199
In stockroom 199
Claims 199
When not to use shelf's bulk writes 199
Exercises 199
13 Trusting the mapping: schema checks and tests 201
13.1 What can drift 201
13.2 `verify` 202
13.3 Where to run it 202
13.4 Testing a derive 202
13.5 Compile-time checking 202
The programs 203
Side by side 203
Claims 203
When not to use `verify` 203
Exercises 203
14 Capstone: stockroom on shelf 204
14.1 The shape 204
14.2 Models for a real schema 205
14.3 Line for line 205
14.4 Where shelf steps aside 205
14.5 What a production ORM would add 206
The programs 206
Side by side 206
The data 206
Claims 206
When not to use shelf 206
Exercises 207
Appendix — attribute reference, generated code and phrasebook 208
A Attribute reference 208
B What each derive generates 209
C Phrasebook 209
D Glossary 210
E Further reading 210

CHAPTER 0

Introduction and setup

Async PostgreSQL in Rust taught tokio-postgres as a driver: you wrote every SQL statement yourself, and you turned every row into a struct with a `TryFrom<&Row>` that named each column again (main book §3.5). This book builds the layer that writes that code for you. **shelf** is a small ORM on tokio-postgres: derive macros that map rows to structs, a model layer that writes the everyday SQL, partial updates with optimistic locking, a typed query builder, relations without N+1 queries, bulk writes, streams and a schema check. You build it one chapter at a time, and every stage of it is in this repository, compiled and tested.

The goal is not to replace diesel or SeaORM. It is to know what an ORM does underneath, by building one on a driver you already understand: what a derive macro receives and returns, how it reports a mistake at the right line, what SQL a model method sends and how many round trips it costs, and where hand-written SQL is still the better tool.

Contents

Chapter	You will learn
0 · Introduction and setup (this page)	What an ORM does, where shelf sits, the three crates, how the book follows the build, and the setup
1 · Mapping rows by hand	A `FromRow` trait, reading by name or by position and what each costs, NULL, nested structs, a fetch helper
2 · Declarative macros	`macro_rules!`, a `model!` macro, `FromRow` for twelve tuple sizes from one macro, hygiene, and where it stops
3 · The first derive macro	Proc-macro crates, syn and quote, the three-crate split, hygiene, seeing the expansion, errors that point at your code
4 · Attributes, errors and generics	`#[shelf(…)]`: `rename`, `skip`, `flatten`, `json`, `try_from`; several errors at once, UI tests, generics
5 · Checking a statement once	Resolving columns and checking their types once per statement instead of on every row, measured
6 · Tables	`#[derive(Table)]`, primary keys, SQL written at compile time, `find` and `all`
7 · The executor and shelf's errors	One trait over `Client`, `Transaction` and deadpool, statement strategies measured, `shelf::Error`
8 · Writing rows	`#[derive(Insert)]`, database defaults, `RETURNING`, updates, deletes and upserts
9 · Partial updates and optimistic locking	`#[derive(Patch)]` three ways, `Option<Option<T>>`, version columns, `updated_at`
10 · A typed query builder	Typed columns, expressions, `$n` numbering, keyset pages, and what fails to compile
11 · Relations without N+1	`belongs_to` and `has_many`, the N+1 problem measured, `= ANY($1)`, pipelined loads, joins
12 · Bulk writes and streams	`insert_many` through `unnest`, binary COPY written by the derive, streams of models

Chapter	You will learn
13 · Trusting the mapping	Checking models against the live schema, testing macros, and what compile-time checks add
14 · Capstone	Part of stockroom rewritten on shelf behind the same routes, compared line for line
A · Appendix	An attribute reference, what each derive generates, a phrasebook, a glossary

The whole book is also a single PDF, shelf-book.pdf. A chapter that is not written yet appears there, and in this repository, as a **sketch**: a plan of the chapter in its place, which the chapter replaces when it is written. orm/README.md shows which is which.

Part I (Chapters 1–5) is the mapping: from a hand-written trait, through `macro_rules!`, to a derive that checks a whole statement before it reads the first row. Part II (6–9) is the model: tables, writing rows, partial updates. Part III (10–14) is queries, relations, bulk work, and the capstone. Read them in order: each chapter grows the crate the previous one left.

0.1 What an ORM does, and what shelf will not do

An object-relational mapper does three jobs, and the main book did all three by hand:

1. **It maps rows to structs**, and structs back to parameters. That is the `TryFrom<&Row>` stockroom has for every type it reads, and the `&[&(dyn ToSql + Sync)]` list it builds for every `INSERT`.
2. **It writes the everyday SQL**: `SELECT … WHERE id = $1`, `INSERT … RETURNING`, `UPDATE … SET`, `DELETE`. Their shape follows from the struct and the table, and nobody enjoys writing them for the twentieth type.
3. **It follows relations** between tables: an order's lines, a line's product. Done naively, that is one query per row, the N+1 problem. Done well, it is one query per relation.

shelf does all three, in that order: Part I maps rows, Part II writes the everyday SQL, and Part III follows relations. It also leaves things out, each for a reason:

Left out	Why
Migrations, and DDL generated from structs	A schema is better written as SQL, in migrations (main book Chapter 14)
Checking queries against a live database at compile time	sqlx's `query!` and Cornucopia do it; Chapter 13 says what it adds and what it costs
Lazy loading, an identity map, change tracking, a unit of work	Each hides round trips or state; Chapter 11 says why async Rust makes lazy loading a poor fit
Other databases, and a blocking API	shelf is an ORM for tokio-postgres; abstracting over drivers would hide what the driver does
Postgres enums, composites and domains	postgres-types' derives already map them (main book Chapter 5), and shelf passes them through
Hooks such as `before_insert`	A plain function call before the write does the same, and you can see it

One thread runs through the whole book: **every shelf method says what SQL it sends and how many round trips it costs**, and every chapter ends by saying when not to use what it built. An ORM you cannot predict is an ORM you end up fighting. Building one is the surest way never to be surprised by the one you use.

0.2 Where shelf sits

Rust already has libraries that do some of these jobs. They differ less in features than in two decisions: who writes the SQL, and when a mistake in the mapping is caught.

Library	Built on	You write	A mapping mistake is caught
diesel, with diesel-async	tokio-postgres (diesel-async's PostgreSQL connection)	Rust: a schema (`table!`, usually generated from the database) and queries in its builder	At compile time, against the `table!` schema
SeaORM	sqlx	Entities (the `DeriveEntityModel` derive), `ActiveModels` for writes, queries in its builder	At run time
sqlx	its own driver	SQL; `#[derive(FromRow)]` maps rows	At compile time with `query!` and `query_as!` (against a live database, or query data saved from one); at run time with `FromRow`, on every row
postgres-from-row	tokio-postgres	SQL; `#[derive(FromRow)]` maps rows	At run time, on every row
Cornucopia	tokio-postgres	SQL in `.sql` files, from which Rust functions are generated	When the code is generated, against a live database
shelf	tokio-postgres	SQL where it matters; derives write the everyday statements	At run time, once per statement, before the first row is read; and against the table at startup, with `verify`

A newer entrant, **toasty**, is the Tokio project's ORM: models are structs with `#[derive(Model)]`, behind one API for SQL databases and DynamoDB, and its PostgreSQL driver holds a `tokio_postgres::Client`. It is young (0.11, with breaking changes in each release). The main book's Appendix F has the versions, dates and details of every crate in this section that is built on tokio-postgres. The sqlx and SeaORM illustrations in this book follow sqlx 0.9 and SeaORM 2.0.

What shelf decides differently, and why it is worth building rather than adopting:

- **The SQL stays visible.** A derive writes statements you can predict, and each chapter prints the ones it generates. Anything else is your SQL, mapped by the same derive. There is no second query language to learn for the hard cases.
- **The mapping is checked at run time, but once.** shelf compares the struct with the statement's column list when the statement is prepared, so a renamed column or a wrong type is an error that names the struct, the field, the column and both types, even for a query that returns no rows (Chapter 5). Checking at compile time catches more, and costs a database at build time; Chapter 13 weighs the two.
- **tokio-postgres is underneath, with nothing hidden.** shelf takes the `Client` and `Transaction` you already have (and deadpool's connections, Chapter 7). Pipelining, COPY, `LISTEN` and everything else in the main book still work on the same connection.

0.3 The finished API in ten lines

By the end of Part III, a model and its everyday SQL look like this. Chapters 6–13 build each piece, and this program runs them all, against temporary tables of its own. The ten lines that matter are the ones in `main` after the setup, each with a comment saying what it sends:

```
use jiff::Timestamp;
use shelf::{FromRow, Insert, Table};
use uuid::Uuid;

#[derive(Debug, FromRow, Table)]
#[shelf(table = "products")]
struct Product {
    #[shelf(primary_key, generated)]
    id: Uuid,
    sku: String,
    price_cents: i64,
    tags: Vec<String>,
    #[shelf(version)]
    version: i32,
    #[shelf(generated)]
    created_at: Timestamp,
}

#[derive(Insert)]
#[shelf(model = Product)]
struct NewProduct {
    sku: String,
    price_cents: i64,
    tags: Vec<String>,
}

#[derive(Debug, FromRow, Table)]
#[shelf(table = "orders", has_many(Line, key = order_id, as = lines))]
struct Order {
    #[shelf(primary_key)]
    id: i64,
}

#[derive(Debug, FromRow, Table)]
#[shelf(table = "lines")]
struct Line {
    #[shelf(primary_key, generated)]
    id: i64,
    order_id: i64,
    sku: String,
}

fn new_product(sku: &str, price_cents: i64, tags: &[&str]) -> NewProduct {
    NewProduct {
        sku: sku.into(),
        price_cents,
        tags: tags.iter().map(|t| t.to_string()).collect(),
    }
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let db = shelf_playground::connect().await?;
    db.batch_execute(
        "CREATE TEMP TABLE products (
             id uuid PRIMARY KEY DEFAULT uuidv7(),
             sku text NOT NULL UNIQUE,
             price_cents bigint NOT NULL,
             tags text[] NOT NULL DEFAULT '{}',
             version integer NOT NULL DEFAULT 1,
             created_at timestamptz NOT NULL DEFAULT now()
         );
```

```
         CREATE TEMP TABLE orders (id bigint PRIMARY KEY);
         CREATE TEMP TABLE lines (
             id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
             order_id bigint NOT NULL REFERENCES orders,
             sku text NOT NULL
         );
         INSERT INTO orders VALUES (1), (2);
         INSERT INTO lines (order_id, sku) VALUES (1, 'MUG-01'), (1, 'TEE-02'), (2,
'MUG-01');",
    )
    .await?;
    let new_mug = new_product("MUG-01", 1299, &["kitchen", "sale"]);
    let batch = [
        new_product("TEE-02", 2499, &["sale"]),
        new_product("VASE-03", 899, &["sale"]),
        new_product("LAMP-04", 4999, &["home"]),
    ];

    let mug = new_mug.insert(&db).await?; // INSERT … RETURNING
    let mug = Product::find(&db, &mug.id).await?; // SELECT … WHERE "id" = $1, or
Error::NotFound
    NewProduct::copy_in(&db, &batch).await?; // the whole batch in one binary COPY
    let query = Product::query()
        .filter(
            Product::PRICE_CENTS
                .gt(1_000)
                .and(Product::TAGS.contains("sale")),
        )
        .order_by(Product::SKU.asc())
        .limit(20);
    let page = query.all(&db).await?; // one SELECT, three parameters
    let orders = Order::all(&db).await?; // SELECT … ORDER BY "id"
    let lines = Order::load_lines(&db, &orders).await?; // one query for every order: =
ANY($1)
    shelf::verify::<Product>(&db).await?; // the struct against the table's columns

    println!("{}", query.sql());
    let skus: Vec<&str> = page.iter().map(|p| p.sku.as_str()).collect();
    let per_order: Vec<usize> = lines.iter().map(Vec::len).collect();
    println!("page: {skus:?}; lines per order: {per_order:?}");
    assert_eq!((mug.sku.as_str(), mug.version), ("MUG-01", 1));
    assert!(mug.created_at <= Timestamp::now());
    assert_eq!(skus, ["MUG-01", "TEE-02"]);
    assert_eq!(per_order, [2, 1]);
    assert_eq!(lines[0][1].sku, "TEE-02");
    assert!(
        lines
            .iter()
            .flatten()
            .all(|line| line.id > 0 && line.order_id > 0)
    );
    Ok(())
}
```

```
SELECT "id", "sku", "price_cents", "tags", "version", "created_at" FROM "products" WHERE
"price_cents" > $1 AND "tags" @> $2 ORDER BY "sku" LIMIT $3
page: ["MUG-01", "TEE-02"]; lines per order: [2, 1]
```

Each call says in its comment what it sends:

- `insert` sends one `INSERT … RETURNING` with the model's columns, and the database fills in `id` (with `uuidv7()`, PG 18+), `version` and `created_at` (Chapter 8).
- `find` sends `SELECT <every column> FROM products WHERE id = $1`, and turns "no row" into an error you can match, `Error::NotFound` (Chapter 6).
- `copy_in` writes the whole batch with one binary `COPY`. Its sibling `insert_many` writes a batch with one `INSERT … SELECT … FROM unnest(…)`, for structs without array columns such as `tags` (Chapter 12).

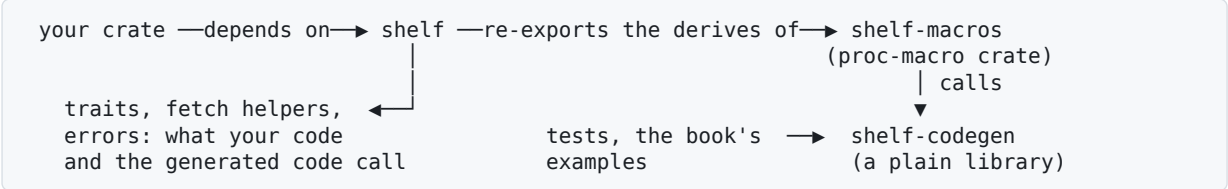
- The query builder renders one `SELECT` with `$1`, `$2` and `$3` as parameters, printed above. Its columns are typed, so comparing `PRICE_CENTS` with a string does not compile (Chapter 10).
- `load_lines` reads the lines of every order in the slice with one query, not one per order (Chapter 11).
- `verify` compares the struct with the table, so drift is caught at startup instead of at the first request that reads the column (Chapter 13).

On a plain `Client`, as here, shelf prepares each statement before it runs it: that is the round trip `query` makes with SQL text anyway, and the moment shelf checks the struct against the statement's columns (Chapter 5). On a deadpool connection, the statement comes from the connection's cache after the first call (Chapter 7).

0.4 Three crates, and why

A derive macro is a function that the compiler calls with the tokens of your struct, and that returns new tokens. Rust requires that function to live in a crate of its own kind, a *proc-macro crate*, which can export macros and nothing else. That is inconvenient: code that generates code is code you want to test and to show, and nothing outside the compiler can call a function in a proc-macro crate, neither a test in another crate nor an example in this book.

So shelf is three crates, and the code that writes code lives in the one that is an ordinary library:



Crate	Kind	What it holds
`shelf`	library	What you depend on: the traits and functions your code calls, the error type, and the derives, re-exported. A hidden module, `__private`, holds what generated code names, so your crate does not need its own dependency on tokio-postgres
`shelf-macros`	proc-macro	A shell: per derive, three lines that parse the input, call shelf-codegen, and return the result or the error
`shelf-codegen`	library	Everything the derives generate, as functions from a parsed struct to tokens. Its own tests, and this book's examples, call them directly

Two more crates sit beside them: `orm/playground`, which holds this book's examples, and `orm/stockroom-shelf`, Chapter 14's capstone, which serves part of stockroom's API on shelf. Chapter 3 builds the split from the inside, and explains the one type that makes it possible, `proc_macro2::TokenStream`.

0.5 How this book follows the build

A build-along book has a problem: each chapter shows the crate at one stage, but the repository holds the finished crate. If the book quoted only the finished code, Chapter 3 would show attribute parsing that Chapter 4 explains, and checks that Chapter 5 motivates. This book handles it in four ways:

1. **Stage programs.** Each new mechanism appears first as a complete, runnable program in one file: a hand-written impl, a `macro_rules!` macro, or a derive's logic written as an ordinary function from a parsed struct to tokens. A proc macro cannot be defined in a single file, but its logic can, because syn and quote work in any program. So even the macro code in this book is compiled, run and checked.
2. **The finished crate, in excerpts.** Then the chapter shows how shelf does it, in excerpts whose first line names the file. A tool checks every excerpt against the file it quotes, and code that later chapters add is elided with `// …`.

3. **Behaviour, through the finished crate.** Claims about what shelf does are runnable programs that use `#[derive(shelf::FromRow)]` and friends against a real database.
4. **Nothing generated is typed by hand.** What a derive expands to is printed by running shelf-codegen, and compiler errors are pasted from a real build, as the main book pasted psql output.

Each chapter then follows the main book's pattern. **Side by side** sections solve one task three ways: **by hand** with tokio-postgres, as the main book would; with **shelf**; and with an established crate where it fits the topic (sqlx for row mapping, SeaORM for models and relations, diesel for the query builder). The first two are runnable programs; the third is an illustration, not compiled, and its first line names the crate. A table of what is the **same** and what is **different** follows. **In shelf** sections show the finished crate, an **In stockroom** section appears where stockroom's hand-written code is the comparison, and a section on **when not to** use what the chapter built comes before the summary and the exercises.

Five kinds of boxes appear where they are earned. The first four are the main book's:

> **Protocol lens:** what travels over the socket.

> **SQL lens:** what the server does with it.

> **Rust refresher:** a language feature you may be hazy on, where it first matters: `PhantomData`, async methods in traits, `LazyLock`.

> **Pitfall:** a mistake that compiles, passes a quick test, and fails in production.

The fifth is new:

> **Macro lens:** what the compiler hands a macro and what it takes back: tokens, spans, hygiene, and the order in which things expand.

Code blocks come in the main book's two kinds. A plain `rust` block is a complete program that was compiled, linted and run against a fresh database, and its `assert!`s say what it produces. A block whose first line names a file, a crate or "illustration" is an excerpt or a sketch, and is not compiled on its own.

0.6 Setup

What you need

The main book's setup, from its Chapter 0: PostgreSQL 18, a database for the examples named by `DATABASE_URL`, and Rust 1.85 or newer for the 2024 edition. This book was written with rustc 1.99.1.

shelf is not published on crates.io. It lives in this repository, next to the main book's workspace, and so do its examples. Work from a clone of the repository; every command below runs from its root.

The playground

The book's examples run in the crate `orm/playground`, whose package is `shelf-playground`. It depends on `shelf` and `shelf-codegen` by path, on the macro crates syn, quote, proc-macro2 and prettyplease, and on what the main book's playground has (tokio, tokio-postgres with its features, serde, jiff, uuid, deadpool-postgres). Unlike the main book's playground, it takes every version from the workspace's `Cargo.toml`, so there is nothing to copy.

Its library re-exports the main book's helpers, so an example connects with `shelf_playground::connect()`, which reads `DATABASE_URL` exactly as the main book's `connect()` does. It adds one helper of its own, `pretty`, which formats generated code; Chapter 3 prints it.

Each runnable block in a chapter becomes one file in `orm/playground/examples/`, named after the chapter and its position: the first program in this chapter, §0.3's, is `shelf_ch00_01`. To run one:

```
python3 tools/book.py --book orm extract                # write orm/playground/examples/
from the book
export DATABASE_URL=postgres://postgres:secret@localhost:5432/tpbook
cargo run -p shelf-playground --example shelf_ch00_01
```

> **Pitfall:** `orm/playground/examples/` is generated, and `extract` (like `check`) deletes every `.rs` file there that the book did not produce. Keep your own programs in `orm/playground/src/bin/` instead: `orm/playground/src/bin/mine.rs` runs with `cargo run -p shelf-playground --bin mine`.

To check a chapter the way it was checked when it was written, let `book.py` build it, lint it with clippy, and run every program against a database it creates for the purpose, `tpbook_shelf_check`. It needs `ADMIN_DATABASE_URL`, a connection with the right to create databases, which the repository's `.env.example` describes:

```
python3 tools/book.py --book orm check --chapter 0      # one chapter's examples
python3 tools/book.py --book orm check                  # the whole book
```

shelf's own tests

shelf is tested apart from the book: unit tests, tests that pin down what each derive expands to, tests that pin down the compiler errors it causes, and integration tests that give each test its own schema in a database named `shelf_test`. Chapter 13 explains each kind.

```
cargo test -p shelf -p shelf-codegen                    # needs ADMIN_DATABASE_URL
```

0.7 The first derive, end to end

Here is shelf as it stands at the end of Part I. `#[derive(FromRow)]` reads each named field from the column of the same name. This program derives it for a struct and reads two rows into it with `shelf::fetch_all`:

```
use shelf::FromRow;

#[derive(Debug, FromRow)]
struct Product {
    id: i64,
    sku: String,
    price_cents: i64,
    tags: Option<Vec<String>>,
}

#[tokio::main]
```

```
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (
                 id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
                 sku text NOT NULL UNIQUE,
                 price_cents bigint NOT NULL,
                 tags text[]
             );
             INSERT INTO products (sku, price_cents, tags)
             VALUES ('MUG-01', 1299, '{kitchen}'), ('TEE-02', 2499, NULL);",
        )
        .await?;

    let products: Vec<Product> = shelf::fetch_all(
        &client,
        "SELECT id, sku, price_cents, tags FROM products ORDER BY id",
        &[],
    )
    .await?;

    println!("{products:#?}");
    assert_eq!(products.len(), 2);
    assert!(products[0].id < products[1].id);
    assert_eq!(products[0].sku, "MUG-01");
    assert_eq!(products[0].price_cents, 1299);
    assert_eq!(products[0].tags, Some(vec!["kitchen".to_string()]));
    assert_eq!(products[1].tags, None);
    Ok(())
}
```

```
[
    Product {
        id: 1,
        sku: "MUG-01",
        price_cents: 1299,
        tags: Some(
            [
                "kitchen",
            ],
        ),
    },
    Product {
        id: 2,
        sku: "TEE-02",
        price_cents: 2499,
        tags: None,
    },
]
```

`use shelf::FromRow` brings in two things with one name: the trait and the derive macro. That works because traits and macros live in different namespaces, and it is how serde's `Serialize` works too. The NULL in `tags` became `None` because the field is an `Option`, exactly as with `row.try_get` in the main book.

What did the derive write? Because the code generation lives in `shelf-codegen`, an ordinary library (§0.4), this program can call it on a struct and print the result:

```
use syn::{DeriveInput, parse_quote};

fn main() {
    let input: DeriveInput = parse_quote! {
        struct Product {
            id: i64,
            sku: String,
        }
```

```
    };
    let generated = shelf_codegen::from_row::expand(&input).expect("a struct with named
fields");
    let code = shelf_playground::pretty(generated);
    println!("{code}");
    assert!(code.contains(r#"column::<String>(columns, "Product", "sku", "sku")?"#));
}
```

It prints the `impl` that the compiler sees next to the struct in place of `#[derive(FromRow)]`:

```
#[automatically_derived]
impl ::shelf::FromRow for Product {
    type Columns = (usize, usize);
    fn resolve(
        columns: &[::shelf::__private::tokio_postgres::Column],
    ) -> ::core::result::Result<Self::Columns, ::shelf::Error> {
        ::core::result::Result::Ok((
            ::shelf::__private::column::<i64>(columns, "Product", "id", "id")?,
            ::shelf::__private::column::<String>(columns, "Product", "sku", "sku")?,
        ))
    }
    fn from_row_at(
        row: &::shelf::__private::tokio_postgres::Row,
        columns: &Self::Columns,
    ) -> ::core::result::Result<Self, ::shelf::Error> {
        ::core::result::Result::Ok(Self {
            id: ::shelf::__private::get::<i64>(row, columns.0, "Product", "id")?,
            sku: ::shelf::__private::get::<String>(row, columns.1, "Product", "sku")?,
        })
    }
}
```

Three things in it are worth noticing now, and each gets a chapter:

- The field names appear as strings, `"id"` and `"sku"`: the derive turned each field's name into the column it reads, the job you did by hand with `row.try_get("sku")`. Chapter 1 writes that by hand once more, and Chapters 2 and 3 make a macro write it.
- Every path is absolute: `::core::result::Result`, never `Result`. The `impl` lands in your module, where `Result` may mean something else. Chapter 3 calls this hygiene.
- There are two steps. `resolve` finds and checks each field's column once per statement, and `from_row_at` reads each row by position. Chapter 5 measures why.

Summary

- An ORM maps rows to structs, writes the everyday SQL, and follows relations. shelf does all three, and leaves out migrations, compile-time query checking, lazy loading, other databases and hooks.
- Libraries differ in who writes the SQL and when a mapping mistake is caught. shelf keeps the SQL visible and checks the mapping at run time, once per statement, before the first row.
- shelf is three crates: `shelf` (what you use), `shelf-macros` (a thin proc-macro shell) and `shelf-codegen` (the code generation, as a plain library that tests and the book call).
- Each mechanism appears first as a stage program, then as excerpts of the finished crate; behaviour is shown through the real derives, and expansions and compiler errors are captured, never typed.
- The book's examples run in `orm/playground`; `book.py --book orm extract` writes them and `check` runs them. Keep your own programs in `orm/playground/src/bin/`.

Exercises

1. **A missing column.** Add a field `weight_grams: i32` to the first program's `Product`, which the table does not have, and run it. What does the error name, and does the query run at all? *Hint:* it names the struct, the field and the column. Try it with `WHERE false` added to the query, which returns no rows.
2. **A renamed column.** Rename the table's `sku` column to `sku_code`, and keep the program working without changing the struct. *Hint:* the derive reads the column names the *query* produces, and `AS` renames a column in the result. (Chapter 4's `#[shelf(rename = "…")]` is the other way.)
3. **A keyword as a field name.** Add a field `r#type: String` to the struct in the second program and print the expansion again. Which column does it read? *Hint:* `type` is a keyword, so the field needs the raw-identifier prefix `r#`; the column does not.

Solutions

1. The error names all three: the struct, the field, and the column it looked for. It is reported before any row is read, because `fetch_all` checks the struct against the statement's column list when it prepares the statement, so it fails the same way when the query returns no rows. Chapter 5 shows how.

```
use shelf::FromRow;

#[derive(Debug, FromRow)]
#[allow(dead_code)] // the program only shows the error
struct Product {
    id: i64,
    sku: String,
    weight_grams: i32,
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (id bigint PRIMARY KEY, sku text NOT NULL);
             INSERT INTO products VALUES (1, 'MUG-01');",
        )
        .await?;

    for sql in [
        "SELECT id, sku FROM products",
        "SELECT id, sku FROM products WHERE false",
    ] {
        let error = shelf::fetch_all::<Product>(&client, sql, &[])
            .await
            .unwrap_err();
        println!("{error}");
        assert_eq!(
            error.to_string(),
            r#"Product.weight_grams: the result has no column "weight_grams""#
        );
    }
    Ok(())
}
```

```
Product.weight_grams: the result has no column "weight_grams"
Product.weight_grams: the result has no column "weight_grams"
```

2. Rename the column back in the query, `SELECT id, sku_code AS sku, …`. The derive never sees the table, only the columns of the result:

```
use shelf::FromRow;
```

```
#[derive(Debug, FromRow)]
struct Product {
    id: i64,
    sku: String,
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (id bigint PRIMARY KEY, sku_code text NOT NULL);
             INSERT INTO products VALUES (1, 'MUG-01');",
        )
        .await?;

    let products: Vec<Product> = shelf::fetch_all(
        &client,
        "SELECT id, sku_code AS sku FROM products ORDER BY id",
        &[],
    )
    .await?;
    assert_eq!(products[0].id, 1);
    assert_eq!(products[0].sku, "MUG-01");
    Ok(())
}
```

3. It reads the column `type`. The `r#` prefix is how Rust spells a keyword used as an identifier, and the derive strips it before it uses the name as a column (Chapter 3, §3.5):

```
use syn::{DeriveInput, parse_quote};

fn main() {
    let input: DeriveInput = parse_quote! {
        struct Product {
            id: i64,
            r#type: String,
        }
    };
    let code = shelf_playground::pretty(shelf_codegen::from_row::expand(&input).unwrap());
    assert!(code.contains(r#"column::<String>(columns, "Product", "type", "type")?"#));
    assert!(code.contains("r#type: ::shelf::__private::get::<String>"));
}
```

CHAPTER 1

Mapping rows by hand

You have written this function before. Every struct that stockroom reads from the database has a `TryFrom<&Row>` that names each field and reads each column: `id: row.try_get("id")?`, `code: row.try_get("code")?`, once per field, for every struct (main book §3.5). The SQL above it names the same columns again. Rename a column and you change three places, and nothing checks any of them against the others until a row arrives.

An ORM's first job is to write that function for you. Before writing the macro that does it, this chapter writes it by hand once more, carefully: as a trait of shelf's own, with the three ways it can fail, nested structs, and helpers that run a query and map its rows. It also measures what reading a column by name actually costs, because that number decides the shape of the derive in Part I.

In this chapter you will learn:

- why shelf has a `FromRow` trait of its own instead of using `TryFrom<&Row>`
- what `Row::try_get("sku")` does in tokio-postgres 0.7.18, and what it costs per row
- the three ways a mapping fails, and what each error says and does not say
- how to read a nested struct from the same row
- `fetch_all`, `fetch_one` and `fetch_opt`: a query and its mapping in one call, on a client or a transaction
- which parts of the hand-written code a macro should take over

1.1 A trait of shelf's own

The mapping needs a name. stockroom uses the standard library's `TryFrom<&Row>`; shelf defines a trait instead, with one method for now:

```
use tokio_postgres::Row;

/// A value built from one result row.
trait FromRow: Sized {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error>;
}

#[derive(Debug, PartialEq)]
struct Product {
    id: i64,
    sku: String,
    name: String,
    price_cents: i64,
    tags: Option<Vec<String>>,
}

impl FromRow for Product {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
        Ok(Product {
            id: row.try_get("id")?,
            sku: row.try_get("sku")?,
            name: row.try_get("name")?,
```

```
            price_cents: row.try_get("price_cents")?,
            tags: row.try_get("tags")?,
        })
    }
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (
                id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
                sku text NOT NULL UNIQUE,
                name text NOT NULL,
                price_cents bigint NOT NULL,
                tags text[]
             );
             INSERT INTO products (sku, name, price_cents, tags)
             VALUES ('MUG-01', 'Mug', 1299, '{kitchen,sale}'),
                    ('TEE-02', 'T-shirt', 2499, NULL);",
        )
        .await?;

    let rows = client
        .query(
            "SELECT id, sku, name, price_cents, tags FROM products ORDER BY id",
            &[],
        )
        .await?;
    let products = rows
        .iter()
        .map(Product::from_row)
        .collect::<Result<Vec<_>, _>>()?;

    assert_eq!(products.len(), 2);
    assert_eq!(
        products[0].tags,
        Some(vec!["kitchen".into(), "sale".into()])
    );
    assert_eq!(
        products[1],
        Product {
            id: products[1].id,
            sku: "TEE-02".into(),
            name: "T-shirt".into(),
            price_cents: 2499,
            tags: None,
        }
    );
    Ok(())
}
```

The body is the same as stockroom's `try_from`. What a trait of shelf's own buys is room to grow, and three things `TryFrom<&Row>` cannot give:

- **Associated items.** Chapter 5 adds an associated type and two methods to `FromRow`, so that a statement's columns are found once instead of on every row. `TryFrom` is the standard library's trait, and nobody can add to it.
- **A short bound.** A generic helper over `TryFrom<&Row>` must say "for a row borrowed for any lifetime", as stockroom's `from_rows` does: `for<'r> T: TryFrom<&'r Row, Error = tokio_postgres::Error>`. Over `FromRow` it says `T: FromRow`.
- **Impls for types that are not yours.** Reading `SELECT sku, price_cents` into a tuple, `(String, i64)`, needs an impl for tuples. Rust's *orphan rule* allows an impl only if the trait or the type belongs to your

crate, and for `TryFrom<&Row> for (A, B)`, neither does. This fails to compile in any crate but tokio-postgres itself:

```
use tokio_postgres::Row;
use tokio_postgres::types::FromSqlOwned;

impl<A: FromSqlOwned, B: FromSqlOwned> TryFrom<&Row> for (A, B) {
    type Error = tokio_postgres::Error;
    fn try_from(row: &Row) -> Result<Self, Self::Error> {
        Ok((row.try_get(0)?, row.try_get(1)?))
    }
}
```

```
error[E0117]: only traits defined in the current crate can be implemented for arbitrary
types
 --> src/bin/orphan.rs:4:1
  |
4 | impl<A: FromSqlOwned, B: FromSqlOwned> TryFrom<&Row> for (A, B) {
  | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  |                                         |                 |
  |                                         |                 this is not defined in the
current crate because tuples are always foreign
  |                                         `tokio_postgres::Row` is not defined in the
current crate
  |
  = note: impl doesn't have any local type before any uncovered type parameters
  = note: for more information see https://doc.rust-lang.org/reference/items/implementations
.html#orphan-rules
  = note: define and implement a trait or new type instead
```

With `FromRow` defined in shelf, `impl<A, B> FromRow for (A, B)` is allowed, because the trait is local. Chapter 2 writes those impls for twelve tuple sizes at once.

> **Rust refresher:** `FromSqlOwned` is tokio-postgres's name for "a type that can be read from a column without borrowing from the row", a shorthand for `for<'a> FromSql<'a>` (main book §3.4). `String` is one; `&str` is not, because it points into the row's buffer. A mapping trait whose method returns `Self` with no lifetime, like `FromRow`, can only build owned values, and shelf keeps it that way: Chapter 4 refuses a struct with a lifetime parameter, with an error that says why.

1.2 By name or by position

`row.try_get("sku")` looks like a map lookup, but a `Row` has no map. In 0.7.18 it walks the row's columns and compares each name with `"sku"`. If none matches exactly, it walks them again, ignoring ASCII case (`row.rs`):

```
// tokio-postgres 0.7.18: src/row.rs, abridged
impl RowIndex for str {
    fn __idx<T>(&self, columns: &[T]) -> Option<usize>
    where
        T: AsName,
    {
        if let Some(idx) = columns.iter().position(|d| d.as_name() == self) {
            return Some(idx);
        };

        // FIXME ASCII-only case insensitivity isn't really the right thing to
        // do. Postgres itself uses a dubious wrapper around tolower and JDBC
        // uses the US locale.
        columns
            .iter()
            .position(|d| d.as_name().eq_ignore_ascii_case(self))
```

```
    }
}
```

Then, whether you asked by name or by position, `try_get` checks that the column's type can be read into the Rust type you asked for, and only then decodes the bytes:

```
// tokio-postgres 0.7.18: src/row.rs, abridged
fn get_inner<'a, I, T>(&'a self, idx: &I) -> Result<T, Error>
where
    I: RowIndex + fmt::Display,
    T: FromSql<'a>,
{
    let idx = match idx.__idx(self.columns()) {
        Some(idx) => idx,
        None => return Err(Error::column(idx.to_string())),
    };

    let ty = self.columns()[idx].type_();
    if !T::accepts(ty) {
        return Err(Error::from_sql(
            Box::new(WrongType::new::<T>(ty.clone())),
            idx,
        ));
    }

    FromSql::from_sql_nullable(ty, self.col_buffer(idx)).map_err(|e| Error::from_sql(e,
idx))
}
```

So a struct of twelve fields read by name does twelve searches per row. A search that finds its column at position p compares $p + 1$ names, so the twelve together compare $1 + 2 + \ldots + 12 = 78$ names on every row, whatever order the columns come in. A missing column costs two full walks, and the second walk means `"SKU"` finds a column named `sku`.

What does that cost? This program reads 100,000 rows of twelve columns (four `bigint`, four `text`, two `integer`, a `boolean` and a `double precision`, two of them sometimes NULL) into the same struct twice: by name, and by position. It fetches the rows once and times only the mapping, best of three rounds:

```
use std::time::{Duration, Instant};
use tokio_postgres::Row;

#[allow(dead_code)] // only the mapping is measured
struct Wide {
    c01: i64,
    c02: String,
    c03: i32,
    c04: Option<String>,
    c05: i64,
    c06: bool,
    c07: String,
    c08: i64,
    c09: Option<i32>,
    c10: String,
    c11: i64,
    c12: f64,
}

fn by_name(r: &Row) -> Result<Wide, tokio_postgres::Error> {
    Ok(Wide {
        c01: r.try_get("c01")?,
        c02: r.try_get("c02")?,
        c03: r.try_get("c03")?,
        c04: r.try_get("c04")?,
        c05: r.try_get("c05")?,
        c06: r.try_get("c06")?,
```

```
        c07: r.try_get("c07")?,
        c08: r.try_get("c08")?,
        c09: r.try_get("c09")?,
        c10: r.try_get("c10")?,
        c11: r.try_get("c11")?,
        c12: r.try_get("c12")?,
    })
}

// The SELECT lists the columns in reverse, so c01 is at position 11.
fn by_position(r: &Row) -> Result<Wide, tokio_postgres::Error> {
    Ok(Wide {
        c01: r.try_get(11)?,
        c02: r.try_get(10)?,
        c03: r.try_get(9)?,
        c04: r.try_get(8)?,
        c05: r.try_get(7)?,
        c06: r.try_get(6)?,
        c07: r.try_get(5)?,
        c08: r.try_get(4)?,
        c09: r.try_get(3)?,
        c10: r.try_get(2)?,
        c11: r.try_get(1)?,
        c12: r.try_get(0)?,
    })
}

fn best_of_three(
    rows: &[Row],
    map: fn(&Row) -> Result<Wide, tokio_postgres::Error>,
) -> Result<Duration, tokio_postgres::Error> {
    let mut best = Duration::MAX;
    for _ in 0..3 {
        let start = Instant::now();
        let mapped: Vec<Wide> = rows.iter().map(map).collect::<Result<_, _>>()?;
        best = best.min(start.elapsed());
        assert_eq!(mapped.len(), rows.len());
    }
    Ok(best)
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = shelf_playground::connect().await?;
    let rows = client
        .query(
            "SELECT i::float8 AS c12, i AS c11, 'name ' || i AS c10,
                    CASE WHEN i % 3 = 0 THEN NULL ELSE i::int END AS c09,
                    i AS c08, 'tag ' || i AS c07, i % 2 = 0 AS c06, i AS c05,
                    CASE WHEN i % 5 = 0 THEN NULL ELSE 'note' END AS c04,
                    i::int AS c03, 'sku-' || i AS c02, i AS c01
             FROM generate_series(1, 100000::bigint) AS i",
            &[],
        )
        .await?;

    let name = best_of_three(&rows, by_name)?;
    let position = best_of_three(&rows, by_position)?;
    println!(
        "{} rows: by name {name:.1?}, by position {position:.1?} ({:.1}×)",
        rows.len(),
        name.as_secs_f64() / position.as_secs_f64()
    );
    assert!(position < name);
    Ok(())
}
```

```
100000 rows: by name 197.8ms, by position 123.2ms (1.6×)
```

That run was a debug build, which is how `book.py check` builds the examples (rustc 1.99.1, Apple Silicon, PostgreSQL 18.6 on localhost); three runs gave 1.5× to 1.7×. In a release build the same comparison is 38 ms by name against 12.5 ms by position, about **3×** (orm/PLAN.md §12, findings 8 and 13). The lookups, not the decoding, are most of the work, and they repeat on every row although the answer cannot change: every row of a statement has the same columns, in the same order.

Reading by position is not the fix, though. Positions break silently when someone adds a column to the `SELECT`, which is why the main book recommends names. The fix is to look each name up **once per statement** and then read every row by position. That needs a trait with somewhere to keep the positions, which is the first thing `FromRow` will grow (Chapter 5).

> **Pitfall:** the second, case-insensitive walk means a misspelt name can still find a column: `row.try_get("SKU")` reads a column named `sku`. PostgreSQL itself folds unquoted identifiers to lower case, and a quoted `"SKU"` is a different column from `sku`, so a mapping that matches names ignoring case can read the wrong one when both exist. shelf matches names exactly.

1.3 Three ways to fail

A row can fail to fit a struct in three ways: a column the struct needs is missing, a column has a type the field cannot hold, or a column is NULL and the field is not an `Option`. This program provokes each with a query that gets it wrong, and prints the error's `Display` and its `Debug`:

```
use tokio_postgres::Row;

trait FromRow: Sized {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error>;
}

#[derive(Debug)]
#[allow(dead_code)] // the program only shows the errors
struct Product {
    sku: String,
    name: String,
    price_cents: i64,
}

impl FromRow for Product {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
        Ok(Product {
            sku: row.try_get("sku")?,
            name: row.try_get("name")?,
            price_cents: row.try_get("price_cents")?,
        })
    }
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = shelf_playground::connect().await?;
    let mistakes = [
        // No `name` column.
        (
            "SELECT 'MUG-01' AS sku, 1299::int8 AS price_cents",
            "invalid column `name`",
        ),
        // `1299` is an int4, and the field is an i64.
        (
            "SELECT 'MUG-01' AS sku, 'Mug' AS name, 1299 AS price_cents",
            "error deserializing column 2",
        ),
```

```
        // A NULL name, and the field is a String.
        (
            "SELECT 'MUG-01' AS sku, NULL::text AS name, 1299::int8 AS price_cents",
            "error deserializing column 1",
        ),
    ];
    for (sql, expected) in mistakes {
        let row = client.query_one(sql, &[]).await?;
        let error = Product::from_row(&row).unwrap_err();
        println!("{error}\n    {error:?}");
        assert_eq!(error.to_string(), expected);
    }
    Ok(())
}
```

```
invalid column `name`
    Error { kind: Column("name"), cause: None }
error deserializing column 2
    Error { kind: FromSql(2), cause: Some(WrongType { postgres: Int4, rust: "i64" }) }
error deserializing column 1
    Error { kind: FromSql(1), cause: Some(WasNull) }
```

tokio-postgres reports what it knows: a column, by name or position, and, one level down in `source()`, the cause. What it cannot report is what you were doing: which struct, and which field. In a service with forty row types, "error deserializing column 2" in a log is the start of a search. Your `from_row` knows the struct and the field, but adding them by hand means a `map_err` on every line (Exercise 1).

This is what shelf reports for the same three queries, with the same struct and `#[derive(shelf::FromRow)]`:

```
#[derive(Debug, shelf::FromRow)]
#[allow(dead_code)] // the program only shows the errors
struct Product {
    sku: String,
    name: String,
    price_cents: i64,
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    let mistakes = [
        (
            "SELECT 'MUG-01' AS sku, 1299::int8 AS price_cents",
            r#"Product.name: the result has no column "name""#,
        ),
        (
            "SELECT 'MUG-01' AS sku, 'Mug' AS name, 1299 AS price_cents",
            r#"Product.price_cents: column "price_cents" has type int4, which cannot be read
into i64"#,
        ),
        (
            "SELECT 'MUG-01' AS sku, NULL::text AS name, 1299::int8 AS price_cents",
            r#"Product.name: column "name" is NULL; read it into an Option"#,
        ),
    ];
    for (sql, expected) in mistakes {
        let error = shelf::fetch_one::<Product>(&client, sql, &[])
            .await
            .unwrap_err();
        println!("{error}");
        assert_eq!(error.to_string(), expected);
    }
    Ok(())
}
```

```
Product.name: the result has no column "name"
Product.price_cents: column "price_cents" has type int4, which cannot be read into i64
Product.name: column "name" is NULL; read it into an Option
```

Each message names the struct, the field, the column and, for a type mismatch, both types. The first two are not even row errors in shelf: they are found when the statement is prepared, before the query runs (Chapter 5). The third can only be found in a row, because a column's type says nothing about whether it holds NULL.

> **SQL lens:** why is `1299` an `int4` and `'Mug'` a `text`? An integer literal that fits in 32 bits is typed `integer`; a quoted literal in a `SELECT` list, with nothing else to go on, is resolved as `text`. The server decides each column's type when it prepares the statement, and the row description carries it to the client. That is the type `accepts` checks: the Rust side never guesses, and never widens an `int4` into an `i64` for you.

1.4 Nested structs

Some structs are better split. A price is an amount and a currency, and it has behaviour of its own (formatting, arithmetic), so it deserves a type. But in the table it is two columns of the `products` row, not a table of its own. Reading it by hand means calling the inner struct's `from_row` from the outer one, on the same row:

```
use tokio_postgres::Row;

trait FromRow: Sized {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error>;
}

#[derive(Debug, PartialEq)]
struct Price {
    cents: i64,
    currency: String,
}

impl FromRow for Price {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
        Ok(Price {
            cents: row.try_get("price_cents")?,
            currency: row.try_get("currency")?,
        })
    }
}

#[derive(Debug, PartialEq)]
struct Product {
    sku: String,
    price: Price,
}

impl FromRow for Product {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
        Ok(Product {
            sku: row.try_get("sku")?,
            price: Price::from_row(row)?, // the same row, read by the inner struct
        })
    }
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
```

```
            "CREATE TEMP TABLE products (
                 sku text PRIMARY KEY,
                 price_cents bigint NOT NULL,
                 currency text NOT NULL
             );
             INSERT INTO products VALUES ('MUG-01', 1299, 'EUR');",
        )
        .await?;

    let row = client
        .query_one("SELECT sku, price_cents, currency FROM products", &[])
        .await?;
    let product = Product::from_row(&row)?;
    assert_eq!(
        product,
        Product {
            sku: "MUG-01".into(),
            price: Price {
                cents: 1299,
                currency: "EUR".into()
            },
        }
    );
    Ok(())
}
```

It works, and it shows the catch: the inner struct and the outer one take their names from the same row. `Price` reads `price_cents` and `currency`, so it fits this query and no other. A product with two prices, a list price and a sale price, cannot use `Price` twice: both would read the same two columns. You would alias them in SQL and give `Price` a second impl, or pass a prefix in at run time and build each name with `format!`, one allocation per column per row.

Chapter 4's `#[shelf(flatten)]` does the first half with one attribute: it reads a nested `FromRow` struct from the same row. The column names stay the inner struct's business, which you set with its own `#[shelf(rename = "…")]` attributes.

1.5 Fetch helpers

Every read so far has had the same three steps: run the query, map each row, collect the results into a `Result<Vec<_>, _>`. A helper does all three, and two siblings cover the other shapes a result can have, exactly one row and at most one row:

```
use tokio_postgres::types::ToSql;
use tokio_postgres::{GenericClient, Row};

trait FromRow: Sized {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error>;
}

/// Every row of `sql`, read into `T`.
async fn fetch_all<T: FromRow>(
    db: &impl GenericClient,
    sql: &str,
    params: &[&(dyn ToSql + Sync)],
) -> Result<Vec<T>, tokio_postgres::Error> {
    let rows = db.query(sql, params).await?;
    rows.iter().map(T::from_row).collect()
}

/// The one row of `sql`; an error if there are none, or several.
async fn fetch_one<T: FromRow>(
    db: &impl GenericClient,
    sql: &str,
```

```
    params: &[&(dyn ToSql + Sync)],
) -> Result<T, tokio_postgres::Error> {
    let row = db.query_one(sql, params).await?;
    T::from_row(&row)
}

/// The row of `sql`, if there is one; an error if there are several.
async fn fetch_opt<T: FromRow>(
    db: &impl GenericClient,
    sql: &str,
    params: &[&(dyn ToSql + Sync)],
) -> Result<Option<T>, tokio_postgres::Error> {
    let row = db.query_opt(sql, params).await?;
    row.as_ref().map(T::from_row).transpose()
}

#[derive(Debug, PartialEq)]
struct Product {
    sku: String,
    price_cents: i64,
}

impl FromRow for Product {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
        Ok(Product {
            sku: row.try_get("sku")?,
            price_cents: row.try_get("price_cents")?,
        })
    }
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let mut client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY, price_cents bigint NOT NULL);
             INSERT INTO products VALUES ('MUG-01', 1299), ('TEE-02', 2499);",
        )
        .await?;

    // On the client.
    let cheap: Vec<Product> = fetch_all(
        &client,
        "SELECT sku, price_cents FROM products WHERE price_cents < $1 ORDER BY sku",
        &[&2000_i64],
    )
    .await?;
    assert_eq!(cheap.len(), 1);
    let one: Product = fetch_one(
        &client,
        "SELECT sku, price_cents FROM products WHERE sku = $1",
        &[&"TEE-02"],
    )
    .await?;
    assert_eq!(one.price_cents, 2499);
    let none: Option<Product> = fetch_opt(
        &client,
        "SELECT sku, price_cents FROM products WHERE sku = $1",
        &[&"VASE-03"],
    )
    .await?;
    assert_eq!(none, None);

    // The same helpers inside a transaction: it sees its own insert, and nothing outside
does.
    let tx = client.transaction().await?;
    tx.execute("INSERT INTO products VALUES ('VASE-03', 3999)", &[])
        .await?;
```

```
    let inside: Vec<Product> = fetch_all(&tx, "SELECT sku, price_cents FROM products",
&[]).await?;
    assert_eq!(inside.len(), 3);
    tx.rollback().await?;
    let after: Vec<Product> =
        fetch_all(&client, "SELECT sku, price_cents FROM products", &[]).await?;
    assert_eq!(after.len(), 2);
    Ok(())
}
```

The helpers take tokio-postgres's `GenericClient`, which `Client` and `Transaction` both implement (main book Chapter 8), so one repository function can run inside or outside a transaction. A deadpool connection derefs, in two steps, to a `Client`, so `&**object` passes too (main book Chapter 13). Chapter 7 replaces `GenericClient` with a trait of shelf's own, because tokio-postgres's is sealed and has no statement cache.

shelf's own `fetch_all` takes the same arguments, and does one thing more: it prepares the statement first and checks the struct against the statement's columns before it runs the query. That is why the first two mistakes in §1.3 were found before any row was read. It costs no extra round trip, because `query` with a SQL string prepares the statement anyway (main book §4.1). Chapter 5 shows how.

1.6 Counting the boilerplate

Look at what you typed for a struct of five fields, and where each name went:

Where the name is written	Written by	Checked against the table
The struct's field (`price_cents: i64`)	you	never: the compiler knows nothing of the table
The `SELECT` list (`SELECT …, price_cents, …`)	you	by the server, when the statement is prepared
The `try_get` call (`row.try_get("price_cents")`)	you	when a row is read, against the `SELECT` list

Each field is named three times, and the second and third are strings that nothing compares with the first. The struct's name is in none of them, which is why §1.3's errors could not name it. And every mapping has the same shape: one line per field, the field's name turned into a string. That is the part a macro can take over:

- **The `from_row` body**: every line follows from the field's name and type.
- **The error context**: the struct's and the field's names are known when the code is generated, so every error can carry them at no cost.
- **The column positions**: the code can find them once per statement and keep them (Chapter 5).

What a macro cannot take over yet is the SQL. A derive that only sees the struct does not know the table; Chapter 6's `#[derive(Table)]` is told it, and writes the column list too.

Side by side — *a row into a struct*

The task: read products, with a nullable array column, into a struct.

```
// sqlx (illustration)
#[derive(sqlx::FromRow)]
struct Product {
    sku: String,
    price_cents: i64,
    tags: Option<Vec<String>>,
}

let products: Vec<Product> =
    sqlx::query_as("SELECT sku, price_cents, tags FROM products ORDER BY sku")
        .fetch_all(&pool)
        .await?;
```

By hand and with shelf, in one program that checks that both read the same values:

```
use tokio_postgres::Row;

trait FromRow: Sized {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error>;
}

// By hand.
#[derive(Debug, PartialEq)]
struct Product {
    sku: String,
    price_cents: i64,
    tags: Option<Vec<String>>,
}

impl FromRow for Product {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
        Ok(Product {
            sku: row.try_get("sku")?,
            price_cents: row.try_get("price_cents")?,
            tags: row.try_get("tags")?,
        })
    }
}

// shelf: the same fields, and one line instead of the impl.
#[derive(Debug, PartialEq, shelf::FromRow)]
struct ShelfProduct {
    sku: String,
    price_cents: i64,
    tags: Option<Vec<String>>,
}

const SQL: &str = "SELECT sku, price_cents, tags FROM products ORDER BY sku";

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY, price_cents bigint, tags
text[]);
             INSERT INTO products VALUES ('MUG-01', 1299, '{kitchen}'), ('TEE-02', 2499,
NULL);",
        )
        .await?;

    let by_hand = client
        .query(SQL, &[])
        .await?
        .iter()
        .map(Product::from_row)
        .collect::<Result<Vec<_>, _>>()?;
    let with_shelf: Vec<ShelfProduct> = shelf::fetch_all(&client, SQL, &[]).await?;

    assert_eq!(by_hand.len(), with_shelf.len());
    for (a, b) in by_hand.iter().zip(&with_shelf) {
        assert_eq!(
            (&a.sku, a.price_cents, &a.tags),
            (&b.sku, b.price_cents, &b.tags)
        );
    }
    assert_eq!(with_shelf[1].tags, None);
    Ok(())
}
```

	by hand	shelf	sqlx
Same	Fields are matched to columns by name; NULL is `Option`; types are checked at run time	The same	The same
Different	One line per field, in an impl you write and can change. Errors name a column, not the struct or field. Every name is looked up on every row	One line, `#[derive(FromRow)]`. The struct is checked against the statement once, before the first row; errors name the struct and field. Owned values only	The derive is generic over sqlx's databases (`R: Row`). Each name is looked up on every row, in a hash map the statement carries. `query_as!` can check the query at compile time

In stockroom

stockroom is the hand-written version at full size. Its `Store` has five fields, and the file that reads it names each of them four times: in the struct, in two `SELECT` lists (one in `list`, one in `create`'s `RETURNING`), and in `try_from`:

```
// stockroom/src/repo/stores.rs (excerpt)
pub struct Store {
    pub id: Uuid,
    pub code: String,
    pub name: String,
    pub address: Address,
    pub created_at: Timestamp,
}

impl TryFrom<&Row> for Store {
    type Error = tokio_postgres::Error;

    fn try_from(row: &Row) -> Result<Self, Self::Error> {
        Ok(Store {
            id: row.try_get("id")?,
            code: row.try_get("code")?,
            name: row.try_get("name")?,
            address: row.try_get("address")?, // the composite, through the derive
            created_at: row.try_get("created_at")?,
        })
    }
}

// …
pub async fn list(db: &impl GenericClient) -> Result<Vec<Store>, AppError> {
    let stmt = db
        .prepare_cached("SELECT id, code, name, address, created_at FROM stores ORDER BY
code")
        .await?;
    from_rows(&db.query(&stmt, &[]).await?)
}
```

stockroom/src/repo/stores.rs

`address` is a PostgreSQL composite type, which `Address` maps through postgres-types' derive (main book Chapter 5). That is one column holding a whole struct, read by `FromSql`, and the opposite of §1.4's nesting, where one struct spans several columns. shelf keeps both: `FromSql` types such as `Address` are ordinary fields, and `flatten` handles the other kind.

Chapter 14 rewrites part of stockroom on shelf and counts the lines this saves.

When not to use a mapping trait

- **One value.** `query_one_scalar::<i64>("SELECT count(*) …")` reads it without a struct (main book §2.5).
- **A one-off query read in place.** In a migration check or a test, `row.get("total")` is clearer than a struct that exists for one line.
- **Hot loops over borrowed data.** Mapping into owned structs allocates a `String` per text column per row. When you only need to look at the values, borrow them from the rows with `row.get::<_, &str>(…)` and keep the rows alive (main book §3.4).

Summary

- shelf maps rows with a trait of its own, `FromRow`, rather than `TryFrom<&Row>`: it can grow associated items, gives generic code a short bound, and can be implemented for tuples, which the orphan rule forbids for `TryFrom<&Row>`.
- In 0.7.18, `try_get("name")` walks the columns comparing names (and walks them again, ignoring ASCII case, if none matches), then checks `accepts`, then decodes. Twelve fields by name compare 78 names per row in any order.
- By name costs about 3× by position in a release build, 1.6× in a debug build, on 100,000 rows of 12 columns. Positions break when the `SELECT` changes; the answer is to find the positions once per statement.
- A mapping fails on a missing column, a wrong type, or a NULL in a non-`Option` field. tokio-postgres's errors name the column but not the struct or field; shelf's name all of them.
- A nested struct can be read from the same row by calling its `from_row`, but its column names are then fixed.
- `fetch_all`, `fetch_one` and `fetch_opt` run a query and map it, on anything that implements `GenericClient`.
- Each field is named three times by hand, twice as strings nothing checks. A macro can write the mapping and its error context; the SQL needs a derive that knows the table.

Exercises

1. **Errors with context.** Change the trait so that `from_row` returns an error naming the struct and the field, as well as tokio-postgres's error and its cause, and implement it for a `Product` with three fields. *Hint:* a small helper, `get(row, "Product", "price_cents")`, keeps each line short. Count how often the struct's name now appears in the impl.
2. **Does the order matter?** Predict how §1.2's timing changes if the `SELECT` lists the columns in the struct's order instead of in reverse. Then measure it. *Hint:* in each order, at which position does each field's search stop?
3. **A tuple.** Implement `FromRow` for `(A, B)`, reading by position, and read `SELECT sku, price_cents FROM products` into a `Vec<(String, i64)>`. *Hint:* each element needs `FromSqlOwned`; §1.1 showed why this impl needs your own trait.

Solutions

1. The error type carries the two names as `&'static str`, so building it costs nothing:

```
use std::fmt;
use tokio_postgres::Row;
use tokio_postgres::types::FromSqlOwned;

#[derive(Debug)]
struct MappingError {
    model: &'static str,
    field: &'static str,
    source: tokio_postgres::Error,
}
```

```
impl fmt::Display for MappingError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "{}.{}: {}", self.model, self.field, self.source)?;
        // The useful part of a tokio-postgres error is often its cause.
        if let Some(cause) = std::error::Error::source(&self.source) {
            write!(f, ": {cause}")?;
        }
        Ok(())
    }
}

impl std::error::Error for MappingError {
    fn source(&self) -> Option<&(dyn std::error::Error + 'static)> {
        Some(&self.source)
    }
}

trait FromRow: Sized {
    fn from_row(row: &Row) -> Result<Self, MappingError>;
}

fn get<T: FromSqlOwned>(
    row: &Row,
    model: &'static str,
    field: &'static str,
) -> Result<T, MappingError> {
    row.try_get(field).map_err(|source| MappingError {
        model,
        field,
        source,
    })
}

#[derive(Debug)]
#[allow(dead_code)] // the program only shows the error
struct Product {
    sku: String,
    name: String,
    price_cents: i64,
}

impl FromRow for Product {
    fn from_row(row: &Row) -> Result<Self, MappingError> {
        Ok(Product {
            sku: get(row, "Product", "sku")?,
            name: get(row, "Product", "name")?,
            price_cents: get(row, "Product", "price_cents")?,
        })
    }
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = shelf_playground::connect().await?;
    let row = client
        .query_one(
            "SELECT 'MUG-01' AS sku, 'Mug' AS name, 1299 AS price_cents",
            &[],
        )
        .await?;
    let error = Product::from_row(&row).unwrap_err();
    println!("{error}");
    assert_eq!(
        error.to_string(),
        "Product.price_cents: error deserializing column 2: \
         cannot convert between the Rust type `i64` and the Postgres type `int4`"
    );
```

```
    Ok(())
}
```

```
Product.price_cents: error deserializing column 2: cannot convert between the Rust type
`i64` and the Postgres type `int4`
```

The struct's name is now written once per field, and each field's name is written twice: as the field and as the string. Both are exactly what a macro knows when it generates the impl.

2. Nothing measurable changes. A search that finds its column at position p compares $p + 1$ names, and every column is looked up once per row, so the total is $1 + 2 + \ldots + 12 = 78$ comparisons per row for *any* order of the `SELECT` list. Measured in a debug build, three rounds each: 202.7–206.9 ms in reverse order and 205.2–206.6 ms in the struct's order, against 124–130 ms by position in both. What does grow the cost is the number of columns: twice as many columns means four times as many comparisons per row.

3. The impl is allowed because `FromRow` is defined in this program:

```
use tokio_postgres::Row;
use tokio_postgres::types::FromSqlOwned;

trait FromRow: Sized {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error>;
}

impl<A: FromSqlOwned, B: FromSqlOwned> FromRow for (A, B) {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
        Ok((row.try_get(0)?, row.try_get(1)?))
    }
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY, price_cents bigint NOT NULL);
             INSERT INTO products VALUES ('MUG-01', 1299), ('TEE-02', 2499);",
        )
        .await?;

    let pairs = client
        .query("SELECT sku, price_cents FROM products ORDER BY sku", &[])
        .await?
        .iter()
        .map(<(String, i64)>::from_row)
        .collect::<Result<Vec<_>, _>>()?;
    assert_eq!(pairs, [("MUG-01".into(), 1299), ("TEE-02".into(), 2499)]);
    Ok(())
}
```

Writing the impl for `(A, B, C)`, then `(A, B, C, D)`, and so on to twelve, is the kind of repetition the next chapter's macros remove.

CHAPTER 2

Declarative macros

Chapter 1's `FromRow` impl is mechanical: the same line for every field, with the field's name in two places, once as a field and once as a string. Mechanical code is what macros are for, and Rust has two kinds. This chapter uses the simpler one, `macro_rules!`, which matches the tokens it is given against patterns and pastes the pieces it matched into a template.

It goes surprisingly far. A `model!` macro can declare a struct and write its `FromRow` impl in one go, and one invocation can write `FromRow` for tuples of every size from one to twelve, which shelf keeps to this day. Then it stops: as soon as a field needs an attribute (read it from another column, skip it, decode it as JSON), the patterns turn into a puzzle, and the errors point at the macro's rules instead of at your mistake. That is the case for the next chapter's procedural macros.

In this chapter you will learn:

- how `macro_rules!` matches tokens: rules, fragment specifiers and repetitions
- how to write a `model!` macro that declares a struct and its `FromRow` impl together
- how one macro invocation writes `FromRow` for tuples of one to twelve elements, and why the positions are matched as `tt`
- what hygiene protects in a macro, what it does not, and what `$crate` is for
- where declarative macros stop, and why shelf's derives are procedural

2.1 Matching tokens

A `macro_rules!` macro is a list of rules. Each rule is a *matcher*, a pattern over tokens, and a *transcriber*, the code to produce when the pattern matches. The compiler tries the rules in order and expands the first one that matches. Here is a macro with one rule, which turns a list of column names into the text of a `SELECT` list, at compile time:

```
/// Joins column names into a `SELECT` list: `columns!(id, sku)` is `"id, sku"`.
macro_rules! columns {
    ($first:ident $(, $rest:ident)* $(,)?) => {
        concat!(stringify!($first) $(, ", ", stringify!($rest))*)
    };
}

const PRODUCT_COLUMNS: &str = columns!(id, sku, price_cents);

fn main() {
    assert_eq!(PRODUCT_COLUMNS, "id, sku, price_cents");
    assert_eq!(columns!(id), "id");
    assert_eq!(columns!(id, sku,), "id, sku"); // a trailing comma is allowed
}
```

The matcher reads like this:

- `$first:ident` matches one identifier and calls it `$first`. The part after the colon is a *fragment specifier*: it says what kind of syntax the metavariable accepts.

- `$(, $rest:ident)*` is a *repetition*: a comma followed by an identifier, zero or more times (`*`). Each identifier it matches is one `$rest`.
- `$(,)?` matches an optional trailing comma (`?` is zero or one).

The transcriber uses the same notation. `$(, ", ", stringify!($rest))*` repeats its contents once for every `$rest` that was matched. `stringify!` turns tokens into a string literal, and `concat!` joins literals into one: both run during expansion, so `PRODUCT_COLUMNS` is a plain `&'static str` literal and the program does no work to build it. The first element is matched on its own because the separator goes *between* elements: there is no way to say "a comma before every element but the first" in one repetition.

The fragment specifiers this book uses:

Specifier	Matches	Example
`ident`	an identifier	`price_cents`, `Product`
`ty`	a type	`Option<Vec<String>>`
`expr`	an expression	`row.len() + 1`
`literal`	a literal	`"sku"`, `0`
`vis`	a visibility, possibly empty	`pub`, `pub(crate)`, or nothing
`meta`	the inside of an attribute	`derive(Debug, PartialEq)`
`tt`	one token tree: a single token, or a group in `()`, `[]` or `{}`	`0`, `(a, b)`

The Rust Reference's Macros By Example lists the others (`path`, `pat`, `block`, `item`, `lifetime`, `stmt`) and the rules about which tokens may follow each one.

> **Rust refresher:** macros expand before the compiler checks types, or even knows what most names refer to. A macro sees `Option<Vec<String>>` as a handful of tokens; the `ty` specifier checks that they *form* a type, but the macro cannot ask whether that type is an `Option`, or whether it implements `FromSql`. This is true of every macro in Rust, procedural ones included. shelf's derive cannot know that your `type Tags = Option<Vec<String>>` is an `Option` either. It does not need to: it generates code that only compiles if the types fit, and lets the compiler check.

2.2 A `model!` macro

A struct definition is tokens too. A macro that accepts one can paste it back unchanged and add the `FromRow` impl after it, writing one `try_get` per field with `stringify!` for the column name:

```
use tokio_postgres::Row;

trait FromRow: Sized {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error>;
}

/// Declares a struct, and implements `FromRow` for it: each field is read from the column
of
/// its name.
macro_rules! model {
    (
        $(#[$meta:meta])*
        $vis:vis struct $name:ident {
            $($field_vis:vis $field:ident : $ty:ty),* $(,)?
        }
    ) => {
        $(#[$meta])*
        $vis struct $name {
```

```
            $($field_vis $field: $ty),*
        }

        impl FromRow for $name {
            fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
                Ok(Self {
                    $($field: row.try_get(stringify!($field))?),*
                })
            }
        }
    };
}

model! {
    #[derive(Debug, PartialEq)]
    pub struct Product {
        pub id: i64,
        pub sku: String,
        pub price_cents: i64,
        pub tags: Option<Vec<String>>,
    }
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (
                 id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
                 sku text NOT NULL,
                 price_cents bigint NOT NULL,
                 tags text[]
             );
             INSERT INTO products (sku, price_cents, tags)
             VALUES ('MUG-01', 1299, '{kitchen}'), ('TEE-02', 2499, NULL);",
        )
        .await?;

    let products = client
        .query(
            "SELECT id, sku, price_cents, tags FROM products ORDER BY id",
            &[],
        )
        .await?
        .iter()
        .map(Product::from_row)
        .collect::<Result<Vec<_>, _>>()?;

    assert_eq!(products[0].sku, "MUG-01");
    assert_eq!(products[0].tags, Some(vec!["kitchen".to_string()]));
    assert_eq!(products[1].price_cents, 2499);
    assert_eq!(products[1].tags, None);
    Ok(())
}
```

Three pieces of the matcher do the work:

- `$(#[$meta:meta])*` accepts any number of attributes on the struct and pastes them back, so `#[derive(Debug, PartialEq)]` still applies. Without it, `model!` would refuse every attribute.
- `$vis:vis` and `$field_vis:vis` accept `pub`, `pub(crate)` or nothing at all, so the macro does not decide the struct's visibility for you.
- The field repetition, `$($field_vis:vis $field:ident : $ty:ty),*`, gives the transcriber two lists that line up, `$field` and `$ty`, and it uses them twice: once to declare the fields and once, `$field` alone, to read them. The field names are written once, by you, and the column names come from them.

The struct's fields are now named once instead of twice. The cost is that the struct lives inside a macro call, which is no longer ordinary Rust to your tools. rustfmt leaves the inside of `model! { … }` alone, so its formatting is yours to keep, and a mistake inside it is reported against the macro's rules, as §2.5 shows. That cost is the reason shelf's mapping is a derive: a derive leaves your struct an ordinary struct, and adds the impl next to it.

2.3 Tuples, one line per arity

A query that returns two columns does not always deserve a struct. Chapter 1 implemented `FromRow` for `(A, B)` by hand, reading by position; shelf has the same for tuples of one to twelve elements, and a single macro writes all twelve impls. Each line of the invocation is one tuple size, with each element's type parameter next to its position:

```
use tokio_postgres::Row;
use tokio_postgres::types::FromSqlOwned;

trait FromRow: Sized {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error>;
}

/// `FromRow` for tuples, read by position: one impl per line of the invocation.
macro_rules! tuple_impls {
    ($( ($($T:ident $i:tt),+) )+) => {
        $(
            impl<$($T: FromSqlOwned),+> FromRow for ($($T,)+) {
                fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
                    Ok(($(row.try_get::<_, $T>($i)?,)+))
                }
            }
        )+
    };
}

tuple_impls! {
    (A 0)
    (A 0, B 1)
    (A 0, B 1, C 2)
    (A 0, B 1, C 2, D 3)
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = shelf_playground::connect().await?;
    let row = client
        .query_one(
            "SELECT 'MUG-01' AS sku, 1299::int8 AS price_cents, true AS active, 3 AS stock",
            &[],
        )
        .await?;

    let (sku,) = <(String,)>::from_row(&row)?;
    let pair = <(String, i64)>::from_row(&row)?;
    let all = <(String, i64, bool, i32)>::from_row(&row)?;
    assert_eq!(sku, "MUG-01");
    assert_eq!(pair, ("MUG-01".to_string(), 1299));
    assert_eq!(all, ("MUG-01".to_string(), 1299, true, 3));
    Ok(())
}
```

The matcher has a repetition inside a repetition. The outer one, `$( (…) )+`, matches each parenthesized line; the inner one, `$($T:ident $i:tt),+`, matches the elements of a line. In the transcriber, `$( impl … )+` repeats per line, and inside it `$($T: FromSqlOwned),+` and `($($T,)+)` repeat per element. The second one writes a

comma after *every* element, including the last, which is not just tidy: a one-element tuple is spelled `(A,)`, and `(A)` would be `A` in parentheses.

Why is the position matched as a `tt`, a single token tree, and not as a `literal`? Here either would do, because `$i` is pasted into expression positions: `row.try_get::<_, $T>($i)`. But the two are pasted differently. A `tt` goes in as the token it was. A `literal` or an `expr` goes in as one sealed piece of syntax, which some positions refuse. A tuple field is one of them:

```
macro_rules! field_tt { ($t:expr, $i:tt) => { $t.$i }; }
macro_rules! field_lit { ($t:expr, $i:literal) => { $t.$i }; }
fn main() {
    let t = (10, 20);
    println!("{}", field_tt!(t, 1));
    println!("{}", field_lit!(t, 1));
}
```

`field_tt!(t, 1)` becomes `t.1`. `field_lit!(t, 1)` does not compile:

```
error: unexpected token: `literal` metavariable
 --> literal.rs:2:56
  |
2 | macro_rules! field_lit { ($t:expr, $i:literal) => { $t.$i }; }
  |                                                        ^^
...
6 |     println!("{}", field_lit!(t, 1));
  |                    ---------------- in this macro invocation
```

So `tt` is the habit for a position: it works everywhere a number does. Exercise 3 needs that.

Writing the positions out by hand looks like something the macro should count for itself. Nightly Rust has *metavariable expressions*, whose `${index()}` gives the position inside a repetition, but in rustc 1.99.1 they are still unstable (error E0658, tracking issue #83527). A recursive macro can avoid repeating the lines instead (Exercise 1), but the flat list is the easiest to read and to check, so shelf keeps it.

2.4 Hygiene, and `$crate`

A macro's output lands in the caller's code, among the caller's names. Two questions follow: can the macro's own variables collide with the caller's, and do the names the macro writes mean what its author intended? `macro_rules!` answers them differently, and this program shows both answers:

```
mod text {
    /// Pads a cell of a plain-text table to eight characters.
    pub fn cell(value: &str) -> String {
        format!("{value:<8}|")
    }

    /// One line of a plain-text table, from any number of cells.
    #[macro_export]
    macro_rules! table_line {
        ($($value:expr),+ $(,)?) => {{
            let mut line = ::std::string::String::new();
            $( line.push_str(&$crate::text::cell($value)); )+
            line
        }};
    }
}

// A name the macro must not depend on: this crate's `String` is not std's.
#[allow(dead_code)]
struct String;

fn main() {
    let line = "MUG-01";
```

```
    let header = table_line!("sku", "price");
    let row = table_line!(line, "1299"); // the caller's `line`, not the macro's
    assert_eq!(header, "sku     |price   |");
    assert_eq!(row, "MUG-01  |1299    |");
    assert_eq!(line, "MUG-01");
}
```

Local variables are hygienic. The macro declares `let mut line`, and the caller passes in its own `line`. Inside the expansion both are in scope, with the same name, yet `$value` still means the caller's `&str` and the macro's `push_str` still targets its own `String`. The compiler remembers where each identifier was written: a variable declared by the macro is visible only to the macro's tokens, and the caller's tokens see only the caller's variables.

Paths are not. Every other name in the expansion, types, functions, traits, constants, is looked up where the macro is *called*. That is why `table_line!` writes `::std::string::String::new()`: in `main`'s module, `String` is the caller's unit struct, and `String::new()` would not compile. A leading `::` means that the first segment names a crate (`std`, `core`, `shelf`), never an item of the caller's module, so nothing the caller declares can shadow it. The Reference calls this mix *mixed-site hygiene*.

`model!` from §2.2 is not that careful. It writes `Result<Self, tokio_postgres::Error>`, and an application with the common alias `type Result<T> = std::result::Result<T, AppError>` in scope breaks it:

```
mod app {
    use super::{FromRow, Row};

    type Result<T> = std::result::Result<T, String>;

    model! {
        pub struct Product {
            pub sku: String,
        }
    }
}
```

```
error[E0107]: type alias takes 1 generic argument but 2 generic arguments were supplied
  --> src/bin/model_alias.rs:20:39
   |
20 |               fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
   |                                         ^^^^^^     ----------------------- help: remove
the unnecessary generic argument
   |                                         |
   |                                         expected 1 generic argument
...
34 | /     model! {
35 | |         pub struct Product {
36 | |             pub sku: String,
37 | |         }
38 | |     }
   | |_____- in this macro invocation
   |
note: type alias defined here, with 1 generic parameter: `T`
  --> src/bin/model_alias.rs:32:10
   |
32 |     type Result<T> = std::result::Result<T, String>;
   |          ^^^^^^ -
   = note: this error originates in the macro `model` (in Nightly builds, run with -Z macro-
backtrace for more info)
```

The fix is the same as in `table_line!`: absolute paths for everything the macro names, `::core::result::Result` and `::core::result::Result::Ok`, never `Result` and `Ok`. Notice also that `app` had to import `FromRow` and `Row` for `model!` to work at all, because the expansion names them as bare identifiers. A careful `model!` would not ask that of its caller.

`$crate` names the macro's own crate. `cell` is a function in this program, so `table_line!` could have written `crate::text::cell`. But a macro exported from a library (`#[macro_export]`) runs in other crates, where `crate` means the *caller's* crate. `$crate` expands to the crate that defined the macro, wherever it is called, and it keeps working when the caller renames the dependency in `Cargo.toml`. It is how a library's macro reaches the library's helpers.

> **Macro lens:** `$crate` exists only in `macro_rules!`. A procedural macro is a crate of its own, and the tokens it returns have no way to say "the crate that re-exports me", so shelf's generated code writes `::shelf::…` and assumes the dependency is called `shelf`. serde does the same, and offers `#[serde(crate = "…")]` for the rare caller who renamed it. Chapter 3 returns to the rest of a derive's hygiene: absolute paths everywhere, and the hidden module `::shelf::__private` that spares your crate a direct dependency on tokio-postgres.

2.5 Where declarative macros stop

`model!` maps each field to the column of its name. Real models need more, and the first thing they need is an attribute: this field reads the column `sku_code`. The natural syntax is an attribute on the field. `model!` does not accept it:

```
model! {
    pub struct Product {
        pub id: i64,
        #[col = "sku_code"]
        pub sku: String,
    }
}
```

```
error: no rules expected `#`
  --> src/bin/model_attr.rs:32:9
   |
 7 | macro_rules! model {
   | ------------------ when calling this macro
...
32 |         #[col = "sku_code"]
   |         ^ no rules expected this token in macro call
   |
note: while trying to match meta-variable `$field_vis:vis`
  --> src/bin/model_attr.rs:11:15
   |
11 |             $($field_vis:vis $field:ident : $ty:ty),* $(,)?
   |               ^^^^^^^^^^^^^^
```

Accepting the attribute is easy: add `$(#[$fattr:meta])*` before each field, as for the struct. *Acting* on it is not. A matcher cannot say "a `col = "…"` attribute if there is one, otherwise the field's name", because the transcriber has no `if`. The way out is a rule per kind of field, and a macro that walks the fields one at a time, calling itself on the rest with what it has built so far: a *tt-muncher*. Its shape, not compiled:

```
// A tt-muncher for model!'s fields (illustration): one field consumed per step.
(@fields [$($done:tt)*] #[col = $col:literal] $field:ident : $ty:ty, $($rest:tt)*) => {
    model!(@fields [$($done)* $field: row.try_get($col)?,] $($rest)*)
};
(@fields [$($done:tt)*] $field:ident : $ty:ty, $($rest:tt)*) => {
    model!(@fields [$($done)* $field: row.try_get(stringify!($field))?,] $($rest)*)
};
(@fields [$($done:tt)*]) => { /* write the impl from everything in $done */ };
```

That handles one attribute. Each new one (`skip`, `json`, `flatten`, `try_from`) adds rules, combinations of them multiply the rules, and an attribute that is misspelt does not match any rule, so the error is "no rules expected" at the `#`, with no word about what was wrong.

Generics go the same way:

```
model! {
    pub struct Page<T> {
        pub items: Vec<T>,
    }
}
```

```
error: no rules expected `<`
  --> src/bin/model_attr.rs:38:20
   |
 7 | macro_rules! model {
   | ------------------ when calling this macro
...
38 |     pub struct Page<T> {
   |                    ^ no rules expected this token in macro call
   |
note: while trying to match `{`
  --> src/bin/model_attr.rs:10:37
   |
10 |         $vis:vis struct $name:ident {
   |                                     ^
```

Matching `<T>` is possible. Matching `<'a, T: Clone, const N: usize>` and a `where` clause, and then adding the bound each field needs, is a parser for Rust's generics written in `macro_rules!`.

Each of these problems has the same root: `macro_rules!` matches tokens against fixed patterns, and a struct is not a fixed pattern. What the job needs is a macro that receives the struct as *data*, already parsed, with a list of fields it can loop over, attributes it can inspect with ordinary `if`s, generics it can modify, and a way to report a mistake at the exact token that caused it, in its own words. That is a procedural macro, and syn is the parser. Chapter 3 writes one.

Side by side — *generating impls for many types*

The task: `FromRow` for tuples, read by position. sqlx has the same impls, written by a macro with one invocation per tuple size:

```
// sqlx (illustration), from sqlx-core 0.9.0's src/from_row.rs, abridged
macro_rules! impl_from_row_for_tuple {
    ($( ($idx:tt) -> $T:ident );+;) => {
        impl<'r, R, $($T,)+> FromRow<'r, R> for ($($T,)+)
        where
            R: Row,
            usize: crate::column::ColumnIndex<R>,
            $($T: crate::decode::Decode<'r, R::Database> +
crate::types::Type<R::Database>,)+
        {
            #[inline]
            fn from_row(row: &'r R) -> Result<Self, Error> {
                Ok(($(row.try_get($idx as usize)?,)+))
            }
        }
    };
}

impl_from_row_for_tuple!(
    (0) -> T1;
    (1) -> T2;
);
// … one invocation per size, up to sixteen elements
```

```
let pairs: Vec<(String, i64)> = sqlx::query_as("SELECT sku, price_cents FROM products")
    .fetch_all(&pool)
    .await?;
```

By hand, two sizes written out, and shelf's own tuples, in one program:

```
use tokio_postgres::Row;
use tokio_postgres::types::FromSqlOwned;

trait FromRow: Sized {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error>;
}

// By hand: one impl per size, each written out.
impl<A: FromSqlOwned> FromRow for (A,) {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
        Ok((row.try_get(0)?,))
    }
}

impl<A: FromSqlOwned, B: FromSqlOwned> FromRow for (A, B) {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
        Ok((row.try_get(0)?, row.try_get(1)?))
    }
}

const SQL: &str = "SELECT sku, price_cents FROM products ORDER BY sku";

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY, price_cents bigint NOT NULL);
             INSERT INTO products VALUES ('MUG-01', 1299), ('TEE-02', 2499);",
        )
        .await?;

    let by_hand = client
        .query(SQL, &[])
        .await?
        .iter()
        .map(<(String, i64)>::from_row)
        .collect::<Result<Vec<_>, _>>()?;

    // shelf: the impls come from one macro invocation in shelf/src/tuples.rs.
    let with_shelf: Vec<(String, i64)> = shelf::fetch_all(&client, SQL, &[]).await?;
    assert_eq!(by_hand, with_shelf);

    // A wrong element type: the error names the position and the column.
    let error = shelf::fetch_all::<(String, i32)>(&client, SQL, &[])
        .await
        .unwrap_err();
    println!("{error}");
    assert_eq!(
        error.to_string(),
        r#"tuple.1: column "price_cents" has type int8, which cannot be read into i32"#
    );
    Ok(())
}
```

```
tuple.1: column "price_cents" has type int8, which cannot be read into i32
```

	by hand	shelf	sqlx
Same	One impl per tuple size, reading by position	The same impls, written by a macro	The same impls, written by a macro
Different	Every impl typed out; tokio-postgres's error names only the column	Twelve sizes from one invocation, a line each; errors name the position (`tuple.1`) and the column, and are found once per statement	Sixteen sizes, one invocation each; generic over sqlx's databases

In shelf

shelf's tuple impls are §2.3's macro, with two differences. `FromRow` has the two methods Chapter 5 introduces, `resolve` (check the statement's columns once) and `from_row_at` (read a row), and the positions are pasted a second time through `stringify!`, so that an error can name the element (`"1"` in `tuple.1` above):

```
// orm/shelf/src/tuples.rs (excerpt)
macro_rules! tuple_impls {
    ($( ($($T:ident $i:tt),+) )+) => {
        $(
            impl<$($T: FromColumn),+> FromRow for ($($T,)+) {
                type Columns = ();

                fn resolve(columns: &[Column]) -> Result<(), Error> {
                    $( column_at::<$T>(columns, "tuple", stringify!($i), $i)?; )+
                    Ok(())
                }

                fn from_row_at(row: &Row, _: &()) -> Result<Self, Error> {
                    Ok(($( get::<$T>(row, $i, "tuple", stringify!($i))?, )+))
                }
            }
        )+
    };
}

tuple_impls! {
    (A 0)
    (A 0, B 1)
    (A 0, B 1, C 2)
    // … one line per size, up to twelve elements
}
```

orm/shelf/src/tuples.rs

The bound is `FromColumn`, shelf's own name for tokio-postgres's `FromSqlOwned`, which exists to make one compiler error read better (Chapter 4). A tuple's `Columns` is `()`: its positions are fixed, so there is nothing to look up, and `resolve` only checks each column's type. This macro lives inside shelf and is not exported, so it can name `FromRow`, `Column` and `get` directly: its expansion is always in `tuples.rs`, where those names are imported. A macro for other crates would need the absolute paths of §2.4.

When not to use `macro_rules!`

- **When a generic impl does the job.** shelf's `FromColumn` has a single impl, `impl<T: FromSqlOwned> FromColumn for T`, which covers every type at once with no macro. A macro is for impls that differ in *shape*, like tuples of different sizes.

- **When a function does the job.** A function is type-checked where it is written, and its errors point at it. Reach for a macro when you need code that a function cannot express: new items, new fields, a variable number of arguments.
- **When the input needs attributes, generics or good error messages.** That is a procedural macro's job (Chapter 3).

Summary

- `macro_rules!` matches tokens against rules, in order, and pastes what it matched into a template. Fragment specifiers (`ident`, `ty`, `expr`, `tt`, `vis`, `meta`, `literal`) say what each metavariable accepts; `*`, `+` and `?` repeat.
- Macros run before type checking: they see tokens, never types, and that holds for procedural macros too.
- A `model!` macro can declare a struct and write its `FromRow` impl, so field names are written once. The struct then lives inside a macro call, which tools handle less well.
- One invocation writes `FromRow` for tuples of one to twelve elements, a line per size. Match positions as `tt`: a captured `literal` cannot be pasted as a tuple index. `${index()}` is still unstable in rustc 1.99.1.
- Local variables in a macro are hygienic; paths are resolved at the call site, so a macro for other people's code writes `::core::result::Result`. `$crate` names the macro's own crate.
- Field attributes, generics and readable errors are beyond what `macro_rules!` does well. A procedural macro gets the struct as parsed data.

Exercises

1. **One line instead of twelve.** Rewrite §2.3's `tuple_impls!` so that a single invocation, `tuple_impls!([] A 0, B 1, C 2, D 3)`, writes all four impls. *Hint:* keep the elements already handled in the brackets, an *accumulator*. Each step moves one element from the list into the accumulator, writes the impl for the accumulator, and calls the macro again with the rest. Peeling elements off the front without an accumulator gets the positions wrong: the rest would start at 1.
2. **The column list too.** Make `model!` also generate a constant `Product::COLUMNS` holding `"id, sku, price_cents, tags"`, and use it to write the query. *Hint:* a macro can call another macro: §2.1's `columns!` is half the work.
3. **Tuples as parameters.** Write a trait `Params` with `fn params(&self) -> Vec<&(dyn ToSql + Sync)>`, implement it for tuples of one to three elements with a macro, and insert a row with `("VASE-03", 3999_i64).params()`. *Hint:* a tuple's fields are `self.0`, `self.1`, and so on, which is where §2.3's `tt` earns its keep.

Solutions

1. Each step writes one impl and recurses, so the invocation is one line. The cost is a macro that is harder to read than the flat list, and a recursion depth, one level per element, that the compiler limits (128 levels by default):

```
use tokio_postgres::Row;
use tokio_postgres::types::FromSqlOwned;

trait FromRow: Sized {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error>;
}

macro_rules! tuple_impls {
    // Nothing left to move: done.
    ([$($T:ident $i:tt),*]) => {};
    // Move the next element into the accumulator, write its impl, and go on with the rest.
    ([$($T:ident $i:tt),*] $U:ident $j:tt $(, $rest_t:ident $rest_i:tt)*) => {
```

```
        impl<$($T: FromSqlOwned,)* $U: FromSqlOwned> FromRow for ($($T,)* $U,) {
            fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
                Ok(($(row.try_get::<_, $T>($i)?,)* row.try_get::<_, $U>($j)?,))
            }
        }
        tuple_impls!([$($T $i,)* $U $j] $($rest_t $rest_i),*);
    };
}

tuple_impls!([] A 0, B 1, C 2, D 3);

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = shelf_playground::connect().await?;
    let row = client
        .query_one("SELECT 'MUG-01' AS sku, 1299::int8, true, 3", &[])
        .await?;
    assert_eq!(<(String,)>::from_row(&row)?, ("MUG-01".to_string(),));
    assert_eq!(
        <(String, i64, bool, i32)>::from_row(&row)?,
        ("MUG-01".to_string(), 1299, true, 3)
    );
    Ok(())
}
```

2. `model!` hands its field names to `columns!`, which joins them at compile time:

```
use tokio_postgres::Row;

trait FromRow: Sized {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error>;
}

macro_rules! columns {
    ($first:ident $(, $rest:ident)* $(,)?) => {
        concat!(stringify!($first) $(, ", ", stringify!($rest))*)
    };
}

macro_rules! model {
    (
        $(#[$meta:meta])*
        $vis:vis struct $name:ident {
            $($field_vis:vis $field:ident : $ty:ty),* $(,)?
        }
    ) => {
        $(#[$meta])*
        $vis struct $name {
            $($field_vis $field: $ty),*
        }

        impl $name {
            /// The columns `from_row` reads, as a `SELECT` list.
            const COLUMNS: &'static str = columns!($($field),*);
        }

        impl FromRow for $name {
            fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
                Ok(Self {
                    $($field: row.try_get(stringify!($field))?),*
                })
            }
        }
    };
}

model! {
    #[derive(Debug)]
```

```
    struct Product {
        id: i64,
        sku: String,
        price_cents: i64,
        tags: Option<Vec<String>>,
    }
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    assert_eq!(Product::COLUMNS, "id, sku, price_cents, tags");

    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (id bigint, sku text, price_cents bigint, tags
text[]);
             INSERT INTO products VALUES (1, 'MUG-01', 1299, NULL);",
        )
        .await?;
    // COLUMNS holds identifiers the macro wrote from the struct, never a value, so it can
    // go into the SQL text; values still travel as $n parameters.
    let sql = format!("SELECT {} FROM products WHERE id = $1", Product::COLUMNS);
    let product = Product::from_row(&client.query_one(&sql, &[&1_i64]).await?)?;
    assert_eq!(product.sku, "MUG-01");
    assert_eq!(
        (product.id, product.price_cents, product.tags),
        (1, 1299, None)
    );
    Ok(())
}
```

The `format!` runs on every call. Chapter 6 builds such statements once, at compile time, from constants the derive writes, and quotes each identifier on the way.

3. The impls use `self.$i`, which compiles only because `$i` is a `tt`:

```
use tokio_postgres::types::ToSql;

/// A tuple's values, as the parameter list `query` and `execute` take.
trait Params {
    fn params(&self) -> Vec<&(dyn ToSql + Sync)>;
}

macro_rules! params_impls {
    ($( ($($T:ident $i:tt),+) )+) => {
        $(
            impl<$($T: ToSql + Sync),+> Params for ($($T,)+) {
                fn params(&self) -> Vec<&(dyn ToSql + Sync)> {
                    vec![$(&self.$i as &(dyn ToSql + Sync)),+]
                }
            }
        )+
    };
}

params_impls! {
    (A 0)
    (A 0, B 1)
    (A 0, B 1, C 2)
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute("CREATE TEMP TABLE products (sku text, price_cents bigint)")
        .await?;
```

```
    let inserted = client
        .execute(
            "INSERT INTO products VALUES ($1, $2)",
            &("VASE-03", 3999_i64).params(),
        )
        .await?;
    assert_eq!(inserted, 1);
    let price: i64 = client
        .query_one_scalar(
            "SELECT price_cents FROM products WHERE sku = $1",
            &("VASE-03",).params(),
        )
        .await?;
    assert_eq!(price, 3999);
    Ok(())
}
```

With `$i:literal` instead, `self.$i` is refused with the error shown in §2.3.

CHAPTER 3

The first derive macro

A procedural macro is a Rust function that runs inside the compiler. `#[derive(FromRow)]` hands it the tokens of your struct; it returns more tokens, an `impl` block, which the compiler compiles as if you had written it. There is no pattern language this time: the function parses the struct into a syntax tree, loops over its fields, looks at whatever it likes, and builds the code it needs with ordinary Rust.

This chapter writes the first version of shelf's derive: every named field read from the column of its name, which is Chapter 1's impl written by a program. Most of the work is not the generator, which is twenty lines of syn and quote, but the setting around it: the kind of crate it must live in, the crate it should live in instead, the paths that keep the generated code working in any module, the ways to see what was generated, and how a derive reports a mistake at the right token.

In this chapter you will learn:

- what a proc-macro crate is, what the compiler hands a derive, and what it takes back
- why shelf splits its macros into a code generator and a thin proc-macro shell, and what `proc_macro2` has to do with it
- how syn 3 parses a struct, and how quote builds tokens with interpolation and repetition
- what hygiene means for a derive: absolute paths, `__private`, raw identifiers
- three ways to see an expansion: printing it, `cargo expand`, and rust-analyzer
- how a derive reports an error as a value, pointing at the token that caused it

3.1 Proc-macro crates

A derive macro lives in a crate whose `Cargo.toml` says so:

```
# orm/shelf-macros/Cargo.toml (excerpt)
[lib]
proc-macro = true

[dependencies]
shelf-codegen = { workspace = true }
syn = { workspace = true }
```

`proc-macro = true` changes what the crate is. It is compiled for the machine running the compiler (not the one your program will run on), loaded into the compiler as a plugin, and its public functions are called during compilation. A derive is one such function, marked with the attribute that names it:

```
// orm/shelf-macros/src/lib.rs (excerpt)
#[proc_macro_derive(FromRow, attributes(shelf))]
pub fn derive_from_row(input: TokenStream) -> TokenStream {
    // …
}
```

`FromRow` is the name you write in `#[derive(…)]`. `attributes(shelf)` declares `#[shelf(…)]` as a *helper attribute* that may appear on the struct and its fields; Chapter 4 uses it. The function takes and returns a

`proc_macro::TokenStream`: a sequence of tokens (identifiers, punctuation, literals, and groups in brackets), not text and not a syntax tree.

What exactly does a derive receive? This derive, a probe written for this chapter and not part of shelf, turns its input into a string and adds it to the type as a constant:

```
#[proc_macro_derive(ShowInput, attributes(shelf))]
pub fn show_input(input: TokenStream) -> TokenStream {
    let text = input.to_string();
    let parsed: syn::DeriveInput = syn::parse(input).unwrap();
    let name = parsed.ident;
    quote! { impl #name { pub const INPUT: &'static str = #text; } }.into()
}
```

Applied to this struct, between two other derives:

```
/// A product in the catalog.
#[derive(Debug, show::ShowInput, Clone)]
#[allow(dead_code)]
struct Product {
    /// The stock-keeping unit.
    #[shelf(rename = "sku_code")]
    sku: String,
    #[cfg(any())]
    gone: i32,
    price_cents: i64,
}
```

`Product::INPUT` holds:

```
/// A product in the catalog.
#[allow(dead_code)] struct Product
{
    /// The stock-keeping unit.
    #[shelf(rename = "sku_code")] sku: String, price_cents: i64,
}
```

> **Macro lens:** a derive gets the whole item: doc comments, other attributes, and the `#[shelf(…)]` helper attributes it will read. It does not get the `#[derive(…)]` line itself, nor the other derives listed in it, and the field under `#[cfg(any())]`, a condition that is never true, is already gone: `cfg` is evaluated before derives run. What the derive returns is *added after* the struct; it does not replace it. That is why `Product::INPUT` compiles: the struct is still there for the `impl` to attach to.

A proc-macro crate has two restrictions, and both shape shelf. It **can export nothing but macros**. A public function next to the derive, say the code generator itself for a test to call, is refused:

```
error: `proc-macro` crate types currently cannot export any items other than functions
tagged with `#[proc_macro]`, `#[proc_macro_derive]`, or `#[proc_macro_attribute]`
  --> show/src/lib.rs:14:1
   |
14 | pub fn expand(input: &syn::DeriveInput) -> proc_macro2::TokenStream {
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

And **its types work only inside the compiler**. `proc_macro::TokenStream` is a handle to data the compiler owns. In an ordinary program, `TokenStream::new()` happens to work, but parsing code into a token stream panics:

```
extern crate proc_macro;
fn main() {
```

```
    let parsed: proc_macro::TokenStream = "struct Product;".parse().unwrap();
}
```

```
thread 'main' (23461715) panicked at /rustc/58cf8f9c35b355199727627d07ae34a4e2846d67/
library/proc_macro/src/bridge/client.rs:200:32:
procedural macro API is used outside of a procedural macro
```

So the function at the heart of the derive can be called by the compiler and by nothing else: not by a unit test in another crate, not by an example in this book.

3.2 Three crates

That is why shelf is three crates (§0.4). `shelf-codegen` is an ordinary library that holds the code generation, as functions from a parsed `syn::DeriveInput` to a `proc_macro2::TokenStream`. `shelf-macros` is the proc-macro crate, and stays a shell of three lines per derive: parse the input, call the generator, and turn an error into a compile error:

```
// orm/shelf-macros/src/lib.rs (excerpt)
use proc_macro::TokenStream;
use syn::{DeriveInput, parse_macro_input};

#[proc_macro_derive(FromRow, attributes(shelf))]
pub fn derive_from_row(input: TokenStream) -> TokenStream {
    let input = parse_macro_input!(input as DeriveInput);
    shelf_codegen::from_row::expand(&input)
        .unwrap_or_else(syn::Error::into_compile_error)
        .into()
}
```

orm/shelf-macros/src/lib.rs

`proc_macro2` makes the split possible. It is a crate with the same types as the compiler's `proc_macro` (`TokenStream`, `Ident`, `Span`, `Literal`, `Group`), which work in two modes. Inside a proc macro, they wrap the compiler's own types; anywhere else, they fall back to an implementation of their own. syn parses `proc_macro2` tokens and quote produces them, so `shelf-codegen` never touches `proc_macro`, and runs equally well in the compiler, in its tests, and in this book's examples. The shell converts at the edges: `parse_macro_input!` parses the compiler's tokens into a `DeriveInput` (or returns the parse error as a compile error), and `.into()` turns the generator's `proc_macro2::TokenStream` back into the compiler's.

`shelf` re-exports the derive (`pub use shelf_macros::FromRow;`), so your crate depends on `shelf` alone and writes `#[derive(shelf::FromRow)]`, or imports it with `use shelf::FromRow`, which brings in the trait and the derive at once (§0.7).

3.3 Parsing with syn 3

syn parses Rust source, as tokens, into a syntax tree. A derive needs one entry point, `DeriveInput`, which is a struct, an enum or a union with its attributes, visibility, name and generics. This program parses a struct the way a derive receives it and walks its fields:

```
use quote::ToTokens;
use syn::{Data, DeriveInput, Fields, parse_quote};

fn main() {
    let input: DeriveInput = parse_quote! {
        /// A product in the catalog.
        pub struct Product {
            id: i64,
```

```
            pub sku: String,
            tags: Option<Vec<String>>,
        }
    };

    println!("name: {}", input.ident);
    println!("attributes: {}", input.attrs.len());
    let Data::Struct(data) = &input.data else {
        panic!("a struct")
    };
    let Fields::Named(fields) = &data.fields else {
        panic!("named fields")
    };
    for field in &fields.named {
        let name = field.ident.as_ref().expect("a named field");
        println!("  field {name}: {}", field.ty.to_token_stream());
    }

    assert_eq!(input.ident, "Product");
    assert_eq!(fields.named.len(), 3);
    assert_eq!(
        fields.named[2].ty.to_token_stream().to_string(),
        "Option < Vec < String > >"
    );
}
```

```
name: Product
attributes: 1
  field id: i64
  field sku: String
  field tags: Option < Vec < String > >
```

`parse_quote!` builds a syntax tree from Rust code written in place; the book's stage programs use it to stand in for the compiler. The doc comment is an attribute (`#[doc = "…"]`), hence `attributes: 1`. And the type of `tags` prints with spaces between its tokens, because it *is* tokens: `Option`, `<`, `Vec`, `<`, `String`, `>`, `>`. syn knows it is a path type with generic arguments, but it does not know what `Option` refers to (Chapter 2's refresher).

The pieces a derive walks through:

Type	What it holds
`DeriveInput`	`attrs`, `vis`, `ident` (the name), `generics`, and `data`
`Data`	`Struct(DataStruct)`, `Enum(DataEnum)` or `Union(DataUnion)`
`Fields`	`Named` (`{ id: i64 }`), `Unnamed` (`(i64, String)`) or `Unit`
`Field`	`attrs`, `vis`, `ident` (`None` in a tuple struct), `ty`, and two items new in syn 3

syn compiles only what you ask for. Its default features (`derive`, `parsing`, `printing`, `clone-impls` and `proc-macro`) are all a derive needs. The workspace also enables `full`, the whole of Rust's syntax, because `pretty` (§3.4) parses generated code into a `syn::File`, which `full` provides.

syn 3, not syn 2. shelf uses syn 3.0.9. Most material online, and most derives on crates.io, are written against syn 2, and for a derive the difference is small. `DeriveInput`, `Data`, `Fields` and `Attribute` are unchanged, as are `parse_nested_meta` (Chapter 4) and `split_for_impl`. `Field` changed in two places: syn 2's `mutability` became `modifiers: FieldModifiers`, and a new `default` holds a field's default value (`x: i32 = 5`, a language feature still unstable in Rust 1.99). Code that *reads* a field's `attrs`, `ident` and `ty`, as shelf does, compiles against either version; code that builds or destructures a whole `Field` does not. The rest of 3.0's breaking changes are in types, expressions, statements, patterns, items, generics and literals; the 3.0.0 release notes list them.

3.4 Generating with quote

quote's `quote!` macro is the other half: it turns Rust code written in place into a `proc_macro2::TokenStream`, and `#name` inside it pastes in any value that implements `ToTokens`. A repetition, `#(…)*`, pastes an iterator, one element at a time, with whatever separators the repetition contains. That is enough for a first derive. Here is its logic as a stage program: the generator as an ordinary function, run on a struct built with `parse_quote!`:

```
use proc_macro2::TokenStream;
use quote::quote;
use syn::{Data, DeriveInput, Fields, parse_quote};

/// The first `#[derive(FromRow)]`: every named field read from the column of its name, as
/// Chapter 1 wrote it by hand.
fn expand(input: &DeriveInput) -> syn::Result<TokenStream> {
    let Data::Struct(data) = &input.data else {
        return Err(syn::Error::new_spanned(
            &input.ident,
            "FromRow needs a struct",
        ));
    };
    let Fields::Named(fields) = &data.fields else {
        return Err(syn::Error::new_spanned(
            &input.ident,
            "FromRow needs named fields",
        ));
    };

    let name = &input.ident;
    let reads = fields.named.iter().map(|field| {
        let ident = field.ident.as_ref().expect("named fields have names");
        let column = ident.to_string();
        quote! { #ident: row.try_get(#column)? }
    });

    Ok(quote! {
        impl FromRow for #name {
            fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
                Ok(Self { #(#reads,)* })
            }
        }
    })
}

fn main() {
    let input: DeriveInput = parse_quote! {
        struct Product {
            id: i64,
            sku: String,
            tags: Option<Vec<String>>,
        }
    };
    let code = shelf_playground::pretty(expand(&input).expect("a struct with named
fields"));
    println!("{code}");
    assert!(code.contains(r#"sku: row.try_get("sku")?"#));
    assert!(code.contains(r#"tags: row.try_get("tags")?"#));
}
```

```
impl FromRow for Product {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
        Ok(Self {
            id: row.try_get("id")?,
            sku: row.try_get("sku")?,
            tags: row.try_get("tags")?,
        })
```

```
    }
}
```

That is Chapter 1's impl, letter for letter. Each value pasted into `quote!` becomes the tokens its type stands for:

- `#name` and `#ident` are `syn::Idents`, pasted as identifiers.
- `#column` is a `String`, and a string is pasted as a string *literal*: `"sku"`, with the quotes. This is how the field's name becomes the column's name.
- `#(#reads,)*` pastes every element of the iterator `reads`, each followed by a comma.

Two helpers come up constantly. `quote::format_ident!` builds a new identifier from parts, as `format!` builds a string (`format_ident!("{}Columns", name)` is `ProductColumns`). And `syn::Index` pastes a number as a tuple index, which the next pitfall is about.

> **Pitfall:** a Rust integer pasted into `quote!` keeps its type as a suffix: `1usize`. That is right in an expression and wrong as a tuple field, which shelf's finished generator writes (`columns.0`, `columns.1`). The derive compiles, and its output fails to parse wherever it is used. `syn::Index::from(position)` pastes an unsuffixed `1`:

```
use quote::quote;
use syn::Index;

fn main() {
    let position: usize = 1;
    let suffixed = quote!(columns.#position);
    assert_eq!(suffixed.to_string(), "columns . 1usize");
    let error = syn::parse2::<syn::Expr>(suffixed)
        .err()
        .expect("a suffixed tuple index does not parse");
    assert_eq!(error.to_string(), "expected unsuffixed integer");

    let at = Index::from(position);
    assert_eq!(quote!(columns.#at).to_string(), "columns . 1");
}
```

The stage program printed its output with `pretty`, a helper in the shelf playground. It parses the tokens as a file of items and formats them with prettyplease, which formats as rustfmt would but works on a syntax tree instead of a file:

```
use proc_macro2::TokenStream;

/// Formats generated code as rustfmt would, so that an example can print what a derive
writes
/// and assert on it. A derive generates items (`impl` blocks, functions, constants), and
that
/// is what `tokens` must hold.
pub fn pretty(tokens: TokenStream) -> String {
    let file: syn::File = syn::parse2(tokens).expect("a derive generates items");
    prettyplease::unparse(&file)
}
```

`pretty` also checks the output: tokens that do not parse as items, such as the `1usize` above, make it panic in the stage program instead of failing later in a user's build.

3.5 Hygiene

The stage generator's output works in Chapter 1's program, and in few others. It names `FromRow`, `Row`, `Result`, `Ok` and `tokio_postgres` as bare paths, and the `impl` lands in the user's module, next to their struct. If that module has no `use tokio_postgres::Row`, or its own `type Result<T>`, or a different `FromRow`, the generated code breaks, or quietly means something else.

A procedural macro has even less protection than Chapter 2's `macro_rules!`. Every token `quote!` writes carries `Span::call_site()`, and the compiler resolves such tokens *as if you had typed them next to the struct*: paths, and local variables too. (`Span::mixed_site()` gives `macro_rules!`'s rules, where a macro's local variables are its own; shelf's generated code has no locals that could collide, so it does not need it.) So the generated code must not depend on any name the user's module defines:

- **Every path is absolute.** `::core::result::Result`, `::core::result::Result::Ok`, `::shelf::FromRow`, `::shelf::Error`. A leading `::` starts at a crate's name, which nothing in the user's module can shadow.
- **tokio-postgres is reached through shelf.** The generated code needs `Row` and `Column`, and the user's crate may not depend on tokio-postgres, or may depend on another version. So shelf gathers what generated code names in a hidden module, tokio-postgres included:

```
// orm/shelf/src/lib.rs (excerpt)
/// What generated code refers to. Not part of the API: it may change in any release.
#[doc(hidden)]
pub mod __private {
    pub use crate::json::Json;
    pub use crate::row::{column, column_at, get, get_try_from};
    // …
    pub use tokio_postgres;
}
```

orm/shelf/src/lib.rs

`::shelf::__private::tokio_postgres::Row` is then the `Row` that shelf itself was compiled against, whatever the user's crate has. The double underscore and `#[doc(hidden)]` are the usual signal that a module is for macros only: it does not appear in the documentation, and it may change in any release. Its helpers, `column` and `get`, keep the generated code short; Chapter 5 explains them. `Json` is shelf's wrapper for `#[shelf(json)]` fields (Chapter 4).

Raw identifiers. A field may be named after a keyword, with Rust's raw-identifier prefix: `r#type`. Its identifier's text includes the prefix, so the stage generator, which uses `ident.to_string()`, would read a column called `r#type`. syn's `IdentExt::unraw` strips it:

```
use syn::ext::IdentExt;
use syn::{DeriveInput, Fields, parse_quote};

fn main() {
    let input: DeriveInput = parse_quote! {
        struct Product { r#type: String }
    };
    let syn::Data::Struct(data) = &input.data else {
        panic!("a struct")
    };
    let Fields::Named(fields) = &data.fields else {
        panic!("named fields")
    };
    let ident = fields.named[0].ident.as_ref().expect("a named field");

    assert_eq!(ident.to_string(), "r#type"); // what the stage generator would read
    assert_eq!(ident.unraw().to_string(), "type"); // the column's name
}
```

The field itself is still written `r#type` in the generated initializer (`r#type: …`), where the prefix is needed; only the column's name drops it.

`#[automatically_derived]` goes on the generated `impl`. The Rust Reference says it "has no direct effect, but it may be used by tools and diagnostic lints to detect these automatically generated implementations". The standard library's derives add it, and so does every careful derive.

This program puts the finished derive in a hostile module: one with its own `Result`, and a constant named `Ok`. It also reads a column named after a keyword:

```
mod catalog {
    // Names that would capture a careless expansion.
    #[allow(dead_code)]
    pub type Result<T> = std::result::Result<T, String>;
    #[allow(dead_code, non_upper_case_globals)]
    const Ok: () = ();

    #[derive(Debug, shelf::FromRow)]
    pub struct Product {
        pub sku: String,
        pub r#type: String,
    }
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY, type text NOT NULL);
             INSERT INTO products VALUES ('MUG-01', 'mug'), ('TEE-02', 'shirt');",
        )
        .await?;

    let products: Vec<catalog::Product> =
        shelf::fetch_all(&client, "SELECT sku, type FROM products ORDER BY sku",
&[]).await?;
    assert_eq!(products[0].r#type, "mug");
    assert_eq!(products[1].sku, "TEE-02");
    Ok(())
}
```

shelf's own tests do the same with a `type Result = ();`, a `const Ok`, and a trait called `Default`, which `#[shelf(skip)]` must not pick up (Chapter 4):

```
// orm/shelf/tests/derive.rs (excerpt)
// The generated code must not depend on what the user's module calls `Result`, `Ok` or
// `Default`.
#[allow(dead_code)]
type Result = ();
#[allow(dead_code, non_upper_case_globals)]
const Ok: () = ();
#[allow(dead_code)]
trait Default {}

#[allow(dead_code)] // only the impls matter here
#[derive(shelf::FromRow)]
struct Product {
    id: i64,
    r#type: String,
    tags: Option<Vec<String>>,
    #[shelf(skip)]
    cached: Vec<u8>,
}
```

orm/shelf/tests/derive.rs

3.6 Seeing the expansion

You will want to see what a derive wrote: to learn, to debug, and to review a change to the generator. There are three ways.

Print it. Because the generator lives in an ordinary library, any program can call it and format the result with `pretty`, as §0.7 and §3.4 did. This is the way this book uses throughout, because it works on stable Rust with nothing to install, and because the output can be asserted on. shelf's expansion tests do exactly this: each file in `orm/shelf-codegen/tests/expand/` holds structs with a derive, and a test compares the pretty-printed expansion of each with a file next to it, so that any change to the generated code shows up as a diff in review (Chapter 13).

Ask the compiler. rustc can print a crate after expanding every macro in it, with the unstable flag `-Zunpretty=expanded`. The tool cargo-expand wraps it (`cargo install cargo-expand`, then `cargo expand`), and formats the result with rustfmt. An unstable flag is normally refused by a stable compiler:

```
error: the option `Z` is only accepted on the nightly compiler
```

cargo-expand gets around that by setting `RUSTC_BOOTSTRAP=1` for the compiler it runs (its `src/main.rs`), so it works on stable Rust. This is what the compiler itself compiled for §3.5's `catalog::Product` (the raw command cargo-expand wraps, run the same way, before formatting):

```
    pub struct Product {
        pub sku: String,
        pub r#type: String,
    }
    #[automatically_derived]
    impl ::core::fmt::Debug for Product {
        #[inline]
        fn fmt(&self, f: &mut ::core::fmt::Formatter) -> ::core::fmt::Result {
            ::core::fmt::Formatter::debug_struct_field2_finish(f, "Product",
                "sku", &self.sku, "type", &&self.r#type)
        }
    }
    #[automatically_derived]
    impl ::shelf::FromRow for Product {
        type Columns = (usize, usize);
        fn resolve(columns: &[::shelf::__private::tokio_postgres::Column])
            -> ::core::result::Result<Self::Columns, ::shelf::Error> {
            ::core::result::Result::Ok((::shelf::__private::column::<String>(columns,
                            "Product", "sku", "sku")?,
                    ::shelf::__private::column::<String>(columns, "Product",
                            "type", "type")?))
        }
        fn from_row_at(row: &::shelf::__private::tokio_postgres::Row,
            columns: &Self::Columns)
            -> ::core::result::Result<Self, ::shelf::Error> {
            ::core::result::Result::Ok(Self {
                    sku: ::shelf::__private::get::<String>(row, columns.0,
                            "Product", "sku")?,
                    r#type: ::shelf::__private::get::<String>(row, columns.1,
                            "Product", "type")?,
                })
        }
    }
```

The compiler's view shows what `pretty` cannot: the expansion in place, next to every other derive. `#[derive(Debug)]`, from the standard library, follows the same rules as shelf: an `#[automatically_derived]` impl, every path starting at `::core`, and the field `r#type` named `"type"`, without its prefix. Expansion to text is lossy, as cargo-expand's README warns (hygiene, for one, does not survive printing), so treat it as a debugging aid.

Ask the editor. rust-analyzer's command "Expand macro recursively at caret" (in VS Code: *rust-analyzer: Expand macro recursively at caret*) shows the expansion of the macro under the cursor.

3.7 Errors as values

A derive that cannot do its job could panic, and the compiler would report it. This is a probe derive that panics on an enum:

```
error: proc-macro derive panicked
 --> user/src/bin/panics.rs:1:10
  |
1 | #[derive(show::ShowInput)]
  |          ^^^^^^^^^^^^^^^
  |
  = help: message: ShowInput needs a struct
```

The message survives, but the error points at the derive's name instead of the code that is wrong, and a panic can report only one problem. So shelf's generator never panics on user input. It returns `syn::Result<TokenStream>`, whose error is a value with a message and a *span*, the place in the user's source it is about; several errors can be combined into one (Chapter 4). `syn::Error::new_spanned(tokens, message)` takes the span from any tokens of the input. The shell turns the error into tokens with `into_compile_error`, and the compiler reports them as an error at that span.

This program runs shelf's generator on two inputs it refuses, and prints the message and the tokens the error becomes:

```
use syn::{DeriveInput, parse_quote};

fn main() {
    let status: DeriveInput = parse_quote! {
        enum Status { Pending, Paid }
    };
    let error = shelf_codegen::from_row::expand(&status).unwrap_err();
    println!("{error}");
    println!("{}", error.to_compile_error());
    assert!(error.to_string().contains("derive postgres_types::FromSql"));

    let pair: DeriveInput = parse_quote! {
        struct Pair(i32, i32);
    };
    let error = shelf_codegen::from_row::expand(&pair).unwrap_err();
    println!("{error}");
    assert!(error.to_string().starts_with("FromRow needs named fields"));
}
```

```
FromRow maps a row to a struct; for a Postgres enum, derive postgres_types::FromSql instead
:: core :: compile_error ! { "FromRow maps a row to a struct; for a Postgres enum, derive
postgres_types::FromSql instead" }
FromRow needs named fields: each one is read from the column of that name
```

An error becomes an invocation of the standard `compile_error!` macro, whose tokens carry the error's span. Here, outside the compiler, the span is a placeholder. In a real build it points at the user's code, and shelf's UI tests (Chapter 4) capture what the compiler then prints. For an enum, a tuple struct and a union:

```
// orm/shelf/tests/ui/shapes.rs (excerpt)
#[derive(FromRow)]
enum Status {
    Pending,
    Paid,
}
```

```
#[derive(FromRow)]
struct Pair(i32, i32);

#[derive(FromRow)]
union Bits {
    int: i32,
    float: f32,
}
```

```
error: FromRow maps a row to a struct; for a Postgres enum, derive postgres_types::FromSql
instead
 --> tests/ui/shapes.rs:5:1
  |
5 | enum Status {
  | ^^^^

error: FromRow needs named fields: each one is read from the column of that name
  --> tests/ui/shapes.rs:11:8
   |
11 | struct Pair(i32, i32);
   |        ^^^^

error: FromRow cannot be derived for a union
  --> tests/ui/shapes.rs:14:1
   |
14 | union Bits {
   | ^^^^^
```

Each error points at the token it was built from: the `enum` keyword (`data.enum_token`), the tuple struct's name (`input.ident`), the `union` keyword. The message says what to do instead, where there is something to do: a Postgres enum is one column, which postgres-types' derive maps (main book Chapter 5).

Spans also decide where errors in the *generated* code land, and the rule is simple: tokens the derive writes itself carry the span of the `#[derive]`, and tokens copied from the input keep their own. A field's type is copied from the struct into `get::<#ty>`, so if that type cannot be read from a column, the compiler's error points at the field's type in your struct, not at the derive. Chapter 4 uses that, and makes the message read well.

Side by side — *attaching generated behaviour to a type*

The task: give a struct a row-mapping impl. sqlx's derive generates one that is generic over every database sqlx supports; this is its shape for a two-field struct, from sqlx 0.9's code generator:

```
// sqlx (illustration): the shape of what #[derive(sqlx::FromRow)] generates in sqlx 0.9
#[automatically_derived]
impl<'a, R: ::sqlx::Row> ::sqlx::FromRow<'a, R> for Product
where
    &'a ::std::primitive::str: ::sqlx::ColumnIndex<R>,
    String: ::sqlx::decode::Decode<'a, R::Database>,
    String: ::sqlx::types::Type<R::Database>,
    i64: ::sqlx::decode::Decode<'a, R::Database>,
    i64: ::sqlx::types::Type<R::Database>,
{
    fn from_row(__row: &'a R) -> ::sqlx::Result<Self> {
        let sku: String = __row.try_get("sku")?;
        let price_cents: i64 = __row.try_get("price_cents")?;
        ::std::result::Result::Ok(Product { sku, price_cents })
    }
}
```

By hand (Chapter 1's trait) and with shelf's derive, on the same struct, in one program:

```
use tokio_postgres::Row;

trait FromRow: Sized {
```

```
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error>;
}

// By hand: the impl is yours to write, read and change.
#[derive(Debug, PartialEq)]
struct Product {
    sku: String,
    price_cents: i64,
}

impl FromRow for Product {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
        Ok(Product {
            sku: row.try_get("sku")?,
            price_cents: row.try_get("price_cents")?,
        })
    }
}

// shelf: the impl is generated next to the struct, at compile time.
#[derive(Debug, PartialEq, shelf::FromRow)]
struct ShelfProduct {
    sku: String,
    price_cents: i64,
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let client = shelf_playground::connect().await?;
    let sql = "SELECT 'MUG-01' AS sku, 1299::int8 AS price_cents";

    let by_hand = Product::from_row(&client.query_one(sql, &[]).await?)?;
    let with_shelf: ShelfProduct = shelf::fetch_one(&client, sql, &[]).await?;
    assert_eq!(
        (by_hand.sku, by_hand.price_cents),
        (with_shelf.sku, with_shelf.price_cents)
    );
    Ok(())
}
```

	by hand	shelf	sqlx
Same	An impl of a row-mapping trait, next to the struct	The same impl, written by a derive at compile time	The same impl, written by a derive at compile time
Different	You write and maintain it, and it can do anything	For tokio-postgres's `Row` only; paths are absolute and reach tokio-postgres through `::shelf::__private`; columns are found once per statement (Chapter 5)	Generic over every sqlx database (`R: Row`), with a `Decode` and a `Type` bound per field; each field is looked up by name on every row

In shelf

The finished generator is §3.4's stage program, grown. The skeleton is the same: refuse what is not a struct with named fields, loop over the fields, collect a piece of code per field, and paste the pieces into one `impl` with absolute paths. What the field loop collects is more, because `FromRow` has the two-step shape Chapter 5 explains, and the attributes of Chapter 4 are elided here:

```
// orm/shelf-codegen/src/from_row.rs (excerpt)
/// The `impl shelf::FromRow` for `input`, or every error found in it.
```

```
pub fn expand(input: &DeriveInput) -> Result<TokenStream> {
    let fields = named_fields(input)?;
    // … the attributes are parsed here, and every error is collected (Chapter 4)

    let model = input.ident.to_string();
    let mut bounds = Bounds::new(&input.generics);
    let mut column_types = Vec::new(); // the tuple `Columns`, one element per field
    let mut resolves = Vec::new(); // `resolve`'s tuple, element by element
    let mut reads = Vec::new(); // `from_row_at`'s field initializers

    for (position, (field, attrs)) in fields.iter().enumerate() {
        let ident = field.ident.as_ref().expect("named fields have identifiers");
        // A raw identifier names the column without its prefix: `r#type` reads `type`.
        let name = ident.unraw().to_string();
        let ty = &field.ty;
        let at = Index::from(position);

        // The type `resolve` checks the column against, and the expression that reads it.
        let (checked, read) = match attrs.read() {
            // … skip, flatten, json and try_from (Chapter 4)
            Read::Column => {
                bounds.add_if_generic(&[ty], parse_quote!(#ty: ::shelf::FromColumn));
                (
                    quote!(#ty),
                    quote!(::shelf::__private::get::<#ty>(row, columns.#at, #model,
#name)?),
                )
            }
        };
        // … `column` is the field's name, or what rename and rename_all make of it (Chapter
4)
        column_types.push(quote!(usize));
        resolves.push(quote! {
            ::shelf::__private::column::<#checked>(columns, #model, #name, #column)?
        });
        reads.push(quote!(#ident: #read));
    }

    // With nothing to read (every field skipped, or none at all), the parameters go unused.
    let reads_any = column_types.iter().any(|t| t.to_string() != "()");
    let (row, columns) = if reads_any {
        (quote!(row), quote!(columns))
    } else {
        (quote!(_row), quote!(_columns))
    };

    let name = &input.ident;
    let generics = bounds.apply(&input.generics);
    let (impl_generics, ty_generics, where_clause) = generics.split_for_impl();

    // Every path is absolute (`::shelf`, `::core`): the expansion lands in the user's
module,
    // where `Result` or even `shelf` may mean something else.
    Ok(quote! {
        #[automatically_derived]
        impl #impl_generics ::shelf::FromRow for #name #ty_generics #where_clause {
            type Columns = ( #(#column_types,)* );

            fn resolve(
                #columns: &[::shelf::__private::tokio_postgres::Column],
            ) -> ::core::result::Result<Self::Columns, ::shelf::Error> {
                ::core::result::Result::Ok(( #(#resolves,)* ))
            }

            fn from_row_at(
                #row: &::shelf::__private::tokio_postgres::Row,
                #columns: &Self::Columns,
            ) -> ::core::result::Result<Self, ::shelf::Error> {
                ::core::result::Result::Ok(Self { #(#reads,)* })
```

```
            }
        }
    })
}

/// The struct's named fields, or an error that says why the input cannot be mapped.
pub(crate) fn named_fields(input: &DeriveInput) -> Result<&Punctuated<Field, Token![,]>> {
    match &input.data {
        Data::Struct(data) => match &data.fields {
            Fields::Named(fields) => Ok(&fields.named),
            _ => Err(Error::new_spanned(
                &input.ident,
                "FromRow needs named fields: each one is read from the column of that name",
            )),
        },
        Data::Enum(data) => Err(Error::new_spanned(
            data.enum_token,
            "FromRow maps a row to a struct; for a Postgres enum, derive \
             postgres_types::FromSql instead",
        )),
        Data::Union(data) => Err(Error::new_spanned(
            data.union_token,
            "FromRow cannot be derived for a union",
        )),
    }
}
```

orm/shelf-codegen/src/from_row.rs

Three details are new since the stage program. The loop builds three lists instead of one, because each field contributes to `Columns`, to `resolve` and to `from_row_at`. `split_for_impl` splits the struct's generics into the three places an `impl` needs them (`impl<T>`, `Product<T>` and the `where` clause), with the bounds Chapter 4 adds. And a struct whose fields are all skipped would leave `row` and `columns` unused, so the generator names them `_row` and `_columns` then, as you would in hand-written code.

When not to write a derive

- **For a handful of types.** Chapter 1's impls are a few lines each, and need no proc-macro crate, and no syn, quote and proc-macro2 to compile before your own code.
- **When a blanket impl does the job.** If every type with some trait should get the behaviour, `impl<T: Trait> Other for T` needs no macro at all.
- **When the generated code would be hard to predict.** A derive's value is that users can guess its output. If they cannot, they will read the expansion every time, and a function they call explicitly would serve them better.

Summary

- A proc-macro crate (`proc-macro = true`) runs inside the compiler. A derive is a function from the item's tokens to new tokens, which are added after the item.
- A derive receives the whole item, attributes and doc comments included, but not the `#[derive]` line, and after `cfg` has been evaluated.
- A proc-macro crate can export only macros, and `proc_macro`'s types do not work outside the compiler. So shelf keeps code generation in `shelf-codegen`, on `proc_macro2`, with `shelf-macros` as a three-line shell per derive.
- syn parses tokens into `DeriveInput` → `Data` → `Fields` → `Field`; quote builds tokens with `#value` and `#(…)*`. A `String` pastes as a literal, a `usize` with a suffix (use `syn::Index` for tuple fields).
- syn 3's derive types are syn 2's, except two fields of `Field`; reading `attrs`, `ident` and `ty` is unchanged.

- A derive's tokens resolve at the call site: write every path absolute (`::core`, `::shelf`), reach tokio-postgres through `::shelf::__private`, strip `r#` with `unraw`, and mark the impl `#[automatically_derived]`.
- See an expansion with `pretty` (stable, assertable), `cargo expand` (the compiler's own view; runs on stable through `RUSTC_BOOTSTRAP`), or rust-analyzer.
- Errors are `syn::Error` values with spans, turned into `compile_error!`; tokens copied from the input keep their spans, so errors in generated code point at the user's code.

Exercises

1. **The column list, in Rust.** Extend §3.4's stage generator so that it also writes `impl Product { pub const COLUMNS: &'static str = "id, sku, tags"; }`. Compare with Chapter 2's Exercise 2, which built the same string with `macro_rules!`. *Hint:* the generator is ordinary Rust: collect the names into a `Vec<String>` and `join` them, then paste the result as one literal.
2. **Make it hygienic.** Rewrite §3.4's generator so that its output compiles in any module: absolute paths, and `r#type` reading `type`. Assume the trait is `::shelf_book::FromRow` and the row type is `::tokio_postgres::Row`. *Hint:* every bare name in the `quote!` is a name the user's module could define.
3. **An expansion test.** Write a test, as a program with an `assert_eq!`, that pins down the whole output of §3.4's generator for a two-field struct. *Hint:* compare `pretty`'s output with the expected text in a raw string literal, `r#"…"#`.

Solutions

1. In a proc macro the string is computed by ordinary Rust while the macro runs, and pasted as one literal; `macro_rules!` had to assemble it from `stringify!` and `concat!`:

```
use proc_macro2::TokenStream;
use quote::quote;
use syn::{Data, DeriveInput, Fields, parse_quote};

fn expand(input: &DeriveInput) -> syn::Result<TokenStream> {
    let Data::Struct(data) = &input.data else {
        return Err(syn::Error::new_spanned(
            &input.ident,
            "FromRow needs a struct",
        ));
    };
    let Fields::Named(fields) = &data.fields else {
        return Err(syn::Error::new_spanned(
            &input.ident,
            "FromRow needs named fields",
        ));
    };

    let name = &input.ident;
    let idents: Vec<_> = fields
        .named
        .iter()
        .filter_map(|f| f.ident.as_ref())
        .collect();
    let columns: Vec<String> = idents.iter().map(|ident| ident.to_string()).collect();
    let list = columns.join(", ");

    Ok(quote! {
        impl #name {
            pub const COLUMNS: &'static str = #list;
        }

        impl FromRow for #name {
            fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
                Ok(Self { #(#idents: row.try_get(#columns)?,)* })
            }
```

```
        }
    })
}

fn main() {
    let input: DeriveInput = parse_quote! {
        struct Product { id: i64, sku: String, tags: Option<Vec<String>> }
    };
    let code = shelf_playground::pretty(expand(&input).unwrap());
    println!("{code}");
    assert!(code.contains(r#"pub const COLUMNS: &'static str = "id, sku, tags";"#));
}
```

The repetition `#(#idents: row.try_get(#columns)?,)*` walks two lists in step, as Chapter 2's `$field` and `$ty` did.

2. Every path starts at a crate, and the column name comes from `unraw`:

```
use proc_macro2::TokenStream;
use quote::quote;
use syn::ext::IdentExt;
use syn::{Data, DeriveInput, Fields, parse_quote};

fn expand(input: &DeriveInput) -> syn::Result<TokenStream> {
    let Data::Struct(data) = &input.data else {
        return Err(syn::Error::new_spanned(
            &input.ident,
            "FromRow needs a struct",
        ));
    };
    let Fields::Named(fields) = &data.fields else {
        return Err(syn::Error::new_spanned(
            &input.ident,
            "FromRow needs named fields",
        ));
    };

    let name = &input.ident;
    let reads = fields.named.iter().map(|field| {
        let ident = field.ident.as_ref().expect("named fields have names");
        let column = ident.unraw().to_string();
        quote! { #ident: row.try_get(#column)? }
    });

    Ok(quote! {
        #[automatically_derived]
        impl ::shelf_book::FromRow for #name {
            fn from_row(
                row: &::tokio_postgres::Row,
            ) -> ::core::result::Result<Self, ::tokio_postgres::Error> {
                ::core::result::Result::Ok(Self { #(#reads,)* })
            }
        }
    })
}

fn main() {
    let input: DeriveInput = parse_quote! {
        struct Product { sku: String, r#type: String }
    };
    let code = shelf_playground::pretty(expand(&input).unwrap());
    println!("{code}");
    assert!(code.contains("impl ::shelf_book::FromRow for Product"));
    assert!(code.contains(r#"r#type: row.try_get("type")?"#));
    assert!(code.contains("::core::result::Result::Ok(Self"));
}
```

`row` is still a bare name, and that is fine: it is a parameter the generated function declares itself, so the user's module cannot change what it means. This version still requires the user's crate to depend on tokio-postgres; shelf's `__private` re-export removes that last dependency.

3. The test fails, and prints both versions, as soon as the generator's output changes in any way, which is what shelf's expansion tests are for:

```
use proc_macro2::TokenStream;
use quote::quote;
use syn::{Data, DeriveInput, Fields, parse_quote};

fn expand(input: &DeriveInput) -> syn::Result<TokenStream> {
    let Data::Struct(data) = &input.data else {
        return Err(syn::Error::new_spanned(
            &input.ident,
            "FromRow needs a struct",
        ));
    };
    let Fields::Named(fields) = &data.fields else {
        return Err(syn::Error::new_spanned(
            &input.ident,
            "FromRow needs named fields",
        ));
    };
    let name = &input.ident;
    let reads = fields.named.iter().map(|field| {
        let ident = field.ident.as_ref().expect("named fields have names");
        let column = ident.to_string();
        quote! { #ident: row.try_get(#column)? }
    });
    Ok(quote! {
        impl FromRow for #name {
            fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
                Ok(Self { #(#reads,)* })
            }
        }
    })
}

const EXPECTED: &str = r#"impl FromRow for Price {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
        Ok(Self {
            cents: row.try_get("cents")?,
            currency: row.try_get("currency")?,
        })
    }
}
"#;

fn main() {
    let input: DeriveInput = parse_quote! {
        struct Price { cents: i64, currency: String }
    };
    let actual = shelf_playground::pretty(expand(&input).unwrap());
    assert_eq!(actual, EXPECTED, "the expansion changed:\n{actual}");
}
```

shelf's version, `orm/shelf-codegen/tests/expand.rs`, keeps the structs and their expected expansions in files, and rewrites the expected files when run with `SHELF_BLESS=1`, so that a deliberate change is accepted by reading a diff rather than by editing strings.

CHAPTER 4

Attributes, errors and generics

A real schema does not match your structs one to one. A column is called `sku_code` but the field is `sku`; a legacy table uses `camelCase`; a field holds a value computed in Rust and is not in the table at all; a `jsonb` column holds a document you want as a struct; an `int` should become a `Grams` that refuses negative values. Each of those is one attribute on a field, and serde has taught every Rust developer to expect them.

This chapter teaches the derive to read `#[shelf(…)]`. Parsing is the easy part. The work is in what happens when the attributes are wrong: a misspelt key, a value where none belongs, two attributes that contradict each other. A good derive reports all of them in one build, each at the token that caused it, in words the user understands. The same care goes into generic structs, which need exactly the bounds their fields require, and no others.

In this chapter you will learn:

- how a derive declares a helper attribute, and what happens if it forgets
- how to parse `#[shelf(…)]` with `parse_nested_meta` into a typed struct
- what `rename_all`, `rename`, `skip`, `flatten`, `json` and `try_from` generate
- how to report every mistake in one build, each at the right token
- how to test a derive's errors with trybuild, and what reading them changed in shelf
- how to give a generic struct the bounds its fields need, and why shelf refuses lifetimes
- what darling would do for you, and why shelf does without it

4.1 Helper attributes

The compiler resolves every attribute it meets. `#[allow(…)]` and `#[doc = "…"]` are built in; `#[tokio::main]` is an attribute macro, found by its path; `#[rustfmt::skip]` and `#[diagnostic::…]` belong to tools. `#[shelf(…)]` is none of those. It is a *helper attribute*: a name that a derive declares, which the compiler then accepts on any item that uses the derive, and leaves in place for the derive to read. It does nothing by itself. shelf declares it in the shell of Chapter 3:

```
// orm/shelf-macros/src/lib.rs (excerpt)
#[proc_macro_derive(FromRow, attributes(shelf))]
```

What happens without the declaration? This probe derive, written for this chapter like §3.1's `ShowInput` and not part of shelf, generates nothing and declares no helper:

```
#[proc_macro_derive(NoHelper)]
pub fn no_helper(_input: TokenStream) -> TokenStream {
    TokenStream::new()
}
```

Put it on a struct with a `#[shelf(skip)]` field, in a crate that depends on shelf, and the build fails:

```
error: cannot find attribute `shelf` in this scope
 --> user/src/bin/no_helper.rs:3:7
  |
3 |     #[shelf(skip)]
  |       ^^^^^
  |
  = note: `shelf` is in scope, but it is a crate, not an attribute
```

The derive never ran. The compiler refused the attribute while resolving names, before any macro saw the struct, and the note says where it looked: the only `shelf` in scope is the crate, and a crate is not an attribute. shelf met exactly this error when its hygiene test first used `#[shelf(skip)]` (PLAN.md §12 #9). Forgetting the `#[derive(FromRow)]` itself gives the same error, with the same note, for the same reason: a helper attribute exists only on an item whose derive declares it.

All of shelf's derives will declare the same helper, so `#[shelf(…)]` is one namespace, shared by `FromRow` and, from Chapter 6 on, `Table`, `Insert` and `Patch`. That decides how shelf parses it.

4.2 Parsing with `parse_nested_meta`

An attribute arrives as a `syn::Attribute`: its path, `shelf`, and the tokens between the parentheses, not yet parsed. syn does not know what `rename = "sku_code"` means to you, but it knows the shape that most attributes share, which its documentation calls the *conventional syntax for structured attributes*: a comma-separated list of items, each a path, optionally followed by `= value` or by a list in parentheses of the same shape. serde's attributes follow it, and so do shelf's.

`Attribute::parse_nested_meta` walks that list. For each item it parses the path and calls your closure with it, as `meta.path`. The closure reads whatever follows the path: `meta.value()?` consumes the `=` and returns the stream to parse the value from. When the closure returns, syn expects a comma or the end of the list. This stage program is a parser for three of shelf's keys, run on a struct built with `parse_quote!`:

```rust
use quote::ToTokens;
use syn::{Attribute, Data, DeriveInput, Fields, LitStr, Type, parse_quote};

/// The stage version of shelf's `FieldAttrs`: three keys, as typed values.
#[derive(Default)]
struct FieldAttrs {
    rename: Option<String>,
    skip: bool,
    try_from: Option<Type>,
}

fn parse(attrs: &[Attribute]) -> syn::Result<FieldAttrs> {
    let mut out = FieldAttrs::default();
    for attr in attrs.iter().filter(|attr| attr.path().is_ident("shelf")) {
        attr.parse_nested_meta(|meta| {
            if meta.path.is_ident("rename") {
                // `rename = "…"`: `value()` reads the `=`, then comes a string literal.
                let lit: LitStr = meta.value()?.parse()?;
                out.rename = Some(lit.value());
            } else if meta.path.is_ident("skip") {
                // A flag: the key is all there is.
                out.skip = true;
            } else if meta.path.is_ident("try_from") {
                // `try_from = i64`: the value is a type.
                out.try_from = Some(meta.value()?.parse()?);
            } else {
                return Err(meta.error("unknown shelf attribute"));
            }
            Ok(())
        })?;
    }
    Ok(out)
```

```
}

fn main() {
    let input: DeriveInput = parse_quote! {
        struct Product {
            #[shelf(rename = "sku_code")]
            sku: String,
            #[shelf(try_from = i64, rename = "price_cents")]
            price: Cents,
            /// Set by the user interface; not stored.
            #[shelf(skip)]
            selected: bool,
        }
    };
    let Data::Struct(data) = &input.data else {
        panic!("a struct")
    };
    let Fields::Named(fields) = &data.fields else {
        panic!("named fields")
    };
    let mut parsed = Vec::new();
    for field in &fields.named {
        let attrs = parse(&field.attrs).expect("valid attributes");
        let try_from = attrs
            .try_from
            .as_ref()
            .map(|ty| ty.to_token_stream().to_string());
        println!(
            "{}: rename {:?}, skip {}, try_from {:?}",
            field.ident.as_ref().expect("a named field"),
            attrs.rename,
            attrs.skip,
            try_from
        );
        parsed.push(attrs);
    }
    assert_eq!(parsed[0].rename.as_deref(), Some("sku_code"));
    assert_eq!(parsed[1].rename.as_deref(), Some("price_cents"));
    assert!(parsed[1].try_from.is_some());
    assert!(parsed[2].skip);

    // Three inputs the stage parser handles badly.
    let unknown: Attribute = parse_quote!(#[shelf(renamed = "sku")]);
    let Err(error) = parse(&[unknown]) else {
        panic!("an unknown key is refused")
    };
    println!("renamed = \"sku\": {error}");

    let flag_with_value: Attribute = parse_quote!(#[shelf(skip = true)]);
    let Err(error) = parse(&[flag_with_value]) else {
        panic!("a value after a flag is refused")
    };
    println!("skip = true: {error}");
    assert_eq!(error.to_string(), "expected `,`");

    let twice: Attribute = parse_quote!(#[shelf(rename = "a", rename = "b")]);
    let attrs = parse(&[twice]).expect("accepted, wrongly");
    println!("rename twice: {:?}", attrs.rename);
    assert_eq!(attrs.rename.as_deref(), Some("b"));
}
```

```
sku: rename Some("sku_code"), skip false, try_from None
price: rename Some("price_cents"), skip false, try_from Some("i64")
selected: rename None, skip true, try_from None
renamed = "sku": unknown shelf attribute
skip = true: expected `,`
rename twice: Some("b")
```

Each kind of key reads its value its own way. `rename` parses a string literal, a `LitStr`, and keeps its contents without the quotes. `skip` is a *flag*: there is nothing after the key to read. `try_from` parses a `syn::Type`, because every node of syn's syntax tree can be parsed from a stream, a type as much as a literal. The doc comment on `selected` is an attribute too (`#[doc = "…"]`), and the filter on `shelf` passes over it. And `price` shows that one attribute can hold several keys; two `#[shelf(…)]` attributes would parse the same.

The last three lines are what the stage parser gets wrong. An unknown key is refused, but the message does not say which keys exist. A flag given a value, `skip = true`, fails with syn's "expected `,`": the closure read nothing after `skip`, so syn looked for the next item and found `=`. That is true, and no help to whoever wrote it. And a key written twice is accepted, with the second value silently winning. shelf's parser keeps the stage's shape and fixes all three:

```
// orm/shelf-codegen/src/attrs.rs (excerpt)
/// A parsed value, with the span of the key that set it, for error messages.
pub struct Keyed<T> {
    pub span: Span,
    pub value: T,
}

// … `FieldAttrs` has an `Option<Keyed<…>>` per key: `Keyed<LitStr>` for `rename`, and so on

impl FieldAttrs {
    pub fn parse(attrs: &[Attribute], derive: Derive, errors: &mut Errors) -> Self {
        let mut out = FieldAttrs::default();
        for attr in shelf_attrs(attrs) {
            // The key being parsed, so that its error goes to the derive that owns it.
            let mut key = None;
            let parsed = attr.parse_nested_meta(|meta| {
                key = known(&meta, FIELD_KEYS);
                let parsed = out.parse_key(&meta);
                if parsed.is_ok() {
                    key = None;
                }
                parsed
            });
            if let Err(error) = parsed
                && derive.reports(key)
            {
                errors.push(error);
            }
        }
        if derive == Derive::FromRow {
            out.check_combinations(errors);
        }
        out
    }

    fn parse_key(&mut self, meta: &ParseNestedMeta) -> Result<()> {
        if meta.path.is_ident("rename") {
            let lit = name_value(meta, "a column name")?;
            once(&mut self.rename, lit, meta)
        } else if meta.path.is_ident("skip") {
            flag(&mut self.skip, meta)
            // … `flatten`, `json`, `primary_key` and `generated` the same way
        } else if meta.path.is_ident("try_from") {
            let ty = type_value(meta)?;
            once(&mut self.try_from, ty, meta)
        } else {
            Err(unknown(meta, "a field", FIELD_KEYS))
        }
    }
    // …
}
```

orm/shelf-codegen/src/attrs.rs

The closure hands each key to `parse_key`, which holds the stage's chain of `if`s. Every value is kept as a `Keyed<T>`: the value, and the span of the key that set it, so that a later check can point at `json` or `rename` in the user's source (§4.4). A flag is a `Keyed<()>`: all that matters is that it was there, and where. `name_value` reads a string literal and refuses an empty one. Errors are not returned but pushed onto `errors`, which collects them (§4.4); the `derive` argument and the `key` the closure records decide whether this derive reports them at all, which the end of this section explains. The rest of the work is in four small functions:

```
// orm/shelf-codegen/src/attrs.rs (excerpt)
/// Stores a key's value, refusing a second one.
fn once<T>(slot: &mut Option<Keyed<T>>, value: T, meta: &ParseNestedMeta) -> Result<()> {
    if slot.is_some() {
        return Err(meta.error(format!("duplicate `{}`", name_of(meta))));
    }
    *slot = Some(Keyed {
        span: meta.path.span(),
        value,
    });
    Ok(())
}

/// A key without a value, such as `skip`.
fn flag(slot: &mut Option<Keyed<()>>, meta: &ParseNestedMeta) -> Result<()> {
    if meta.input.peek(Token![=]) || meta.input.peek(syn::token::Paren) {
        return Err(meta.error(format!("`{}` takes no value", name_of(meta))));
    }
    once(slot, (), meta)
}

/// A type after `=`: `try_from = i64`, or quoted as serde writes it, `try_from = "i64"`.
fn type_value(meta: &ParseNestedMeta) -> Result<Type> {
    let value = meta.value()?;
    if value.peek(LitStr) {
        let lit: LitStr = value.parse()?;
        lit.parse()
            .map_err(|_| Error::new_spanned(&lit, "expected a type, as in `try_from =
i64`"))
    } else {
        value.parse()
    }
}

fn unknown(meta: &ParseNestedMeta, on: &str, known: &[&str]) -> Error {
    let known: Vec<String> = known.iter().map(|k| format!("`{k}`")).collect();
    meta.error(format!(
        "unknown shelf attribute `{}` on {on}; expected {}",
        name_of(meta),
        known.join(", ")
    ))
}
```

orm/shelf-codegen/src/attrs.rs

- `once` stores a value and refuses a second one.
- `flag` looks ahead before anything else (`peek` reads a token without consuming it): if an `=` or a parenthesis follows the key, it says that the key takes no value, instead of leaving syn to complain about a comma.
- `type_value` accepts a type as it is, `try_from = i64`, or quoted, `try_from = "i64"`, as serde writes its own `try_from`. A quoted type is parsed from the literal's contents by `LitStr::parse`.
- `unknown` lists the keys that do exist.

`meta.error(…)` builds a `syn::Error` that covers everything from the key to the last token the closure has read. So a duplicate `rename = "e2"` is underlined whole, because `once` refused it after its value was parsed, while `skip = true` underlines only `skip`, because `flag` refused it before reading on. One of shelf's UI tests (§4.5) holds six such mistakes, and this is what the compiler prints for them:

```
error: expected `=`
 --> tests/ui/bad_values.rs:6:19
  |
6 |     #[shelf(rename)]
  |                   ^

error: expected string literal
 --> tests/ui/bad_values.rs:8:22
  |
8 |     #[shelf(rename = 1)]
  |                      ^

error: a column name cannot be empty
  --> tests/ui/bad_values.rs:10:22
   |
10 |     #[shelf(rename = "")]
   |                      ^^

error: `skip` takes no value
  --> tests/ui/bad_values.rs:12:13
   |
12 |     #[shelf(skip = true)]
   |             ^^^^

error: duplicate `rename`
  --> tests/ui/bad_values.rs:14:28
   |
14 |     #[shelf(rename = "e1", rename = "e2")]
   |                            ^^^^^^^^^^^^

error: expected a type, as in `try_from = i64`
  --> tests/ui/bad_values.rs:16:24
   |
16 |     #[shelf(try_from = "not a type")]
   |                        ^^^^^^^^^^^^
```

The first two errors come from syn itself, at the token where parsing failed: after `rename`, syn wanted an `=` and found the end of the list (so it points at the closing parenthesis), and `LitStr` wanted a string where there was a number. The other four are shelf's.

> **Macro lens:** a proc macro on stable Rust cannot merge two spans into one (`Span::join` is unstable), so how does syn underline `rename = "e2"` from its first token to its last? It keeps both ends. `to_compile_error` writes `::core::compile_error!` with the first span on every token of the path, and the braces around the message with the last span, and the compiler underlines the whole invocation, which now runs from one to the other (syn 3.0.9, `error.rs`).

A key whose value is itself a list, `key(…)`, is parsed by calling `meta.parse_nested_meta` inside the closure: it walks the inner list with the same rules. `FromRow` has no such key; the relations of Chapter 11 will.

One namespace, every key. All of shelf's derives share `#[shelf(…)]`, so each of them parses every key: `FromRow` and Chapter 6's `Table` sit on the same struct, and must read the same column names. Reporting is the problem. A derive cannot see which other derives the struct has (§3.1), so if each reported every mistake, a struct with two derives would hear about each one twice. shelf gives every key an owner, the derive that reports its errors:

```
// orm/shelf-codegen/src/attrs.rs (excerpt)
impl Derive {
    /// Whether this derive reports an error found while parsing `key`. `None` is a key no
    /// derive knows, or a mistake in the attribute's syntax: `FromRow` reports those,
because a
```

```
    /// `Table` always comes with it.
    pub fn reports(self, key: Option<&str>) -> bool {
        match (self, key) {
            (Derive::Insert | Derive::Patch, _) => true,
            (_, None) => self == Derive::FromRow,
            (_, Some(key)) => KEYS
                .iter()
                .any(|(name, owner)| *name == key && *owner == self),
        }
    }
}

/// Every key, and the derive that owns it: the one that reports its errors. A derive may
read
/// a key it does not own (`Table` reads `rename` to name its columns).
const KEYS: &[(&str, Derive)] = &[
    ("rename_all", Derive::FromRow),
    ("table", Derive::Table),
    // …
    ("primary_key", Derive::Table),
    ("generated", Derive::Table),
    // …
];
```

orm/shelf-codegen/src/attrs.rs

That is why the closure records which key it is parsing, and forgets it once the key has parsed: an error then belongs to that key's owner, and an error with no key (an unknown key, or a syntax mistake between keys) to `FromRow`. Chapter 8's `Insert` and Chapter 9's `Patch` are each the only shelf derive on their struct, so they report every mistake themselves. An unknown key is still refused, so a typo cannot hide behind another derive: `tabel = "products"` is reported, with the keys that do exist (`tests/ui/unknown_attributes.rs`). The combinations of §4.4 are `FromRow`'s, so only it checks them.

4.3 The six attributes

What each attribute changes is easiest to see in the generated code. This program runs shelf's generator on a struct that uses all six:

```
use syn::{DeriveInput, parse_quote};

fn main() {
    let input: DeriveInput = parse_quote! {
        #[shelf(rename_all = "camelCase")]
        struct Listing {
            #[shelf(rename = "sku_code")]
            sku: String,
            unit_price: i64,
            #[shelf(flatten)]
            stock: Stock,
            #[shelf(json)]
            attributes: Attributes,
            #[shelf(try_from = i32)]
            weight: Grams,
            #[shelf(skip)]
            selected: bool,
        }
    };
    let expanded = shelf_codegen::from_row::expand(&input).expect("valid attributes");
    let code = shelf_playground::pretty(expanded);
    println!("{code}");

    assert!(code.contains(r#""sku", "sku_code")"#)); // rename
    assert!(code.contains(r#""unit_price", "unitPrice")"#)); // rename_all
    assert!(code.contains("<Stock as ::shelf::FromRow>::resolve(columns)?")); // flatten
```

```
    assert!(code.contains("::shelf::__private::Json<Attributes>")); // json
    assert!(code.contains("get_try_from")); // try_from
    assert!(code.contains("selected: ::core::default::Default::default()")); // skip
}
```

```
#[automatically_derived]
impl ::shelf::FromRow for Listing {
    type Columns = (
        usize,
        usize,
        <Stock as ::shelf::FromRow>::Columns,
        usize,
        usize,
        (),
    );
    fn resolve(
        columns: &[::shelf::__private::tokio_postgres::Column],
    ) -> ::core::result::Result<Self::Columns, ::shelf::Error> {
        ::core::result::Result::Ok((
            ::shelf::__private::column::<String>(columns, "Listing", "sku", "sku_code")?,
            ::shelf::__private::column::<
                i64,
            >(columns, "Listing", "unit_price", "unitPrice")?,
            <Stock as ::shelf::FromRow>::resolve(columns)?,
            ::shelf::__private::column::<
                ::shelf::__private::Json<Attributes>,
            >(columns, "Listing", "attributes", "attributes")?,
            ::shelf::__private::column::<i32>(columns, "Listing", "weight", "weight")?,
            (),
        ))
    }
    fn from_row_at(
        row: &::shelf::__private::tokio_postgres::Row,
        columns: &Self::Columns,
    ) -> ::core::result::Result<Self, ::shelf::Error> {
        ::core::result::Result::Ok(Self {
            sku: ::shelf::__private::get::<String>(row, columns.0, "Listing", "sku")?,
            unit_price: ::shelf::__private::get::<
                i64,
            >(row, columns.1, "Listing", "unit_price")?,
            stock: <Stock as ::shelf::FromRow>::from_row_at(row, &columns.2)?,
            attributes: ::shelf::__private::get::<
                ::shelf::__private::Json<Attributes>,
            >(row, columns.3, "Listing", "attributes")?
                .0,
            weight: ::shelf::__private::get_try_from::<
                i32,
                Grams,
            >(row, columns.4, "Listing", "weight")?,
            selected: ::core::default::Default::default(),
        })
    }
}
```

Against the plain field of Chapter 3, which `resolve` finds with `column::<T>` and `from_row_at` reads with `get::<T>` (Chapter 5 explains the pair), each attribute changes one thing:

Attribute	`Columns` holds	`resolve` calls	`from_row_at` reads with
none	`usize`	`column::<T>`	`get::<T>`
`rename`, `rename_all`	`usize`	`column::<T>`, with another name	`get::<T>`
`skip`	`()`	nothing	`Default::default()`
`flatten`	`<T as FromRow>::Columns`	`T::resolve`	`T::from_row_at`
`json`	`usize`	`column::<Json<T>>`	`get::<Json<T>>`, then `.0`

Attribute	`Columns` holds	`resolve` calls	`from_row_at` reads with
`try_from = S`	`usize`	`column::<S>`	`get_try_from::<S, T>`

In the generator, this is the `match` that Chapter 3's excerpt elided. `attrs.read()` says how a field is read, once its attributes are known to be consistent (§4.4):

```
// orm/shelf-codegen/src/from_row.rs (excerpt)
// The type `resolve` checks the column against, and the expression that reads it.
let (checked, read) = match attrs.read() {
    Read::Skip => {
        bounds.add_if_generic(&[ty], parse_quote!(#ty: ::core::default::Default));
        column_types.push(quote!(()));
        resolves.push(quote!(()));
        reads.push(quote!(#ident: ::core::default::Default::default()));
        continue;
    }
    Read::Flatten => {
        bounds.add_if_generic(&[ty], parse_quote!(#ty: ::shelf::FromRow));
        column_types.push(quote!(<#ty as ::shelf::FromRow>::Columns));
        resolves.push(quote!(<#ty as ::shelf::FromRow>::resolve(columns)?));
        reads.push(quote! {
            #ident: <#ty as ::shelf::FromRow>::from_row_at(row, &columns.#at)?
        });
        continue;
    }
    // … Read::Column, as in Chapter 3
    Read::Json => {
        let json = quote!(::shelf::__private::Json<#ty>);
        bounds.add_if_generic(&[ty], parse_quote!(#json: ::shelf::FromColumn));
        (
            json.clone(),
            quote!(::shelf::__private::get::<#json>(row, columns.#at, #model, #name)?.0),
        )
    }
    Read::TryFrom(source) => {
        // … the bounds a generic struct needs (§4.6)
        (
            quote!(#source),
            quote! {
                ::shelf::__private::get_try_from::<#source, #ty>(
                    row, columns.#at, #model, #name,
                )?
            },
        )
    }
};
let column = match &attrs.rename {
    Some(rename) => rename.value.value(),
    None => container.column_for(&name),
};
```

orm/shelf-codegen/src/from_row.rs

`rename` and `rename_all`

`rename` replaces the column name; `rename_all` on the struct sets a convention for every field without one. Both work on the field's name after `unraw` (§3.5), and `rename` wins. shelf knows four conventions, written by hand in `case.rs` rather than taken from the `heck` crate, because four is all a PostgreSQL schema needs. `snake_case` is the default, because PostgreSQL folds unquoted names to lower case, so most schemas are written that way. The other three turn up in tables created with quoted names, often by tools in other languages:

```
// orm/shelf-codegen/src/case.rs (excerpt)
pub fn apply(self, field: &str) -> String {
    match self {
        Case::Snake => field.to_owned(),
```

```
        Case::ScreamingSnake => field.to_ascii_uppercase(),
        Case::Camel | Case::Pascal => {
            let mut out = String::with_capacity(field.len());
            let mut upper = self == Case::Pascal;
            for c in field.chars() {
                if c == '_' {
                    upper = !out.is_empty() || self == Case::Pascal;
                } else if upper {
                    out.extend(c.to_uppercase());
                    upper = false;
                } else {
                    out.push(c);
                }
            }
            out
        }
    }
}
```

orm/shelf-codegen/src/case.rs

The conversion runs inside the derive, so the generated code holds the finished name as a literal (`"unitPrice"` above) and pays nothing for it at run time. Names are then matched exactly, as PostgreSQL reports them: in SQL a camelCase column must be quoted, `"unitPrice"`, or the server folds it to `unitprice`.

`skip`

A skipped field is not read: it gets `::core::default::Default::default()`, with the absolute path that kept §3.5's hostile module, with its own trait called `Default`, from capturing it. Its element of `Columns` is `()` rather than nothing, so that every other field's element keeps the index of its field: `columns.4` is still the fifth field's.

`flatten`

A flattened field is a struct with its own `FromRow`, read from the same row. Its `Columns` is nested whole in the outer tuple, its `resolve` searches the same column list, and its `from_row_at` gets the same row with its own part of `columns`. The inner struct's derive does its own naming: the outer struct's `rename_all` does not reach into it. Chapter 1's limitation remains (§1.4): a struct flattened twice reads the same columns twice, and the way out is still to alias the columns in SQL.

`json`

`json` reads the column through `Json<T>`, whose `FromSql` accepts `json` and `jsonb` and deserializes with serde, so `T` must implement `Deserialize`; the generated code unwraps it with `.0`. postgres-types has a wrapper of that name (main book §3.7), and shelf's is a thin layer over it, for one reason: NULL.

postgres-types' `Json<T>` refuses an SQL `NULL` whatever `T` is. In postgres-types 0.2.14 it keeps `FromSql`'s default `from_sql_null`, which returns a `WasNull` error; only `Option<T>` overrides it. So `#[shelf(json)]` on an `Option<T>` field, read through `Json<Option<T>>`, would fail on a NULL although the field can hold one: the `Option` is inside the wrapper. Writing this chapter found that, and shelf now reads an SQL `NULL` as JSON `null`, and lets `T` decide:

```
// orm/shelf/src/json.rs (excerpt)
#[doc(hidden)]
pub struct Json<T>(pub T);

impl<'a, T: Deserialize<'a>> FromSql<'a> for Json<T> {
    fn from_sql(ty: &Type, raw: &'a [u8]) -> Result<Self, Box<dyn Error + Sync + Send>> {
        tokio_postgres::types::Json::<T>::from_sql(ty, raw).map(|json| Json(json.0))
    }

    fn from_sql_null(_: &Type) -> Result<Self, Box<dyn Error + Sync + Send>> {
        // As for a JSON `null`; a type that refuses it makes this a NULL error.
```

```
        T::deserialize(serde_json::Value::Null)
            .map(Json)
            .map_err(|_| Box::new(WasNull) as _)
    }
    // … `accepts` is postgres-types' own: `json` and `jsonb`
}
```

orm/shelf/src/json.rs

`serde_json::Value::Null` is itself a serde deserializer, so `T::deserialize` reads it as it would read `null` in a document. This program reads a document, a JSON `null` and an SQL `NULL`, into a field that can hold null and into one that cannot:

```
use serde::Deserialize;
use shelf::{Error, FromRow, Mapping};
use tokio_postgres::types::Json;

#[derive(Debug, PartialEq, Deserialize)]
struct Attributes {
    colour: String,
}

#[derive(Debug, FromRow)]
struct Listing {
    #[shelf(json)]
    attributes: Option<Attributes>,
}

#[derive(Debug, FromRow)]
struct Required {
    #[shelf(json)]
    attributes: Attributes,
}

#[tokio::main]
async fn main() -> Result<(), Error> {
    let client = shelf_playground::connect().await?;
    let document = r#"SELECT '{"colour": "blue"}'::jsonb AS attributes"#;
    let json_null = "SELECT 'null'::jsonb AS attributes";
    let sql_null = "SELECT NULL::jsonb AS attributes";

    // Into an Option: a document is Some, and either kind of null is None.
    let blue = Some(Attributes {
        colour: "blue".into(),
    });
    for (sql, expected) in [(document, blue), (json_null, None), (sql_null, None)] {
        let listing: Listing = shelf::fetch_one(&client, sql, &[]).await?;
        assert_eq!(listing.attributes, expected);
    }

    // Into a struct, which refuses JSON null: an SQL NULL is the error any NULL gives.
    let required: Required = shelf::fetch_one(&client, document, &[]).await?;
    assert_eq!(required.attributes.colour, "blue");
    let error = shelf::fetch_one::<Required>(&client, sql_null, &[])
        .await
        .unwrap_err();
    println!("{error}");
    assert!(matches!(
        error,
        Error::Mapping {
            problem: Mapping::Null { .. },
            ..
        }
    ));

    // To tell the two nulls apart, leave the attribute off.
    for (sql, expected) in [(json_null, Some(None)), (sql_null, None)] {
```

```
        let row = client.query_one(sql, &[]).await?;
        let value: Option<Json<Option<Attributes>>> = row.try_get(0)?;
        assert_eq!(value.map(|json| json.0), expected);
    }
    Ok(())
}
```

```
Required.attributes: column "attributes" is NULL; read it into an Option
```

An `Option<T>` field reads both kinds of null as `None`, as any `Option` field reads a NULL. A type that refuses `null` fails with the error a NULL gives any other field, and its advice fits. `serde_json::Value` accepts `null`, so it reads an SQL `NULL` as `Value::Null`. When the difference between the two nulls matters, leave the attribute off and read `Option<tokio_postgres::types::Json<Option<T>>>`, as the program's last lines do. sqlx keeps the two apart by default and has a second key for the nullable case, `#[sqlx(json(nullable))]`.

`try_from`

`try_from = S` reads the column as `S`, then converts with `TryFrom<S>`. The type to read is written as a type, `try_from = i32`, or quoted as serde writes it. A conversion that fails, a negative weight for a `Grams`, fails its row with a `Mapping::Decode` error that names the struct, the field and the column, and carries the conversion's own error as its source. For that, the conversion's error type must implement `std::error::Error + Send + Sync + 'static`, and the generated code says so in its bounds.

The side by side at the end of the chapter reads all six attributes from a table, by hand and with shelf, and shows that error next to the one hand-written code gives.

4.4 Every error at once

A derive that stops at its first complaint makes you rebuild once per mistake. A `syn::Error` can hold several messages instead: `combine` appends one error to another, `into_compile_error` turns each into its own `compile_error!`, and iterating an error yields them one by one. shelf collects them in a small wrapper:

```
// orm/shelf-codegen/src/attrs.rs (excerpt)
/// Errors gathered while parsing, so that one build reports all of them, not only the
first.
#[derive(Default)]
pub struct Errors(Option<Error>);

impl Errors {
    pub fn push(&mut self, error: Error) {
        match &mut self.0 {
            Some(all) => all.combine(error),
            None => self.0 = Some(error),
        }
    }

    pub fn finish(self) -> Result<()> {
        self.0.map_or(Ok(()), Err)
    }
}
```

orm/shelf-codegen/src/attrs.rs

and the generator parses everything before it gives up. This is the beginning of `from_row::expand`, which Chapter 3 elided:

```
// orm/shelf-codegen/src/from_row.rs (excerpt)
pub fn expand(input: &DeriveInput) -> Result<TokenStream> {
    let fields = named_fields(input)?;
```

```
    let mut errors = Errors::default();
    for lifetime in input.generics.lifetimes() {
        errors.push(Error::new_spanned(
            lifetime,
            "FromRow reads owned values, so the struct cannot borrow from the row: use
`String` \
             instead of `&str`, and `Vec<u8>` instead of `&[u8]`",
        ));
    }
    let container = ContainerAttrs::parse(&input.attrs, Derive::FromRow, &mut errors);
    let fields: Vec<(&Field, FieldAttrs)> = fields
        .iter()
        .map(|field| {
            (
                field,
                FieldAttrs::parse(&field.attrs, Derive::FromRow, &mut errors),
            )
        })
        .collect();
    errors.finish()?;
    // …
}
```

orm/shelf-codegen/src/from_row.rs

Only a struct of the wrong shape stops it at once (`named_fields(input)?`), because then there are no fields to look at. Everything else is pushed: the lifetimes (§4.6), the struct's attributes, and every field's. This program gives the generator a struct with five mistakes:

```
use syn::{DeriveInput, parse_quote};

fn main() {
    let input: DeriveInput = parse_quote! {
        #[shelf(rename_all = "kebab-case")]
        struct Product {
            #[shelf(renamed = "sku_code", skip = 1)]
            sku: String,
            #[shelf(skip, rename = "price_cents")]
            price: i64,
            #[shelf(json, try_from = i32)]
            weight: Grams,
        }
    };
    let error = shelf_codegen::from_row::expand(&input).expect_err("mistakes to report");
    let messages: Vec<String> = error.into_iter().map(|e| e.to_string()).collect();
    for message in &messages {
        println!("{message}");
    }
    assert_eq!(messages.len(), 4);
    assert!(messages[1].starts_with("unknown shelf attribute `renamed`"));
}
```

```
unknown case: expected "snake_case", "camelCase", "PascalCase" or "SCREAMING_SNAKE_CASE"
unknown shelf attribute `renamed` on a field; expected `rename`, `skip`, `flatten`, `json`,
`try_from`, `primary_key`, `generated`, `version`, `updated_at`, `conflict`
`rename` cannot apply to a field marked `skip`, which is not read
`try_from` and `json` cannot be combined: read the JSON into a type that converts itself
```

Five mistakes, four errors. `skip = 1` is not reported, because it follows `renamed` in the same attribute: `parse_nested_meta` returns as soon as the closure returns an error, and the rest of that attribute is never parsed (syn 3.0.9, `meta.rs`). So the unit of collection is the attribute: each `#[shelf(…)]` reports at most one parse error, and every attribute of every field is parsed. Fix the first error in an attribute, and the next one shows up on the next build.

The last two errors are of another kind. Each key was valid alone, and the combination is not. Those checks run after a field's attributes are parsed, and point at the key that does not fit:

```
// orm/shelf-codegen/src/attrs.rs (excerpt)
/// Refuses keys that cannot go together, pointing at the one that does not fit.
fn check_combinations(&self, errors: &mut Errors) {
    let others = [
        ("rename", self.rename.as_ref().map(|a| a.span)),
        ("flatten", self.flatten.as_ref().map(|a| a.span)),
        ("json", self.json.as_ref().map(|a| a.span)),
        ("try_from", self.try_from.as_ref().map(|a| a.span)),
    ];
    if self.skip.is_some() {
        for (name, span) in others {
            if let Some(span) = span {
                errors.push(Error::new(
                    span,
                    format!(
                        "`{name}` cannot apply to a field marked `skip`, which is not read"
                    ),
                ));
            }
        }
    } else if self.flatten.is_some() {
        // … the same, for every key but `flatten`
    } else if let (Some(_), Some(try_from)) = (&self.json, &self.try_from) {
        errors.push(Error::new(
            try_from.span,
            "`try_from` and `json` cannot be combined: read the JSON into a type that \
             converts itself",
        ));
    }
}
```

orm/shelf-codegen/src/attrs.rs

Each refusal has a reason the message gives. A skipped field is not read, so a `rename`, `json` or `try_from` on it would be dead. A flattened struct reads its own columns, so a name or a wrapper on the outer field would mean nothing. And `json` with `try_from` has two plausible meanings (convert the JSON value, or convert after decoding), so shelf refuses it and points at the answer serde already has: a type that converts itself while it is deserialized. The UI test for these three, with the span of each error on the key that does not fit:

```
error: `rename` cannot apply to a field marked `skip`, which is not read
  --> tests/ui/conflicts.rs:11:19
   |
11 |     #[shelf(skip, rename = "a")]
   |                   ^^^^^^

error: `json` cannot apply to a `flatten` field, whose struct reads its own columns
  --> tests/ui/conflicts.rs:13:22
   |
13 |     #[shelf(flatten, json)]
   |                      ^^^^

error: `try_from` and `json` cannot be combined: read the JSON into a type that converts
itself
  --> tests/ui/conflicts.rs:15:19
   |
15 |     #[shelf(json, try_from = i64)]
   |                   ^^^^^^^^
```

So an error lands at one of three places (PLAN.md §12 #12): at a key, when the key is the problem (`Keyed::span`, or `meta.error` before the value is read); at a value, when the value is (`Error::new_spanned(&lit, …)`, as for an empty name or an unknown case); or at a field's type, when the compiler finds that the generated code does not type-check. That last kind is the subject of the next section.

4.5 UI tests

An error message is part of a derive's interface, so shelf tests it like one. trybuild compiles each file in `tests/ui/`, expects it to fail, and compares the compiler's output with the `.stderr` file beside it. One test function covers the whole directory:

```
// orm/shelf/tests/ui.rs (excerpt)
#[test]
fn derive_errors() {
    trybuild::TestCases::new().compile_fail("tests/ui/*.rs");
}
```

orm/shelf/tests/ui.rs

Each file is a small program that should not compile, with its mistakes on separate lines so that each error has its own line number, and the `.stderr` beside it holds the compiler's output: `conflicts.rs` holds the three fields of §4.4, and `conflicts.stderr` is what §4.4 printed. There is a pair for each kind of mistake: the shapes of §3.7, unknown keys, the bad values of §4.2, the conflicts of §4.4, an unreadable type and a lifetime (below).

trybuild builds the files in a project of its own, which depends on shelf, and normalizes what the compiler prints before it compares, so that the files do not depend on the machine (trybuild 1.0.122, `normalize.rs`). Paths become relative to the crate (`tests/ui/…`). Line numbers are kept for the test file and erased for every other file, so that editing shelf's sources does not break every test. And a list that rustc cuts short with "and 20 others" says "and $N others".

The first run of a new test has no `.stderr` to compare with. trybuild then fails the test and writes what the compiler said into `wip/`, for you to read and move into place. Run with `TRYBUILD=overwrite` and it writes the file in place, and also replaces any `.stderr` that no longer matches:

```
TRYBUILD=overwrite cargo test -p shelf --test ui
git diff orm/shelf/tests/ui/
```

The expected files are rustc's words, so a new toolchain may change them without any change to shelf; shelf's are rustc 1.99.1's. Regenerate them deliberately, and read the diff as you would read a change to an API.

What reading them changed

Reading those files for the first time changed shelf. For a field whose type tokio-postgres cannot read, such as `std::time::Instant`, the error named the bound the generated code had used: `shelf::__private::tokio_postgres::types::FromSqlOwned`, a path that no user should ever have to see. shelf now bounds field types by a trait of its own, implemented for every type the driver can read, and carrying a message:

```
// orm/shelf/src/row.rs (excerpt)
#[diagnostic::on_unimplemented(
    message = "`{Self}` cannot be read from a PostgreSQL column",
    label = "tokio-postgres has no `FromSql` for this type",
    note = "use a type tokio-postgres reads, add `#[shelf(json)]` or `#[shelf(try_from =
…)]`, \
            or implement `FromSql` for it"
)]
pub trait FromColumn: FromSqlOwned {}

impl<T: FromSqlOwned> FromColumn for T {}
```

orm/shelf/src/row.rs

`#[diagnostic::on_unimplemented]` was stabilized in Rust 1.78.0 (the release notes: "Stabilize the `#[diagnostic]` namespace and `#[diagnostic::on_unimplemented]` attribute"). When a type does not implement the trait, the compiler uses `message` as the error's headline, `label` under the offending code and `note` as a note, and then adds

its own explanation. It works through the blanket impl: `Instant` fails `FromColumn` because it fails `FromSqlOwned`, and the headline is still shelf's (PLAN.md §12 #11). For this struct

```
// A field type that tokio-postgres cannot read: the compiler's error names the field's
type.
use shelf::FromRow;

#[derive(FromRow)]
struct Timing {
    started: std::time::Instant,
}

fn main() {}
```

the compiler now says:

```
error[E0277]: `Instant` cannot be read from a PostgreSQL column
 --> tests/ui/not_readable.rs:6:14
  |
6 |     started: std::time::Instant,
  |              ^^^^^^^^^^^^^^^^^^ tokio-postgres has no `FromSql` for this type
  |
  = help: the trait `for<'a> shelf::__private::tokio_postgres::types::FromSql<'a>` is not
implemented for `Instant`
  = note: use a type tokio-postgres reads, add `#[shelf(json)]` or `#[shelf(try_from = …)]`,
or implement `FromSql` for it
  = help: the following other types implement trait
`shelf::__private::tokio_postgres::types::FromSql<'a>`:
            &'a [u8]
            &'a str
            Box<T>
            Box<[T]>
            Box<str>
            Cow<'a, str>
            HashMap<String, Option<String>, S>
            IpAddr
          and $N others
  = note: required for `Instant` to implement
`shelf::__private::tokio_postgres::types::FromSqlOwned`
  = note: required for `Instant` to implement `FromColumn`
note: required by a bound in `shelf::__private::column`
 --> src/row.rs
  |
  | pub fn column<T: FromColumn>(
  |                  ^^^^^^^^^^ required by this bound in `column`
```

The error points at the field's type in your struct, not at the derive, because quote keeps the spans of the tokens it copies from the input (§3.7): `column::<#ty>` carries the span of the `std::time::Instant` you wrote.

That is the first of two errors (`tests/ui/not_readable.stderr` has both). The second is the same, except for its last note, `required by a bound in shelf::__private::get`: the type is used twice in the generated code, by `column` in `resolve` and by `get` in `from_row_at`, and each use is checked. shelf accepts the repetition, because the one way to report it once loses both the message and the position (§4.6).

4.6 Generics

A generic struct needs a generic impl: `impl<T, M> FromRow for Page<T, M>`, with whatever bounds make its body compile. syn's `Generics::split_for_impl` gives the three pieces such an impl needs, `<T, M>` after `impl`, `<T, M>` after the type's name and the `where` clause, and the generator adds its own predicates to the `where` clause first. Which predicates? It depends on how each field is read, so shelf adds one per field, and only for a field whose type mentions a type parameter:

```
use syn::{DeriveInput, parse_quote};

fn main() {
    let input: DeriveInput = parse_quote! {
        struct Page<T, M> {
            total: i64,
            item: T,
            #[shelf(flatten)]
            meta: M,
            #[shelf(json)]
            extra: Vec<T>,
        }
    };
    let expanded = shelf_codegen::from_row::expand(&input).expect("valid attributes");
    let code = shelf_playground::pretty(expanded);
    // The impl's header: everything before its first item.
    let header: Vec<&str> = code
        .lines()
        .take_while(|line| !line.contains("type Columns"))
        .collect();
    println!("{}", header.join("\n"));

    assert!(code.contains("T: ::shelf::FromColumn,"));
    assert!(code.contains("M: ::shelf::FromRow,"));
    assert!(code.contains("::shelf::__private::Json<Vec<T>>: ::shelf::FromColumn,"));
    assert!(!code.contains("i64: ")); // no bound for a concrete type
}
```

```
#[automatically_derived]
impl<T, M> ::shelf::FromRow for Page<T, M>
where
    T: ::shelf::FromColumn,
    M: ::shelf::FromRow,
    ::shelf::__private::Json<Vec<T>>: ::shelf::FromColumn,
{
```

`item: T` is read from a column, so `T: FromColumn`. `meta: M` is flattened, so `M: FromRow`. `extra: Vec<T>` is read through `Json`, so its bound is on `Json<Vec<T>>`, not on `T`: for that field alone, `T` would only have to be deserializable. And `total: i64` gets no bound.

The standard library's derives work differently: `#[derive(Clone)]` on `Page<T, M>` requires `T: Clone` and `M: Clone`, one bound per parameter, whatever the fields are. For `FromRow` that rule would be wrong, because the right bound depends on the attribute: a parameter bounded by `FromColumn` could not be flattened. This program uses a generic wrapper that adds a total to any row type, a derived struct and a tuple included:

```
use shelf::FromRow;

/// A row of a page, with the number of rows the whole query has.
#[derive(Debug, FromRow)]
struct Counted<T> {
    total: i64,
    #[shelf(flatten)]
    item: T,
}

#[derive(Debug, PartialEq, FromRow)]
struct Product {
    sku: String,
    price_cents: i64,
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    client
```

```
        .batch_execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY, price_cents bigint NOT NULL);
             INSERT INTO products VALUES ('MUG-01', 1299), ('TEE-02', 2500), ('CAP-03',
1800);",
        )
        .await?;

    // count(*) OVER () is computed before LIMIT: every row carries the full count.
    let page: Vec<Counted<Product>> = shelf::fetch_all(
        &client,
        "SELECT sku, price_cents, count(*) OVER () AS total FROM products ORDER BY sku LIMIT
2",
        &[],
    )
    .await?;
    assert_eq!(page.len(), 2);
    assert_eq!(page[0].total, 3);
    assert_eq!(
        page[0].item,
        Product {
            sku: "CAP-03".into(),
            price_cents: 1800
        }
    );

    // A tuple is a FromRow too; it reads by position, the first column here.
    let skus: Vec<Counted<(String,)>> = shelf::fetch_all(
        &client,
        "SELECT sku, count(*) OVER () AS total FROM products ORDER BY sku",
        &[],
    )
    .await?;
    assert_eq!(skus[2].item.0, "TEE-02");
    Ok(())
}
```

`Counted<Product>` needs `Product: FromRow`, which holds; `T: FromColumn`, the `Clone`-style bound, would have refused it.

The code that decides is short. `mentions_param` walks a type's tokens looking for the name of a type parameter, inside groups too, so `Vec<T>` and `Option<(T, i64)>` count:

```
// orm/shelf-codegen/src/generics.rs (excerpt)
/// Adds `predicate` if any of `types` mentions a type parameter. A bound on a concrete
/// type would be checked where the derive is, far from the field, so it is left out: the
/// generated code reports a concrete type that does not fit at the field itself.
pub fn add_if_generic(&mut self, types: &[&Type], predicate: WherePredicate) {
    if types.iter().any(|ty| self.mentions_param(ty)) {
        self.predicates.push(predicate);
    }
}

// … `apply` adds the predicates to a clone of the struct's `where` clause
fn mentions(tokens: TokenStream, params: &[Ident]) -> bool {
    tokens.into_iter().any(|tree| match tree {
        TokenTree::Ident(ident) => params.contains(&ident),
        TokenTree::Group(group) => mentions(group.stream(), params),
        _ => false,
    })
}
```

orm/shelf-codegen/src/generics.rs

The check works on tokens, not on meaning, so it can be fooled: a concrete type that happens to share a parameter's name, say a path ending in `::T`, would get a bound too. That costs nothing when the bound holds, and darling's `usage` module (§4.7) is the precise alternative.

Why no bounds on concrete types

Why not bound every field's type, `std::time::Instant: FromColumn` included, so that §4.5's error appears once instead of twice? This probe derive, written for this chapter, does that for a struct's first field, in an empty impl of a marker trait that is there only to carry the `where` clause:

```
#[proc_macro_derive(BoundFirst)]
pub fn bound_first(input: TokenStream) -> TokenStream {
    // … `ty`, the first field's type, and `name`, the struct's
    quote! {
        impl ::core::marker::Unpin for #name where #ty: ::shelf::FromColumn {}
    }
    .into()
}
```

On §4.5's `Timing`, rustc 1.99.1 reports one error, as hoped, and loses both things §4.5 gained:

```
error[E0277]: the trait bound `Instant:
shelf::__private::tokio_postgres::types::FromSqlOwned` is not satisfied
 --> user/src/bin/bound_first.rs:2:10
  |
2 | #[derive(show::BoundFirst)]
  |          ^^^^^^^^^^^^^^^^ the trait `for<'a>
shelf::__private::tokio_postgres::types::FromSql<'a>` is not implemented for `Instant`
  |
  = help: the following other types implement trait
`shelf::__private::tokio_postgres::types::FromSql<'a>`:
            &'a [u8]
            &'a str
            Box<T>
            Box<[T]>
            Box<str>
            Cow<'a, str>
            HashMap<String, Option<String>, S>
            IpAddr
          and 20 others
  = note: required for `Instant` to implement
`shelf::__private::tokio_postgres::types::FromSqlOwned`
  = note: this error originates in the derive macro `show::BoundFirst` (in Nightly builds,
run with -Z macro-backtrace for more info)
```

The error points at the derive, not at the field, although `#ty` carries the field's span. And it is in the driver's words: to accept the `where` clause, the compiler first needs `Instant` to meet `FromColumn`'s supertrait, `FromSqlOwned`, and it reports that failure, without `FromColumn`'s message. Two errors at the field that say what to do are better than one at the derive that does not, so shelf bounds only what is generic, and leaves concrete types to be checked where the generated code uses them.

Lifetimes are refused

A struct that borrows from the row, `struct Product<'a> { sku: &'a str }`, is refused, with an error at the lifetime that says what to use instead:

```
error: FromRow reads owned values, so the struct cannot borrow from the row: use `String`
instead of `&str`, and `Vec<u8>` instead of `&[u8]`
 --> tests/ui/lifetime.rs:5:16
  |
5 | struct Product<'a> {
  |                ^^
```

Borrowing would save copying each string out of the row's buffer (main book §3.4). But `FromRow` returns `Self` with no lifetime tied to the row, and making it `FromRow<'r>` would spread that lifetime to every function that maps rows: `fetch_all` could no longer return the values, because the rows they borrow from would be dropped as it returns (Exercise 3). shelf keeps the trait simple and maps owned values. The generator does not try to tell a harmless lifetime from a borrow: every lifetime parameter is refused, each with its own error.

4.7 What darling would save

darling is the crate most derives use for this chapter's work. Version 0.24.1 is built on syn 3. You declare the attributes as a struct and derive a parser for it, much as serde derives a deserializer. This is shelf's parser in darling, written for this chapter and compiled against darling 0.24.1 in a probe:

```
// darling 0.24.1 (illustration): shelf's attributes as darling receivers
use darling::util::{Flag, SpannedValue};
use darling::{FromDeriveInput, FromField, FromMeta, ast};

#[derive(FromDeriveInput)]
#[darling(attributes(shelf), supports(struct_named))]
struct Input {
    data: ast::Data<(), FieldAttrs>,
    rename_all: Option<Case>,
}

#[derive(FromMeta)]
enum Case {
    #[darling(rename = "snake_case")]
    Snake,
    #[darling(rename = "camelCase")]
    Camel,
    // … PascalCase and SCREAMING_SNAKE_CASE the same way
}

#[derive(FromField)]
#[darling(attributes(shelf), and_then = FieldAttrs::check)]
struct FieldAttrs {
    rename: Option<SpannedValue<String>>,
    skip: Flag,
    flatten: Flag,
    json: Flag,
    try_from: Option<SpannedValue<syn::Type>>,
}

impl FieldAttrs {
    /// The combinations shelf refuses: darling cannot know them.
    fn check(self) -> darling::Result<Self> {
        let mut errors = darling::Error::accumulator();
        if self.json.is_present()
            && let Some(try_from) = &self.try_from
        {
            errors.push(
                darling::Error::custom("`try_from` and `json` cannot be combined")
                    .with_span(&try_from.span()),
            );
        }
        // … the other combinations, the same way
        errors.finish_with(self)
    }
}

// In the derive: Input::from_derive_input(&input), and on failure, errors.write_errors().
```

On a struct with the mistakes of shelf's UI tests, plus one attribute with two, this is what the probe's build reports:

```
error: Unknown value: `kebab-case`. Available values: `PascalCase`, `SCREAMING_SNAKE_CASE`,
`camelCase`, `snake_case`
 --> userd/src/bin/darling.rs:4:22
  |
4 | #[shelf(rename_all = "kebab-case")]
  |                      ^^^^^^^^^^^^

error: Unknown field: `renamed`. Did you mean `rename`?
 --> userd/src/bin/darling.rs:6:13
```

```
  |
6 |     #[shelf(renamed = "sku")]
  |             ^^^^^^^

error: `rename` cannot apply to a field marked `skip`
 --> userd/src/bin/darling.rs:8:28
  |
8 |     #[shelf(skip, rename = "a")]
  |                           ^^^

error: `try_from` and `json` cannot be combined
  --> userd/src/bin/darling.rs:10:30
   |
10 |     #[shelf(json, try_from = i64)]
   |                              ^^^

error: Duplicate field `rename`
  --> userd/src/bin/darling.rs:12:28
   |
12 |     #[shelf(rename = "e1", rename = "e2")]
   |                            ^^^^^^

error: Unexpected type `bool`
  --> userd/src/bin/darling.rs:14:20
   |
14 |     #[shelf(skip = true)]
   |                    ^^^^

error: Unexpected type `int`
  --> userd/src/bin/darling.rs:16:22
   |
16 |     #[shelf(rename = 1)]
   |                      ^

error: Unknown field: `renamed`. Did you mean `rename`?
  --> userd/src/bin/darling.rs:24:13
   |
24 |     #[shelf(renamed = "x", skip = 1, json)]
   |             ^^^^^^^

error: Unexpected type `int`
  --> userd/src/bin/darling.rs:24:35
   |
24 |     #[shelf(renamed = "x", skip = 1, json)]
   |                                   ^
```

What darling does for you:

- **Unknown keys, with a suggestion.** "Did you mean `rename`?" comes from comparing the unknown name with the known ones (the default `suggestions` feature).
- **Duplicates, wrong kinds of value, and enum values**, each with its message and the list of what would have been accepted.
- **Every error in an attribute.** darling goes on after an error inside one attribute, so the attribute on line 24 gets two errors (`renamed` and `skip = 1`) where shelf's parser would report one.
- **Spans.** `SpannedValue<T>` and `Flag` keep where each value came from, as shelf's `Keyed` does; the span is the value's rather than the key's.
- **`darling::usage`**, which finds where a struct's type parameters are used, the precise version of §4.6's token check.

What it does not do: shelf's combinations are shelf's rules, so they are written by hand in `and_then` either way. The wording of its own messages is darling's ("Unexpected type `bool`" where shelf says "`skip` takes no value"). And it is five more crates to compile before your derive: darling, darling_core, darling_macro, ident_case and, for the suggestions, strsim.

shelf parses by hand because this book is about the mechanism: `attrs.rs` is about 250 lines, and every one of them is something darling would otherwise be doing for you. In a derive of your own with more than a few keys, darling is the better choice. Either way, once you have written the parser yourself, darling no longer looks like magic.

Side by side — *attribute-driven mapping*

The task: read a legacy table whose names and types do not match the struct. Its columns were created with quoted camelCase names, except `sku_code`, added later; two of them belong in a nested struct; a `jsonb` column holds a document; an `int` must become a `Grams` that refuses negative values; and one field is not in the table at all. sqlx's derive takes nearly the same attributes:

```
// sqlx (illustration): the same struct with sqlx 0.9's #[sqlx(…)] attributes
#[derive(sqlx::FromRow)]
#[sqlx(rename_all = "camelCase")]
struct Listing {
    #[sqlx(rename = "sku_code")]
    sku: String,
    unit_price: i64,
    #[sqlx(flatten)]
    stock: Stock,
    #[sqlx(json)]
    attributes: Attributes,
    #[sqlx(try_from = "i32")]
    weight: Grams,
    #[sqlx(skip)]
    selected: bool,
}
```

By hand and with shelf, on the same table, in one program:

```
use std::error::Error;

use serde::Deserialize;
use shelf::FromRow;
use tokio_postgres::Row;
use tokio_postgres::types::Json;

/// A document in a `jsonb` column.
#[derive(Debug, PartialEq, Deserialize)]
struct Attributes {
    colour: String,
    ml: Option<u32>,
}

/// A weight that is never negative, although the column would allow it.
#[derive(Debug, PartialEq)]
struct Grams(u32);

impl TryFrom<i32> for Grams {
    type Error = std::num::TryFromIntError;

    fn try_from(value: i32) -> Result<Self, Self::Error> {
        u32::try_from(value).map(Grams)
    }
}

/// Two columns of the same row, in a struct of their own.
#[derive(Debug, PartialEq, FromRow)]
#[shelf(rename_all = "camelCase")]
struct Stock {
    on_hand: i32,
    reserved: i32,
}

// By hand: every name, wrapper and conversion, written out in the body.
```

```
struct HandListing {
    sku: String,
    unit_price: i64,
    stock: Stock,
    attributes: Attributes,
    weight: Grams,
    selected: bool,
}

impl HandListing {
    fn from_row(row: &Row) -> Result<Self, Box<dyn Error + Send + Sync>> {
        Ok(HandListing {
            sku: row.try_get("sku_code")?,
            unit_price: row.try_get("unitPrice")?,
            stock: Stock {
                on_hand: row.try_get("onHand")?,
                reserved: row.try_get("reserved")?,
            },
            attributes: row.try_get::<_, Json<Attributes>>("attributes")?.0,
            weight: Grams::try_from(row.try_get::<_, i32>("weight")?)?,
            selected: false,
        })
    }
}

// shelf: the same mapping, as attributes.
#[derive(Debug, FromRow)]
#[shelf(rename_all = "camelCase")]
struct Listing {
    #[shelf(rename = "sku_code")]
    sku: String,
    unit_price: i64,
    #[shelf(flatten)]
    stock: Stock,
    #[shelf(json)]
    attributes: Attributes,
    #[shelf(try_from = i32)]
    weight: Grams,
    #[shelf(skip)]
    selected: bool,
}

const SELECT: &str = r#"SELECT sku_code, "unitPrice", "onHand", reserved, attributes, weight
                        FROM listings ORDER BY sku_code"#;

#[tokio::main]
async fn main() -> Result<(), Box<dyn Error + Send + Sync>> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            r#"CREATE TEMP TABLE listings (
                   sku_code    text PRIMARY KEY,
                   "unitPrice" bigint NOT NULL,
                   "onHand"    int NOT NULL,
                   reserved    int NOT NULL,
                   attributes  jsonb NOT NULL,
                   weight      int NOT NULL
               );
               INSERT INTO listings VALUES
                   ('MUG-01', 1299, 40, 3, '{"colour": "blue", "ml": 350}', 410),
                   ('TEE-02', 2500, 12, 0, '{"colour": "black"}', 180);"#,
        )
        .await?;

    let rows = client.query(SELECT, &[]).await?;
    let by_hand = rows
        .iter()
        .map(HandListing::from_row)
        .collect::<Result<Vec<_>, _>>()?;
```

```
    let with_shelf: Vec<Listing> = shelf::fetch_all(&client, SELECT, &[]).await?;
    for (hand, shelf) in by_hand.iter().zip(&with_shelf) {
        assert_eq!(hand.sku, shelf.sku);
        assert_eq!(hand.unit_price, shelf.unit_price);
        assert_eq!(hand.stock, shelf.stock);
        assert_eq!(hand.attributes, shelf.attributes);
        assert_eq!(hand.weight, shelf.weight);
        assert_eq!(hand.selected, shelf.selected);
    }
    assert_eq!(with_shelf[0].stock.on_hand, 40);
    assert_eq!(with_shelf[1].attributes.ml, None);

    // A weight the conversion refuses: the same failure, reported both ways.
    client
        .execute(
            "UPDATE listings SET weight = -5 WHERE sku_code = 'TEE-02'",
            &[],
        )
        .await?;
    let rows = client.query(SELECT, &[]).await?;
    let Err(hand_error) = rows
        .iter()
        .map(HandListing::from_row)
        .collect::<Result<Vec<_>, _>>()
    else {
        panic!("a negative weight is refused")
    };
    let Err(shelf_error) = shelf::fetch_all::<Listing>(&client, SELECT, &[]).await else {
        panic!("a negative weight is refused")
    };
    println!("by hand: {hand_error}");
    println!("shelf:   {shelf_error}");
    assert!(matches!(
        shelf_error,
        shelf::Error::Mapping {
            field: "weight",
            ..
        }
    ));
    Ok(())
}
```

```
by hand: number too small to fit in target type
shelf:   Listing.weight: column "weight": number too small to fit in target type
```

The message after shelf's colon is the conversion's own, from the standard library's `TryFromIntError`, captured rather than remembered: shelf's tests once expected a different wording, which an older Rust had used (PLAN.md §12 #10).

	by hand	shelf	sqlx
Same	The column's name, the nested struct, the JSON wrapper and the conversion, each stated once per field	The same, as attributes on the fields	The same, as attributes with the same names (`rename_all`, `rename`, `flatten`, `json`, `try_from`, `skip`)
Different	Any logic at all, in the function body; an error is whatever the failing call returned, without the field or the column	Unknown keys, duplicates and contradictions are compile errors at the token, all in one build; a failed conversion names the struct, the field and the column; `try_from` takes a type or a quoted type; `json` reads an SQL `NULL` as JSON `null`	`try_from` takes only a quoted type; `json(nullable)` and a `default` key exist; it has no branch for an unknown field key, and `unwraps` the result of parsing field attributes, so a mistake there stops the derive with

by hand	shelf	sqlx
		a panic (§3.7; sqlx-macros-core 0.9.0's source)

In shelf

Parsing and generating meet at one enum. Once a field's attributes are known to be consistent, `read` reduces them to the single way the field is read, and the generator matches on that (§4.3) instead of on the combination of keys:

```
// orm/shelf-codegen/src/attrs.rs (excerpt)
/// How a field is read, once its attributes are known to be consistent.
pub enum Read<'a> {
    /// From one column, by its own `FromSql`.
    Column,
    /// From one column, through `Json<T>`.
    Json,
    /// From one column as the given type, then `TryFrom`.
    TryFrom(&'a Type),
    /// As a nested `FromRow`, from the same row.
    Flatten,
    /// Not at all: `Default::default()`.
    Skip,
}
```

orm/shelf-codegen/src/attrs.rs

`FieldAttrs::read` returns the first that applies, in the order `skip`, `flatten`, `json`, `try_from`, and `Column` otherwise; after `check_combinations`, at most one of them can. `rename` is not part of `Read`, because it changes the column's name, not how its value is read.

Adding an attribute to shelf touches these places, which is also a map of the chapter:

1. **`attrs.rs`**: the key in `FieldAttrs` (or `ContainerAttrs`), a branch in `parse_key`, its owner in `KEYS` and its place in `FIELD_KEYS` (or `CONTAINER_KEYS`), and any combination it cannot join in `check_combinations`.
2. **`from_row.rs`**: what the field generates, usually a new `Read` arm, with the bound `add_if_generic` should add for a generic field (`generics.rs`).
3. **An expansion test**, `orm/shelf-codegen/tests/expand/NAME.rs`, which pins down the code it generates (Chapter 13).
4. **An integration test** in `orm/shelf/tests/from_row.rs`, against PostgreSQL.
5. **A UI test** for each error it can report, in `orm/shelf/tests/ui/`.
6. **The attribute table** in the documentation of `shelf::FromRow` (orm/shelf/src/lib.rs).

When not to use an attribute

- **When the SQL can do it.** `SELECT sku_code AS sku` needs no `rename`, and keeps the mapping where the query is.
- **When only one query needs it.** An attribute applies to every query that reads the struct: a `rename` for one report's alias binds the struct to that name everywhere. Alias in that query instead.
- **When the conversion is business logic.** `try_from` is for conversions that only check and wrap, such as a non-negative amount or a validated code. If the conversion needs other fields, or a lookup, read the raw values and build the type in a constructor.

Summary

- A derive declares its helper attributes, `attributes(shelf)`. Without that, or without the `#[derive]` on the struct, the compiler refuses `#[shelf(…)]` before any macro runs, with a note that `shelf` is a crate.
- `Attribute::parse_nested_meta` walks the conventional `key`, `key = value`, `key(…)` syntax; the closure reads each key's value with `meta.value()?` and a parse of the right type (`LitStr`, `Type`), or nothing for a flag.
- shelf's parser keeps each value with the span of its key (`Keyed`), refuses duplicates, flags with values and unknown keys (listing the known ones), and accepts a type quoted or not.
- `rename_all`, `rename`, `skip`, `flatten`, `json` and `try_from` each change one element of `Columns`, one call in `resolve` and one read in `from_row_at`. `#[shelf(json)]` reads an SQL `NULL` as JSON `null`, so an `Option<T>` field takes both as `None`.
- Errors are collected with `syn::Error::combine` and reported together. `parse_nested_meta` stops at the first error in an attribute, so each attribute reports at most one parse error; combinations that cannot go together are refused at the key that does not fit.
- trybuild's UI tests pin down the compiler's output, normalized; regenerate them with `TRYBUILD=overwrite` and read the diff. Reading them gave shelf `FromColumn`, whose `#[diagnostic::on_unimplemented]` (Rust 1.78+) puts shelf's words on a type error.
- A generic struct gets one bound per field whose type mentions a type parameter, chosen by how the field is read; concrete types get none, so their errors keep shelf's message. Lifetime parameters are refused.
- darling would parse the same attributes from a declared struct, with better errors for free; shelf parses by hand to show the mechanism.

Exercises

1. **A default for a missing column.** Design `#[shelf(default)]`: read the column if the result has it, and use `Default::default()` if it does not (sqlx has the same key). Write, by hand, the `impl FromRow` the derive should generate for a struct with one such field, and check it against a query with the column and one without. *Hint:* `resolve` cannot return an index for a column that is not there; its element of `Columns` can be an `Option<usize>`.
2. **Two fields, one column.** Write a check, as a stage function on a `DeriveInput`, that refuses two fields that read the same column, and reports the second at its field. Then decide whether shelf should have it. *Hint:* collect each column name in a `HashMap` with the field that claimed it first, and combine the errors.
3. **Borrowing from the row.** Suppose `FromRow` became `FromRow<'r>`, with `fn from_row(row: &'r Row) -> Result<Self, Error>`, so that a field could be a `&'r str`. What goes wrong in `fetch_all`? *Hint:* who owns the rows, and when are they dropped?

Solutions

1. The field's element of `Columns` becomes `Option<usize>`: `resolve` records whether the column is there, and `from_row_at` reads it or falls back. This is the impl the derive should generate, written by hand without the type checks that `column` would add:

```
use shelf::{FromRow, Mapping};
use tokio_postgres::{Column, Row};

/// A product whose `note` column only some queries select: `#[shelf(default)]` on `note`.
#[derive(Debug, PartialEq)]
struct Product {
    sku: String,
    note: String,
}

impl FromRow for Product {
    /// `None` when the result has no `note` column.
```

```
    type Columns = (usize, Option<usize>);

    fn resolve(columns: &[Column]) -> Result<Self::Columns, shelf::Error> {
        let find = |name: &str| columns.iter().position(|c| c.name() == name);
        let sku = find("sku").ok_or_else(|| shelf::Error::Mapping {
            model: "Product",
            field: "sku",
            problem: Mapping::MissingColumn {
                column: "sku".into(),
            },
        })?;
        Ok((sku, find("note")))
    }

    fn from_row_at(row: &Row, columns: &Self::Columns) -> Result<Self, shelf::Error> {
        Ok(Product {
            sku: row.try_get(columns.0)?,
            note: match columns.1 {
                Some(at) => row.try_get(at)?,
                None => Default::default(),
            },
        })
    }
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;

    let with_note: Product =
        shelf::fetch_one(&client, "SELECT 'MUG-01' AS sku, 'fragile' AS note", &[]).await?;
    assert_eq!(with_note.note, "fragile");

    let without: Product = shelf::fetch_one(&client, "SELECT 'MUG-01' AS sku", &[]).await?;
    assert_eq!(without.note, "");
    Ok(())
}
```

In the derive, `default` is a flag in `FieldAttrs`, orthogonal to `Read`, like `rename`: a defaulted field can still be read through `json` or `try_from`, so it should not become a `Read` arm of its own. It cannot go with `skip` or `flatten`. The generator then wraps the field's `resolve` call and its read, and `__private` needs a version of `column` that returns `Ok(None)` for a missing column but still checks the type of one that is present.

2. The check needs each field's column name, so it reads `rename` (only `rename`, for brevity: a full version would apply `rename_all` too):

```
use std::collections::HashMap;

use syn::{Data, DeriveInput, Error, Field, Fields, Ident, LitStr, parse_quote};

/// The column a field reads: its `rename`, or its own name.
fn column_of(field: &Field) -> syn::Result<String> {
    let mut column = field.ident.as_ref().expect("a named field").to_string();
    for attr in field.attrs.iter().filter(|a| a.path().is_ident("shelf")) {
        attr.parse_nested_meta(|meta| {
            if meta.path.is_ident("rename") {
                column = meta.value()?.parse::<LitStr>()?.value();
            }
            Ok(())
        })?;
    }
    Ok(column)
}

/// Refuses a second field that reads a column another field already reads.
fn check_unique(input: &DeriveInput) -> syn::Result<()> {
    let Data::Struct(data) = &input.data else {
```

```
        return Ok(());
    };
    let Fields::Named(fields) = &data.fields else {
        return Ok(());
    };
    let mut first_reader: HashMap<String, &Ident> = HashMap::new();
    let mut errors: Option<Error> = None;
    for field in &fields.named {
        let ident = field.ident.as_ref().expect("a named field");
        let column = column_of(field)?;
        if let Some(first) = first_reader.get(&column) {
            let error = Error::new_spanned(
                ident,
                format!("`{first}` already reads the column {column:?}"),
            );
            match &mut errors {
                Some(all) => all.combine(error),
                None => errors = Some(error),
            }
        } else {
            first_reader.insert(column, ident);
        }
    }
    errors.map_or(Ok(()), Err)
}

fn main() {
    let input: DeriveInput = parse_quote! {
        struct Product {
            sku: String,
            #[shelf(rename = "sku")]
            code: String,
            price_cents: i64,
        }
    };
    let error = check_unique(&input).expect_err("two fields read sku");
    println!("{error}");
    assert_eq!(error.to_string(), "`sku` already reads the column \"sku\"");

    let fine: DeriveInput = parse_quote! {
        struct Product { sku: String, #[shelf(rename = "sku_code")] code: String }
    };
    assert!(check_unique(&fine).is_ok());
}
```

shelf does not have this check, on purpose: reading one column twice is odd, but not wrong. A struct may want a `jsonb` column both as a typed value and as a raw `serde_json::Value`, and `resolve` finds the same index for both without complaint.

3. `fetch_all` runs the query, which gives it a `Vec<Row>` that it owns, and maps every row. Each value would borrow from its row, so the values could not outlive the vector, and the vector is dropped when `fetch_all` returns: the borrow checker refuses to return them. The caller would have to own the rows instead, calling the query and then the mapping, two steps every time, with the rows kept alive beside the values. Every helper built on `FromRow` (`fetch_one`, and later the model's `find` and `all`) would inherit the same lifetime and the same two steps. The main book shows where borrowing does pay: a loop over rows that only looks at each value briefly (main book §3.4). shelf maps into owned values, and leaves that loop to hand-written code.

CHAPTER 5

Checking a statement once, not every row

Every expansion since Chapter 0 has had the same two methods. `resolve` takes a list of columns and returns a tuple of positions; `from_row_at` takes a row and that tuple, and builds the struct. Chapter 2's tuple impls have them too, with `type Columns = ()`. This chapter explains that shape. Chapter 1 measured what it replaces: a lookup by name for every field of every row, about three times the cost of reading by position, repeated although its answer cannot change, because every row of a statement has the same columns in the same order.

So shelf splits the work in two. What depends only on the statement is done once, when the statement is prepared: find each field's column, and check that its type can be read into the field. What depends on the row is done per row: decode the values. The first half has a second payoff, which matters more than the speed. A struct that does not fit its query is reported before the query runs, with the struct's name, the field's, the column's and both types, even when the query would return no rows, and even when it is an `INSERT` that would have written one.

In this chapter you will learn:

- why every row of a statement shares one column list, and what that list holds before any row exists
- how `resolve` finds each field's column by its exact name and checks its type, once per statement
- which mistakes are found before the query runs, and which can only be found in a row
- what the derive generates for this, and how `from_row` keeps a single row simple
- what it is worth, measured on 100,000 rows, and what `resolve` itself costs per call
- how sqlx finds its columns, for comparison

5.1 The answer belongs to the statement

Chapter 1's numbers, briefly: on 100,000 rows of twelve columns, a struct read by name took about three times as long as the same struct read by position in a release build, and 1.6 times as long in a debug build (§1.2). The decoding was the same in both. The difference was the searching: twelve walks along the column list per row, 78 name comparisons, each walk finding the position it found on the row before.

It has to find the same position, because a row has no column names of its own. In tokio-postgres 0.7.18 a `Row` holds the `Statement` it came from, and `Row::columns()` returns that statement's list. This program checks it with pointer equality: not two equal lists, but the same list.

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = shelf_playground::connect().await?;

    let statement = client
        .prepare(
            "SELECT 'sku-' || g AS sku, g * 100 AS price_cents FROM generate_series(1, 3) AS
```

```
g",
        )
        .await?;
    let rows = client.query(&statement, &[]).await?;
    // Not three equal lists: the statement's one list, three times.
    for row in &rows {
        assert!(std::ptr::eq(row.columns(), statement.columns()));
    }

    let names: Vec<&str> = statement.columns().iter().map(|c| c.name()).collect();
    println!("{} rows, one column list: {names:?}", rows.len());

    // SQL text is prepared too (main book §4.1), and its rows share that statement's list.
    let series = client
        .query("SELECT g FROM generate_series(1, 5) AS g", &[])
        .await?;
    assert!(
        series
            .iter()
            .all(|row| std::ptr::eq(row.columns(), series[0].columns()))
    );
    Ok(())
}
```

```
3 rows, one column list: ["sku", "price_cents"]
```

So the work of reading a row splits cleanly:

Work	Depends on	How often shelf does it
Find each field's column	the statement	once per statement
Check that the column's type can be read into the field	the statement	once per statement
Decode each value, and notice a NULL	the row	once per row

Reading by position moved the first line of that table out of the loop in Chapter 1, at the price of positions written into the code, which break silently when someone changes the `SELECT`. Finding them once per statement keeps the names and drops the cost.

5.2 A statement knows its columns

`Client::prepare` sends the SQL to the server, which parses it and describes it, and returns a `Statement` (main book §4.2). Its `columns()` are known at that point, before anything has run, so a query that will return no rows still has them:

```
use tokio_postgres::types::Type;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (
                 id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
                 sku text NOT NULL,
                 price_cents bigint NOT NULL
             );",
        )
        .await?;

    let statement = client
        .prepare(
```

```
            "SELECT id, sku, price_cents, price_cents * 2 AS doubled FROM products WHERE
false",
        )
        .await?;
    let rows = client.query(&statement, &[]).await?;

    println!(
        "{} rows, {} columns:",
        rows.len(),
        statement.columns().len()
    );
    for column in statement.columns() {
        let name = column.name();
        let sql_type = column.type_().to_string();
        let source = match column.column_id() {
            Some(n) => format!("column {n} of a table"),
            None => "an expression".to_string(),
        };
        println!("  {name:<12} {sql_type:<5} {source}");
    }
    assert!(rows.is_empty());
    assert_eq!(statement.columns()[3].name(), "doubled");
    assert_eq!(statement.columns()[3].type_(), &Type::INT8);
    Ok(())
}
```

```
0 rows, 4 columns:
  id           int8  column 1 of a table
  sku          text  column 2 of a table
  price_cents  int8  column 3 of a table
  doubled      int8  an expression
```

> **Protocol lens:** the columns come from the *RowDescription* message, the server's answer to the *Describe* that `prepare` sends (main book §4.1). For each column it carries a name; the OID of the table and the column's number in it, when the column comes straight from a table, and zero otherwise; the type's OID, size and modifier; and a format code. tokio-postgres keeps the name, the table OID and column number (`None` for zero), the type modifier, and the type, turning its OID into a `Type`. For a type it does not know, such as an enum or a domain, it asks the catalog, once per connection. What a row description does not carry is whether a column can be NULL: Chapter 13 goes to the catalog for that.

Finding a field's column in that list, and checking its type, is everything `resolve` does. Chapter 1's `FromRow` had one method, `from_row`. To do the statement's work once, the trait needs somewhere to keep what that work found, and a way to read a row with it: an associated type, `Columns`, and two methods. This stage program writes the two-step trait by hand, a `Product` that implements it, and a `fetch_all` that prepares the statement, resolves once, and then reads every row:

```
use tokio_postgres::types::{FromSqlOwned, ToSql};
use tokio_postgres::{Client, Column, Row};

type Error = Box<dyn std::error::Error + Send + Sync>;

/// Chapter 1's trait, in two steps.
trait FromRow: Sized {
    /// Where this type's values are in a result.
    type Columns;
    /// Once per statement: find each field's column, and check its type.
    fn resolve(columns: &[Column]) -> Result<Self::Columns, Error>;
    /// Once per row: read the values at the positions `resolve` found.
    fn from_row_at(row: &Row, columns: &Self::Columns) -> Result<Self, Error>;
```

```
}

/// The position of the column `name`, checked to be readable as `T`.
fn column<T: FromSqlOwned>(columns: &[Column], name: &str) -> Result<usize, Error> {
    let index = columns
        .iter()
        .position(|c| c.name() == name) // the exact name, nothing else
        .ok_or_else(|| format!("the result has no column {name:?}"))?;
    let sql_type = columns[index].type_();
    if !T::accepts(sql_type) {
        let rust_type = std::any::type_name::<T>();
        return Err(format!(
            "column {name:?} has type {sql_type}, which cannot be read into {rust_type}"
        )
        .into());
    }
    Ok(index)
}

#[derive(Debug)]
struct Product {
    sku: String,
    price_cents: i64,
}

impl FromRow for Product {
    type Columns = (usize, usize);

    fn resolve(columns: &[Column]) -> Result<Self::Columns, Error> {
        Ok((
            column::<String>(columns, "sku")?,
            column::<i64>(columns, "price_cents")?,
        ))
    }

    fn from_row_at(row: &Row, columns: &Self::Columns) -> Result<Self, Error> {
        Ok(Product {
            sku: row.try_get(columns.0)?,
            price_cents: row.try_get(columns.1)?,
        })
    }
}

/// Every row of `sql`, read into `T`: prepare, resolve once, then read each row.
async fn fetch_all<T: FromRow>(
    client: &Client,
    sql: &str,
    params: &[&(dyn ToSql + Sync)],
) -> Result<Vec<T>, Error> {
    let statement = client.prepare(sql).await?;
    let columns = T::resolve(statement.columns())?; // before the query runs
    let rows = client.query(&statement, params).await?;
    rows.iter()
        .map(|row| T::from_row_at(row, &columns))
        .collect()
}

#[tokio::main]
async fn main() -> Result<(), Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY, price_cents bigint NOT NULL);
             INSERT INTO products VALUES ('MUG-01', 1299), ('TEE-02', 2499);",
        )
        .await?;

    // The columns in another order than the fields: the positions take care of it.
    let products: Vec<Product> = fetch_all(
```

```
        &client,
        "SELECT price_cents, sku FROM products ORDER BY sku",
        &[],
    )
    .await?;
    assert_eq!(products[0].sku, "MUG-01");
    assert_eq!(products[1].price_cents, 2499);

    // Two queries that do not fit: both are refused before they run, with no row to look
at.
    let mistakes = [
        (
            "SELECT sku FROM products WHERE false",
            r#"the result has no column "price_cents""#,
        ),
        (
            "SELECT sku, price_cents::int AS price_cents FROM products WHERE false",
            r#"column "price_cents" has type int4, which cannot be read into i64"#,
        ),
    ];
    for (sql, expected) in mistakes {
        let error = fetch_all::<Product>(&client, sql, &[]).await.unwrap_err();
        println!("{error}");
        assert_eq!(error.to_string(), expected);
    }
    Ok(())
}
```

```
the result has no column "price_cents"
column "price_cents" has type int4, which cannot be read into i64
```

`column` compares names with ==, so it finds a column by its exact name only. That is a deliberate difference from `Row::try_get(&str)`, whose second walk ignores ASCII case (§1.2): by the time PostgreSQL describes a statement it has already folded unquoted names to lower case, so the name in the description is the name, and a column that matches only when case is ignored is a different column.

> **Rust refresher:** an *associated type* is a type each impl of a trait chooses, as each `Iterator` chooses its `Item`. Here `Product` chooses `(usize, usize)`, and a tuple impl can choose `()`. Generic code names it through its type parameter, `T::Columns`, without knowing what it is: `fetch_all` keeps the `T::Columns` that `resolve` returns and hands it back to `from_row_at`, and never looks inside.

5.3 Types checked once

The second half of `column` is one call: `T::accepts(column.type_())`. Every `FromSql` type has `accepts`, which says whether it can decode a given PostgreSQL type. In postgres-types 0.2.14, `i64` accepts `int8` and nothing else; `String` accepts `text`, `varchar`, `bpchar`, `name` and `unknown`, and the extension types `citext`, `ltree`, `lquery` and `ltxtquery`; `Option<T>` accepts what `T` accepts. The answer depends only on the column's type, so like the position it belongs to the statement. tokio-postgres's `try_get` asks it on every row anyway (§1.2); shelf asks once, in `resolve`, where a mismatch can still be reported with the names of the struct and the field. This is shelf's version of the stage program's `column`:

```
// orm/shelf/src/row.rs (excerpt)
/// The position of the column `name` in `columns`, checked to be readable as `T`.
///
/// Generated code calls this from `resolve`; it is public for that reason only.
#[doc(hidden)]
pub fn column<T: FromColumn>(
```

```
    columns: &[Column],
    model: &'static str,
    field: &'static str,
    name: &str,
) -> Result<usize, Error> {
    let mapping = |problem| Error::Mapping {
        model,
        field,
        problem,
    };
    // Exact names only. `Row::try_get(&str)` would also try again ignoring ASCII case, but
    // PostgreSQL itself folds unquoted names to lower case, so the exact name is the rule.
    let Some(index) = columns.iter().position(|c| c.name() == name) else {
        return Err(mapping(Mapping::MissingColumn {
            column: name.to_owned(),
        }));
    };
    check::<T>(&columns[index], index).map_err(mapping)
}

// …

fn check<T: FromColumn>(column: &Column, index: usize) -> Result<usize, Mapping> {
    if T::accepts(column.type_()) {
        Ok(index)
    } else {
        Err(Mapping::WrongType {
            column: column.name().to_owned(),
            sql_type: column.type_().clone(),
            rust_type: short_type_name(std::any::type_name::<T>()),
        })
    }
}
```

orm/shelf/src/row.rs

Two things differ from the stage. The error is a value a program can match on, `Error::Mapping { model, field, problem }`, with `Mapping::MissingColumn` or `Mapping::WrongType` as the problem. And the Rust type is shortened by `short_type_name`, so that a field of type `Option<String>` is reported as `Option<String>`, not as `type_name`'s `core::option::Option<alloc::string::String>`. (`check` is shared with `column_at`, which the tuple impls call with a fixed position.)

Because the check needs only the statement, it happens before the query runs, and a query that does not fit its struct never runs at all. This program shows it twice, through the real derive: a `SELECT` that would return no rows, and an `INSERT … RETURNING` whose result lacks a column the struct reads:

```
use shelf::{FromRow, Mapping};

#[derive(Debug, FromRow)]
#[allow(dead_code)] // the program only shows the errors
struct Product {
    sku: String,
    price_cents: i64,
    note: Option<String>,
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (
                 sku text PRIMARY KEY,
                 price_cents bigint NOT NULL,
                 note integer -- a mistake: notes are text
             );
```

```
                INSERT INTO products VALUES ('MUG-01', 1299, NULL);",
            )
            .await?;

        // No row, and still an error: the type is the statement's, not a row's.
        let error = shelf::fetch_all::<Product>(
            &client,
            "SELECT sku, price_cents, note FROM products WHERE false",
            &[],
        )
        .await
        .unwrap_err();
        println!("{error}");
        assert!(matches!(
            error,
            shelf::Error::Mapping {
                field: "note",
                problem: Mapping::WrongType { .. },
                ..
            }
        ));

        // An INSERT whose RETURNING has no `note`: prepared, checked, and never run.
        let error = shelf::fetch_one::<Product>(
            &client,
            "INSERT INTO products (sku, price_cents) VALUES ('TEE-02', 2499) RETURNING sku,
price_cents",
            &[],
        )
        .await
        .unwrap_err();
        println!("{error}");
        let count: i64 = client
            .query_one("SELECT count(*) FROM products", &[])
            .await?
            .get(0);
        assert_eq!(count, 1, "the INSERT never ran");
        Ok(())
    }
```

```
Product.note: column "note" has type int4, which cannot be read into Option<String>
Product.note: the result has no column "note"
```

> **SQL lens:** preparing a statement parses and analyses it, which is how the server knows the columns of an `INSERT … RETURNING` that has not run. Nothing is executed until a *Bind* and an *Execute* follow, so an `INSERT` that is only prepared writes nothing. shelf's check sits between the two exchanges, which is why the count above is still 1.

Two problems are left for the rows, because a statement cannot reveal them:

- **A NULL in a field that is not an `Option`** (`Mapping::Null`). A column's type says nothing about whether it holds NULL, so this is found only when one arrives (§1.3 showed the message). Chapter 13's `verify` narrows it down, from the catalog, for a table's own columns.
- **A value that cannot be decoded or converted** (`Mapping::Decode`): a `try_from` that refuses it, or a `jsonb` document that does not fit the field's type.

Both still name the struct, the field and the column, because the per-row helper that reads a value is given the same names as `column` (In shelf, below). And tokio-postgres's own `accepts` check still runs inside `try_get(index)` on every row: Chapter 1 found that skipping it, through a raw-bytes wrapper, saves nothing

measurable (orm/PLAN.md §12, finding 8), and it turns a misuse of `from_row_at` into an error instead of garbage (Exercise 3).

5.4 What the derive generates

The derive writes the stage program's impl. You have seen its output since §0.7, and each piece has a counterpart in the stage:

The stage program (§5.2)	`#[derive(FromRow)]`
`type Columns = (usize, usize);`	`type Columns = (usize, usize);`, one element per field
`column::<String>(columns, "sku")?`	`::shelf::__private::column::<String>(columns, "Product", "sku", "sku")?`
`row.try_get(columns.0)?`	`::shelf::__private::get::<String>(row, columns.0, "Product", "sku")?`

The extra arguments are the error's context, written as literals at compile time: the struct, the field and the column, which differ when the field is renamed (Chapter 4).

`Columns` is a tuple, not an array, because its elements are not all `usize`: a skipped field contributes `()`, and a flattened one the nested struct's own `Columns`, whole (Chapter 4, §4.3). A resolved `Columns` is therefore a small tree of positions, and for a concrete struct you can print it. This program resolves a `Product` with a flattened `Price` and a skipped field against a statement whose columns come in another order:

```
use shelf::FromRow;

#[derive(Debug, PartialEq, FromRow)]
struct Price {
    #[shelf(rename = "price_cents")]
    cents: i64,
    currency: String,
}

#[derive(Debug, PartialEq, FromRow)]
struct Product {
    sku: String,
    #[shelf(flatten)]
    price: Price,
    #[shelf(skip)]
    selected: bool,
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    let statement = client
        .prepare("SELECT 'EUR' AS currency, 1299::int8 AS price_cents, 'MUG-01' AS sku")
        .await?;

    let columns = Product::resolve(statement.columns())?;
    println!("{columns:?}");
    assert_eq!(columns, (2, (1, 0), ()));

    let row = client.query_one(&statement, &[]).await?;
    let product = Product::from_row_at(&row, &columns)?;
    assert_eq!(
        product,
        Product {
            sku: "MUG-01".into(),
            price: Price {
                cents: 1299,
                currency: "EUR".into()
            },
            selected: false,
        }
```

```
    );
    Ok(())
}
```

```
(2, (1, 0), ())
```

`sku` is at position 2. `Price`'s own `resolve`, searching the same list, found `price_cents` at 1 and `currency` at 0. `selected` had nothing to find. `from_row_at` then walks the same tree: `columns.0` for `sku`, and `&columns.1` handed to `Price::from_row_at` with the same row.

This is the trait in shelf:

```
// orm/shelf/src/row.rs (excerpt)
pub trait FromRow: Sized {
    /// Where this type's values are in a result, as found by `resolve`. For a derived
struct,
    /// a tuple with one element per field.
    type Columns: Send + Sync + 'static;

    /// Finds and checks this type's columns in `columns`: a statement's, or a row's.
    fn resolve(columns: &[Column]) -> Result<Self::Columns, Error>;

    /// Reads `row`, whose columns `columns` was resolved from.
    fn from_row_at(row: &Row, columns: &Self::Columns) -> Result<Self, Error>;

    /// Reads a single row: `resolve`, then `from_row_at`. For many rows of one statement,
    /// resolve once instead ([`from_rows`] does).
    fn from_row(row: &Row) -> Result<Self, Error> {
        Self::from_row_at(row, &Self::resolve(row.columns())?)
    }
}
```

orm/shelf/src/row.rs

Three things in it go beyond the stage:

- **`from_row` stays**, as a provided method. It resolves against the row's own columns, which are its statement's (§5.1), and reads the row. For one row, that is the same work as the two steps, in one call. In a loop over many rows it repeats `resolve` for each, which §5.5 measures.
- **`Columns` is `Send`.** shelf's generic code holds a `T::Columns` across an `.await`: `fetch_all` resolves, then awaits the query, then reads the rows. Its future, and the `+ Send` futures of the model methods built on it (Chapter 6's `find` and `all`, for one), can be `Send` for an unknown `T` only if the trait promises it; removing `Send` from the bound stops shelf compiling. `Sync` and `'static` are not needed by any code in shelf today. They cost nothing, since a tuple of positions has both, and they promise generic code that a resolved `Columns` can be shared between tasks and kept as long as its statement, which is what a cache of them would need (§5.6).
- **Tuples choose `()`**, as Chapter 2 showed: their positions are fixed, so their `resolve` only checks each column's type, with `column_at`.

For rows you already have, `from_rows` resolves against the first row and reads them all:

```
// orm/shelf/src/row.rs (excerpt)
/// Reads every row of one statement into `T`, resolving the columns once.
pub fn from_rows<T: FromRow>(rows: &[Row]) -> Result<Vec<T>, Error> {
    let Some(first) = rows.first() else {
        return Ok(Vec::new());
    };
    let columns = T::resolve(first.columns())?;
    rows.iter()
        .map(|row| T::from_row_at(row, &columns))
        .collect()
}
```

orm/shelf/src/row.rs

> **Pitfall:** `from_rows` gets rows, not a statement, so on an empty result it has no columns to check, and returns an empty `Vec` whatever the struct. A test against an empty table passes, and the first row in production fails. When you have the SQL, call `fetch_all`, which checks the struct against the statement whether or not any row comes back.

5.5 Measured

The shelf crate has a benchmark for this, `bench_mapping`. It reads 100,000 rows with the twelve column types of Chapter 1's program, again in the reverse of the struct's order, four ways, and times only the mapping: by name and by index written by hand, and through shelf, resolved once (`from_rows`) and resolved again for every row (`from_row` in a loop). It runs in release mode and prints the median of five rounds as a table:

```
cargo run -p shelf --release --example bench_mapping
```

One run printed:

Mapping, 100000 rows × 12 columns	Median of 5	×
by name, by hand (`try_get("c01")`)	40.6 ms	3.0
by index, by hand (`try_get(i)`)	13.5 ms	1.0
shelf, resolved once (`from_rows`)	14.1 ms	1.1
shelf, resolved per row (`from_row`)	41.3 ms	3.1

Three runs on Apple Silicon (rustc 1.99.1, PostgreSQL 18.6 on localhost) gave 40.6–41.7 ms by name, 13.2–13.5 ms by index, 14.1–14.4 ms resolved once and 41.3–42.7 ms resolved per row (orm/PLAN.md §12, findings 13 and 58). Resolved once, shelf costs 4–8 per cent more than the hand-written index code, which in exchange you would have to keep in step with every `SELECT`. Resolved for every row, it costs what lookups by name cost, because that is what it does: `resolve` walks the column list once per field. That is why `fetch_all` and `from_rows` resolve once, and why `from_row` is for a single row.

The book's examples run as debug builds, where the gap is smaller but still clear. This program measures the two shelf ways on the same rows, and then the cost of one `resolve` on its own, against the cheapest round trip there is:

```
use std::hint::black_box;
use std::time::{Duration, Instant};

use shelf::FromRow;

#[derive(FromRow)]
#[allow(dead_code)] // only the mapping is measured
struct Wide {
    c01: i64,
    c02: String,
    c03: i32,
    c04: Option<String>,
    c05: i64,
    c06: bool,
    c07: String,
    c08: i64,
    c09: Option<i32>,
    c10: String,
    c11: i64,
    c12: f64,
```

```
}

const SQL: &str = "
    SELECT i::float8 AS c12, i AS c11, 'name ' || i AS c10,
           CASE WHEN i % 3 = 0 THEN NULL ELSE i::int END AS c09,
           i AS c08, 'tag ' || i AS c07, i % 2 = 0 AS c06, i AS c05,
           CASE WHEN i % 5 = 0 THEN NULL ELSE 'note' END AS c04,
           i::int AS c03, 'sku-' || i AS c02, i AS c01
    FROM generate_series(1, 100000::bigint) AS i";

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    let rows = client.query(SQL, &[]).await?;

    // The same rows, read twice: resolving once, and resolving for every row.
    let mut once = Duration::MAX;
    let mut per_row = Duration::MAX;
    for _ in 0..3 {
        let start = Instant::now();
        let a: Vec<Wide> = shelf::from_rows(&rows)?;
        once = once.min(start.elapsed());

        let start = Instant::now();
        let b = rows
            .iter()
            .map(Wide::from_row)
            .collect::<Result<Vec<_>, _>>()?;
        per_row = per_row.min(start.elapsed());
        assert_eq!(a.len(), b.len());
    }
    println!(
        "{} rows: resolved once {once:.1?}, per row {per_row:.1?} ({:.1}×)",
        rows.len(),
        per_row.as_secs_f64() / once.as_secs_f64()
    );
    assert!(once < per_row);

    // One resolve on its own, against one round trip of a prepared `SELECT 1`.
    let statement = client.prepare(SQL).await?;
    let start = Instant::now();
    for _ in 0..10_000 {
        black_box(Wide::resolve(black_box(statement.columns()))?);
    }
    let resolve = start.elapsed() / 10_000;
    let ping = client.prepare("SELECT 1").await?;
    let start = Instant::now();
    for _ in 0..100 {
        client.query(&ping, &[]).await?;
    }
    let round_trip = start.elapsed() / 100;
    println!("one resolve {resolve:.1?}, one round trip {round_trip:.1?}");
    assert!(resolve * 10 < round_trip);
    Ok(())
}
```

```
100000 rows: resolved once 134.5ms, per row 226.7ms (1.7×)
one resolve 874.0ns, one round trip 116.2µs
```

Three runs gave 1.7× each time (134.5–135.6 ms resolved once, 225.6–227.2 ms per row): smaller than release's 3×, as Chapter 1's debug ratio was, and with a wide margin for the assertion. One `resolve` of the twelve fields took 815–874 ns, and one round trip 77–116 µs; the assertion asks only for a factor of ten.

5.6 Where shelf resolves

`fetch_all` is the stage program's helper, with shelf's executor (Chapter 7) in place of a `Client`:

```
// orm/shelf/src/fetch.rs (excerpt)
/// Every row of `sql`, read into `T`.
pub async fn fetch_all<T: FromRow>(
    db: &impl Executor,
    sql: &str,
    params: &[&(dyn ToSql + Sync)],
) -> Result<Vec<T>, Error> {
    let statement = db.prepare(sql).await?;
    let columns = T::resolve(statement.columns())?;
    let rows = db.query(&statement, params).await?;
    rows.iter()
        .map(|row| T::from_row_at(row, &columns))
        .collect()
}
```

orm/shelf/src/fetch.rs

`fetch_one` and `fetch_opt` differ only in the call that runs the query. The `Statement` comes from the executor's `prepare`. On a plain `Client` or `Transaction`, that is the first of the two round trips `query` would make with SQL text anyway (main book §4.1), so the check costs no round trip of its own. On a deadpool connection it is `prepare_cached`, which after the first call returns the statement from the connection's cache without asking the server (main book §13.4; Chapter 7 measures the strategies).

Everything in shelf that reads rows goes through one of these, or resolves the same way itself. The model methods (`find`, `all`, `insert`) and the query builder call `fetch_*`. The `update` of a versioned model prepares and resolves by itself, because its result carries two more columns that say whether the row was written (Chapter 9). `fetch_stream` resolves before the first row and keeps the `Columns` in the stream for as long as rows arrive (Chapter 12). And `verify` goes a step further: it calls the same `column` for every field against the table itself, at startup, rather than against one query (Chapter 13).

Once per call, not once per connection

`resolve` runs on every call, even when deadpool's cache hands back a statement that was resolved a minute ago. shelf could keep the `Columns` next to the cached statement, in a map per connection keyed by the statement and the struct's type, and skip the work. §5.5 says what that would save: one `resolve`, under a microsecond even in a debug build. In a release build, one `resolve` of the twelve-column struct took 262–268 ns, and one round trip of a prepared `SELECT 1` to a server on the same machine 46–47 µs (PLAN.md §12, finding 59): resolving is about half a per cent of the cheapest call there is, and less over a real network. A cache would cost a map lookup per call (hashing the SQL or the statement's name, and the type), a place to keep the map per connection, and a rule for when a statement goes stale (main book §4.6). shelf does not have one. Chapter 7's exercise builds a statement cache for a plain `Client`, and a `Columns` cache would sit next to it.

Side by side — *reading 100,000 rows*

The task: read 100,000 rows of four columns, one of them nullable, into a struct.

```
// sqlx (illustration)
#[derive(sqlx::FromRow)]
struct Product {
    id: i64,
    sku: String,
    price_cents: i64,
    note: Option<String>,
}
```

```
// The derive reads each field with row.try_get("sku"): a hash-map lookup per field per row.
let products: Vec<Product> = sqlx::query_as(SQL).fetch_all(&pool).await?;
```

By hand, finding the positions once from the prepared statement, and with shelf, in one program that checks that both read the same values:

```
use shelf::FromRow;
use tokio_postgres::Statement;

#[derive(Debug, PartialEq, FromRow)]
struct Product {
    id: i64,
    sku: String,
    price_cents: i64,
    note: Option<String>,
}

const SQL: &str = "
    SELECT i AS id, 'sku-' || i AS sku, i * 100 AS price_cents,
           CASE WHEN i % 10 = 0 THEN 'clearance' END AS note
    FROM generate_series(1, 100000::bigint) AS i";

/// By hand: a column's position in the statement's result.
fn position(statement: &Statement, name: &str) -> usize {
    statement
        .columns()
        .iter()
        .position(|c| c.name() == name)
        .expect("the column is in the result")
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let client = shelf_playground::connect().await?;

    // By hand: prepare, find the positions once, then read every row by position.
    let statement = client.prepare(SQL).await?;
    let [id, sku, price_cents, note] =
        ["id", "sku", "price_cents", "note"].map(|name| position(&statement, name));
    let by_hand = client
        .query(&statement, &[])
        .await?
        .iter()
        .map(|row| {
            Ok(Product {
                id: row.try_get(id)?,
                sku: row.try_get(sku)?,
                price_cents: row.try_get(price_cents)?,
                note: row.try_get(note)?,
            })
        })
        .collect::<Result<Vec<_>, tokio_postgres::Error>>()?;

    // shelf: the same steps, in one call.
    let with_shelf: Vec<Product> = shelf::fetch_all(&client, SQL, &[]).await?;

    assert_eq!(by_hand.len(), 100_000);
    assert_eq!(by_hand, with_shelf);
    assert_eq!(with_shelf[9].note.as_deref(), Some("clearance"));
    Ok(())
}
```

	by hand	shelf	sqlx
Same	Each field's column is found by its exact name in the statement's column list, and NULL is `Option`	The same	The same
Different	You find the positions once, in each function that reads; nothing checks the types before the first row, and a missing column is whatever you make of it (a panic here)	The derive finds the positions and checks every type once per statement, before the query runs, even when no row comes back; errors name the struct, the field, the column and both types	Each field is looked up on every row, in a `HashMap` of names that the statement carries (sqlx-postgres 0.9.0's `PgRow`); a type is checked per value and not for a NULL, so a type mismatch in an empty result, or in a column that holds only NULLs, is never reported; a name that appears twice finds the last column

The last cell is worth a sentence. sqlx builds its map by inserting each column's name in order, so for a result with two columns called `id` the second insert wins; tokio-postgres's `position`, and so shelf's `column`, finds the first. Exercise 2 is about that.

In shelf

The per-row half is `get`, which every field's read in `from_row_at` calls, and the function that turns tokio-postgres's error into shelf's:

```
// orm/shelf/src/row.rs (excerpt)
/// Reads the value at `index` (found by [`column`]) as `T`.
#[doc(hidden)]
pub fn get<T: FromColumn>(
    row: &Row,
    index: usize,
    model: &'static str,
    field: &'static str,
) -> Result<T, Error> {
    row.try_get(index)
        .map_err(|e| decode_error(row, index, model, field, e))
}

// …

fn decode_error(
    row: &Row,
    index: usize,
    model: &'static str,
    field: &'static str,
    error: tokio_postgres::Error,
) -> Error {
    let column = column_name(row, index);
    let message = error.to_string();
    let problem = match error.into_source() {
        Some(source) if source.is::<WasNull>() => Mapping::Null { column },
        Some(source) => Mapping::Decode { column, source },
        // No cause: the index is not in the row, which `resolve` rules out unless the row
        // comes from another statement.
        None => Mapping::Decode {
            column,
            source: message.into(),
        },
    };
    Error::Mapping {
        model,
```

```
            field,
            problem,
        }
    }
```

orm/shelf/src/row.rs

`try_get(index)` does tokio-postgres's per-row work: the bounds check, `accepts`, and the decoding. Its error carries the cause one level down, in `source()` (§1.3), and `decode_error` sorts it: a `WasNull` cause is `Mapping::Null`, any other cause is `Mapping::Decode`. An error with no cause at all means the position does not exist in the row, which `resolve` rules out for every row of the statement it resolved: it can only happen when the row comes from another statement with fewer columns. (A row of another statement whose column at that position has the wrong type fails `accepts`, which is a cause: Exercise 3.) The model and field names travel as `&'static str`, so the context costs nothing until something fails.

When not to resolve once

- **One row.** `Product::from_row(&row)` resolves and reads in one call. For a single row it is the same work as the two steps, and nothing is gained by keeping the `Columns`.
- **Rows of different statements.** A `Columns` belongs to the statement it was resolved from. Rows of two queries, even queries that look alike, are each read with their own, or with `from_row`.
- **When you need only a value or two.** As in Chapter 1, `query_one_scalar` or `row.get` in place needs no struct and no `resolve`.

Summary

- In tokio-postgres 0.7.18 a `Row` holds its `Statement`, and every row of a statement returns the same column list. Where a field's column is, and whether its type can be read into the field, depend only on the statement.
- `Client::prepare` gives a `Statement` whose `columns()` (name, type, table OID and column number) are known before anything runs, from the server's *RowDescription*.
- `FromRow` has an associated type, `Columns`, and two steps: `resolve` finds each field's column by its exact name and checks `accepts` once per statement; `from_row_at` reads a row by position. `from_row` does both, for one row.
- A missing column or a wrong type is reported before the query runs, even when it would return no rows; an `INSERT … RETURNING` that does not fit its struct is never executed. A NULL in a non-`Option` field and a value that fails to decode can only be found in a row.
- `Columns` is a tuple, one element per field: `usize`, `()` for `skip`, the nested `Columns` for `flatten`. It is `Send` because shelf's futures hold it across an `.await`.
- Resolving once costs 4–8 per cent over hand-written index code; resolving per row costs what lookups by name cost (about 3× in release, 1.7× in a debug build). One `resolve` costs well under a microsecond, so shelf resolves on every call and caches nothing.
- tokio-postgres still checks `accepts` on every row. It costs nothing measurable, and it turns a row read with another statement's positions into an error instead of garbage.
- `from_rows` on an empty slice checks nothing; `fetch_all` checks against the statement.

Exercises

1. **One statement, two executions.** Prepare a statement with a parameter, resolve a struct against it once, and read the rows of two executions with different parameters. *Hint:* `Columns` belongs to the statement, not to the execution; `Product::resolve` and `Product::from_row_at` are public.
2. **Two columns, one name.** A join such as `SELECT o.id, l.id, l.sku FROM orders o JOIN lines l ON l.order_id = o.id` returns two columns named `id`. Which one does shelf read into a struct with an `id` field, and which would sqlx read? Write a `resolve_unique::<T>` that refuses such a result before calling `T::resolve`. Then decide whether shelf should refuse it. *Hint:* compare each column's name with the names before it. §5.6's side by side says what each library does.
3. **Positions from another statement.** Resolve a `Product { sku: String, price_cents: i64 }` against `SELECT sku, price_cents …`, then call `from_row_at` on a row of `SELECT price_cents, sku …`. What happens? What would happen if tokio-postgres did not call `accepts` on every row? *Hint:* the positions point at the other column. `String`'s `FromSql` reads any bytes that are valid UTF-8, and an `int8` is eight bytes.

Solutions

1. Resolve once, after `prepare`, and keep the `Columns` for as long as the statement:

```
use shelf::FromRow;

#[derive(Debug, FromRow)]
struct Product {
    sku: String,
    price_cents: i64,
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY, price_cents bigint NOT NULL);
             INSERT INTO products VALUES ('MUG-01', 1299), ('TEE-02', 2499), ('VASE-03',
3999);",
        )
        .await?;

    let statement = client
        .prepare("SELECT sku, price_cents FROM products WHERE price_cents < $1 ORDER BY
sku")
        .await?;
    let columns = Product::resolve(statement.columns())?; // once, for every execution

    for (limit, expected) in [(2000_i64, vec!["MUG-01"]), (3000, vec!["MUG-01", "TEE-02"])]
{
        let rows = client.query(&statement, &[&limit]).await?;
        let products = rows
            .iter()
            .map(|row| Product::from_row_at(row, &columns))
            .collect::<Result<Vec<_>, _>>()?;
        let skus: Vec<&str> = products.iter().map(|p| p.sku.as_str()).collect();
        assert_eq!(skus, expected);
        assert!(products.iter().all(|p| p.price_cents < limit));
    }
    Ok(())
}
```

This is what a repository function that runs one query many times can do by hand. shelf's `fetch_*` take SQL text rather than a `Statement`, so each call prepares (or asks the cache) and resolves again, which §5.6 found not worth avoiding.

2. shelf reads the first `id`, the order's, as `Row::try_get("id")` would; sqlx would read the last, the line's. Neither says anything. A check that refuses the result:

```
use shelf::FromRow;
use tokio_postgres::Column;

#[derive(Debug, FromRow)]
struct Line {
    id: i64,
    sku: String,
}

/// `T`'s columns, refusing a result in which two columns have the same name.
fn resolve_unique<T: FromRow>(columns: &[Column]) -> Result<T::Columns, String> {
    for (i, column) in columns.iter().enumerate() {
        if columns[..i].iter().any(|c| c.name() == column.name()) {
            return Err(format!(
                "the result has two columns named {:?}; rename one with AS",
                column.name()
            ));
        }
    }
    T::resolve(columns).map_err(|e| e.to_string())
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE orders (id bigint PRIMARY KEY);
             CREATE TEMP TABLE lines (
                 id bigint PRIMARY KEY,
                 order_id bigint NOT NULL,
                 sku text NOT NULL
             );
             INSERT INTO orders VALUES (7);
             INSERT INTO lines VALUES (70, 7, 'MUG-01');",
        )
        .await?;
    let join = "SELECT o.id, l.id, l.sku FROM orders o JOIN lines l ON l.order_id = o.id";

    // shelf reads the first column called `id`: the order's.
    let line: Line = shelf::fetch_one(&client, join, &[]).await?;
    assert_eq!((line.id, line.sku.as_str()), (7, "MUG-01"));

    let statement = client.prepare(join).await?;
    let error = resolve_unique::<Line>(statement.columns()).unwrap_err();
    println!("{error}");
    assert!(error.starts_with(r#"the result has two columns named "id""#));

    // With an alias, the line's own id is the only `id`.
    let line: Line = shelf::fetch_one(
        &client,
        "SELECT o.id AS order_id, l.id, l.sku FROM orders o JOIN lines l ON l.order_id =
o.id",
        &[],
    )
    .await?;
    assert_eq!(line.id, 70);
    Ok(())
}
```

```
the result has two columns named "id"; rename one with AS
```

shelf does not refuse such a result. Two columns of one name are legal SQL, and common in a join, and the struct may read neither of them; refusing every such result would refuse queries that work. The precise

check refuses only a name the struct reads, and belongs in `column`, which knows the name: after `position` finds the first match, a second search of the rest of the list finds an ambiguous one, at the cost of walking the whole list for every field. shelf leaves it out (PLAN.md §10); the alias is the fix either way.

3. The positions say `sku` is column 0, which in the other statement is `price_cents`, an `int8`. tokio-postgres's per-row `accepts` refuses to read it as a `String`, and shelf reports it as a decoding problem of the field `sku`, naming the column it actually found:

```
use shelf::{FromRow, Mapping};
use tokio_postgres::types::{FromSql, Type};

#[derive(Debug, FromRow)]
#[allow(dead_code)] // the program only shows the error
struct Product {
    sku: String,
    price_cents: i64,
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    let first = client
        .prepare("SELECT 'MUG-01' AS sku, 1299::int8 AS price_cents")
        .await?;
    let columns = Product::resolve(first.columns())?;
    assert_eq!(columns, (0, 1));

    // A row of another statement, whose columns are the other way round.
    let row = client
        .query_one("SELECT 1299::int8 AS price_cents, 'MUG-01' AS sku", &[])
        .await?;
    let error = Product::from_row_at(&row, &columns).unwrap_err();
    println!("{error}");
    assert!(matches!(
        error,
        shelf::Error::Mapping {
            field: "sku",
            problem: Mapping::Decode { .. },
            ..
        }
    ));

    // Without the per-row check, `String` would decode the int8's eight bytes as text.
    let text = <String as FromSql>::from_sql(&Type::TEXT, &1299_i64.to_be_bytes())
        .expect("eight bytes that happen to be valid UTF-8");
    assert_eq!(text, "\0\0\0\0\0\0\u{5}\u{13}");
    Ok(())
}
```

```
Product.sku: column "price_cents": cannot convert between the Rust type
`alloc::string::String` and the Postgres type `int8`
```

Without the per-row check, the read would succeed: the eight bytes of 1299 as a big-endian `int8` are all valid UTF-8, so `sku` would hold six NUL characters and two control characters, and nothing would report it. That is the case for keeping tokio-postgres's check on every row, although `resolve` makes it redundant for correct code: it costs nothing measurable, and it turns a misuse into an error.

CHAPTER 6

Tables: the model derive

`FromRow` reads the rows of any query, but you still write every query, and most of them are the same few: fetch one row by its key, fetch them all, count them. The struct already knows its columns. What it lacks is the table they come from and which of them is the key. Two attributes supply those, and with them a second derive, `Table`, can write the everyday `SELECT`s itself.

It writes them at compile time. The macro knows every name, so it assembles each statement as a string literal, with every identifier quoted, and leaves nothing to build at run time. The statements are constants you can print, so the SQL a call sends is never a guess. Values never go into those strings: they stay `$1` parameters, as they have since the main book's second chapter. This chapter builds the derive and the four reads it makes free, and finds where SQL written at compile time stops.

In this chapter you will learn:

- what a table adds to a row type: a name, a key and a column list
- how `#[derive(Table)]` reads `table`, `schema`, `primary_key` and `generated`, and what it refuses
- how a derive writes SQL at compile time, quoting identifiers so that a table called `order` works
- `find`, `find_opt`, `all` and `count`, and the SQL each one sends
- why shelf never writes `SELECT *`
- how a constant can be built from another type's constants, and where that stops

6.1 What a table adds

A `FromRow` struct describes a result: these columns, read into these fields. It does not say where the result comes from, and Chapters 1–5 never needed it to, because you wrote the query. A *model* is a row type that also knows where its rows live. That takes three more facts:

Fact	What it is	For a `Product`
The table	where the rows live	`"products"`
The key	the columns that pick out one row	`"id"`
The column list	the columns the struct reads, in field order	`"id", "sku", "price_cents"`

The column list is the one `FromRow` already reads; the other two are new. From the three, the everyday statements follow mechanically: the row with a key is `SELECT` the list `FROM` the table `WHERE` the key `= $1`, and so on. shelf's `Table` trait holds them, as constants:

```
// orm/shelf/src/table.rs (excerpt)
pub trait Table: FromRow + Send + Sync {
    /// The key's type: the key column's field type, or a tuple of them for a composite key.
    type Key: Send + Sync;

    /// The struct's name, for errors.
    const MODEL: &'static str;
```

```
    /// The table, quoted, and qualified when it has a schema: `"inventory"."products"`.
    const TABLE: &'static str;
    /// The columns the model reads, quoted, in field order.
    const COLUMNS: &'static str;
    /// The row with a given key: `SELECT … WHERE "id" = $1`.
    const FIND: &'static str;
    /// Every row, ordered by the key.
    const ALL: &'static str;
    /// The number of rows: `SELECT count(*) FROM …`.
    const COUNT: &'static str;
    // … UPDATE, DELETE, KEY_COLUMNS, VERSION, UPDATED_AT (Chapters 8 and 9), FIELDS
(Chapter 13)

    /// This row's key.
    fn key(&self) -> Self::Key;

    /// `key` as statement parameters, in the order of the key's columns.
    fn key_params(key: &Self::Key) -> Vec<&(dyn ToSql + Sync)>;

    // … find, find_opt, all and count (§6.4); update and delete (Chapter 8)
}
```

orm/shelf/src/table.rs

`FromRow` is a supertrait: a model is always readable, so `find` can hand its statement to Chapter 5's `fetch_opt::<Self>` and get a `Self` back. `Send + Sync` let a model's methods hold it across an `.await` in a future that must be `Send`, which Chapter 7 explains. And `Key` is whatever identifies a row: the key field's own type, or a tuple of them.

The derive writes all of it. It needs the table's name and the key's field, and the constants can be printed without a database:

```
use shelf::{FromRow, Table};

#[derive(Debug, FromRow, Table)]
#[shelf(table = "products")]
#[allow(dead_code)] // the program only prints the SQL
struct Product {
    #[shelf(primary_key, generated)]
    id: i64,
    sku: String,
    price_cents: i64,
}

fn main() {
    println!("TABLE   {}", Product::TABLE);
    println!("COLUMNS {}", Product::COLUMNS);
    println!("FIND    {}", Product::FIND);
    println!("ALL     {}", Product::ALL);
    println!("COUNT   {}", Product::COUNT);

    assert_eq!(Product::TABLE, r#""products""#);
    assert_eq!(
        Product::FIND,
        r#"SELECT "id", "sku", "price_cents" FROM "products" WHERE "id" = $1"#
    );
    // The key's type is the key field's.
    let key: <Product as Table>::Key = 7_i64;
    assert_eq!(Product::key_params(&key).len(), 1);
}
```

```
TABLE   "products"
COLUMNS "id", "sku", "price_cents"
FIND    SELECT "id", "sku", "price_cents" FROM "products" WHERE "id" = $1
ALL     SELECT "id", "sku", "price_cents" FROM "products" ORDER BY "id"
COUNT   SELECT count(*) FROM "products"
```

Every statement names its columns, quotes every identifier, and leaves the key as `$1`. Nothing here runs at run time: `Product::FIND` is a `&'static str` in the binary, as fixed as any literal you could write by hand, which is what makes it safe to print, to log and to assert on.

6.2 `#[derive(Table)]`

`Table` goes beside `FromRow` and reads the column names `FromRow` reads, `rename` and `rename_all` included. It adds four keys of its own, and two that Chapter 9 uses:

Attribute	On	Meaning
`table = "products"`	the struct	The table (required)
`schema = "inventory"`	the struct	Its schema, when it is not found on the `search_path`
`primary_key`	a field	Part of the key (required on at least one field); several make a composite key
`generated`	a field	The database fills it (`DEFAULT uuidv7()`, `now()`, an identity): writes leave it out
`version`, `updated_at`	a field	Optimistic locking and a modification time (Chapter 9)

`generated` changes nothing about reading. A generated column is read like any other; the flag is for Chapter 8, where `update` must not write a column the database owns, and the struct a new row is built from leaves it out.

A key spread over several columns is a tuple of their types, in field order. stockroom's `stock` table is keyed by store and product, and so is its model:

```
#[derive(FromRow, Table)]
#[shelf(table = "stock")]
struct Stock {
    #[shelf(primary_key)]
    store_id: i64,
    #[shelf(primary_key)]
    product_id: i64,
    on_hand: i32,
}

// type Key = (i64, i64), and FIND ends WHERE "store_id" = $1 AND "product_id" = $2
let stock = Stock::find(&client, &(store_id, product_id)).await?;
```

The key is owned: `key()` clones the key fields out of a row, and `find` borrows a `&(i64, i64)`. A tuple of references would save the clone and cost a lifetime on the trait's `Key`, which every generic caller would then have to name; for keys, which are integers and UUIDs, the clone is free.

Who reports a mistake

`#[shelf(…)]` is one namespace (Chapter 4, §4.2): both derives on a model parse every key, and each key has one owner that reports its mistakes. `table`, `schema`, `primary_key` and `generated` belong to `Table`; `FromRow` passes over them, and `Table` passes over a mistake in `rename`, which `FromRow` reports. A struct with both derives hears about each mistake once. Unknown keys go to `FromRow`, which every model has.

What `Table` itself refuses, each at the token that causes it, is pinned by a UI test:

```
// orm/shelf/tests/ui/table.rs (excerpt)
#[derive(FromRow, Table)]
struct NoTable {
    #[shelf(primary_key)]
    id: i64,
}

#[derive(FromRow, Table)]
#[shelf(table = "products")]
```

```
struct NoKey {
    id: i64,
}

#[derive(FromRow, Table)]
#[shelf(table = "products", schema = "")]
struct Mistakes {
    #[shelf(primary_key)]
    id: i64,
    #[shelf(flatten)]
    price: Price,
    #[shelf(skip, generated)]
    cached: i64,
}

#[derive(FromRow, Table)]
#[shelf(table = "products")]
struct Generic<T> {
    #[shelf(primary_key)]
    id: i64,
    value: T,
}
```

orm/shelf/tests/ui/table.rs

```
error: Table needs the table's name: #[shelf(table = "…")]
 --> tests/ui/table.rs:5:8
  |
5 | struct NoTable {
  |        ^^^^^^^

error: Table needs a key: mark its column, or columns, #[shelf(primary_key)]
  --> tests/ui/table.rs:12:8
   |
12 | struct NoKey {
   |        ^^^^^

error: a schema name cannot be empty
  --> tests/ui/table.rs:22:38
   |
22 | #[shelf(table = "products", schema = "")]
   |                                      ^^

error: a Table model lists its own columns, and a flattened struct's are not known here:
write them out as fields, or read this struct with FromRow and SQL
  --> tests/ui/table.rs:26:13
   |
26 |     #[shelf(flatten)]
   |             ^^^^^^^

error: `generated` cannot apply to a field marked `skip`, which is not a column
  --> tests/ui/table.rs:28:19
   |
28 |     #[shelf(skip, generated)]
   |                   ^^^^^^^^^

error: a Table model cannot be generic: its SQL names one table
  --> tests/ui/table.rs:34:16
   |
34 | struct Generic<T> {
   |                ^
```

Every one of these is reported in a single build, combined as Chapter 4 combined `FromRow`'s (§4.4). The last two need a word each.

No type parameters. A model's SQL names one table. `Page<T>` would send the same `SELECT … FROM "pages"` for every `T`, and the `T` would have nothing to do with the rows. The refusal also keeps a door open that §6.5 needs: SQL that joins another model's constants cannot be written in a generic impl.

No `flatten`. `Table` writes its statements at macro time, and a macro sees only the tokens of the item it is attached to. A `#[shelf(flatten)] price: Price` field says that some of the columns are `Price`'s, and `Price`'s fields are in another item, which the macro never sees. `FromRow` copes because it does not need the names: its generated `resolve` calls `Price::resolve` at run time. `Table` needs them in a string literal.

It could be done at compile time all the same. If `FromRow` published its column list as a constant, `Table` could join `Price`'s list into its statements with the technique of §6.5. The catch is that a trait constant must exist for every impl, and a generic `FromRow` struct that flattens a type parameter would have to join `T`'s list in a generic impl, which §6.5 shows the compiler refuses. So shelf keeps the refusal, and the error says what to do instead: write the nested struct's columns out as fields of the model, or read the nested shape with `FromRow` and a query of your own.

6.3 SQL at compile time

A derive is a function from tokens to tokens, and nothing stops that function from formatting a string. This stage program is a small `Table` derive: it reads `table` and `primary_key` with Chapter 4's `parse_nested_meta`, quotes every name, writes three statements with `format!`, and emits them as constants. Run on a struct whose table is called `order` and whose field is called `type`, two words SQL reserves:

```
use proc_macro2::TokenStream;
use quote::quote;
use syn::ext::IdentExt;
use syn::{Data, DeriveInput, Fields, LitStr, parse_quote};

/// `name` as an SQL identifier: in double quotes, with any `"` inside it doubled.
fn quote_ident(name: &str) -> String {
    format!("\"{}\"", name.replace('"', "\"\""))
}

/// The stage `Table` derive: the table from `#[shelf(table = "…")]`, the key from
/// `#[shelf(primary_key)]`, and three statements written as string literals.
fn expand(input: &DeriveInput) -> syn::Result<TokenStream> {
    let mut table = None;
    for attr in input.attrs.iter().filter(|a| a.path().is_ident("shelf")) {
        attr.parse_nested_meta(|meta| {
            if meta.path.is_ident("table") {
                table = Some(meta.value()?.parse::<LitStr>()?.value());
                Ok(())
            } else {
                Err(meta.error("the stage knows only `table`"))
            }
        })?;
    }
    let Some(table) = table else {
        return Err(syn::Error::new_spanned(
            &input.ident,
            "Table needs #[shelf(table = \"…\")]",
        ));
    };
    let Data::Struct(data) = &input.data else {
        return Err(syn::Error::new_spanned(&input.ident, "a struct, please"));
    };
    let Fields::Named(fields) = &data.fields else {
        return Err(syn::Error::new_spanned(
            &data.fields,
            "named fields, please",
        ));
    };
```

```
    let mut columns = Vec::new();
    let mut key = Vec::new();
    for field in &fields.named {
        let mut primary_key = false;
        for attr in field.attrs.iter().filter(|a| a.path().is_ident("shelf")) {
            attr.parse_nested_meta(|meta| {
                if meta.path.is_ident("primary_key") {
                    primary_key = true;
                    Ok(())
                } else {
                    Err(meta.error("the stage knows only `primary_key`"))
                }
            })?;
        }
        let ident = field.ident.as_ref().expect("named fields have names");
        let column = quote_ident(&ident.unraw().to_string()); // r#type is the column "type"
        if primary_key {
            key.push(column.clone());
        }
        columns.push(column);
    }
    if key.is_empty() {
        return Err(syn::Error::new_spanned(&input.ident, "Table needs a key"));
    }

    // Each statement is formatted here, while the macro runs, and becomes a literal.
    let table = quote_ident(&table);
    let columns = columns.join(", ");
    let key_is: Vec<String> = key
        .iter()
        .enumerate()
        .map(|(i, column)| format!("{column} = ${}", i + 1))
        .collect();
    let find = format!(
        "SELECT {columns} FROM {table} WHERE {}",
        key_is.join(" AND ")
    );
    let all = format!("SELECT {columns} FROM {table} ORDER BY {}", key.join(", "));
    let count = format!("SELECT count(*) FROM {table}");

    let name = &input.ident;
    Ok(quote! {
        impl Table for #name {
            const FIND: &'static str = #find;
            const ALL: &'static str = #all;
            const COUNT: &'static str = #count;
        }
    })
}

fn main() {
    let input: DeriveInput = parse_quote! {
        #[shelf(table = "order")]
        struct Order {
            #[shelf(primary_key)]
            id: i64,
            r#type: String,
            total_cents: i64,
        }
    };
    let code = shelf_playground::pretty(expand(&input).expect("a valid model"));
    println!("{code}");
    assert!(code.contains(
        r#""SELECT \"id\", \"type\", \"total_cents\" FROM \"order\" WHERE \"id\" = $1""#
    ));

    let keyless: DeriveInput = parse_quote! {
        #[shelf(table = "order")]
```

```
        struct Order { id: i64 }
    };
    assert_eq!(
        expand(&keyless).unwrap_err().to_string(),
        "Table needs a key"
    );
}
```

```
impl Table for Order {
    const FIND: &'static str = "SELECT \"id\", \"type\", \"total_cents\" FROM \"order\"
WHERE \"id\" = $1";
    const ALL: &'static str = "SELECT \"id\", \"type\", \"total_cents\" FROM \"order\" ORDER
BY \"id\"";
    const COUNT: &'static str = "SELECT count(*) FROM \"order\"";
}
```

Three things happened at macro time. `unraw` turned the raw identifier `r#type` into the column name `type` (Chapter 3, §3.5). `quote_ident` put every name in double quotes. And `quote!` turned each `String` into a string literal token, escaping its quotes, so the generated code holds the finished statement.

> **Macro lens:** interpolating a `String` with `#find` does not paste its characters into the output as code: quote's `ToTokens` impl for `String` makes one `Literal`, a string literal token, escaped as Rust requires. That is why a statement full of double quotes comes out as one valid literal, and why no `concat!` or `format!` is left for run time. A `&str` or a `String` becomes a literal; an `Ident` becomes a name; a `TokenStream` is spliced in as it is.

shelf's derive does the same in more detail. The statements are written by `format!` in `expand`, from a quoted table and column list:

```
// orm/shelf-codegen/src/table.rs (excerpt)
    let table = match &container.schema {
        Some(schema) => format!(
            "{}.{}",
            quote_ident(&schema.value.value()),
            quote_ident(&table.value.value())
        ),
        None => quote_ident(&table.value.value()),
    };
    let column_sql = column_list(&columns);
    let key_columns: Vec<String> = key.iter().map(|(_, column)| column.clone()).collect();
    let find = format!(
        "SELECT {column_sql} FROM {table} WHERE {}",
        equal_params(&key_columns, 1)
    );
    let all = format!(
        "SELECT {column_sql} FROM {table} ORDER BY {}",
        column_list(&key_columns)
    );
    let count = format!("SELECT count(*) FROM {table}");
    // …
    Ok(quote! {
        #[automatically_derived]
        impl ::shelf::Table for #name {
            type Key = #key_type;

            const MODEL: &'static ::core::primitive::str = #model;
            const TABLE: &'static ::core::primitive::str = #table;
            const COLUMNS: &'static ::core::primitive::str = #column_sql;
            const FIND: &'static ::core::primitive::str = #find;
            const ALL: &'static ::core::primitive::str = #all;
```

```
            const COUNT: &'static ::core::primitive::str = #count;
            // …
```

orm/shelf-codegen/src/table.rs

The column names in `columns` are the ones `FromRow` reads: `rename` if a field has one, else the field's name through `rename_all`, unrawed. The helpers that quote them are in their own module, `sql.rs`, which Chapter 8's `Insert` and Chapter 9's `Patch` share:

```
// orm/shelf-codegen/src/sql.rs (excerpt)
/// `name` as a quoted SQL identifier: `"name"`, with any `"` inside it doubled. Quoting
keeps
/// the name exactly as written (PostgreSQL folds unquoted names to lower case) and lets a
table
/// be called `order` or a column `type`. The main book's `quote_ident` (its Chapter 2), run
by
/// the compiler.
pub fn quote_ident(name: &str) -> String {
    format!("\"{}\"", name.replace('"', "\"\""))
}

/// The columns, quoted and separated by commas: `"id", "sku"`.
pub fn column_list(columns: &[String]) -> String {
    let quoted: Vec<String> = columns.iter().map(|c| quote_ident(c)).collect();
    quoted.join(", ")
}

/// `"a" = $1 AND "b" = $2`, numbering from `first`.
pub fn equal_params(columns: &[String], first: usize) -> String {
    let terms: Vec<String> = columns
        .iter()
        .enumerate()
        .map(|(i, c)| format!("{} = ${}", quote_ident(c), first + i))
        .collect();
    terms.join(" AND ")
}
```

orm/shelf-codegen/src/sql.rs

Quoting every name, not only the awkward ones, costs nothing and settles two problems for good. A key word such as `order` or `type` is a name again. And case survives: PostgreSQL folds an unquoted `unitPrice` to `unitprice`, so a table created with a quoted camel-case column can only be reached by quoting it. This program reads such a table through the real derive, and sends the same `SELECT` unquoted to show what quoting saved:

```
use shelf::{FromRow, Table};
use tokio_postgres::error::SqlState;

#[derive(Debug, FromRow, Table)]
#[shelf(table = "order", rename_all = "camelCase")]
struct Order {
    #[shelf(primary_key)]
    id: i64,
    r#type: String,
    unit_price: i64,
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            r#"CREATE TEMP TABLE "order" (
                   id bigint PRIMARY KEY,
                   type text NOT NULL,
```

```
                    "unitPrice" bigint NOT NULL
                );
                INSERT INTO "order" VALUES (1, 'retail', 1299);"#,
        )
        .await?;

    println!("{}", Order::FIND);
    let order = Order::find(&client, &1).await?;
    assert_eq!((order.r#type.as_str(), order.unit_price), ("retail", 1299));

    // The same statement without the quotes: `order` is a key word.
    let unquoted = Order::FIND.replace('"', "");
    println!("{unquoted}");
    let error = client.query(&unquoted, &[&1_i64]).await.unwrap_err();
    println!(
        "{}",
        error.as_db_error().expect("from the server").message()
    );
    assert_eq!(error.code(), Some(&SqlState::SYNTAX_ERROR));
    Ok(())
}
```

```
SELECT "id", "type", "unitPrice" FROM "order" WHERE "id" = $1
SELECT id, type, unitPrice FROM order WHERE id = $1
syntax error at or near "order"
```

> **Pitfall:** the derive quotes *identifiers*, which it knows at compile time, and writes `$n` for every *value*. Nothing a program receives at run time reaches the SQL: there is no code path from a request body to a statement's text. Keep it that way when you extend a model: a table or column name that must vary at run time is chosen from a fixed list (main book §2.3), and a value is always a parameter, never `format!`ted in.

6.4 Reading

With the statements written, the four reads are a few lines each. They are provided methods of the trait, so the derive generates none of them; it generates the constants they use:

Method	Statement	Returns
`find(&db, &key)`	`FIND`, through `fetch_opt`	the row, or `Error::NotFound`
`find_opt(&db, &key)`	`FIND`, through `fetch_opt`	`Option<Self>`
`all(&db)`	`ALL`, through `fetch_all`	every row, ordered by the key
`count(&db)`	`COUNT`	the number of rows, an `i64`

`find` and `find_opt` send the same statement. The difference is what a missing row means: for `find_opt` it is an answer, `None`; for `find`, which a caller uses when the row must exist, it is an error, `NotFound`, with the model's name in it. `all` orders by the key, so two calls on an unchanged table return the rows in the same order (without an `ORDER BY`, PostgreSQL returns them in whatever order it reads them, which an `UPDATE` changes). This program runs the four on a table with a single key and one with a composite key:

```
use shelf::{FromRow, Table};

#[derive(Debug, PartialEq, FromRow, Table)]
#[shelf(table = "products")]
struct Product {
```

```
    #[shelf(primary_key, generated)]
    id: i64,
    sku: String,
    price_cents: i64,
}

#[derive(Debug, FromRow, Table)]
#[shelf(table = "stock")]
struct Stock {
    #[shelf(primary_key)]
    store_id: i64,
    #[shelf(primary_key)]
    product_id: i64,
    on_hand: i32,
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (
                 id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
                 sku text NOT NULL UNIQUE,
                 price_cents bigint NOT NULL
             );
             INSERT INTO products (sku, price_cents)
                 VALUES ('MUG-01', 1299), ('TEE-02', 2499), ('VASE-03', 3999);
             CREATE TEMP TABLE stock (
                 store_id bigint,
                 product_id bigint,
                 on_hand integer NOT NULL,
                 PRIMARY KEY (store_id, product_id)
             );
             INSERT INTO stock VALUES (1, 1, 10), (1, 2, 0), (2, 1, 4);",
        )
        .await?;

    let mug = Product::find(&client, &1).await?;
    assert_eq!(mug.sku, "MUG-01");
    assert_eq!(Product::find_opt(&client, &99).await?, None);

    let skus: Vec<String> = Product::all(&client)
        .await?
        .into_iter()
        .map(|p| p.sku)
        .collect();
    assert_eq!(skus, ["MUG-01", "TEE-02", "VASE-03"]);
    assert_eq!(Product::count(&client).await?, 3);

    // A composite key is a tuple, in field order.
    let stock = Stock::find(&client, &(2, 1)).await?;
    assert_eq!(stock.on_hand, 4);
    println!("{}", Stock::FIND);

    // A key with no row: `find` says which model it was looking for.
    let error = Product::find(&client, &99).await.unwrap_err();
    println!("{error}");
    assert!(matches!(error, shelf::Error::NotFound { model: "Product" }));
    Ok(())
}
```

```
SELECT "store_id", "product_id", "on_hand" FROM "stock" WHERE "store_id" = $1 AND
"product_id" = $2
Product not found
```

On a plain `Client` each of these calls costs two round trips, one to prepare the statement and one to run it, which is what `query` with SQL text costs anyway; Chapter 5 used the first to check the struct against the result. On a pooled deadpool connection the prepared statement comes from the connection's cache after the first call, and each call costs one. Chapter 7 measures both.

Never `SELECT *`

Every statement the derive writes lists its columns. `SELECT *` would be shorter, and `resolve` would find the fields' columns in it all the same, so the list needs a reason. There are two.

The first is the statement cache. A prepared statement's result columns are fixed when it is prepared. Add a column to the table and a cached `SELECT *` would now return one more, which PostgreSQL refuses to do behind the client's back: the statement fails with SQLSTATE 0A000, "cached plan must not change result type", until it is prepared again (main book §4.6). A statement that names its columns returns the same columns after the migration, and keeps working. This program caches both on one pooled connection, adds a column, and runs both again:

```
use shelf::{FromRow, Table};
use tokio_postgres::error::SqlState;

#[derive(Debug, FromRow, Table)]
#[shelf(table = "products")]
struct Product {
    #[shelf(primary_key)]
    id: i64,
    sku: String,
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    // One connection, so every call below uses the same session and the same cache.
    let pool = shelf_playground::pool(1);
    let db = pool.get().await?;
    db.batch_execute(
        "CREATE TEMP TABLE products (id bigint PRIMARY KEY, sku text NOT NULL);
         INSERT INTO products VALUES (1, 'MUG-01');",
    )
    .await?;

    // Both statements prepared and cached on this connection.
    let everything = db.prepare_cached("SELECT * FROM products").await?;
    db.query(&everything, &[]).await?;
    Product::find(&db, &1).await?;

    // A migration adds a column.
    db.batch_execute("ALTER TABLE products ADD COLUMN note text")
        .await?;

    let everything = db.prepare_cached("SELECT * FROM products").await?;
    let error = db.query(&everything, &[]).await.unwrap_err();
    println!(
        "SELECT *: {}",
        error.as_db_error().expect("from the server").message()
    );
    assert_eq!(error.code(), Some(&SqlState::FEATURE_NOT_SUPPORTED));

    // The model's cached statement names its columns: it still returns the same two.
    let mug = Product::find(&db, &1).await?;
    println!("Product::find: {mug:?}");
    assert_eq!(mug.sku, "MUG-01");
    Ok(())
}
```

```
SELECT *: cached plan must not change result type
Product::find: Product { id: 1, sku: "MUG-01" }
```

The second reason is what `*` would hand `resolve`. With an explicit list the result holds exactly the columns the struct reads; with `*` it holds every column of the table, including a large `jsonb` document or a `bytea` the struct has no field for, which the server would still send in every row and the client would still receive. The list costs nothing to write, because the macro writes it.

6.5 Constants, and where they stop

Everything `Table` generates so far is a literal, because the macro knew every piece of it. That stops being true in Chapter 8. A `NewProduct` struct derives `Insert` and names its model, `#[shelf(model = Product)]`, and its statement needs the model's table and columns: `INSERT INTO "products" (…) VALUES (…) RETURNING "id", "sku", …`. Those live in `Product`'s constants, in another item, which the `Insert` derive cannot see. It sees the tokens `Product`, nothing more.

The obvious tool is `concat!`, which joins literals at compile time. It takes literals only:

```
const COUNT: &str = concat!("SELECT count(*) FROM ", Product::TABLE);
```

```
error: expected a literal
 --> src/bin/concat.rs:5:54
  |
5 | const COUNT: &str = concat!("SELECT count(*) FROM ", Product::TABLE);
  |                                                      ^^^^^^^^^^^^^^
  |
  = note: only literals (like `"foo"`, `-42` and `3.14`) can be passed to `concat!()`
```

`concat!` is a macro, so it runs before the compiler knows what `Product::TABLE` holds. What can read a constant's value is const evaluation, and a `const fn` can join strings at compile time if it is written with the loops const evaluation allows.

> **Rust refresher:** a `const` item is evaluated by the compiler, and its value is stored in the binary; using it costs what using a literal costs. A `const fn` is a function the compiler can run while evaluating one. Const evaluation allows `while` and `loop`, `if` and `match`, indexing, and calls to other `const fn`s. It does not allow `for` loops, which call `IntoIterator::into_iter` and `Iterator::next`, trait methods that are not `const` (rustc 1.99.1 refuses one with E0015, "cannot use `for` loop on `&[&str]` in constant functions"), and it does not allow allocation: the result must be a fixed-size value, such as an array whose length is itself a constant.

So the join takes two steps: one `const fn` computes the total length, and a second fills an array of that length, which a third views as a `&str`. This program builds a statement of its own from `Product`'s constants that way, and runs it:

```
use shelf::{FromRow, Table};

#[derive(Debug, FromRow, Table)]
#[shelf(table = "products")]
struct Product {
    #[shelf(primary_key)]
    id: i64,
    sku: String,
    price_cents: i64,
}
```

```
/// The length of `parts` joined.
const fn total_len(parts: &[&str]) -> usize {
    let mut len = 0;
    let mut i = 0;
    while i < parts.len() {
        len += parts[i].len();
        i += 1;
    }
    len
}

/// `parts` joined, as bytes; `N` is their `total_len`.
const fn join<const N: usize>(parts: &[&str]) -> [u8; N] {
    let mut out = [0; N];
    let mut at = 0;
    let mut i = 0;
    while i < parts.len() {
        let bytes = parts[i].as_bytes();
        let mut j = 0;
        while j < bytes.len() {
            out[at] = bytes[j];
            at += 1;
            j += 1;
        }
        i += 1;
    }
    out
}

/// The cheapest product: a statement of this program's, made of the model's constants.
const CHEAPEST: &str = {
    const PARTS: &[&str] = &[
        "SELECT ",
        Product::COLUMNS,
        " FROM ",
        Product::TABLE,
        " ORDER BY \"price_cents\" LIMIT 1",
    ];
    const BYTES: [u8; total_len(PARTS)] = join(PARTS);
    match std::str::from_utf8(&BYTES) {
        Ok(sql) => sql,
        Err(_) => panic!("joined &strs are UTF-8"),
    }
};

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    println!("{CHEAPEST}");
    assert_eq!(
        CHEAPEST,
        r#"SELECT "id", "sku", "price_cents" FROM "products" ORDER BY "price_cents" LIMIT
1"#
    );

    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (
                 id bigint PRIMARY KEY, sku text NOT NULL, price_cents bigint NOT NULL
             );
             INSERT INTO products VALUES (1, 'VASE-03', 3999), (2, 'MUG-01', 1299);",
        )
        .await?;
    let cheapest: Product = shelf::fetch_one(&client, CHEAPEST, &[]).await?;
    assert_eq!(cheapest.sku, "MUG-01");
    Ok(())
}
```

```
SELECT "id", "sku", "price_cents" FROM "products" ORDER BY "price_cents" LIMIT 1
```

`CHEAPEST` is as constant as `Product::FIND`: the compiler ran `total_len` and `join`, and the binary holds the finished statement. shelf has the same three functions in `shelf/src/sql.rs`, under `__private` for generated code to call, and Chapter 8's `Insert` derive generates exactly this block, with its own statement's pieces around `<Product as Table>::TABLE` (orm/shelf/src/sql.rs). The inner `const` items matter: an array's length must be a constant, so `PARTS` must be one before `total_len(PARTS)` can size `BYTES`.

Where it stops

The inner constants are also where the technique stops. An inner `const` is an item of its own, which shares nothing with the impl around it but names, so it cannot use that impl's type parameters. A generic model, if `Table` allowed one, would need its statement joined inside a generic impl:

```
impl<T> Counted for Page<T> {
    const COUNT: &'static str = {
        const PARTS: &[&str] = &["SELECT count(*) FROM ", <Page<T> as Named>::TABLE];
        const BYTES: [u8; total_len(PARTS)] = join(PARTS);
        utf8(&BYTES)
    };
}
```

```
error[E0401]: can't use generic parameters from outer item
  --> src/bin/generic.rs:16:65
   |
14 | impl<T> Counted for Page<T> {
   |      - type parameter from outer item
15 |     const COUNT: &'static str = {
16 |         const PARTS: &[&str] = &["SELECT count(*) FROM ", <Page<T> as Named>::TABLE];
   |                ----                                                ^ use of generic
parameter from outer item
   |               |
   |               generic parameter used in this inner constant item
   |
   = note: nested items are independent from their parent item for everything except for
privacy and name resolution
   = note: a `const` is a separate item from the item that contains it
help: try using a local `let` binding instead
   |
16 -         const PARTS: &[&str] = &["SELECT count(*) FROM ", <Page<T> as Named>::TABLE];
16 +         let PARTS: &[&str] = &["SELECT count(*) FROM ", <Page<T> as Named>::TABLE];
   |
```

The help does not help: a `let` binding is not a constant, so it cannot size the array. A `static` in a generic function has the same limit, so moving the statement into a `LazyLock` would not help either. That is one more reason `Table` refuses type parameters (§6.2), and the reason Chapter 8's `Insert` refuses a model that mentions its own.

The other limit is computing. Joining puts finished pieces side by side; it cannot decide what a piece says. Chapter 9's `Patch` writes an `UPDATE` whose key parameters are numbered after its own (`WHERE "id" = $7`, with the 7 depending on how many fields the patch has and how many columns the model's key has), and that numbering is a computation over another type's constants. There, shelf builds the statement once, at first use, in a `LazyLock<String>`: the case where a run-time constant is the right tool.

Side by side — *fetching by primary key*

The task: read the product with a given id, or learn that there is none.

```
// SeaORM (illustration)
mod product {
    use sea_orm::entity::prelude::*;
```

```
    #[sea_orm::model]
    #[derive(Clone, Debug, PartialEq, Eq, DeriveEntityModel)]
    #[sea_orm(table_name = "products")]
    pub struct Model {
        #[sea_orm(primary_key)]
        pub id: i64,
        pub sku: String,
        pub price_cents: i64,
    }

    impl ActiveModelBehavior for ActiveModel {}
}

// SELECT "products"."id", "products"."sku", "products"."price_cents" FROM "products"
//     WHERE "products"."id" = $1 LIMIT $2
let product: Option<product::Model> = product::Entity::find_by_id(7).one(&db).await?;
```

By hand, with Chapter 5's `fetch_opt` and the SQL written out, and with the model, in one program that checks both find the same row:

```
use shelf::{FromRow, Table};

#[derive(Debug, PartialEq, FromRow, Table)]
#[shelf(table = "products")]
struct Product {
    #[shelf(primary_key)]
    id: i64,
    sku: String,
    price_cents: i64,
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (
                 id bigint PRIMARY KEY, sku text NOT NULL, price_cents bigint NOT NULL
             );
             INSERT INTO products VALUES (7, 'MUG-01', 1299);",
        )
        .await?;

    // By hand: the statement written out, the key passed as a parameter.
    let by_hand: Option<Product> = shelf::fetch_opt(
        &client,
        "SELECT id, sku, price_cents FROM products WHERE id = $1",
        &[&7_i64],
    )
    .await?;

    // shelf: the derive wrote the statement.
    let with_shelf = Product::find_opt(&client, &7).await?;

    assert_eq!(by_hand, with_shelf);
    assert_eq!(with_shelf.map(|p| p.sku).as_deref(), Some("MUG-01"));
    assert!(Product::find_opt(&client, &8).await?.is_none());
    Ok(())
}
```

	by hand	shelf	SeaORM
Same	One `SELECT` with the key as `$1`, the columns listed, the row read into a struct	The same	The same

	by hand	shelf	SeaORM
Different	You write the statement in each function that needs it, and keep its column list in step with the struct	The statement is a constant written at compile time from the struct, which you can print; `find` turns a missing row into `NotFound`, `find_opt` into `None`	The statement is built at run time by sea-query on every call, qualified by table name and with `LIMIT $2` added by `one()`; the model is a struct in a module with its entity, and `find_by_id` takes anything that converts into the key

SeaORM builds its SQL from a description of the entity at run time, which is what lets its query builder compose anything; shelf builds the fixed statements once, at compile time, and leaves composition to Chapter 10's builder. Neither ever puts a value in the SQL.

In shelf

The reads are provided methods of `Table`, written once in shelf for every model:

```
// orm/shelf/src/table.rs (excerpt)
    /// The row whose key is `key`, or [`Error::NotFound`].
    fn find(
        db: &impl Executor,
        key: &Self::Key,
    ) -> impl Future<Output = Result<Self, Error>> + Send {
        async move {
            Self::find_opt(db, key)
                .await?
                .ok_or(Error::NotFound { model: Self::MODEL })
        }
    }

    /// The row whose key is `key`, if there is one.
    fn find_opt(
        db: &impl Executor,
        key: &Self::Key,
    ) -> impl Future<Output = Result<Option<Self>, Error>> + Send {
        async move { fetch_opt(db, Self::FIND, &Self::key_params(key)).await }
    }

    /// Every row, ordered by the key.
    fn all(db: &impl Executor) -> impl Future<Output = Result<Vec<Self>, Error>> + Send {
        async move { fetch_all(db, Self::ALL, &[]).await }
    }

    // …

    /// The number of rows.
    fn count(db: &impl Executor) -> impl Future<Output = Result<i64, Error>> + Send {
        async move {
            let statement = db.prepare(Self::COUNT).await?;
            Ok(db.query_one(&statement, &[]).await?.try_get(0)?)
        }
    }
```

orm/shelf/src/table.rs

Two details. The methods take `&impl Executor`, shelf's own trait for "something that can run a statement", which a `Client`, a `Transaction` and a deadpool connection all implement; and they return `impl Future<Output = …> + Send` with an `async move` block inside, rather than being `async fn`s. Chapter 7 is about both. And `key_params` is generated: for a single key it is one parameter, `param(key)`; for a composite key, one per element of the tuple, in the order of the key's columns, which is the order of the `$n` in `FIND`.

When not to use `Table`

- **Reports, joins and aggregates.** A result that is not one table's rows has no key to find by. Write the SQL and read it with a `FromRow` struct and `fetch_all`, as in Chapters 1–5.
- **A table with no key.** A log table without a primary key can still be read with `FromRow`; `Table` needs a column, or columns, that pick out one row.
- **A view.** `find`, `find_opt`, `all` and `count` are `SELECT`s, so they work on a view if you mark a column `primary_key`. PostgreSQL does not check that the column identifies one row, so make sure it does; and Chapter 8's writes reach the table only through a view PostgreSQL can update.

Summary

- A model is a `FromRow` struct that also knows its table, its key and its column list. `#[derive(Table)]` takes `table` (and `schema`) on the struct, `primary_key` on one or more fields, and `generated` on the columns the database fills.
- A composite key is a tuple of the key fields' types, in field order, passed as `&(a, b)` of owned values.
- The derive writes every statement at macro time with `format!`, quoting every identifier, and emits it as a string literal constant: `TABLE`, `COLUMNS`, `FIND`, `ALL`, `COUNT`. Values are always `$n` parameters.
- `find` (`Error::NotFound` when no row has the key), `find_opt`, `all` (ordered by the key) and `count` are provided methods of `Table`, built on Chapter 5's `fetch_*`.
- shelf never writes `SELECT *`: a cached `SELECT *` fails with 0A000 after a column is added, and `*` sends columns the struct does not read.
- `Table` refuses a struct without a table or a key, type parameters, and `flatten`, each in one build, at the token that causes it.
- `concat!` takes literals only. `const fn`s can join another type's constants into a constant statement, with inner `const` items to size the array; that cannot be done in a generic impl (E0401), and computing a statement, rather than joining one, needs a `LazyLock`.

Exercises

1. **`exists`.** Write `exists(&db, &key) -> Result<bool, shelf::Error>` for any model, without reading the row. *Hint:* `SELECT EXISTS (SELECT 1 FROM … WHERE …)`. A trait with a provided method and a blanket impl for every `T: Table` adds it to every model; `TABLE` and `KEY_COLUMNS` are public constants.
2. **Several keys, one statement.** Write `find_many(&db, &[i64])` that returns every product whose key is in the slice, in one round trip, for a model with a single-column key. *Hint:* `= ANY($1)` with an array parameter (main book §3.8). Chapter 11's relation loaders are built on the same statement.
3. **Why `all` orders.** Copy `Product::ALL` without its `ORDER BY`, insert three rows, update the first, and read them all again. In what order do they come back? *Hint:* an `UPDATE` in PostgreSQL writes a new version of the row; a table scan returns rows in the order it finds them on the page.

Solutions

1. The statement can be written at run time from the model's constants: they are quoted identifiers, not values, so formatting them in is safe. It is built on every call here; a real version would cache it per model, as Chapter 9's `Patch` does with a `LazyLock`.

```
use shelf::{Executor, FromRow, Table};

/// `exists` for every model.
trait Exists: Table {
    async fn exists(db: &impl Executor, key: &Self::Key) -> Result<bool, shelf::Error> {
        let key_is: Vec<String> = Self::KEY_COLUMNS
```

```
            .iter()
            .enumerate()
            .map(|(i, column)| format!("{column} = ${}", i + 1))
            .collect();
        let sql = format!(
            "SELECT EXISTS (SELECT 1 FROM {} WHERE {})",
            Self::TABLE,
            key_is.join(" AND ")
        );
        let statement = db.prepare(&sql).await?;
        let row = db.query_one(&statement, &Self::key_params(key)).await?;
        Ok(row.try_get(0)?)
    }
}

impl<T: Table> Exists for T {}

#[derive(Debug, FromRow, Table)]
#[shelf(table = "stock")]
#[allow(dead_code)] // the program only asks whether rows exist
struct Stock {
    #[shelf(primary_key)]
    store_id: i64,
    #[shelf(primary_key)]
    product_id: i64,
    on_hand: i32,
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE stock (
                 store_id bigint, product_id bigint, on_hand integer NOT NULL,
                 PRIMARY KEY (store_id, product_id)
             );
             INSERT INTO stock VALUES (1, 1, 10);",
        )
        .await?;
    assert!(Stock::exists(&client, &(1, 1)).await?);
    assert!(!Stock::exists(&client, &(1, 2)).await?);
    Ok(())
}
```

The method is an `async fn` in a trait. That is fine for a program that awaits it directly; a caller that needs the future to be `Send`, such as an axum handler generic over the executor, would need the `impl Future + Send` form shelf uses (Chapter 7).

2. One statement with the keys as one array parameter:

```
use shelf::{Executor, FromRow, Table};

#[derive(Debug, FromRow, Table)]
#[shelf(table = "products")]
struct Product {
    #[shelf(primary_key)]
    id: i64,
    sku: String,
}

/// Every product whose id is in `ids`, in one round trip, ordered by id.
async fn find_many(db: &impl Executor, ids: &[i64]) -> Result<Vec<Product>, shelf::Error> {
    let [key] = Product::KEY_COLUMNS else {
        unreachable!("Product has a single-column key");
    };
    let sql = format!(
```

```
        "SELECT {} FROM {} WHERE {key} = ANY($1) ORDER BY {key}",
        Product::COLUMNS,
        Product::TABLE
    );
    shelf::fetch_all(db, &sql, &[&ids]).await
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (id bigint PRIMARY KEY, sku text NOT NULL);
             INSERT INTO products VALUES (1, 'MUG-01'), (2, 'TEE-02'), (3, 'VASE-03');",
        )
        .await?;
    let found = find_many(&client, &[3, 1, 42]).await?;
    let skus: Vec<&str> = found.iter().map(|p| p.sku.as_str()).collect();
    assert_eq!(skus, ["MUG-01", "VASE-03"]); // 42 has no row
    Ok(())
}
```

A key with no row is simply absent from the result; whether that is an error is the caller's decision. For a composite key the condition becomes a row comparison against two arrays, `("store_id", "product_id") IN (SELECT * FROM unnest($1, $2))`, which is why shelf's relation loaders support single-column keys only (Chapter 11).

3. The updated row comes back last:

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (id bigint PRIMARY KEY, sku text NOT NULL);
             INSERT INTO products VALUES (1, 'MUG-01'), (2, 'TEE-02'), (3, 'VASE-03');",
        )
        .await?;
    let unordered = "SELECT id FROM products";
    let ids = |rows: Vec<tokio_postgres::Row>| -> Vec<i64> {
        rows.iter().map(|row| row.get(0)).collect()
    };

    assert_eq!(ids(client.query(unordered, &[]).await?), [1, 2, 3]);
    client
        .execute("UPDATE products SET sku = 'MUG-01B' WHERE id = 1", &[])
        .await?;
    assert_eq!(ids(client.query(unordered, &[]).await?), [2, 3, 1]);

    // With the key's ORDER BY, as Product::ALL has it, the order does not move.
    let ordered = "SELECT id FROM products ORDER BY id";
    assert_eq!(ids(client.query(ordered, &[]).await?), [1, 2, 3]);
    Ok(())
}
```

PostgreSQL never updates a row in place: the `UPDATE` wrote a new version of row 1 after the others on the page, and the sequential scan found it there. Nothing in SQL promises any order without `ORDER BY`, so even this one is an accident of the storage, not a rule; a page that fills up, a `VACUUM` or an index scan gives another. A model's `all` orders by the key so that the order means something.

CHAPTER 7

The executor and shelf's errors

Chapter 1's fetch helpers took tokio-postgres's `GenericClient`, which a `Client` and a `Transaction` both implement. A service holds neither: it holds a pooled deadpool connection, which reaches a `Client` only through two dereferences, and loses on the way the one thing a model layer wants most from it, the connection's statement cache. deadpool has a `GenericClient` of its own, with that cache, but it covers deadpool's two types and not tokio-postgres's. Both traits are sealed, so shelf can implement neither for anything, and code generic over one does not accept the other.

The way out is a trait of shelf's own, with only the calls shelf needs, implemented for the four types a program actually has. Choosing what goes in it means choosing how each statement reaches the server, which is a question of round trips the main book taught you to measure. This chapter writes the trait, measures three ways to send a statement through lagproxy, runs shelf inside transactions and the main book's retry helper, and gives shelf an error type that tells an application what it can do about each failure.

In this chapter you will learn:

- why neither `GenericClient` will do, and what a trait of shelf's own must contain
- how to write async methods in a trait whose futures are `Send`, and why that matters to `tokio::spawn` and axum
- three ways to send a statement (`prepare`, `prepare_cached`, `query_typed`) and their round trips, measured
- how shelf calls run inside a transaction, and inside the main book's retry helper
- `shelf::Error`, and how an application maps it to its own errors and HTTP statuses

7.1 Why not `GenericClient`

Two traits already abstract over "something that can run a statement":

Trait	Implemented for	`prepare_cached`	Can you implement it?
`tokio_postgres::GenericClient` (0.7.18)	`Client`, `Transaction<'_>`	no	no: sealed
`deadpool_postgres::GenericClient` (0.14.2)	`deadpool_postgres::Client` (a pooled `Object`), deadpool's `Transaction<'_>`	yes	no: sealed

Code written against the first runs inside or outside a transaction (main book §8.5). A pooled connection derefs, in two steps, to a `Client`, so `&**object` passes too, and every statement is then prepared on each call as if there were no cache. Code written against the second uses the cache, and refuses a plain `Client`. Neither can be extended to cover the other, because both are *sealed*: each has a supertrait that lives in a private module of its crate, so no other crate can implement it. Trying it on a wrapper of your own:

```
use tokio_postgres::{Client, GenericClient};

/// A client that logs what it sends.
```

```
struct Logged(Client);

impl GenericClient for Logged {}
```

```
error[E0277]: the trait bound `Logged: tokio_postgres::generic_client::private::Sealed` is
not satisfied
   --> user/src/bin/sealed.rs:6:24
    |
  6 | impl GenericClient for Logged {}
    |                        ^^^^^^ unsatisfied trait bound
    |
help: the trait `tokio_postgres::generic_client::private::Sealed` is not implemented for
`Logged`
   --> user/src/bin/sealed.rs:4:1
    |
  4 | struct Logged(Client);
    | ^^^^^^^^^^^^^^
help: the following other types implement trait
`tokio_postgres::generic_client::private::Sealed`
   --> /Users/luciano/.cargo/registry/src/index.crates.io-1949cf8c6b5b557f/tokio-
postgres-0.7.18/src/generic_client.rs:116:1
    |
116 | impl private::Sealed for Client {}
    | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ `Client`
...
246 | impl private::Sealed for Transaction<'_> {}
    | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ `Transaction<'_>`
note: required by a bound in `GenericClient`
   --> /Users/luciano/.cargo/registry/src/index.crates.io-1949cf8c6b5b557f/tokio-
postgres-0.7.18/src/generic_client.rs:14:26
    |
 14 | pub trait GenericClient: private::Sealed {
    |                          ^^^^^^^^^^^^^^^ required by this bound in `GenericClient`
    = note: `GenericClient` is a "sealed trait", because to implement it you also need to
implement `tokio_postgres::generic_client::private::Sealed`, which is not accessible; this
is usually done to force you to use one of the provided types that already implement it
    = help: the following types implement the trait:
              tokio_postgres::Client
              tokio_postgres::Transaction<'_>
```

(The same build also lists, as E0046, the seventeen methods the impl is missing, from `execute` to `client`.) Sealing is a deliberate choice, and a reasonable one: a crate can add methods to a sealed trait in a minor release without breaking anyone, and tokio-postgres did, adding `query_typed` in 0.7.11 and `execute_typed`, `query_typed_one` and `query_typed_opt` in 0.7.17. The price is paid by crates like shelf, which need one trait for all four types.

So shelf defines its own. It needs few methods: something that prepares (from a cache when there is one), and the four ways to run a prepared statement it uses. Everything else a connection can do (`batch_execute`, `simple_query`, starting a transaction, `LISTEN`) stays on the connection, where the program already has it.

7.2 The `Executor` trait

The interesting part of the trait is not which methods it has but how they are declared. Here is a stage version with two of shelf's methods, implemented for a plain `Client` and for a pooled connection, and called by a generic repository function on tasks of its own:

```
use std::future::Future;
use std::sync::Arc;
use tokio_postgres::types::ToSql;
use tokio_postgres::{Error, Row, Statement};

/// The stage executor: two of shelf's methods.
```

```
trait Executor: Sync {
    fn prepare(&self, sql: &str) -> impl Future<Output = Result<Statement, Error>> + Send;

    fn query(
        &self,
        statement: &Statement,
        params: &[&(dyn ToSql + Sync)],
    ) -> impl Future<Output = Result<Vec<Row>, Error>> + Send;
}

impl Executor for tokio_postgres::Client {
    fn prepare(&self, sql: &str) -> impl Future<Output = Result<Statement, Error>> + Send {
        tokio_postgres::Client::prepare(self, sql) // every time
    }

    fn query(
        &self,
        statement: &Statement,
        params: &[&(dyn ToSql + Sync)],
    ) -> impl Future<Output = Result<Vec<Row>, Error>> + Send {
        tokio_postgres::Client::query(self, statement, params)
    }
}

impl Executor for deadpool_postgres::Client {
    fn prepare(&self, sql: &str) -> impl Future<Output = Result<Statement, Error>> + Send {
        self.prepare_cached(sql) // from the connection's cache after the first time
    }

    fn query(
        &self,
        statement: &Statement,
        params: &[&(dyn ToSql + Sync)],
    ) -> impl Future<Output = Result<Vec<Row>, Error>> + Send {
        tokio_postgres::Client::query(self, statement, params) // `self` derefs to a Client
    }
}

/// A repository function, generic over the executor.
async fn count(db: &impl Executor) -> Result<i64, Error> {
    let statement = db
        .prepare("SELECT count(*) FROM generate_series(1, 3)")
        .await?;
    db.query(&statement, &[]).await?[0].try_get(0)
}

/// Runs `count` on a task of its own: its future must be `Send` for every `E`.
fn spawn_count<E: Executor + Send + 'static>(
    db: Arc<E>,
) -> tokio::task::JoinHandle<Result<i64, Error>> {
    tokio::spawn(async move { count(&*db).await })
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let client = Arc::new(shelf_playground::connect().await?);
    assert_eq!(spawn_count(client).await??, 3);

    let pool = shelf_playground::pool(1);
    let object = Arc::new(pool.get().await?);
    for _ in 0..3 {
        assert_eq!(spawn_count(object.clone()).await??, 3);
    }
    assert_eq!(object.statement_cache.size(), 1, "prepared once");
    Ok(())
}
```

`spawn_count` is the case that matters. An axum handler is the same case: axum requires a handler's future to be `Send`, so that the runtime can move it between threads, and a repository function generic over its

executor is only as `Send` as the futures its executor returns. The trait says they are, in so many words: `-> impl Future<Output = …> + Send`. Written the natural way instead, as `async fn prepare(&self, sql: &str) -> Result<Statement, Error>`, the trait makes no such promise, and the same `spawn_count` fails to compile:

```
error[E0277]: `impl Future<Output = Result<Statement, tokio_postgres::Error>>` cannot be
sent between threads safely
   --> user/src/bin/async_fn.rs:29:5
    |
 29 |     tokio::spawn(async move { count(&*db).await });
    |     ^^^^^^^^^^^^^----------^^^^^^^^^^^^^^^^^^^^^^
    |     |            |
    |     |            within this `{async block@user/src/bin/async_fn.rs:29:18: 29:28}`
    |     `impl Future<Output = Result<Statement, tokio_postgres::Error>>` cannot be sent
between threads safely
    |
    = help: within `{async block@user/src/bin/async_fn.rs:29:18: 29:28}`, the trait `Send`
is not implemented for `impl Future<Output = Result<Statement, tokio_postgres::Error>>`
note: required because it's used within this `async` fn body
   --> user/src/bin/async_fn.rs:22:58
    |
 22 |   async fn count(db: &impl Executor) -> Result<i64, Error> {
    |  __________________________________________________________^
 23 | |     let statement = db.prepare("SELECT count(*) FROM products").await?;
 24 | |     Ok(db.query(&statement, &[]).await?[0].get(0))
 25 | | }
    | |_^
note: required because it's used within this `async` block
   --> user/src/bin/async_fn.rs:29:18
    |
 29 |     tokio::spawn(async move { count(&*db).await });
    |                  ^^^^^^^^^^
note: required by a bound in `tokio::spawn`
   --> /Users/luciano/.cargo/registry/src/index.crates.io-1949cf8c6b5b557f/tokio-1.54.1/src/
task/spawn.rs:176:21
    |
174 |     pub fn spawn<F>(future: F) -> JoinHandle<F::Output>
    |            ----- required by a bound in this function
175 |     where
176 |         F: Future + Send + 'static,
    |                     ^^^^ required by this bound in `spawn`
help: `Send` can be made part of the associated future's guarantees for all implementations
of `Executor::prepare`
    |
  6 -     async fn prepare(&self, sql: &str) -> Result<Statement, Error>;
  6 +     fn prepare(&self, sql: &str) -> impl std::future::Future<Output =
Result<Statement, Error>> + Send;
    |
```

(A second, identical error follows for `query`.) rustc's own help proposes the form shelf uses.

> **Rust refresher:** `async fn` in a trait (stable since Rust 1.75) is shorthand for a method returning `impl Future<Output = …>`, and that future's type is chosen by each impl. When the caller knows the concrete type (`count(&client)` with a `Client`), the compiler sees the impl's future and knows whether it is `Send`. When the caller is generic (`E: Executor`), it knows only what the trait declares, and an `async fn` declares nothing about `Send`. Writing the return type out, `-> impl Future<Output = …> + Send`, puts the promise in the trait: every impl must return a `Send` future, and every generic caller may rely on it. An impl may still be written as an `async fn` (Exercise 2's is), or return another `Send` future, as these return tokio-postgres's; either way the compiler checks that impl's future against the trait's `Send`. Before Rust 1.75 the usual answer was `#[async_trait]`, which boxes every future; tokio-postgres's and deadpool's `GenericClients` still use it.

shelf's trait is the stage's with the rest of its methods. `prepare_uncached` has a default and exists for Chapter 13's `verify`; `query_raw` and `copy_in` serve Chapter 12's streams and COPY:

```
// orm/shelf/src/executor.rs (excerpt)
pub trait Executor: Sync {
    /// A prepared statement for `sql`, from the cache when the executor has one.
    fn prepare(&self, sql: &str) -> impl Future<Output = Result<Statement, Error>> + Send;

    /// A prepared statement for `sql`, never from a cache: the server's description now.
    /// [`verify`](crate::verify) needs it, since a cached statement describes a table as it
was
    /// when it was prepared. By default, [`prepare`](Executor::prepare).
    fn prepare_uncached(&self, sql: &str) -> impl Future<Output = Result<Statement, Error>>
+ Send {
        self.prepare(sql)
    }

    /// Every row `statement` returns.
    fn query(
        &self,
        statement: &Statement,
        params: &[&(dyn ToSql + Sync)],
    ) -> impl Future<Output = Result<Vec<Row>, Error>> + Send;

    // … query_one, query_opt, execute: the same shape

    /// Starts `statement` and returns its rows as they arrive, for streams. The `RowStream`
does
    /// not borrow the connection, which serves no one else until the stream is finished or
    /// dropped.
    fn query_raw(
        &self,
        statement: &Statement,
        params: &[&(dyn ToSql + Sync)],
    ) -> impl Future<Output = Result<RowStream, Error>> + Send;

    // … copy_in (Chapter 12)
}
```

orm/shelf/src/executor.rs

The four impls are one `macro_rules!` macro, Chapter 2's tool for impls that differ in a few tokens. Each forwards to the target type's own method, except `prepare`, which is the argument that differs:

```
// orm/shelf/src/executor.rs (excerpt)
/// Implements [`Executor`] for `$ty` by calling `$target`'s methods (which `$ty` derefs
to),
/// with `$prepare` for [`Executor::prepare`].
macro_rules! executor {
    ($ty:ty, $target:ty, $prepare:ident) => {
        impl Executor for $ty {
            fn prepare(&self, sql: &str) -> impl Future<Output = Result<Statement, Error>> +
Send {
                self.$prepare(sql)
            }

            fn prepare_uncached(
                &self,
                sql: &str,
            ) -> impl Future<Output = Result<Statement, Error>> + Send {
                <$target>::prepare(self, sql)
            }

            fn query(
                &self,
                statement: &Statement,
                params: &[&(dyn ToSql + Sync)],
            ) -> impl Future<Output = Result<Vec<Row>, Error>> + Send {
```

```
                <$target>::query(self, statement, params)
            }

            // …
        }
    };
}

executor!(tokio_postgres::Client, tokio_postgres::Client, prepare);
executor!(
    tokio_postgres::Transaction<'_>,
    tokio_postgres::Transaction<'_>,
    prepare
);
#[cfg(feature = "deadpool")]
executor!(
    deadpool_postgres::Client,
    tokio_postgres::Client,
    prepare_cached
);
#[cfg(feature = "deadpool")]
executor!(
    deadpool_postgres::Transaction<'_>,
    tokio_postgres::Transaction<'_>,
    prepare_cached
);
```

orm/shelf/src/executor.rs

The deadpool impls sit behind the default feature `deadpool`, so a program without a pool does not compile deadpool. `prepare_uncached` names the target type's `prepare` explicitly, `<$target>::prepare(self, sql)`: on a deadpool connection, `self.prepare(sql)` would find the trait's own `prepare` first and use the cache.

`&object`, not `&**object`

A pooled connection is three types deep. `deadpool_postgres::Client` is deadpool's `Object`, which derefs to deadpool's `ClientWrapper` (where the statement cache lives), which derefs to tokio-postgres's `Client`. shelf implements `Executor` for the outermost and the innermost:

You pass	Type	What shelf does
`&object`	`&deadpool_postgres::Client`	prepares through the connection's cache
`&*object`	`&ClientWrapper`	does not compile: no impl
`&**object`	`&tokio_postgres::Client`	prepares on every call, as if there were no pool

The middle one is refused, with the four types that do implement the trait listed in the error:

```
error[E0277]: the trait bound `ClientWrapper: Executor` is not satisfied
   --> user/src/bin/deref.rs:5:32
    |
  5 |     shelf::fetch_all::<(i64,)>(&*object, "SELECT 1::int8", &[]).await?;
    |     ------------------------- ^^^^^^^^ the trait `Executor` is not implemented for
`ClientWrapper`
    |     |
    |     required by a bound introduced by this call
```

The last one compiles and works, which is the trap: nothing reports the lost cache. Chapter 1's helpers needed `&**object`, because they took tokio-postgres's `GenericClient`; with shelf, pass the pooled connection itself.

The trait is not sealed. A wrapper that counts or times what passes through it implements it in a few lines per method (Exercise 1), and Chapter 11 counts round trips with one.

7.3 Three ways to send a statement

A statement reaches PostgreSQL in one of three ways, and the difference is round trips (main book §4.1 to §4.6):

- **Prepare, then run.** *Parse* and *Describe*, then wait for the description; then *Bind* and *Execute*. Two round trips on every call. It is also what `client.query(sql_text, …)` costs, since tokio-postgres prepares SQL text the same way. The `Statement` is closed when it is dropped.
- **Prepare once per connection, from a cache.** deadpool's `prepare_cached` prepares on the first call and keeps the `Statement`; every later call on that connection is one round trip. The statement stays open on the session for as long as the connection lives.
- **`query_typed`** (tokio-postgres 0.7.11+). *Parse*, *Bind* and *Execute* in one go, with an unnamed statement: one round trip, nothing left behind. The price is that the caller must give each parameter's PostgreSQL `Type`, and that no `Statement` exists before the query runs.

This program times each through lagproxy, which adds 5 ms each way, and counts the statements each leaves open on the session. All three use the same connection, so they read the same temporary table: `&**object` is the plain `Client` inside the pooled one.

```
use shelf::{FromRow, Table};
use std::time::{Duration, Instant};
use tokio_postgres::types::Type;

#[derive(Debug, FromRow, Table)]
#[shelf(table = "products")]
struct Product {
    #[shelf(primary_key)]
    id: i64,
    sku: String,
}

const CALLS: u32 = 20;

/// The named statements open on this session. `query_typed` sends an unnamed one, which the
/// view does not list, so the count does not count itself.
async fn open_statements(client: &tokio_postgres::Client) -> Result<i64,
tokio_postgres::Error> {
    client
        .query_typed_one("SELECT count(*) FROM pg_prepared_statements", &[])
        .await?
        .try_get(0)
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    // Through lagproxy: every round trip costs at least twice the delay.
    let delay: u64 = std::env::var("LAGPROXY_DELAY_MS").map_or(5, |v| v.parse().unwrap());
    let pool = shelf_playground::pool(1);
    let object = pool.get().await?;
    let client: &tokio_postgres::Client = &object; // the same session, without the cache
    client
        .batch_execute(
            "CREATE TEMP TABLE products (id bigint PRIMARY KEY, sku text NOT NULL);
             INSERT INTO products VALUES (1, 'MUG-01');",
        )
        .await?;

    // 1. A plain Client: prepare, then run, on every call.
    let started = Instant::now();
    for _ in 0..CALLS {
        Product::find(client, &1).await?;
    }
    let prepared = started.elapsed() / CALLS;
    let open_after_prepared = open_statements(client).await?;
```

```
    // 2. The pooled connection: prepared on the first call, from the cache after.
    let started = Instant::now();
    Product::find(&object, &1).await?;
    let first_cached = started.elapsed();
    let started = Instant::now();
    for _ in 0..CALLS {
        Product::find(&object, &1).await?;
    }
    let cached = started.elapsed() / CALLS;
    let open_after_cached = open_statements(client).await?;

    // 3. query_typed: one round trip, with the parameter's type stated by hand.
    let started = Instant::now();
    for _ in 0..CALLS {
        let row = client
            .query_typed_one(Product::FIND, &[(&1_i64, Type::INT8)])
            .await?;
        Product::from_row(&row)?;
    }
    let typed = started.elapsed() / CALLS;

    println!("prepare, then run   {prepared:>8.1?} a call, {open_after_prepared} left
open");
    println!(
        "prepare_cached      {cached:>8.1?} a call (the first {first_cached:.1?}),
{open_after_cached} left open"
    );
    println!("query_typed         {typed:>8.1?} a call");

    let round_trip = Duration::from_millis(2 * delay);
    assert!(cached >= round_trip, "at least one round trip");
    assert!(prepared > cached * 3 / 2, "two round trips against one");
    assert!(prepared > typed * 3 / 2, "two round trips against one");
    assert!(first_cached > cached * 3 / 2, "the first call prepares");
    assert_eq!(
        open_after_prepared, 0,
        "each Statement is closed when dropped"
    );
    assert_eq!(open_after_cached, 1, "the cache keeps one");
    Ok(())
}
```

```
prepare, then run     27.9ms a call, 0 left open
prepare_cached        14.2ms a call (the first 27.2ms), 1 left open
query_typed           13.9ms a call
```

Three runs in a debug build and three in a release build (20 calls each, lagproxy at 5 ms each way, PostgreSQL 18.6 on the same machine, Apple Silicon) gave the same picture, because the time is the network's:

Strategy	Round trips	Per call, debug	Per call, release	Left open on the session
prepare, then run (a plain `Client`)	2	27.0–27.9 ms	27.5–27.6 ms	none
`prepare_cached`, first call	2	27.2–28.8 ms	26.4–28.0 ms	one statement
`prepare_cached`, after	1	13.7–14.2 ms	13.6 ms	one statement
`query_typed`	1	13.6–13.9 ms	13.7–13.9 ms	none

Prepare-then-run costs two round trips, a cached statement and `query_typed` one each, and a cached statement's first call costs two. Each `Statement` that `prepare` returns is closed when it is dropped, at the end of the call, so a plain `Client` leaves nothing behind; the cache keeps one statement per distinct SQL text, per connection, for the connection's life.

`query_typed` is as fast as the cache without a cache. shelf still does not use it, for two reasons. It needs each parameter's PostgreSQL `Type`, and a Rust type does not say which: an `i64` may be bound to an `int8` column, a `String` to `text`, `varchar` or a domain over either, and only the server knows which the statement needs. And it gives no `Statement` before the query runs, so Chapter 5's check, which resolves the struct against the statement's columns before anything executes, would come after the fact: an `INSERT … RETURNING` that does not fit its struct would already have inserted. So each executor gets the strategy that keeps the check: a plain `Client` or `Transaction` prepares on every call, which costs nothing that `query` with SQL text would not; a deadpool connection or transaction uses `prepare_cached`. For a service, that means one round trip per call after warm-up.

> **Protocol lens:** in tokio-postgres 0.7.18, `prepare` sends *Parse* (with a name from a process-wide counter: `s0`, `s1` and so on), *Describe* of the statement and *Sync*, and waits for *ParseComplete*, *ParameterDescription* and *RowDescription*: the column list `resolve` reads. Running it sends *Bind*, *Execute* and *Sync*. Dropping the last clone of a `Statement` queues *Close* and *Sync* for its name on the connection's request channel, so they go out before the next request. `query_typed` writes *Parse* with the empty name and the types' OIDs, *Bind*, *Describe* of that unnamed statement, *Execute* and *Sync* at once, and the server answers all of it in one go; an unnamed statement lasts until the next *Parse*, needs no *Close*, and `pg_prepared_statements` does not list it.

7.4 Transactions

`Transaction` is an executor too, so every shelf call that takes `&impl Executor` runs inside a transaction when it is given one. It then sees the transaction's own writes, and nobody else does until the commit:

```
use shelf::{FromRow, Table};

#[derive(Debug, FromRow, Table)]
#[shelf(table = "products", schema = "shelf_ch07_tx")]
struct Product {
    #[shelf(primary_key)]
    id: i64,
    sku: String,
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let mut client = shelf_playground::connect().await?;
    let other = shelf_playground::connect().await?;
    // A schema both connections see, dropped at the end.
    client
        .batch_execute(
            "DROP SCHEMA IF EXISTS shelf_ch07_tx CASCADE;
             CREATE SCHEMA shelf_ch07_tx;
             CREATE TABLE shelf_ch07_tx.products (id bigint PRIMARY KEY, sku text NOT
NULL);",
        )
        .await?;

    let tx = client.transaction().await?;
    tx.execute(
        "INSERT INTO shelf_ch07_tx.products VALUES (1, 'MUG-01')",
        &[],
    )
    .await?;
    // Inside the transaction, shelf sees the new row; outside, it does not exist yet.
    assert_eq!(Product::find(&tx, &1).await?.sku, "MUG-01");
    assert!(Product::find_opt(&other, &1).await?.is_none());
    tx.commit().await?;
```

```
    assert_eq!(Product::count(&other).await?, 1);

    client
        .batch_execute("DROP SCHEMA shelf_ch07_tx CASCADE")
        .await?;
    Ok(())
}
```

The `Executor` has no method that starts a transaction. `Client::transaction` takes `&mut self` (main book §8.2), and an executor is a shared borrow, which is what lets several shelf calls run on one connection at once (Exercise 3). So the program starts the transaction on the connection it owns, and hands `&tx` to everything inside.

The retry helper

At `SERIALIZABLE`, a transaction can fail because of another one, with SQLSTATE 40001, and the cure is to run it again from `BEGIN` (main book §8.6). The main book wrote a helper that does that, taking the transaction's body as an async closure. It works with shelf calls inside, with one change: the body now returns `shelf::Error`, so the helper finds the SQLSTATE inside `Error::Db`. This is the main book's two doctors, of whom one must stay on call, with the reads done by a model:

```
use shelf::{FromRow, Table};
use std::time::Duration;
use tokio_postgres::error::SqlState;
use tokio_postgres::{Client, IsolationLevel, Transaction};

#[derive(Debug, FromRow, Table)]
#[shelf(table = "doctors", schema = "shelf_ch07_retry")]
struct Doctor {
    #[shelf(primary_key)]
    name: String,
    on_call: bool,
}

/// Runs `work` in a SERIALIZABLE transaction, again from the start after a serialization
/// failure or a deadlock, up to 10 times: the main book's helper, with shelf's error.
async fn serializable<T>(
    client: &mut Client,
    mut work: impl AsyncFnMut(&Transaction<'_>) -> Result<T, shelf::Error>,
) -> Result<T, shelf::Error> {
    let mut attempt: u32 = 1;
    loop {
        let tx = client
            .build_transaction()
            .isolation_level(IsolationLevel::Serializable)
            .start()
            .await?;
        let outcome = match work(&tx).await {
            Ok(value) => tx
                .commit()
                .await
                .map(|()| value)
                .map_err(shelf::Error::from),
            Err(error) => Err(error), // `tx` is dropped here: rolled back
        };
        match outcome {
            Err(error) if attempt < 10 && retryable(&error) => {
                tokio::time::sleep(Duration::from_millis(5 * u64::from(attempt))).await;
                attempt += 1;
            }
            other => return other,
        }
    }
}
```

```
/// A serialization failure or a deadlock: shelf passes the database's errors on as `Db`.
fn retryable(error: &shelf::Error) -> bool {
    let shelf::Error::Db(error) = error else {
        return false;
    };
    matches!(
        error.code(),
        Some(&SqlState::T_R_SERIALIZATION_FAILURE | &SqlState::T_R_DEADLOCK_DETECTED)
    )
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let setup = shelf_playground::connect().await?;
    setup
        .batch_execute(
            "DROP SCHEMA IF EXISTS shelf_ch07_retry CASCADE; CREATE SCHEMA shelf_ch07_retry;
             CREATE TABLE shelf_ch07_retry.doctors (name text PRIMARY KEY, on_call bool NOT
NULL);
             INSERT INTO shelf_ch07_retry.doctors VALUES ('alice', true), ('bob', true);",
        )
        .await?;

    // Both doctors try to leave at once, each on a task of its own.
    let mut tasks = Vec::new();
    for name in ["alice", "bob"] {
        let name = name.to_owned(); // owned: the closure must own what it uses
        tasks.push(tokio::spawn(async move {
            let mut client = shelf_playground::connect().await?;
            serializable(&mut client, async move |tx| {
                let on_call = Doctor::all(tx).await?.iter().filter(|d| d.on_call).count();
                if on_call < 2 {
                    return Ok(false); // someone must stay
                }
                tokio::time::sleep(Duration::from_millis(20)).await; // a slow decision
                tx.execute(
                    "UPDATE shelf_ch07_retry.doctors SET on_call = false WHERE name = $1",
                    &[&name],
                )
                .await?;
                Ok(true)
            })
            .await
        }));
    }
    let mut left = 0;
    for task in tasks {
        if task.await.expect("no panic")? {
            left += 1;
        }
    }
    assert_eq!(left, 1, "exactly one doctor left");
    let still = Doctor::all(&setup)
        .await?
        .iter()
        .filter(|d| d.on_call)
        .count();
    assert_eq!(still, 1);

    setup
        .batch_execute("DROP SCHEMA shelf_ch07_retry CASCADE")
        .await?;
    Ok(())
}
```

Two things made this compile. `Doctor::all` returns a `Send` future (§7.2), so the body can run inside `tokio::spawn`. And the closure owns `name`, so each attempt's future borrows from the closure rather than from the task; a closure that borrowed the task's locals would fail with "implementation of `Send` is not

general enough", as the main book's §8.6 explains. The `?` on `tx.execute` works because `shelf::Error` has `From<tokio_postgres::Error>`, the `Db` variant.

stockroom's own helper, `db::in_transaction`, takes a closure that returns stockroom's `AppError`, which classifies SQLSTATEs itself. The capstone calls it unchanged, with a body that calls shelf and converts shelf's errors (§7.5):

```
// orm/stockroom-shelf/src/http/orders.rs (excerpt)
pub async fn place(
    Db(mut db): Db,
    AppJson(order): AppJson<NewOrder>,
) -> Result<impl IntoResponse, AppError> {
    // `async move`: the closure owns the request, and each attempt borrows it from there.
    let order = db::in_transaction(&mut db, IsolationLevel::ReadCommitted, async move |tx| {
        orders::place(tx, &order).await
    })
    .await?;
    Ok((StatusCode::CREATED, etag(order.version), Json(order)))
}
```

orm/stockroom-shelf/src/http/orders.rs

There `tx` is deadpool's `Transaction`, which shares its connection's statement cache: a statement prepared through the connection is found again inside the transaction, and the other way round.

7.5 `shelf::Error`

Everything shelf returns fails with one type. Its variants say what went wrong in terms a caller can act on:

Variant	From	What it means	What a caller does
`Mapping { model, field, problem }`	reading (Chapters 4, 5)	the struct does not fit the result	fix the code: a bug, not a request's fault
`NotFound { model }`	`find`, writes by key (Chapter 8)	no row has the key	report it: 404
`Stale { model, current_version }`	a versioned update (Chapter 9)	someone changed the row since it was read	report it with the version: 412, or read again
`Schema { model, mismatches }`	`verify` (Chapter 13)	the model and its table disagree	refuse to start
`Unsupported { model, message }`	the bulk writes (Chapter 12)	a call shelf cannot make for this model	use the other method the message names
`Db(tokio_postgres::Error)`	everything else	the database or the connection	look inside: a SQLSTATE and a constraint name

The enum is `#[non_exhaustive]`, so a `match` on it needs a `_` arm, and shelf can add a variant in a minor release. Its `Display` is meant for logs, not for API clients; the `Db` variant is `#[error(transparent)]`, so it displays and sources as the driver's error does, and `source()` leads to the server's `DbError` (main book §6.2).

What shelf does not do is decide what a constraint violation means. A duplicate SKU is a 409 in one service and a validation message in another, and the knowledge of which constraint guards what lives in the application, keyed by constraint name, as stockroom's `AppError` does (main book §6.5). An application error type with a `From<shelf::Error>` impl lets `?` do the conversion:

```
use shelf::{FromRow, Table};
use std::error::Error as _;
use tokio_postgres::error::SqlState;

#[derive(Debug, FromRow, Table)]
```

```
#[shelf(table = "products")]
struct Product {
    #[shelf(primary_key)]
    id: i64,
    sku: String,
    price_cents: i64,
}

/// What the application's API reports.
#[derive(Debug, thiserror::Error)]
enum AppError {
    #[error("{0} not found")]
    NotFound(&'static str),
    #[error("{0}")]
    Conflict(&'static str),
    #[error("{0}")]
    Invalid(&'static str),
    #[error("changed since it was read; it is now at version {0}")]
    Stale(i64),
    #[error("internal error")]
    Internal(#[source] shelf::Error),
}

impl AppError {
    fn status(&self) -> u16 {
        match self {
            AppError::NotFound(_) => 404,
            AppError::Conflict(_) => 409,
            AppError::Stale(_) => 412,
            AppError::Invalid(_) => 422,
            AppError::Internal(_) => 500,
        }
    }
}

/// Constraint names are a contract between the schema and this function.
fn constraint_message(constraint: &str) -> Option<&'static str> {
    match constraint {
        "products_sku_key" => Some("a product with this SKU already exists"),
        "products_price_cents_check" => Some("a price cannot be negative"),
        _ => None,
    }
}

impl From<shelf::Error> for AppError {
    fn from(error: shelf::Error) -> Self {
        match error {
            shelf::Error::NotFound { model } => AppError::NotFound(model),
            shelf::Error::Stale {
                current_version, ..
            } => AppError::Stale(current_version),
            shelf::Error::Db(db) => {
                let known = db.as_db_error().and_then(|server| {
                    let message = constraint_message(server.constraint()?)?;
                    Some((server.code().clone(), message))
                });
                match known {
                    Some((SqlState::UNIQUE_VIOLATION, message)) =>
AppError::Conflict(message),
                    Some((SqlState::CHECK_VIOLATION, message)) =>
AppError::Invalid(message),
                    _ => AppError::Internal(shelf::Error::Db(db)),
                }
            }
            // A mapping or schema error is a bug in this program, not in the request.
            other => AppError::Internal(other),
        }
    }
}
```

```
/// A repository function: shelf's errors become the application's at the `?`.
async fn create(
    db: &impl shelf::Executor,
    sku: &str,
    price_cents: i64,
) -> Result<Product, AppError> {
    let sql = "INSERT INTO products (sku, price_cents) VALUES ($1, $2)
               RETURNING id, sku, price_cents";
    Ok(shelf::fetch_one(db, sql, &[&sku, &price_cents]).await?)
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (
                 id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
                 sku text NOT NULL UNIQUE,
                 price_cents bigint NOT NULL CHECK (price_cents >= 0)
             );",
        )
        .await?;
    create(&client, "MUG-01", 1299).await?;

    let duplicate = create(&client, "MUG-01", 999).await.unwrap_err();
    let negative = create(&client, "TEE-02", -1).await.unwrap_err();
    let missing = AppError::from(Product::find(&client, &99).await.unwrap_err());
    // A struct that does not fit its query: the client hears 500, the log hears why.
    let mistake = AppError::from(
        shelf::fetch_one::<Product>(&client, "SELECT id, sku FROM products", &[])
            .await
            .unwrap_err(),
    );
    for error in [&duplicate, &negative, &missing, &mistake] {
        println!("{} {error}", error.status());
    }
    assert_eq!(
        [&duplicate, &negative, &missing, &mistake].map(AppError::status),
        [409, 422, 404, 500]
    );

    let mut cause = mistake.source();
    while let Some(error) = cause {
        println!("  caused by: {error}");
        cause = error.source();
    }
    Ok(())
}
```

```
409 a product with this SKU already exists
422 a price cannot be negative
404 Product not found
500 internal error
  caused by: Product.price_cents: the result has no column "price_cents"
  caused by: the result has no column "price_cents"
```

The 500's message is "internal error" and nothing more, because an API client should not learn what a struct is called; the chain under it is for the log. The `Db` arm keeps the original error when it does not recognise the constraint, so the log still has the SQLSTATE.

In stockroom-shelf: the orphan rule

The capstone cannot write that `From` impl. Its `AppError` is stockroom's, unchanged, and `shelf::Error` is shelf's: an impl of a foreign trait (`From`) for a foreign type (`AppError`) in a third crate breaks the orphan rule, and

rustc refuses it with E0117 (PLAN.md §12, finding 49). So the capstone converts with a function, at every call, with `map_err`:

```
// orm/stockroom-shelf/src/error.rs (excerpt)
/// What a client sees when shelf fails: a missing row is 404, a stale version 412, a
database
/// error whatever stockroom makes of it (its SQLSTATE and constraint names), and a mapping
or
/// schema error, which is a bug, 500.
pub fn app_error(error: shelf::Error) -> AppError {
    match error {
        shelf::Error::NotFound { model } => AppError::NotFound(noun(model)),
        shelf::Error::Stale {
            current_version, ..
        } => match i32::try_from(current_version) {
            Ok(current) => AppError::StaleVersion { current },
            Err(_) => AppError::Internal(format!("version {current_version} is not an
i32")),
        },
        shelf::Error::Db(error) => error.into(),
        other => AppError::Internal(other.to_string()),
    }
}
```

orm/stockroom-shelf/src/error.rs

`shelf::Error::Db(error) => error.into()` hands the driver's error to stockroom's own `From<tokio_postgres::Error>`, which maps SQLSTATEs and constraint names exactly as it does for stockroom's hand-written code, so a duplicate SKU is the same 409 through either. In a crate that owns its error type, prefer the `From` impl; in one that borrows it, the function is the price.

Side by side — *one repository function over a client, a transaction and a pool*

The task: count the products on sale, in a function that runs on whatever the caller holds.

```
// sqlx (illustration)
async fn on_sale(db: impl PgExecutor<'_>) -> sqlx::Result<i64> {
    sqlx::query_scalar("SELECT count(*) FROM products WHERE on_sale")
        .fetch_one(db)
        .await
}

on_sale(&pool).await?;      // any connection of the pool, per query
on_sale(&mut *conn).await?; // a PoolConnection, dereferenced to the PgConnection
on_sale(&mut *tx).await?;   // a Transaction, dereferenced the same way
// The executor is taken by value: to run two queries on one, take `impl Acquire` instead.
```

By hand, generic over tokio-postgres's `GenericClient`, and with shelf, generic over its `Executor`, in one program that calls each with all four types and checks what reaches deadpool's statement cache:

```
use shelf::Executor;
use tokio_postgres::GenericClient;

const ON_SALE: &str =
    "SELECT count(*) FROM (VALUES (true), (false), (true)) AS products (on_sale) WHERE
on_sale";

/// By hand: anything tokio-postgres's GenericClient covers.
async fn on_sale_by_hand(db: &impl GenericClient) -> Result<i64, tokio_postgres::Error> {
    db.query_one(ON_SALE, &[]).await?.try_get(0)
}

/// shelf: anything shelf's Executor covers.
async fn on_sale(db: &impl Executor) -> Result<i64, shelf::Error> {
    let (count,): (i64,) = shelf::fetch_one(db, ON_SALE, &[]).await?;
    Ok(count)
```

```
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let mut client = shelf_playground::connect().await?;
    let pool = shelf_playground::pool(1);
    let mut object = pool.get().await?;

    // A plain Client and its Transaction: both functions take them.
    assert_eq!(on_sale_by_hand(&client).await?, 2);
    assert_eq!(on_sale(&client).await?, 2);
    let tx = client.transaction().await?;
    assert_eq!(on_sale_by_hand(&tx).await?, 2);
    assert_eq!(on_sale(&tx).await?, 2);
    tx.commit().await?;

    // A pooled connection: by hand needs the Client inside it, and skips the cache.
    assert_eq!(on_sale_by_hand(&**object).await?, 2);
    assert_eq!(object.statement_cache.size(), 0);
    // shelf takes the connection itself, and its transaction, through the cache.
    assert_eq!(on_sale(&object).await?, 2);
    let tx = object.transaction().await?;
    assert_eq!(on_sale(&tx).await?, 2);
    assert_eq!(on_sale_by_hand(&*tx).await?, 2); // deadpool's Transaction derefs to tokio-
postgres's
    tx.commit().await?;
    assert_eq!(
        object.statement_cache.size(),
        1,
        "one statement, prepared once"
    );
    Ok(())
}
```

	by hand	shelf	sqlx
Same	One function for a connection and a transaction; the caller decides which	The same	The same
Different	tokio-postgres's `GenericClient` covers a `Client` and a `Transaction`; a pooled connection must be passed as `&**object`, which bypasses deadpool's cache (and deadpool's own `GenericClient` refuses a plain `Client`)	One trait for all four types; a pooled connection is passed as `&object` and prepares through its cache; shelf's calls check the struct against the prepared statement before running	The executor is consumed by value: `&Pool` (each query may take a different connection) or `&mut PgConnection` (a pool connection or transaction dereferenced to it); a function that runs two queries on one generic executor takes `Acquire` instead; statements are cached per connection

In shelf

Reading code that uses the executor, three patterns repeat. A provided method or helper takes `db: &impl Executor` and returns `impl Future<…> + Send` with an `async move` block inside (the `Table` methods of Chapter 6); a free function in shelf is a plain `async fn`, because an inherent or free `async fn`'s future is `Send` whenever what it holds is, which the compiler can see (`fetch_all`); and every call goes through `prepare` first, then `query`, `query_one`, `query_opt` or `execute` with the `Statement`:

```rust
// orm/shelf/src/fetch.rs (excerpt)
/// The row of `sql`, read into `T`, if there is one; fails if there are several.
pub async fn fetch_opt<T: FromRow>(
    db: &impl Executor,
    sql: &str,
    params: &[&(dyn ToSql + Sync)],
) -> Result<Option<T>, Error> {
    let statement = db.prepare(sql).await?;
    let columns = T::resolve(statement.columns())?;
    let row = db.query_opt(&statement, params).await?;
    row.map(|row| T::from_row_at(&row, &columns)).transpose()
}
```

orm/shelf/src/fetch.rs

The difference between the two forms is where the `Send` promise is made. Inside a trait, the caller cannot see the impl, so the promise must be written in the signature; outside one, `fetch_opt`'s future is a concrete type the compiler inspects, and it is `Send` because the executor's futures are. clippy's `manual_async_fn` lint prefers the `async fn` wherever it is enough.

When not to use the executor

- **Statements the trait does not model.** `batch_execute`, `simple_query`, `COPY … TO STDOUT`, `LISTEN`, `cancel_token` and `query_typed` are on the connection; use it. The executor is a view of the calls shelf makes, not a replacement for the client.
- **Starting a transaction.** That takes `&mut` on the connection (§7.4). Start it there, then pass `&tx` to shelf.
- **Many independent reads on one connection.** These work through the executor: shelf's futures borrow it shared, so `tokio::try_join!` on two shelf calls pipelines them on one connection (Exercise 3), as the main book's Chapter 7 pipelines raw queries. What the executor cannot do is send one batch of many statements in a single write; for that, use the `Client` directly.

Summary

- tokio-postgres's `GenericClient` covers a `Client` and a `Transaction`; deadpool's covers a pooled connection and its transaction and adds `prepare_cached`. Both are sealed, so no third crate can implement either, and neither covers all four types.
- shelf's `Executor` is a small trait of its own, `: Sync`, implemented by one `macro_rules!` for the four types; a plain `Client` or `Transaction` prepares on every call, a deadpool connection or transaction through its cache.
- Its methods return `impl Future<Output = …> + Send`, so a function generic over the executor is `Send` and runs in `tokio::spawn` and axum handlers. `async fn` in the trait would not promise that, and a generic caller in `tokio::spawn` fails to compile.
- Pass a pooled connection as `&object`. `&*object` does not compile; `&**object` compiles and silently skips the cache.
- Through lagproxy: prepare-then-run costs two round trips a call; `prepare_cached` two the first time and one after; `query_typed` one, but it needs PostgreSQL types and gives no statement to check the struct against before the query runs.
- A shelf call on a `&Transaction` sees the transaction's writes. The main book's retry helper runs shelf calls with one change: it finds the SQLSTATE inside `Error::Db`.
- `shelf::Error` separates bugs (`Mapping`, `Schema`), outcomes (`NotFound`, `Stale`) and the database (`Db`). Mapping SQLSTATEs and constraint names to statuses stays the application's job: a `From<shelf::Error>` impl where the error type is the application's own, a function with `map_err` where it is not (the orphan rule).

Exercises

1. **A counting executor.** Write `Counting<'a, E>`, an executor around another that counts the statements prepared and the statements run, and use it to show what `Product::find` sends. *Hint:* implement each required method by delegating to the inner executor; count before returning its future. `copy_in`'s future yields a `CopyInSink<bytes::Bytes>`.
2. **A statement cache for a plain `Client`.** Write `Cached`, which owns a `Client` and a map from SQL text to `Statement`, and implements `Executor` so that `prepare` prepares each text once. Check with `pg_prepared_statements` that five `find`s leave one statement. *Hint:* the main book's §4.6 wrote the cache; the new part is the `Send` future, which must not hold the map's lock across an `.await`.
3. **Two calls at once.** Through lagproxy, time `Product::find` followed by `Product::count` on one `Client`, then the same two inside `tokio::try_join!`. How many round trips does each take? *Hint:* each call is two round trips on a plain `Client`, and the client sends requests in the order it receives them, without waiting for answers (main book Chapter 7).

Solutions

1. Each method counts, then returns the inner executor's future unchanged:

```
use bytes::Bytes;
use shelf::{Executor, FromRow, Table};
use std::future::Future;
use std::sync::atomic::{AtomicU32, Ordering};
use tokio_postgres::types::ToSql;
use tokio_postgres::{CopyInSink, Error, Row, RowStream, Statement};

/// An executor that counts what passes through it on the way to another.
struct Counting<'a, E> {
    inner: &'a E,
    prepared: AtomicU32,
    run: AtomicU32,
}

impl<'a, E: Executor> Counting<'a, E> {
    fn new(inner: &'a E) -> Self {
        Counting {
            inner,
            prepared: AtomicU32::new(0),
            run: AtomicU32::new(0),
        }
    }

    fn counts(&self) -> (u32, u32) {
        (
            self.prepared.load(Ordering::Relaxed),
            self.run.load(Ordering::Relaxed),
        )
    }

    fn ran(&self) {
        self.run.fetch_add(1, Ordering::Relaxed);
    }
}

impl<E: Executor> Executor for Counting<'_, E> {
    fn prepare(&self, sql: &str) -> impl Future<Output = Result<Statement, Error>> + Send {
        self.prepared.fetch_add(1, Ordering::Relaxed);
        self.inner.prepare(sql)
    }

    fn query(
        &self,
        statement: &Statement,
        params: &[&(dyn ToSql + Sync)],
```

```
    ) -> impl Future<Output = Result<Vec<Row>, Error>> + Send {
        self.ran();
        self.inner.query(statement, params)
    }

    fn query_one(
        &self,
        statement: &Statement,
        params: &[&(dyn ToSql + Sync)],
    ) -> impl Future<Output = Result<Row, Error>> + Send {
        self.ran();
        self.inner.query_one(statement, params)
    }

    fn query_opt(
        &self,
        statement: &Statement,
        params: &[&(dyn ToSql + Sync)],
    ) -> impl Future<Output = Result<Option<Row>, Error>> + Send {
        self.ran();
        self.inner.query_opt(statement, params)
    }

    fn execute(
        &self,
        statement: &Statement,
        params: &[&(dyn ToSql + Sync)],
    ) -> impl Future<Output = Result<u64, Error>> + Send {
        self.ran();
        self.inner.execute(statement, params)
    }

    fn query_raw(
        &self,
        statement: &Statement,
        params: &[&(dyn ToSql + Sync)],
    ) -> impl Future<Output = Result<RowStream, Error>> + Send {
        self.ran();
        self.inner.query_raw(statement, params)
    }

    fn copy_in(
        &self,
        statement: &Statement,
    ) -> impl Future<Output = Result<CopyInSink<Bytes>, Error>> + Send {
        self.ran();
        self.inner.copy_in(statement)
    }
}

#[derive(Debug, FromRow, Table)]
#[shelf(table = "products")]
struct Product {
    #[shelf(primary_key)]
    id: i64,
    sku: String,
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (id bigint PRIMARY KEY, sku text NOT NULL);
             INSERT INTO products VALUES (1, 'MUG-01'), (2, 'TEE-02');",
        )
        .await?;

    let counting = Counting::new(&client);
```

```
    Product::find(&counting, &1).await?;
    assert_eq!(counting.counts(), (1, 1), "one prepare, one query");
    assert_eq!(Product::all(&counting).await?.len(), 2);
    assert_eq!(Product::count(&counting).await?, 2);
    assert_eq!(counting.counts(), (3, 3));
    Ok(())
}
```

The wrapper counts calls to `prepare`, not round trips: around a deadpool connection it would count a `prepare` for every call, cached or not. To count what reaches the server, count inside the connection (the capstone's `line_for_line` counts the client's *Parse* and *Execute* messages), or time the calls through lagproxy, as §7.3 does.

2. The lock is taken twice, each time in a statement of its own, so no guard lives across the `.await`:

```
use bytes::Bytes;
use shelf::{Executor, FromRow, Table};
use std::collections::HashMap;
use std::future::Future;
use std::sync::Mutex;
use tokio_postgres::types::ToSql;
use tokio_postgres::{Client, CopyInSink, Error, Row, RowStream, Statement};

/// A plain `Client` with a statement cache. One per connection: statements are per session.
struct Cached {
    client: Client,
    statements: Mutex<HashMap<String, Statement>>,
}

impl Executor for Cached {
    // An `async fn` in the impl: the compiler checks that its future is `Send`, as the
trait
    // requires.
    async fn prepare(&self, sql: &str) -> Result<Statement, Error> {
        let hit = self.statements.lock().unwrap().get(sql).cloned();
        if let Some(statement) = hit {
            return Ok(statement);
        }
        let statement = self.client.prepare(sql).await?;
        self.statements
            .lock()
            .unwrap()
            .insert(sql.to_owned(), statement.clone());
        Ok(statement)
    }

    fn prepare_uncached(&self, sql: &str) -> impl Future<Output = Result<Statement, Error>>
+ Send {
        self.client.prepare(sql)
    }

    fn query(
        &self,
        statement: &Statement,
        params: &[&(dyn ToSql + Sync)],
    ) -> impl Future<Output = Result<Vec<Row>, Error>> + Send {
        self.client.query(statement, params)
    }

    fn query_one(
        &self,
        statement: &Statement,
        params: &[&(dyn ToSql + Sync)],
    ) -> impl Future<Output = Result<Row, Error>> + Send {
        self.client.query_one(statement, params)
    }

    fn query_opt(
```

```
        &self,
        statement: &Statement,
        params: &[&(dyn ToSql + Sync)],
    ) -> impl Future<Output = Result<Option<Row>, Error>> + Send {
        self.client.query_opt(statement, params)
    }

    fn execute(
        &self,
        statement: &Statement,
        params: &[&(dyn ToSql + Sync)],
    ) -> impl Future<Output = Result<u64, Error>> + Send {
        self.client.execute(statement, params)
    }

    fn query_raw(
        &self,
        statement: &Statement,
        params: &[&(dyn ToSql + Sync)],
    ) -> impl Future<Output = Result<RowStream, Error>> + Send {
        self.client.query_raw(statement, params.iter().copied())
    }

    fn copy_in(
        &self,
        statement: &Statement,
    ) -> impl Future<Output = Result<CopyInSink<Bytes>, Error>> + Send {
        self.client.copy_in(statement)
    }
}

#[derive(Debug, FromRow, Table)]
#[shelf(table = "products")]
struct Product {
    #[shelf(primary_key)]
    id: i64,
    sku: String,
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let db = Cached {
        client: shelf_playground::connect().await?,
        statements: Mutex::new(HashMap::new()),
    };
    db.client
        .batch_execute(
            "CREATE TEMP TABLE products (id bigint PRIMARY KEY, sku text NOT NULL);
             INSERT INTO products VALUES (1, 'MUG-01');",
        )
        .await?;

    for _ in 0..5 {
        assert_eq!(Product::find(&db, &1).await?.sku, "MUG-01");
    }
    // An unnamed statement (query_typed) does not count itself.
    let open: i64 = db
        .client
        .query_typed_one("SELECT count(*) FROM pg_prepared_statements", &[])
        .await?
        .try_get(0)?;
    assert_eq!(open, 1, "five finds, one statement");
    Ok(())
}
```

Written as `if let Some(statement) = self.statements.lock().unwrap().get(sql) { … }`, the guard would live to the end of the `if let`, and in a version that awaited inside it the future would hold a `MutexGuard`, which is not `Send`. A cache like this one could keep each statement's resolved `Columns` beside it, keyed by the struct's

type too, and skip Chapter 5's `resolve`; §5.6 measured what that would save, under a microsecond per call, and shelf does without.

3. Run one after the other, the two calls take four round trips; joined, two:

```
use shelf::{FromRow, Table};
use std::time::Instant;

#[derive(Debug, FromRow, Table)]
#[shelf(table = "products")]
struct Product {
    #[shelf(primary_key)]
    id: i64,
    sku: String,
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?; // through lagproxy
    client
        .batch_execute(
            "CREATE TEMP TABLE products (id bigint PRIMARY KEY, sku text NOT NULL);
             INSERT INTO products VALUES (1, 'MUG-01'), (2, 'TEE-02');",
        )
        .await?;

    let started = Instant::now();
    let mug = Product::find(&client, &1).await?;
    let count = Product::count(&client).await?;
    let sequential = started.elapsed();

    let started = Instant::now();
    let (joined_mug, joined_count) =
        tokio::try_join!(Product::find(&client, &1), Product::count(&client))?;
    let joined = started.elapsed();

    assert_eq!((mug.sku, count), (joined_mug.sku, joined_count));
    println!("one after the other {sequential:.1?}, joined {joined:.1?}");
    assert!(joined < sequential * 3 / 4, "about half the round trips");
    Ok(())
}
```

Three runs took 53.5–56.6 ms one after the other and 26.0–27.9 ms joined. Both calls borrow the client shared, so `try_join!` polls them together: their two *Parse* requests leave in one round trip, and their two queries in the next. The executor needs nothing special for this; a plain `Client` accepts requests from any number of futures and keeps their answers in order.

CHAPTER 8

Writing rows: insert, update, delete, upsert

Reading is half of a model layer. The other half is writing, and the first question is what type you write. A `Product` read from the database has an `id` and a `created_at`, but a product you are about to create has neither: the database will assign them. Making those fields `Option`s would spread `unwrap()`s through every reader of the model. shelf does what stockroom already does by hand, and gives creation its own struct, `NewProduct`, with only the columns you supply.

`#[derive(Insert)]` on that struct writes the `INSERT`, and `RETURNING` hands back the full model, defaults filled in, in the same round trip. Updates and deletes go through the model itself, by its key, and an upsert is one more clause. Each method's SQL is printed, and each one is run until it fails, so that you know what its failures look like.

In this chapter you will learn:

- why creation gets its own struct, and how `#[derive(Insert)]` builds a statement from another type's constants
- how `RETURNING` gives back the whole model, with the database's defaults, in one round trip
- `update`, `delete` and `delete_by_key`, and what zero rows affected means
- upserts with `ON CONFLICT`, why shelf has no `DO NOTHING` method, and when `MERGE` is the better statement
- what constraint violations from shelf's writes look like, and where they become HTTP statuses

8.1 `#[derive(Insert)]`

A `New…` struct holds the columns a new row supplies, and names the model the row becomes:

```
#[derive(Insert)]
#[shelf(model = Product)]
struct NewProduct<'a> {
    sku: &'a str,
    price_cents: i64,
}
```

`model = Product` gives the derive two things: the table the row goes into, and the columns `RETURNING` reads back, so that `insert` returns a `Product`. Both are the model's constants, `Product::TABLE` and `Product::COLUMNS` from Chapter 6, and here §6.5's problem arrives: the `Insert` derive sees the tokens `Product`, not the model's fields, so it cannot write the table or the column list into a literal. It writes what it knows, its own columns and their placeholders, and joins the model's constants in at compile time with §6.5's `const fn`s. This program prints the `INSERT` the derive generates, by running shelf-codegen, and then the statement the compiler made of it:

```
use shelf::{FromRow, Insert, Table};
use syn::{DeriveInput, parse_quote};
```

```
#[derive(Debug, FromRow, Table)]
#[shelf(table = "products")]
#[allow(dead_code)] // the program only prints the SQL
struct Product {
    #[shelf(primary_key, generated)]
    id: i64,
    sku: String,
    price_cents: i64,
}

#[derive(Insert)]
#[shelf(model = Product)]
#[allow(dead_code)]
struct NewProduct<'a> {
    sku: &'a str,
    price_cents: i64,
}

fn main() {
    // What the derive writes, by running it...
    let input: DeriveInput = parse_quote! {
        #[shelf(model = Product)]
        struct NewProduct<'a> {
            sku: &'a str,
            price_cents: i64,
        }
    };
    let code = shelf_playground::pretty(shelf_codegen::insert::expand(&input).unwrap());
    let start = code.find("    const INSERT").expect("an INSERT constant");
    let end = start + code[start..].find("    };").expect("its end") + "    };".len();
    println!("{}", &code[start..end]);

    // ...and the statement the compiler joined from it.
    println!("{}", NewProduct::INSERT);
    assert_eq!(
        NewProduct::INSERT,
        r#"INSERT INTO "products" ("sku", "price_cents") VALUES ($1, $2) RETURNING "id",
"sku", "price_cents""#
    );
}
```

```
    const INSERT: &'static ::core::primitive::str = {
        const PARTS: &[&::core::primitive::str] = &[
            "INSERT INTO ",
            <Product as ::shelf::Table>::TABLE,
            " (\"sku\", \"price_cents\") VALUES ($1, $2) RETURNING ",
            <Product as ::shelf::Table>::COLUMNS,
        ];
        const BYTES: [::core::primitive::u8; ::shelf::__private::total_len(PARTS)]
= ::shelf::__private::join(
            PARTS,
        );
        ::shelf::__private::utf8(&BYTES)
    };
INSERT INTO "products" ("sku", "price_cents") VALUES ($1, $2) RETURNING "id", "sku",
"price_cents"
```

The block is §6.5's, generated: the derive's own text (`" (\"sku\", \"price_cents\") VALUES ($1, $2) RETURNING "`) between the model's two constants, joined into one `&'static str`. The `id` is in the `RETURNING` list and not in the column list, because the model reads it and the `New…` struct does not supply it. The values are `$1` and `$2`, and `params()` lists the fields in the same order.

The struct may borrow. `params()` returns references to its fields, `&self.sku` and `&self.price_cents`, and `insert` takes `&self`, so a `NewProduct<'a>` built from a request's `&str`s is written without copying them. What it may not have is a type parameter: the statement is a constant built from the model's constants, and §6.5 showed

that such a constant cannot be written in a generic impl. A lifetime is not a problem, because nothing in the statement mentions it.

`insert` itself is a provided method of the `Insert` trait, two lines on top of Chapter 5's `fetch_one`:

```
// orm/shelf/src/write.rs (excerpt)
pub trait Insert: Sync {
    /// The model the row becomes: its table is where the row goes, and its columns what
    /// `RETURNING` reads back.
    type Model: Table;

    /// `INSERT INTO … (…) VALUES ($1, …) RETURNING …`, the model's columns.
    const INSERT: &'static str;

    /// The columns it writes, quoted, in field order.
    const COLUMNS: &'static [&'static str];

    // … COPY and arrays (Chapter 12)

    /// The fields, as `INSERT`'s parameters, in order.
    fn params(&self) -> Vec<&(dyn ToSql + Sync)>;

    /// Inserts the row and returns it as the model, with every default filled in: one round
    /// trip.
    fn insert(
        &self,
        db: &impl Executor,
    ) -> impl Future<Output = Result<Self::Model, Error>> + Send {
        async move { fetch_one(db, Self::INSERT, &self.params()).await }
    }

    // … insert_many and copy_in (Chapter 12)
}
```

orm/shelf/src/write.rs

Because it goes through `fetch_one`, the `INSERT` is prepared first, and `Product` is resolved against its `RETURNING` columns before anything is inserted (Chapter 5): a model that does not fit its table fails without writing a row. "One round trip" is the cost on a deadpool connection with the statement cached; on a plain `Client`, the prepare adds one more (Chapter 7).

What `Insert` refuses

`Insert` is the only shelf derive on its struct, so it reports every mistake in `#[shelf(…)]` itself, unknown keys included (Chapter 4, §4.2). Its own refusals are pinned by a UI test:

```
// orm/shelf/tests/ui/insert.rs (excerpt)
#[derive(Insert)]
struct NoModel {
    sku: String,
}

#[derive(Insert)]
#[shelf(model = Product, table = "products")]
struct Misplaced {
    #[shelf(primary_key)]
    id: i64,
    #[shelf(skip, conflict)]
    note: String,
}

#[derive(FromRow)]
struct NotATable {
    sku: String,
}

#[derive(Insert)]
```

```
#[shelf(model = NotATable)]
struct NewThing {
    sku: String,
}
```

orm/shelf/tests/ui/insert.rs

```
error: Insert needs the model the row becomes: #[shelf(model = Product)]
 --> tests/ui/insert.rs:6:8
  |
6 | struct NoModel {
  |        ^^^^^^^

error: `table` belongs on the model: an Insert struct writes to its model's table
  --> tests/ui/insert.rs:19:26
   |
19 | #[shelf(model = Product, table = "products")]
   |                          ^^^^^

error: `primary_key` has no meaning on an Insert struct, which writes each field to its
column; it belongs on the model
  --> tests/ui/insert.rs:21:13
   |
21 |     #[shelf(primary_key)]
   |             ^^^^^^^^^^^

error: `conflict` cannot apply to a field marked `skip`, which is not written
  --> tests/ui/insert.rs:23:19
   |
23 |     #[shelf(skip, conflict)]
   |                   ^^^^^^^^

error[E0277]: `NotATable` is not a shelf model
  --> tests/ui/insert.rs:33:17
   |
33 | #[shelf(model = NotATable)]
   |                 ^^^^^^^^^ not a `Table`
   |
help: the trait `Table` is not implemented for `NotATable`
  --> tests/ui/insert.rs:28:1
   |
28 | struct NotATable {
   | ^^^^^^^^^^^^^^^^
   = note: derive it: `#[derive(FromRow, Table)]`, with `#[shelf(table = "…")]` on the
struct and `#[shelf(primary_key)]` on its key
help: the trait `Table` is implemented for `Product`
  --> tests/ui/insert.rs:10:19
   |
10 | #[derive(FromRow, Table)]
   |                   ^^^^^
note: required by a bound in `shelf::Insert::Model`
  --> src/write.rs
   |
   |     type Model: Table;
   |                 ^^^^^ required by this bound in `Insert::Model`
   = note: this error originates in the derive macro `Table` (in Nightly builds, run with -Z
macro-backtrace for more info)
```

The first four come from the derive. The last one cannot: whether `NotATable` is a model is a fact about another item, which the derive never sees.

> **Macro lens:** the derive emits `type Model = NotATable;`, and the trait declares `type Model: Table;`, so the compiler checks the bound when it type-checks the impl, long after the macro has returned. The error lands on `NotATable` inside the attribute because the derive builds that line from the attribute's own tokens, which carry their spans into the output. The wording is shelf's: `Table` has a `#[diagnostic::on_unimplemented]` with a message, a label and a note, as Chapter 4 gave `FromColumn`. (The build reports it twice, the second time for the `<NotATable as Table>::TABLE` the joined statement uses; the UI test's `.stderr` has both.)

8.2 Database defaults

The columns a `New…` struct leaves out are filled by their defaults. Three kinds are common, and the model marks each one `generated`:

Column	Default	What fills it
`id uuid DEFAULT uuidv7()`	a version-7 UUID, which sorts by creation time	`uuidv7()`, PG 18+ (main book §3.9)
`id bigint GENERATED ALWAYS AS IDENTITY`	the next value of the column's sequence	the identity
`created_at timestamptz DEFAULT now()`	the transaction's start time	`now()`

`generated` tells Chapter 6's `update` not to write the column; the `Insert` struct leaves it out simply by not having the field. `RETURNING` reads the values back with the rest of the row:

```
use jiff::Timestamp;
use shelf::{FromRow, Insert, Table};
use uuid::Uuid;

#[derive(Debug, FromRow, Table)]
#[shelf(table = "products")]
struct Product {
    #[shelf(primary_key, generated)]
    id: Uuid,
    sku: String,
    price_cents: i64,
    #[shelf(generated)]
    created_at: Timestamp,
}

#[derive(Insert)]
#[shelf(model = Product)]
struct NewProduct<'a> {
    sku: &'a str,
    price_cents: i64,
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (
                 id uuid PRIMARY KEY DEFAULT uuidv7(),
                 sku text NOT NULL UNIQUE,
                 price_cents bigint NOT NULL,
                 created_at timestamptz NOT NULL DEFAULT now()
             );",
        )
        .await?;
```

```
    let before = Timestamp::now();
    let mug = NewProduct {
        sku: "MUG-01",
        price_cents: 1299,
    }
    .insert(&client)
    .await?;
    let tee = NewProduct {
        sku: "TEE-02",
        price_cents: 2499,
    }
    .insert(&client)
    .await?;

    // The database's values, read back by the same statement.
    assert_eq!(mug.id.get_version_num(), 7);
    assert!(tee.id > mug.id, "version-7 UUIDs sort by creation time");
    assert!(mug.created_at >= before - jiff::SignedDuration::from_secs(5));
    assert_eq!(Product::find(&client, &mug.id).await?.sku, "MUG-01");
    println!(
        "{} {} {}",
        mug.sku,
        mug.price_cents,
        mug.id.get_version_num()
    );
    Ok(())
}
```

```
MUG-01 1299 7
```

The `Insert` derive does not know which of the model's columns are generated (that is in the model's attributes, another item again). So a `New…` struct that names one is not refused at compile time; the database refuses it, if the column says so. An identity declared `GENERATED ALWAYS` does:

```
use shelf::{FromRow, Insert, Table};
use tokio_postgres::error::SqlState;

#[derive(Debug, FromRow, Table)]
#[shelf(table = "products")]
#[allow(dead_code)] // the program only shows the error
struct Product {
    #[shelf(primary_key, generated)]
    id: i64,
    sku: String,
}

/// A mistake: `id` belongs to the database.
#[derive(Insert)]
#[shelf(model = Product)]
struct NewProduct {
    id: i64,
    sku: String,
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (
                 id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
                 sku text NOT NULL
             );",
        )
        .await?;
```

```
    let error = NewProduct {
        id: 42,
        sku: "MUG-01".into(),
    }
    .insert(&client)
    .await
    .unwrap_err();
    let shelf::Error::Db(error) = error else {
        panic!("a database error");
    };
    let server = error.as_db_error().expect("from the server");
    println!("{}", server.message());
    println!("{}", server.detail().unwrap_or_default());
    println!("{}", server.hint().unwrap_or_default());
    assert_eq!(server.code(), &SqlState::GENERATED_ALWAYS);

    // The server refuses the statement as it parses it: preparing alone fails.
    let error = client.prepare(NewProduct::INSERT).await.unwrap_err();
    assert_eq!(error.code(), Some(&SqlState::GENERATED_ALWAYS));
    Ok(())
}
```

```
cannot insert a non-DEFAULT value into column "id"
Column "id" is an identity column defined as GENERATED ALWAYS.
Use OVERRIDING SYSTEM VALUE to override.
```

The server refuses the statement as it parses it, so the error comes from the *Parse* that `prepare` sends, and nothing runs. A column declared `GENERATED BY DEFAULT AS IDENTITY`, or with a plain `DEFAULT`, would accept the value and skip the default, which is usually a bug that nothing reports; a model's `generated` columns deserve `GENERATED ALWAYS` where PostgreSQL offers it.

> **Pitfall:** a parameter that goes into a domain column is typed as the domain, in an `INSERT`'s `VALUES` and in an `UPDATE`'s `SET` alike, and a `String` refuses it on the client before anything is sent (main book §5.5). A `New…` struct for a table with a `sku` domain needs a field type that accepts the domain: a derived domain newtype, or a type with a hand-written `accepts`, as stockroom's `Sku` has. The model can read the column into a `String`, because a domain result column is described as its base type. This program shows both halves.

```
use postgres_types::{FromSql, ToSql};
use shelf::{FromRow, Insert, Table};

/// The `sku` domain, as a derived newtype.
#[derive(Debug, Clone, ToSql, FromSql)]
#[postgres(name = "sku")]
struct Sku(String);

#[derive(Debug, FromRow, Table)]
#[shelf(table = "products", schema = "shelf_ch08_domain")]
#[allow(dead_code)]
struct Product {
    #[shelf(primary_key, generated)]
    id: i64,
    sku: String, // read back as text: a domain result column is its base type
}

#[derive(Insert)]
#[shelf(model = Product)]
struct NewProduct {
    sku: String,
}
```

```
#[derive(Insert)]
#[shelf(model = Product)]
struct NewProductSku {
    sku: Sku,
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "DROP SCHEMA IF EXISTS shelf_ch08_domain CASCADE;
             CREATE SCHEMA shelf_ch08_domain;
             CREATE DOMAIN shelf_ch08_domain.sku AS text
                 CHECK (VALUE ~ '^[A-Z0-9]{3,12}(-[A-Z0-9]{1,6})?$');
             CREATE TABLE shelf_ch08_domain.products (
                 id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
                 sku shelf_ch08_domain.sku NOT NULL
             );",
        )
        .await?;

    // A String for a domain parameter: refused by the client, nothing inserted.
    let error = NewProduct {
        sku: "MUG-01".into(),
    }
    .insert(&client)
    .await
    .unwrap_err();
    println!("{error}");
    let cause = std::error::Error::source(&error).expect("the conversion's error");
    println!("  caused by: {cause}");
    assert!(matches!(&error, shelf::Error::Db(e) if e.as_db_error().is_none()));

    // The domain's own type is accepted, and the model reads the value as text.
    let mug = NewProductSku {
        sku: Sku("MUG-01".into()),
    }
    .insert(&client)
    .await?;
    assert_eq!(mug.sku, "MUG-01");

    client
        .batch_execute("DROP SCHEMA shelf_ch08_domain CASCADE")
        .await?;
    Ok(())
}
```

```
error serializing parameter 0
  caused by: cannot convert between the Rust type `alloc::string::String` and the Postgres
type `shelf_ch08_domain.sku`
```

8.3 Update and delete

Updates and deletes need a row that exists, so they go through the model, by its key. Chapter 6's derive writes two more statements for them:

Method	Statement	Zero rows means
`product.update(&db)`	`UPDATE` every column that is neither key nor `generated`, `WHERE` the key, `RETURNING` the model's columns	`Error::NotFound`
`product.delete(&db)`	`DELETE … WHERE` the key	`Error::NotFound`

Method	Statement	Zero rows means
`Product::delete_by_key(&db, &key)`	the same, with a key you have	`Error::NotFound`

`update` writes the value you hold over the row and returns the row as it now is, which matters when the database changes something itself: a trigger, a `version` that goes up by one, an `updated_at` (Chapter 9). Zero rows affected has one meaning for all three: no row has the key, because someone deleted it, or because it never existed. shelf reports that as `NotFound` rather than as success, since a write that wrote nothing is not what the caller asked for:

```
use shelf::{FromRow, Insert, Table};

#[derive(Debug, Clone, FromRow, Table)]
#[shelf(table = "products")]
struct Product {
    #[shelf(primary_key, generated)]
    id: i64,
    sku: String,
    price_cents: i64,
}

#[derive(Insert)]
#[shelf(model = Product)]
struct NewProduct<'a> {
    sku: &'a str,
    price_cents: i64,
}

/// Where a row's current version lives in the table, as text.
async fn ctid(client: &tokio_postgres::Client, id: i64) -> Result<String, shelf::Error> {
    let row = client
        .query_one("SELECT ctid::text FROM products WHERE id = $1", &[&id])
        .await?;
    Ok(row.try_get(0)?)
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (
                 id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
                 sku text NOT NULL UNIQUE,
                 price_cents bigint NOT NULL
             );",
        )
        .await?;
    println!("{}", Product::UPDATE);
    println!("{}", Product::DELETE);

    let mut mug = NewProduct {
        sku: "MUG-01",
        price_cents: 1299,
    }
    .insert(&client)
    .await?;
    mug.price_cents = 999;
    let updated = mug.update(&client).await?;
    assert_eq!(Product::find(&client, &mug.id).await?.price_cents, 999);

    // An update that changes nothing still writes a new version of the row.
    let before = ctid(&client, mug.id).await?;
    updated.update(&client).await?;
    let after = ctid(&client, mug.id).await?;
    println!("ctid {before} -> {after}");
    assert_ne!(before, after);
```

```
    // Gone: every write by its key is NotFound.
    mug.delete(&client).await?;
    for error in [
        mug.update(&client).await.unwrap_err(),
        mug.delete(&client).await.unwrap_err(),
        Product::delete_by_key(&client, &mug.id).await.unwrap_err(),
    ] {
        assert!(matches!(error, shelf::Error::NotFound { model: "Product" }));
    }
    Ok(())
}
```

```
UPDATE "products" SET "sku" = $1, "price_cents" = $2 WHERE "id" = $3 RETURNING "id", "sku",
"price_cents"
DELETE FROM "products" WHERE "id" = $1
ctid (0,2) -> (0,3)
```

> **SQL lens:** PostgreSQL never changes a row in place. An `UPDATE` writes a new version of the row and marks the old one dead, to be cleaned up by `VACUUM`, whatever the new version holds; `ctid`, the row version's physical address, moved above although no value changed (Routine Vacuuming). So writing every column costs what writing one costs, as far as the table is concerned: the whole row is rewritten either way. Indexes are another matter: a *HOT* update, which needs no new index entries, is possible only when no indexed column changes value and the new version fits on the same page (Heap-Only Tuples). The real cost of writing every column is elsewhere: it overwrites what someone else changed since you read the row (Exercise 2), which is Chapter 9's subject.

`update`'s statement for a model whose every column is key or generated would have nothing to `SET`. The derive then uses `FIND` as its `UPDATE`, so `update` returns the row as it is, or `NotFound`, and never sends invalid SQL.

8.4 Upsert

An upsert inserts a row, or updates the one already there when the new row collides with a unique key. PostgreSQL's statement for it is `INSERT … ON CONFLICT (…) DO UPDATE SET …`, where `excluded` names the row that could not be inserted (main book §2.6). In shelf, the fields marked `#[shelf(conflict)]` name the key, and the derive then also implements `Upsert`, with one more statement:

```
use shelf::{FromRow, Insert, Table, Upsert};
use tokio_postgres::error::SqlState;

#[derive(Debug, FromRow, Table)]
#[shelf(table = "products")]
struct Product {
    #[shelf(primary_key, generated)]
    id: i64,
    sku: String,
    price_cents: i64,
}

/// A price list's line: insert the SKU, or reprice it.
#[derive(Insert)]
#[shelf(model = Product)]
struct PriceLine<'a> {
    #[shelf(conflict)]
    sku: &'a str,
    price_cents: i64,
}
```

```rust
/// A mistake: no unique key is on `price_cents` alone.
#[derive(Insert)]
#[shelf(model = Product)]
struct ByPrice<'a> {
    sku: &'a str,
    #[shelf(conflict)]
    price_cents: i64,
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (
                 id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
                 sku text NOT NULL CONSTRAINT products_sku_key UNIQUE,
                 price_cents bigint NOT NULL
             );",
        )
        .await?;
    println!("{}", PriceLine::UPSERT);

    let first = PriceLine {
        sku: "MUG-01",
        price_cents: 1299,
    }
    .upsert(&client)
    .await?;
    let again = PriceLine {
        sku: "MUG-01",
        price_cents: 999,
    }
    .upsert(&client)
    .await?;
    assert_eq!(
        (again.id, again.price_cents),
        (first.id, 999),
        "the same row, repriced"
    );
    assert_eq!(Product::count(&client).await?, 1);

    // The upsert that updated still drew an identity value: the next row skips it.
    let tee = PriceLine {
        sku: "TEE-02",
        price_cents: 2499,
    }
    .upsert(&client)
    .await?;
    println!("ids: {}, {}", first.id, tee.id);
    assert_eq!(tee.id, first.id + 2);

    // A conflict target that matches no unique index: preparing it succeeds...
    client.prepare(ByPrice::UPSERT).await?;
    // ...and running it fails, when PostgreSQL plans it.
    let error = ByPrice {
        sku: "VASE-03",
        price_cents: 3999,
    }
    .upsert(&client)
    .await
    .unwrap_err();
    let shelf::Error::Db(error) = &error else {
        panic!("a database error: {error}");
    };
    println!(
        "{}",
        error.as_db_error().expect("from the server").message()
```

```
    );
    assert_eq!(error.code(), Some(&SqlState::INVALID_COLUMN_REFERENCE));
    Ok(())
}
```

```
INSERT INTO "products" ("sku", "price_cents") VALUES ($1, $2) ON CONFLICT ("sku") DO UPDATE
SET "price_cents" = excluded."price_cents" RETURNING "id", "sku", "price_cents"
ids: 1, 3
there is no unique or exclusion constraint matching the ON CONFLICT specification
```

`SET` names every column of the struct except the conflict key, each set to its `excluded` value. The target must match a unique index or constraint, column for column, and `ByPrice` shows what happens when it does not: SQLSTATE 42P10, but only when the statement runs. PostgreSQL looks for the index that arbitrates the conflict when it *plans* the statement, and a *Parse* only parses and analyses it, so preparing the `UPSERT` succeeds and Chapter 5's check cannot catch this one. (The `GENERATED ALWAYS` mistake of §8.2 is caught earlier, while the statement is parsed and rewritten, which is why `prepare` failed there.) And the gap in the ids is the price of upserting into a table with an identity or a sequence: the row's defaults are computed before PostgreSQL knows whether it will insert, and a sequence value, once drawn, is never given back.

Why no `DO NOTHING`

The other form, `ON CONFLICT DO NOTHING`, skips the row on a conflict. With `RETURNING`, it then returns no row at all:

```
 id | sku | price_cents
----+-----+-------------
(0 rows)

INSERT 0 0
```

(That is psql 18 running `INSERT … VALUES ('MUG-01', 999) ON CONFLICT DO NOTHING RETURNING id, sku, price_cents` when `MUG-01` already exists.) A method for it would return `Option<Product>`, and `None` would say only that some row was in the way, not which one, or what it holds; a caller that needs the existing row must read it with a second query, and between the two another transaction can change or delete it. shelf's derive does something else when the `Insert` struct has no column besides its conflict key: it writes `DO UPDATE SET "sku" = excluded."sku"`, an update that changes nothing, so that the existing row is still returned, in the same statement. (It is still an update: it takes the row's lock and writes a new version.) A `DO NOTHING` method is easy to add on top of the public constants, which Exercise 1 does; shelf leaves it out because what it returns is rarely what the caller wanted.

`MERGE`

`MERGE` (PG 15+) is the SQL standard's statement for the same job, with more to say: it can update, insert or delete depending on conditions, and since PostgreSQL 17 `RETURNING merge_action()` reports which happened to each row. The main book uses it to apply a price list loaded with COPY (main book §10.6). For one row from a request, `ON CONFLICT` is the better statement, and the PostgreSQL documentation says why: when `MERGE` runs concurrently with other changes the usual isolation rules apply, so two `MERGE`s that both decide "not matched" can both insert and one fails with a unique violation, while `INSERT … ON CONFLICT` "offers the ability to run an `UPDATE` if a concurrent `INSERT` occurs" (MERGE). Exercise 3 writes the upsert as a `MERGE`.

8.5 Constraint errors

A write that breaks a constraint fails on the server, after the statement has been prepared and the struct checked, and shelf passes the error on as `Error::Db`. Everything an application needs to report it is inside: the SQLSTATE says which kind of rule, and the constraint's name says which rule (main book §6.3):

```rust
use shelf::{FromRow, Insert, Table};

#[derive(Debug, FromRow, Table)]
#[shelf(table = "products")]
#[allow(dead_code)]
struct Product {
    #[shelf(primary_key, generated)]
    id: i64,
    sku: String,
    price_cents: i64,
    store_id: Option<i64>,
}

#[derive(Insert)]
#[shelf(model = Product)]
struct NewProduct<'a> {
    sku: &'a str,
    price_cents: i64,
    store_id: Option<i64>,
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE stores (id bigint PRIMARY KEY);
             CREATE TEMP TABLE products (
                 id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
                 sku text NOT NULL CONSTRAINT products_sku_key UNIQUE,
                 price_cents bigint NOT NULL CONSTRAINT products_price_positive CHECK
(price_cents > 0),
                 store_id bigint CONSTRAINT products_store_fkey REFERENCES stores
             );",
        )
        .await?;
    let new = |sku, price_cents, store_id| NewProduct {
        sku,
        price_cents,
        store_id,
    };
    new("MUG-01", 1299, None).insert(&client).await?;

    for row in [
        new("MUG-01", 999, None),      // a second MUG-01
        new("TEE-02", 0, None),        // a price that is not positive
        new("VASE-03", 3999, Some(7)), // a store that does not exist
    ] {
        let error = row.insert(&client).await.unwrap_err();
        let shelf::Error::Db(error) = &error else {
            panic!("a database error: {error}");
        };
        let server = error.as_db_error().expect("from the server");
        println!(
            "{} {:<24} {}",
            server.code().code(),
            server.constraint().unwrap_or("-"),
            server.message()
        );
    }
    assert_eq!(Product::count(&client).await?, 1);
    Ok(())
}
```

```
23505 products_sku_key         duplicate key value violates unique constraint
"products_sku_key"
23514 products_price_positive  new row for relation "products" violates check constraint
"products_price_positive"
```

```
23503 products_store_fkey      insert or update on table "products" violates foreign key
constraint "products_store_fkey"
```

Turning these into statuses is the application's job, keyed by constraint name, as Chapter 7's `AppError` did: 23505 on `products_sku_key` is a 409, 23514 and 23503 are 422s, each with a message the API's client can act on. A constraint renamed in a migration silently stops matching that table, which is why stockroom treats constraint names as a contract (its `error::constraint_message`), and why naming constraints yourself, as these do, beats the names PostgreSQL makes up.

One more consequence follows from the error being the server's. Inside a transaction, a failed statement aborts the transaction: every statement after it fails with 25P02 until a rollback (main book §6.6). A `NotFound` or a `Stale` from shelf is different: shelf decides it from the rows that came back, nothing failed on the server, and the transaction can go on.

Side by side — *inserting a row and reading it back*

The task: create a store from a request and return it, with the id and timestamp the database assigned.

```
// SeaORM (illustration)
let store = store::ActiveModel {
    code: Set("ber-01".to_owned()),
    name: Set("Berlin Mitte".to_owned()),
    ..Default::default() // id and created_at: NotSet, so the database's defaults
};
// INSERT INTO "stores" ("code", "name") VALUES ($1, $2) RETURNING "id", "code", "name",
"created_at"
let store: store::Model = store.insert(&db).await?;
```

By hand, as stockroom's `stores::create` writes it, and with shelf, in one program that checks both return the same kind of row:

```
use jiff::Timestamp;
use shelf::{FromRow, Insert, Table};
use uuid::Uuid;

#[derive(Debug, FromRow, Table)]
#[shelf(table = "stores")]
struct Store {
    #[shelf(primary_key, generated)]
    id: Uuid,
    code: String,
    name: String,
    #[shelf(generated)]
    created_at: Timestamp,
}

#[derive(Insert)]
#[shelf(model = Store)]
struct NewStore<'a> {
    code: &'a str,
    name: &'a str,
}

/// By hand: the statement and its column list written out, the row read with FromRow.
async fn create_by_hand(
    db: &tokio_postgres::Client,
    new: &NewStore<'_>,
) -> Result<Store, shelf::Error> {
    let statement = db
        .prepare(
            "INSERT INTO stores (code, name) VALUES ($1, $2)
             RETURNING id, code, name, created_at",
        )
        .await?;
```

```
    let row = db.query_one(&statement, &[&new.code, &new.name]).await?;
    Store::from_row(&row)
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE stores (
                 id uuid PRIMARY KEY DEFAULT uuidv7(),
                 code text NOT NULL UNIQUE,
                 name text NOT NULL,
                 created_at timestamptz NOT NULL DEFAULT now()
             );",
        )
        .await?;

    let by_hand = create_by_hand(
        &client,
        &NewStore {
            code: "ber-01",
            name: "Berlin Mitte",
        },
    )
    .await?;
    let with_shelf = NewStore {
        code: "ham-01",
        name: "Hamburg Altona",
    }
    .insert(&client)
    .await?;

    for store in [&by_hand, &with_shelf] {
        assert_eq!(store.id.get_version_num(), 7);
        assert!(store.created_at <= Timestamp::now());
    }
    assert_eq!(Store::find(&client, &by_hand.id).await?.code, "ber-01");
    assert_eq!(
        Store::find(&client, &with_shelf.id).await?.name,
        "Hamburg Altona"
    );
    Ok(())
}
```

	by hand	shelf	SeaORM
Same	A struct of the columns supplied; one `INSERT … RETURNING` that writes them and reads the whole row back, defaults included	The same	The same
Different	You write the statement, its column list and its `RETURNING` list, and keep all three in step with two structs	A `New…` struct with `#[derive(Insert)]` names its model; the statement is a constant joined at compile time from both, and the model is checked against its `RETURNING` columns before the row is inserted	The same `ActiveModel` type serves insert and update: each field is `Set`, `NotSet` or `Unchanged`, and only `Set` fields are written; `insert` runs the model's `before_save` and `after_save` hooks around the statement, which sea-query builds at run time

In shelf

The upsert's statement is built beside the `INSERT`, from the same pieces, when any field is marked `conflict`:

```
// orm/shelf-codegen/src/insert.rs (excerpt)
    let conflict: Vec<String> = columns
        .iter()
        .filter(|c| c.conflict)
        .map(|c| c.name.clone())
        .collect();
    if !conflict.is_empty() {
        // Every other column takes the new value. With none, the conflict key is set to
itself,
        // so that the existing row is still returned (`DO NOTHING` would return no row).
        let mut updated: Vec<&String> = names.iter().filter(|n| !
conflict.contains(n)).collect();
        if updated.is_empty() {
            updated = conflict.iter().take(1).collect();
        }
        let set: Vec<String> = updated
            .iter()
            .map(|n| format!("{0} = excluded.{0}", quote_ident(n)))
            .collect();
        let on_conflict = format!(
            " ON CONFLICT ({}) DO UPDATE SET {} RETURNING ",
            column_list(&conflict),
            set.join(", ")
        );
        let middle = format!("{values}{on_conflict}");
        let upsert = joined(&[quote!("INSERT INTO "), table, quote!(#middle), returning]);
        out.extend(quote! {
            #[automatically_derived]
            impl #impl_generics ::shelf::Upsert for #name #ty_generics #where_clause {
                const UPSERT: &'static ::core::primitive::str = #upsert;
            }
        });
    }
```

orm/shelf-codegen/src/insert.rs

`joined` is the block §8.1 printed, wrapped in a function of the derive. `Upsert` is a separate trait, a subtrait of `Insert`, so that a struct without a conflict key has no `upsert` method to call by mistake: the compiler, not a run-time error, says it cannot upsert.

`update` is a provided method of `Table`. It puts the key's parameters after the values it sets, in the order of the `$n` in `UPDATE`, and hands the statement to `written`, which for a model without a version is `fetch_opt` with `NotFound` for no row:

```
// orm/shelf/src/table.rs (excerpt)
    fn update(&self, db: &impl Executor) -> impl Future<Output = Result<Self, Error>> + Send
{
        async move {
            let key = self.key();
            let mut params = self.update_params();
            params.extend(Self::key_params(&key));
            written(db, Self::UPDATE, &params).await
        }
    }
```

orm/shelf/src/table.rs

The rest of `written`, for a model with a `version`, is Chapter 9's.

When not to use shelf's writes

- **A guarded update.** stockroom's reservation, `UPDATE stock SET reserved = reserved + $3 WHERE store_id = $1 AND product_id = $2 AND available >= $3`, is correct because of its `WHERE`: the condition and the change happen in one statement, under the row's lock. Read, check in Rust, then `update` would let two orders sell the last chair. Write it by hand (main book §8.9).
- **Counters and expressions.** `SET stock = stock - 1` computes from the row's current value; `update` writes the value you hold, which may be stale.
- **Bulk loads.** One `insert` per row costs a round trip per row; Chapter 12's `insert_many` and `copy_in` cost one per batch.
- **Several outcomes per row.** Inserting, updating and deleting from one source by conditions is what `MERGE` is for.

Summary

- Creation gets its own struct. `#[derive(Insert)]` with `#[shelf(model = Product)]` writes `INSERT INTO … (its columns) VALUES ($1, …) RETURNING (the model's columns)`, joining the model's `TABLE` and `COLUMNS` in at compile time. The struct may borrow; it may not have type parameters.
- `insert` returns the model with every default filled in, in one round trip on a cached connection, and checks the model against `RETURNING` before inserting.
- `generated` marks the columns the database fills (`uuidv7()`, PG 18+; identities; `now()`). The `Insert` derive cannot see them: a `New…` struct that names one is refused by the database, if the column is `GENERATED ALWAYS`.
- A domain column's parameter is typed as the domain, so a `String` is refused on the client; write it with a type that accepts the domain.
- `update` writes every column that is neither key nor generated and returns the row; `delete` and `delete_by_key` delete by key. Zero rows is `Error::NotFound` for all three.
- `#[shelf(conflict)]` fields make an `Upsert`: `ON CONFLICT (…) DO UPDATE SET` every other column, returning the row whether it was inserted or updated. An upsert that updates still draws an identity value.
- shelf has no `DO NOTHING` method: on a conflict it returns no row, and the caller is left with a second query and a race. `MERGE … RETURNING merge_action()` (PG 17+) suits batches with several outcomes; `ON CONFLICT` suits one row under concurrency.
- A constraint violation is `Error::Db` with the SQLSTATE and the constraint's name; mapping it to a status is the application's job, by name.

Exercises

1. **`insert_if_absent`.** Write `insert_if_absent(&db, &row) -> Result<Option<T::Model>, _>` for any `T: Insert`, with `ON CONFLICT DO NOTHING`. What does it return for a row that collides? *Hint:* `Insert::COLUMNS`, `Table::TABLE` and `Table::COLUMNS` are public; `fetch_opt` reads zero or one row.
2. **Two tabs.** Read the same product twice, as two admins' browser tabs would. Change the price in one copy and the name in the other, and `update` both. What is in the table? *Hint:* `update` writes every column it is given.
3. **Upsert with `MERGE`.** Write the price-list upsert of §8.4 as a `MERGE` that returns the row and `merge_action()`, and read both into one struct. *Hint:* `USING (VALUES ($1::text, $2::bigint)) AS new (sku, price_cents)`; the casts give the parameters their types. A `FromRow` struct can `flatten` a model.

Solutions

1. The statement is assembled at run time from constants, which are quoted identifiers, never values:

```
use shelf::{Executor, FromRow, Insert, Table};

#[derive(Debug, FromRow, Table)]
#[shelf(table = "products")]
struct Product {
    #[shelf(primary_key, generated)]
    id: i64,
    sku: String,
    price_cents: i64,
}

#[derive(Insert)]
#[shelf(model = Product)]
struct NewProduct<'a> {
    sku: &'a str,
    price_cents: i64,
}

/// Inserts `row` unless it collides with a unique key; then nothing is written, and nothing
/// comes back.
async fn insert_if_absent<T: Insert>(
    db: &impl Executor,
    row: &T,
) -> Result<Option<T::Model>, shelf::Error> {
    let placeholders: Vec<String> = (1..=T::COLUMNS.len()).map(|n| format!
("${n}")).collect();
    let sql = format!(
        "INSERT INTO {} ({}) VALUES ({}) ON CONFLICT DO NOTHING RETURNING {}",
        <T::Model as Table>::TABLE,
        T::COLUMNS.join(", "),
        placeholders.join(", "),
        <T::Model as Table>::COLUMNS
    );
    shelf::fetch_opt(db, &sql, &row.params()).await
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (
                 id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
                 sku text NOT NULL UNIQUE,
                 price_cents bigint NOT NULL
             );",
        )
        .await?;
    let mug = |price_cents| NewProduct {
        sku: "MUG-01",
        price_cents,
    };

    let first = insert_if_absent(&client, &mug(1299)).await?;
    assert_eq!(first.map(|p| p.price_cents), Some(1299));
    // The same SKU again: no error, no row, and the table unchanged.
    assert!(insert_if_absent(&client, &mug(999)).await?.is_none());
    assert_eq!(Product::find(&client, &1).await?.price_cents, 1299);
    Ok(())
}
```

`None` means "a row was in the way", without saying which row or which unique constraint: a table with two unique keys gives the same `None` for both. To return the existing row, read it after the `None`, by the key you meant, and accept that it may have changed in between; or use `upsert`, whose `RETURNING` reads it under the row's lock.

2. The second `update` writes back the price its copy still holds, so the first change is lost:

```
use shelf::{FromRow, Table};

#[derive(Debug, Clone, FromRow, Table)]
#[shelf(table = "products")]
struct Product {
    #[shelf(primary_key)]
    id: i64,
    name: String,
    price_cents: i64,
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (
                 id bigint PRIMARY KEY, name text NOT NULL, price_cents bigint NOT NULL
             );
             INSERT INTO products VALUES (1, 'Mug', 1299);",
        )
        .await?;

    let mut tab_a = Product::find(&client, &1).await?;
    let mut tab_b = Product::find(&client, &1).await?;
    tab_a.price_cents = 999;
    tab_a.update(&client).await?;
    tab_b.name = "Blue mug".into();
    tab_b.update(&client).await?;

    let now = Product::find(&client, &1).await?;
    assert_eq!(now.name, "Blue mug");
    assert_eq!(now.price_cents, 1299, "tab A's price is lost");
    Ok(())
}
```

Nothing failed, and nothing reported the loss. Two things would have prevented it, and Chapter 9 builds both: writing only the columns that changed (a patch), and refusing a write made against an old version of the row (a version column).

3. A `FromRow` struct reads `merge_action()` and the model, flattened, from one row:

```
use shelf::{FromRow, Table};

#[derive(Debug, FromRow, Table)]
#[shelf(table = "products")]
struct Product {
    #[shelf(primary_key, generated)]
    id: i64,
    sku: String,
    price_cents: i64,
}

/// One row of the MERGE's RETURNING: what it did, and the row it did it to.
#[derive(Debug, FromRow)]
struct Merged {
    action: String,
    #[shelf(flatten)]
    product: Product,
}

const MERGE: &str = "MERGE INTO products AS p
    USING (VALUES ($1::text, $2::bigint)) AS new (sku, price_cents) ON p.sku = new.sku
    WHEN MATCHED THEN UPDATE SET price_cents = new.price_cents
    WHEN NOT MATCHED THEN INSERT (sku, price_cents) VALUES (new.sku, new.price_cents)
    RETURNING merge_action() AS action, p.id, p.sku, p.price_cents";
```

```
#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (
                 id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
                 sku text NOT NULL UNIQUE,
                 price_cents bigint NOT NULL
             );",
        )
        .await?;

    let first: Merged = shelf::fetch_one(&client, MERGE, &[&"MUG-01", &1299_i64]).await?;
    let again: Merged = shelf::fetch_one(&client, MERGE, &[&"MUG-01", &999_i64]).await?;
    println!("{} {:?}", first.action, first.product);
    println!("{} {:?}", again.action, again.product);
    assert_eq!(
        (first.action.as_str(), again.action.as_str()),
        ("INSERT", "UPDATE")
    );
    assert_eq!(again.product.id, first.product.id);
    Ok(())
}
```

```
INSERT Product { id: 1, sku: "MUG-01", price_cents: 1299 }
UPDATE Product { id: 1, sku: "MUG-01", price_cents: 999 }
```

`MERGE` says what it did, which `ON CONFLICT` does not, and it can do more (a `WHEN MATCHED AND … THEN DELETE`, or no action at all for an unchanged price). It needs casts where `ON CONFLICT` needed none, because a `VALUES` list in `USING` has no column to take its types from. And under concurrency it is the weaker statement: two sessions that both find `MUG-01` missing both take the `NOT MATCHED` branch, and the second insert fails with 23505, where `ON CONFLICT` would have turned it into an update.

CHAPTER 9

Partial updates and optimistic locking

An admin edits a product's price and nothing else. The request body is `{"price_cents": 990}`, and the update should touch that column only: another admin's change to the name, made a second earlier, must survive. Chapter 8's `update` writes every column, so it would undo that change, as its Exercise 2 showed. A partial update needs to know which fields were sent, and JSON has a third state that `Option` cannot express: a field sent as `null`, which means "set it to NULL", is not a field that was left out.

Then there is the edit that arrives late. Two tabs load version 7 of the product; both save. Without a check, the second save silently overwrites the first. With a version column, the second `UPDATE` finds no row at version 7, and the application can say so. This chapter builds both: `#[derive(Patch)]`, in the one of three designs that measurement picked, and a `version` attribute that turns a lost update into an error.

In this chapter you will learn:

- how to tell "absent" from "null" in a request body, with `Option<Option<T>>` and serde
- three ways to write a partial `UPDATE`, and what each costs in statements and round trips
- how `#[derive(Patch)]` generates the one shelf chose, and why its statement lives in a `LazyLock`
- how a version column turns a lost update into `Error::Stale`, told apart from `NotFound` in the same round trip
- what `updated_at = now()` records: the transaction's start, not the statement's
- how ETags and `If-Match` fit on top, in an axum handler

9.1 Absent or null

A partial update's fields have one more state than a model's:

Column	Field type	Absent	`null`	A value
`price_cents bigint NOT NULL`	`Option<i64>`	`None`: leave it	`None` too	`Some(990)`
`note text` (nullable)	`Option<Option<String>>`	`None`: leave it	`Some(None)`: set NULL	`Some(Some("…"))`

For a column that cannot be NULL, `Option<T>` is enough: `None` leaves the column as it is. For a nullable column, the outer `Option` says whether to touch the column, the inner one what to put there. Rust has the type; the catch is serde, which reads a JSON `null` into an `Option<Option<T>>` as the outer `None`, the same as a missing field. A one-line deserializer fixes that: it reads the field as an `Option<T>` and wraps whatever it gets in `Some`, so a `null` becomes `Some(None)`; `#[serde(default)]` keeps a missing field `None`, since the deserializer then never runs:

```
use serde::{Deserialize, Deserializer};

/// A field that may be absent, `null` or a value: `null` becomes `Some(None)`. With
/// `#[serde(default)]`, an absent field stays `None`.
fn double_option<'de, T, D>(deserializer: D) -> Result<Option<Option<T>>, D::Error>
where
```

```
    T: Deserialize<'de>,
    D: Deserializer<'de>,
{
    Option::<T>::deserialize(deserializer).map(Some)
}

/// serde's own reading of `Option<Option<T>>`.
#[derive(Debug, PartialEq, Deserialize)]
struct Naive {
    note: Option<Option<String>>,
}

#[derive(Debug, PartialEq, Deserialize)]
#[serde(deny_unknown_fields)]
struct ProductPatch {
    price_cents: Option<i64>,
    #[serde(default, deserialize_with = "double_option")]
    note: Option<Option<String>>,
}

fn main() {
    let read = |json| serde_json::from_str::<ProductPatch>(json).unwrap();
    let naive = |json| serde_json::from_str::<Naive>(json).unwrap();

    for body in [r#"{}"#, r#"{"note": null}"#, r#"{"note": "fragile"}"#] {
        println!(
            "{body:<22} naive {:?}, patch {:?}",
            naive(body).note,
            read(body).note
        );
    }
    assert_eq!(
        naive(r#"{"note": null}"#),
        naive(r#"{}"#),
        "serde cannot tell them apart"
    );
    assert_eq!(read(r#"{}"#).note, None);
    assert_eq!(read(r#"{"note": null}"#).note, Some(None));
    assert_eq!(
        read(r#"{"note": "fragile"}"#).note,
        Some(Some("fragile".into()))
    );

    // A null for a column that cannot be NULL reads as `None`: "leave it".
    assert_eq!(read(r#"{"price_cents": null}"#).price_cents, None);
    // And a misspelt field is refused rather than ignored.
    assert!(serde_json::from_str::<ProductPatch>(r#"{"price": 990}"#).is_err());
}
```

```
{}                     naive None, patch None
{"note": null}         naive None, patch Some(None)
{"note": "fragile"}    naive Some(Some("fragile")), patch Some(Some("fragile"))
```

The function is what serde_with 3.24 ships as `serde_with::rust::double_option` (with a serializer beside it), used as `#[serde(default, with = "::serde_with::rust::double_option")]`; for one function, a crate is optional. `deny_unknown_fields` matters more in a patch than anywhere else: without it, `{"price": 990}` deserializes to an empty patch, which succeeds and changes nothing, and the client is told it worked. A `null` sent for a column that cannot be NULL is read as "leave it"; refusing it instead takes a deserializer of its own, which reads the field as a `T` and so fails on `null`.

9.2 Three designs

There are three ways to send a partial update, and each gives something up. The first builds the `SET` list at run time from the fields that are present: the SQL is exactly what you would write by hand, but every combination

of fields is a different statement, prepared and cached separately. The second sends every column, and writes `col = coalesce($n, col)`: one statement for every patch, but `coalesce` cannot tell a NULL you want from a value you did not send, so it can never set a column to NULL. The third sends a flag beside each value, `col = CASE WHEN $f THEN $v ELSE col END`: one statement, NULLs included, at the price of twice the parameters.

Design	Statement for `price_cents` only	Statements per model	Can set NULL
(a) a `SET` list per shape	`UPDATE … SET price_cents = $1 WHERE id = $2`	up to $2^n - 1$ for n fields	yes
(b) `coalesce`	`UPDATE … SET name = coalesce($1, name), price_cents = coalesce($2, price_cents), … WHERE id = $n`	1	no
(c) a flag per field	`UPDATE … SET name = CASE WHEN $1 THEN $2 ELSE name END, … WHERE id = $n`	1	yes

This program runs all three on one pooled connection, through lagproxy, with fourteen patches: every non-empty combination of three fields, once setting the nullable `note` to a value and once to NULL. It counts the statements each design leaves in deadpool's cache and the NULLs it managed to write, and times the patches:

```
use std::time::{Duration, Instant};
use tokio_postgres::types::ToSql;

/// A patch of three columns, one nullable: `Some(None)` sets `note` to NULL.
struct Patch {
    name: Option<String>,
    price_cents: Option<i64>,
    note: Option<Option<String>>,
}

/// Every non-empty shape of the three fields, twice: `note` set to a value, then to NULL.
fn patches() -> Vec<Patch> {
    let mut all = Vec::new();
    for shape in 1..8 {
        for null in [false, true] {
            all.push(Patch {
                name: (shape & 1 != 0).then(|| "Blue mug".to_string()),
                price_cents: (shape & 2 != 0).then_some(990),
                note: (shape & 4 != 0).then(|| (!null).then(|| "fragile".to_string())),
            });
        }
    }
    all
}

const ID: i64 = 1;

/// (a) The `SET` list of the fields that are present: one statement per shape.
fn set_list(patch: &Patch) -> (String, Vec<&(dyn ToSql + Sync)>) {
    let mut set = Vec::new();
    let mut params: Vec<&(dyn ToSql + Sync)> = Vec::new();
    if let Some(name) = &patch.name {
        params.push(name);
        set.push(format!("name = ${}", params.len()));
    }
    if let Some(price_cents) = &patch.price_cents {
        params.push(price_cents);
        set.push(format!("price_cents = ${}", params.len()));
    }
    if let Some(note) = &patch.note {
        params.push(note);
        set.push(format!("note = ${}", params.len()));
    }
    params.push(&ID);
    let sql = format!(
        "UPDATE products SET {} WHERE id = ${}",
        set.join(", "),
```

```
        params.len()
    );
    (sql, params)
}

/// (b) Every column, `coalesce`d: NULL means "leave it", so NULL cannot be written.
const COALESCE: &str = "UPDATE products SET
    name = coalesce($1, name),
    price_cents = coalesce($2, price_cents),
    note = coalesce($3, note)
    WHERE id = $4";

/// (c) Every column, with a flag that says whether to set it.
const FLAGS: &str = "UPDATE products SET
    name = CASE WHEN $1 THEN $2 ELSE name END,
    price_cents = CASE WHEN $3 THEN $4 ELSE price_cents END,
    note = CASE WHEN $5 THEN $6 ELSE note END
    WHERE id = $7";

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let pool = shelf_playground::pool(1);
    let db = pool.get().await?;
    db.batch_execute(
        "CREATE TEMP TABLE products (
             id bigint PRIMARY KEY, name text NOT NULL, price_cents bigint NOT NULL, note
text
         );
         INSERT INTO products VALUES (1, 'Mug', 1299, 'new');",
    )
    .await?;

    let mut results = Vec::new();
    for design in ["(a) SET list", "(b) coalesce", "(c) flags"] {
        let cached_before = db.statement_cache.size();
        let (mut nulls_wanted, mut nulls_written) = (0, 0);
        let mut elapsed = Duration::ZERO;
        for patch in patches() {
            db.batch_execute("UPDATE products SET name = 'Mug', price_cents = 1299, note =
'new'")
                .await?;
            let started = Instant::now();
            match design {
                "(a) SET list" => {
                    let (sql, params) = set_list(&patch);
                    let statement = db.prepare_cached(&sql).await?;
                    db.execute(&statement, &params).await?;
                }
                "(b) coalesce" => {
                    let note = patch.note.clone().flatten(); // absent and null: both None
                    let statement = db.prepare_cached(COALESCE).await?;
                    db.execute(&statement, &[&patch.name, &patch.price_cents, &note, &ID])
                        .await?;
                }
                _ => {
                    let flags = [
                        patch.name.is_some(),
                        patch.price_cents.is_some(),
                        patch.note.is_some(),
                    ];
                    let statement = db.prepare_cached(FLAGS).await?;
                    let params: [&(dyn ToSql + Sync); 7] = [
                        &flags[0],
                        &patch.name,
                        &flags[1],
                        &patch.price_cents,
                        &flags[2],
                        &patch.note,
                        &ID,
```

```
                    ];
                    db.execute(&statement, &params).await?;
                }
            }
            elapsed += started.elapsed();
            if patch.note == Some(None) {
                nulls_wanted += 1;
                let note: Option<String> = db
                    .query_one("SELECT note FROM products WHERE id = 1", &[])
                    .await?
                    .get(0);
                nulls_written += i32::from(note.is_none());
            }
        }
        let statements = db.statement_cache.size() - cached_before;
        let per_patch = elapsed / patches().len() as u32;
        println!(
            "{design:<13} {statements} statement(s), {nulls_written} of {nulls_wanted} NULLs
written, {per_patch:.1?} a patch"
        );
        results.push((statements, nulls_written, per_patch));
    }

    let [
        (a_statements, a_nulls, a_time),
        (b_statements, b_nulls, _),
        (c_statements, c_nulls, c_time),
    ] = results[..]
    else {
        unreachable!("three designs");
    };
    assert_eq!((a_statements, b_statements, c_statements), (7, 1, 1));
    assert_eq!((a_nulls, b_nulls, c_nulls), (4, 0, 4));
    assert!(a_time > c_time * 6 / 5, "each new shape costs a prepare");
    Ok(())
}
```

```
(a) SET list  7 statement(s), 4 of 4 NULLs written, 22.1ms a patch
(b) coalesce  1 statement(s), 0 of 4 NULLs written, 15.6ms a patch
(c) flags     1 statement(s), 4 of 4 NULLs written, 15.8ms a patch
```

Three runs (a debug build, lagproxy at 5 ms each way) gave 22.0–22.7 ms a patch for (a), 15.6–16.4 ms for (b) and 15.5–15.8 ms for (c): half of (a)'s fourteen patches are a shape it has not seen, and each of those costs a prepare. The measurement that chose shelf's design (PLAN.md §12, finding 33) ran longer, 70 patches cycling through the seven shapes, 20 of them writing NULL, three runs in a release build through lagproxy at 5 ms each way, so that (a)'s prepares were spread thinner:

Design	Statements cached	NULLs written	Per patch
(a) a `SET` list per shape	7	20 of 20	15.0–15.8 ms
(b) `coalesce`	1	0 of 20	13.8–14.6 ms
(c) a flag per field	1	20 of 20	13.9–14.7 ms

(b) is out: a patch API that cannot clear a field is a bug report waiting to happen. Between (a) and (c), the numbers say that a flag per field costs nothing measurable in time, while a `SET` list per shape costs a prepare per new shape on every pooled connection, and a cache that grows with the number of shapes in use: with n patchable fields, up to $2^n - 1$ statements, on each connection of the pool. shelf uses (c).

Design (c) has a catch that had to be checked: it sends a typed value for every field, even the fields a patch leaves alone. An unset `Option<i64>` is sent as a NULL of type `int8` with its flag false, and the server never looks at it. That held for every column type tried (text, integers, `bool`, `float8`, `jsonb`, `text[]`, `timestamptz`, `uuid`, an enum and a domain), and the domain had a bonus: in a `CASE` or a `coalesce`, PostgreSQL types the

parameter as the domain's base type, so a patch can write a `String` to a `sku` column that Chapter 8's `INSERT` and `update` refuse it for. The domain's check still runs when the value is assigned (PLAN.md §12, finding 30, and shelf's test `a_domain_column_takes_a_string_in_a_patch_but_not_in_an_update`).

9.3 `#[derive(Patch)]`

A patch is a struct of `Option` fields that names its model. It usually derives `Deserialize` too, so that the request body is the patch:

```
#[derive(Debug, Default, Deserialize, Patch)]
#[serde(deny_unknown_fields)]
#[shelf(model = Product)]
struct ProductPatch {
    name: Option<String>,
    price_cents: Option<i64>,
    #[serde(default, deserialize_with = "double_option")]
    note: Option<Option<String>>,
}
```

The derive generates design (c). Running it on a two-field patch shows what it emits:

```
use syn::{DeriveInput, parse_quote};

fn main() {
    let input: DeriveInput = parse_quote! {
        #[shelf(model = Product)]
        struct ProductPatch {
            price_cents: Option<i64>,
            note: Option<Option<String>>,
        }
    };
    let code = shelf_playground::pretty(shelf_codegen::patch::expand(&input).unwrap());
    println!("{code}");
    assert!(code.contains("::shelf::__private::flag(&self.price_cents)"));
    assert!(code.contains("static SQL: ::std::sync::LazyLock<::std::string::String>"));
}
```

```
#[automatically_derived]
impl ::shelf::Patch for ProductPatch {
    type Model = Product;
    const COLUMNS: &'static [&'static ::core::primitive::str] = &[
        "\"price_cents\"",
        "\"note\"",
    ];
    fn sql() -> &'static ::core::primitive::str {
        static SQL: ::std::sync::LazyLock<::std::string::String>
= ::std::sync::LazyLock::new(||
        { ::shelf::__private::patch_sql::<Product>(&["\"price_cents\"", "\"note\""]) });
        ::std::string::String::as_str(&SQL)
    }
    fn params(
        &self,
    ) -> ::std::vec::Vec<
        &(dyn ::shelf::__private::tokio_postgres::types::ToSql + ::core::marker::Sync),
    > {
        ::std::vec::Vec::from([
            ::shelf::__private::flag(&self.price_cents),
            ::shelf::__private::param(&self.price_cents),
            ::shelf::__private::flag(&self.note),
            ::shelf::__private::param(&self.note),
        ])
    }
}
const _: () = {
```

```
    if <Product as ::shelf::Table>::VERSION.is_some() {
        ::core::panic!(
            "ProductPatch: the model Product has a `version` column, so the patch needs a
#[shelf(version)] field holding the version it was made against"
        );
    }
};
```

Three things are generated. `COLUMNS` lists the columns the patch can set, quoted at macro time. `params()` lists a flag and a value per field, in the order of the `$n` in the statement, each borrowed from the patch: `flag` returns a `&'static` reference to `true` or `false`, so the flags need no storage of their own. And `sql()` returns the statement, which is not a constant.

Why a `LazyLock`

The statement is `UPDATE "products" SET "price_cents" = CASE WHEN $1 THEN $2 ELSE "price_cents" END, … WHERE "id" = $5`. The derive knows its own columns, but the table, the key's columns and whether the model has a version and an `updated_at` are the model's constants, and the key's placeholder numbers depend on how many parameters the patch has before them. That is the computing §6.5 said joining cannot do: the `$5` is a number derived from another type's constants. So the derive emits a `static LazyLock`, which calls shelf's `patch_sql::<Product>` the first time `sql()` is called, and keeps the `String` for the rest of the program.

> **Rust refresher:** `std::sync::LazyLock<T>` (stable since Rust 1.80.0) holds a value that is computed on first access, by the closure given to `LazyLock::new`, and is the same value for every access after, from every thread: the first caller runs the closure while any others wait. A `static` must be initialised by a constant expression, and `LazyLock::new(closure)` is one, which is what lets a `static` hold something built at run time. Before 1.80 the `once_cell` and `lazy_static` crates did this job. Here, each patch type's `sql()` has its own `static`, since each impl is a separate function; a `static` inside a generic function would be one for every instantiation, which is one reason `Patch`, like `Table` and `Insert`, refuses type parameters.

`patch_sql` is ordinary run-time code, the counterpart of Chapter 6's `format!`s, run once per patch type:

```
// orm/shelf/src/patch.rs (excerpt)
pub fn patch_sql<M: Table>(columns: &[&str]) -> String {
    let mut set: Vec<String> = columns
        .iter()
        .enumerate()
        .map(|(i, c)| {
            format!(
                "{c} = CASE WHEN ${} THEN ${} ELSE {c} END",
                2 * i + 1,
                2 * i + 2
            )
        })
        .collect();
    let mut next = 2 * columns.len() + 1;
    if let Some(version) = M::VERSION {
        set.push(format!("{version} = {version} + 1"));
    }
    if let Some(updated_at) = M::UPDATED_AT {
        set.push(format!("{updated_at} = now()"));
    }
    // … the version's parameter, then the key's: `"id" = $n` from M::KEY_COLUMNS
    let key_where = key_where.join(" AND ");
    let (table, set) = (M::TABLE, set.join(", "));
    let columns = M::COLUMNS;
    match (M::VERSION, version_param) {
        // … with a version: §9.4's statement
```

```
        _ => format!("UPDATE {table} SET {set} WHERE {key_where} RETURNING {columns}"),
    }
}
```

orm/shelf/src/patch.rs

`apply(&db, &key)` appends the key's parameters to `params()` and sends `sql()` through the same `written` helper as Chapter 8's `update`, so a patch returns the row as it now is, and a key with no row is `NotFound`. This program deserializes three request bodies into one patch type and applies each:

```
use serde::{Deserialize, Deserializer};
use shelf::{FromRow, Patch, Table};

fn double_option<'de, T, D>(deserializer: D) -> Result<Option<Option<T>>, D::Error>
where
    T: Deserialize<'de>,
    D: Deserializer<'de>,
{
    Option::<T>::deserialize(deserializer).map(Some)
}

#[derive(Debug, FromRow, Table)]
#[shelf(table = "products")]
struct Product {
    #[shelf(primary_key)]
    id: i64,
    name: String,
    price_cents: i64,
    note: Option<String>,
}

#[derive(Debug, Deserialize, Patch)]
#[serde(deny_unknown_fields)]
#[shelf(model = Product)]
struct ProductPatch {
    name: Option<String>,
    price_cents: Option<i64>,
    #[serde(default, deserialize_with = "double_option")]
    note: Option<Option<String>>,
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (
                 id bigint PRIMARY KEY, name text NOT NULL, price_cents bigint NOT NULL,
note text
             );
             INSERT INTO products VALUES (1, 'Mug', 1299, 'fragile');",
        )
        .await?;
    println!("{}", ProductPatch::sql());

    for body in [
        r#"{"price_cents": 990}"#,
        r#"{"note": null}"#,
        r#"{"name": "Blue mug", "note": "chipped"}"#,
    ] {
        let patch: ProductPatch = serde_json::from_str(body)?;
        let product = patch.apply(&client, &1).await?;
        println!(
            "{body:<40} -> {} | {} | {:?}",
            product.name, product.price_cents, product.note
        );
    }
```

```
    let product = Product::find(&client, &1).await?;
    assert_eq!(
        (
            product.name.as_str(),
            product.price_cents,
            product.note.as_deref()
        ),
        ("Blue mug", 990, Some("chipped"))
    );

    let missing = ProductPatch {
        name: None,
        price_cents: Some(1),
        note: None,
    };
    let error = missing.apply(&client, &2).await.unwrap_err();
    assert!(matches!(error, shelf::Error::NotFound { model: "Product" }));
    Ok(())
}
```

```
UPDATE "products" SET "name" = CASE WHEN $1 THEN $2 ELSE "name" END, "price_cents" = CASE
WHEN $3 THEN $4 ELSE "price_cents" END, "note" = CASE WHEN $5 THEN $6 ELSE "note" END WHERE
"id" = $7 RETURNING "id", "name", "price_cents", "note"
{"price_cents": 990}                    -> Mug | 990 | Some("fragile")
{"note": null}                          -> Mug | 990 | None
{"name": "Blue mug", "note": "chipped"}  -> Blue mug | 990 | Some("chipped")
```

Each body touched only what it named, and the `null` cleared the note. One statement served all three, prepared on each call on this plain `Client`, and once per connection on a pooled one.

What `Patch` refuses

The UI test pins what the derive reports:

```
// orm/shelf/tests/ui/patch.rs (excerpt)
#[derive(FromRow, Table)]
#[shelf(table = "products")]
struct Product {
    #[shelf(primary_key)]
    id: i64,
    sku: String,
    #[shelf(version)]
    version: i32,
}

#[derive(Patch)]
#[shelf(model = Product)]
struct NotOptions {
    sku: String,
    #[shelf(json)]
    note: Option<String>,
    #[shelf(version)]
    version: i32,
}

#[derive(Patch)]
#[shelf(model = Product)]
struct NoVersion {
    sku: Option<String>,
}
```

orm/shelf/tests/ui/patch.rs

```
error: a Patch field is an `Option`: `None` leaves the column as it is and `Some` sets it
(`Option<Option<T>>` for a nullable column, whose `Some(None)` sets NULL)
  --> tests/ui/patch.rs:18:10
```

```
   |
18 |     sku: String,
   |          ^^^^^^

error: a Patch writes each value as it is, so `json` has no meaning here: for a json column,
write the field as `Option<tokio_postgres::types::Json<T>>` or `Option<serde_json::Value>`
  --> tests/ui/patch.rs:19:13
   |
19 |     #[shelf(json)]
   |             ^^^^

error[E0080]: evaluation panicked: NoVersion: the model Product has a `version` column, so
the patch needs a #[shelf(version)] field holding the version it was made against
  --> tests/ui/patch.rs:25:10
   |
25 | #[derive(Patch)]
   |          ^^^^^ evaluation of `_` failed here
```

A `json` field on a patch needs no attribute: a field of type `Option<tokio_postgres::types::Json<T>>`, or `Option<serde_json::Value>`, writes JSON through its own `ToSql`, which is what the attribute does on a model (shelf's test `a_json_column_is_patched_through_json`; the capstone's `ProductPatch` has an `Option<Value>`).

> **Macro lens:** the last error is a rule that spans two types: a patch of a model with a `version` must carry the version it was made against, and a patch of a model without one must not. The `Patch` derive cannot see the model's fields, but the compiler can evaluate the model's constant, so the derive emits `const _: () = { if <Product as Table>::VERSION.is_none() { panic!("…") } };` (printed above, with `is_some` for a patch without a version field). A panic during constant evaluation is a compile error, E0080, with the panic's message, and the derive points it at `Patch` in the `#[derive]`, through the span of the code it generates. It needs `Option::is_none` to be a `const fn`, which it is.

9.4 Version columns

A version column makes every write say which version of the row it was made against. Mark the model's field `#[shelf(version)]`, and three things change:

- `update` and every patch add `"version" = "version" + 1` to their `SET`;
- they write only `WHERE "id" = $n AND "version" = $m`, with the version the caller holds;
- the patch type needs a `#[shelf(version)]` field, the version it was made against.

When the row is not at that version, the `UPDATE` changes nothing. Zero rows then has two causes: the row is gone (`NotFound`), or someone else wrote it first (`Stale`). They need different answers, a 404 and a 412, and a 412 is more useful with the current version in it, so that the client can fetch the row again and retry. stockroom tells them apart with a second query when the update finds nothing. shelf does it in the same statement:

```
WITH updated AS (
    UPDATE "products" SET … , "version" = "version" + 1
    WHERE "id" = $7 AND "version" = $6
    RETURNING "id", "name", …
)
SELECT "id", "name", …,
       (SELECT count(*) FROM updated) AS shelf_updated,
       (SELECT "version"::bigint FROM "products" WHERE "id" = $7) AS shelf_version
FROM (VALUES (1)) AS one LEFT JOIN updated ON true
```

The statement always returns exactly one row: `VALUES (1)` provides it, and the `LEFT JOIN` fills in the updated row's columns when there is one, and NULLs when there is not. `shelf_updated` says which, and `shelf_version` is the row's version as the statement found it, NULL when there is no row. So:

shelf_updated	shelf_version	Result
1	the old version	the updated row
0	a version	`Error::Stale { current_version }`
0	NULL	`Error::NotFound`

> **SQL lens:** every part of a statement with a data-modifying `WITH` sees the same snapshot, taken before any of it ran, so the outer `SELECT … FROM "products"` does not see the `UPDATE`'s new row version; that is why `shelf_version` is the old version after a success (WITH queries, data-modifying statements). It is also why the `RETURNING` of the `UPDATE`, not a second read of the table, supplies the row: only `RETURNING` has the new values.

`written`, in shelf, reads that row: the model from its columns when `shelf_updated` is 1, else `Stale` or `NotFound` from `shelf_version`.

```rust
// orm/shelf/src/write.rs (excerpt)
pub(crate) async fn written<M: Table>(
    db: &impl Executor,
    sql: &str,
    params: &[&(dyn ToSql + Sync)],
) -> Result<M, Error> {
    let not_found = Error::NotFound { model: M::MODEL };
    if M::VERSION.is_none() {
        return fetch_opt(db, sql, params).await?.ok_or(not_found);
    }
    let statement = db.prepare(sql).await?;
    let columns = M::resolve(statement.columns())?;
    let row = db.query_one(&statement, params).await?;
    if row.try_get::<_, i64>("shelf_updated")? == 1 {
        return M::from_row_at(&row, &columns);
    }
    match row.try_get::<_, Option<i64>>("shelf_version")? {
        Some(current_version) => Err(Error::Stale {
            model: M::MODEL,
            current_version,
        }),
        None => Err(not_found),
    }
}
```

orm/shelf/src/write.rs

The model is resolved against the statement's columns as always; the two extra columns are simply not fields of it. Two tabs, then, each holding version 1:

```rust
use shelf::{FromRow, Patch, Table};

#[derive(Debug, Clone, FromRow, Table)]
#[shelf(table = "products")]
struct Product {
    #[shelf(primary_key)]
    id: i64,
    name: String,
    price_cents: i64,
    #[shelf(version)]
    version: i32,
}

#[derive(Debug, Default, Patch)]
#[shelf(model = Product)]
struct ProductPatch {
```

```
    name: Option<String>,
    price_cents: Option<i64>,
    #[shelf(version)]
    version: i32,
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (
                 id bigint PRIMARY KEY, name text NOT NULL, price_cents bigint NOT NULL,
                 version integer NOT NULL DEFAULT 1
             );
             INSERT INTO products (id, name, price_cents) VALUES (1, 'Mug', 1299);",
        )
        .await?;

    // Both tabs load the product at version 1.
    let tab_a = Product::find(&client, &1).await?;
    let tab_b = Product::find(&client, &1).await?;

    let saved = ProductPatch {
        price_cents: Some(990),
        version: tab_a.version,
        ..ProductPatch::default()
    }
    .apply(&client, &1)
    .await?;
    assert_eq!(saved.version, 2);

    // Tab B saves against version 1, which is gone.
    let error = ProductPatch {
        name: Some("Blue mug".into()),
        version: tab_b.version,
        ..ProductPatch::default()
    }
    .apply(&client, &1)
    .await
    .unwrap_err();
    println!("{error}");
    let shelf::Error::Stale {
        current_version, ..
    } = error
    else {
        panic!("stale");
    };

    // Tab B reads again, sees the new price, and saves at the current version.
    let fresh = Product::find(&client, &1).await?;
    assert_eq!(
        (i64::from(fresh.version), fresh.price_cents),
        (current_version, 990)
    );
    let saved = ProductPatch {
        name: Some("Blue mug".into()),
        version: fresh.version,
        ..ProductPatch::default()
    }
    .apply(&client, &1)
    .await?;
    assert_eq!(
        (saved.name.as_str(), saved.price_cents, saved.version),
        ("Blue mug", 990, 3)
    );

    // Chapter 8's `update` checks the version too: tab B's stale copy is refused.
    let mut stale_copy = tab_b.clone();
```

```
    stale_copy.price_cents = 1;
    assert!(matches!(
        stale_copy.update(&client).await,
        Err(shelf::Error::Stale {
            current_version: 3,
            ..
        })
    ));
    // And a row that does not exist is NotFound, from the same statement.
    let error = ProductPatch {
        version: 1,
        price_cents: Some(1),
        ..ProductPatch::default()
    }
    .apply(&client, &2)
    .await
    .unwrap_err();
    assert!(matches!(error, shelf::Error::NotFound { .. }));
    Ok(())
}
```

```
Product has changed since it was read: it is now at version 2
```

The loss of Chapter 8's Exercise 2 is now an error, and the error carries what the client needs to recover. For a model with a version, `update` runs the same kind of statement, so a stale copy of the whole model is refused as well; and since `update`'s `SET` comes from the copy's fields, the version is the only guard it has against overwriting a newer row.

9.5 `updated_at`

`#[shelf(updated_at)]` on a timestamp field adds `"updated_at" = now()` to every update and every patch. The field is read like any other, so the row a write returns has the new time in it.

> **SQL lens:** `now()` is the start time of the current *transaction*, not of the statement, so its value does not change while the transaction lasts, and PostgreSQL's documentation calls that a feature; `clock_timestamp()` is the one that changes from call to call (Current Date/Time). So two writes in one transaction record the same instant, which is usually what you want: the rows a transaction changed all say when it happened. Outside a transaction, each statement is a transaction of its own, and each gets its own time.

```
use jiff::Timestamp;
use shelf::{FromRow, Patch, Table};
use std::time::Duration;

#[derive(Debug, FromRow, Table)]
#[shelf(table = "products")]
struct Product {
    #[shelf(primary_key)]
    id: i64,
    price_cents: i64,
    #[shelf(updated_at)]
    updated_at: Timestamp,
}

#[derive(Debug, Patch)]
#[shelf(model = Product)]
struct ProductPatch {
```

```
    price_cents: Option<i64>,
}

fn price(price_cents: i64) -> ProductPatch {
    ProductPatch {
        price_cents: Some(price_cents),
    }
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let mut client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (
                 id bigint PRIMARY KEY, price_cents bigint NOT NULL,
                 updated_at timestamptz NOT NULL DEFAULT now()
             );
             INSERT INTO products (id, price_cents) VALUES (1, 1299);",
        )
        .await?;

    // Two patches in one transaction, 20 ms apart: one instant.
    let tx = client.transaction().await?;
    let first = price(990).apply(&tx, &1).await?;
    tokio::time::sleep(Duration::from_millis(20)).await;
    let second = price(980).apply(&tx, &1).await?;
    tx.commit().await?;
    assert_eq!(first.updated_at, second.updated_at);

    // The same two outside a transaction: each statement has its own time.
    let first = price(970).apply(&client, &1).await?;
    tokio::time::sleep(Duration::from_millis(20)).await;
    let second = price(960).apply(&client, &1).await?;
    assert!(
        second.updated_at.duration_since(first.updated_at) >=
jiff::SignedDuration::from_millis(20)
    );
    Ok(())
}
```

9.6 ETags and `If-Match` in axum

HTTP has the version check built in. A `GET` returns the version as an `ETag` header; a client that wants to change the resource sends the tag back in `If-Match`; and the server answers 412 Precondition Failed when the resource no longer has that tag (main book Chapter 15). With a version column, the tag is the version, and `If-Match` is the patch's `version` field. This program serves `GET` and `PATCH` for products with axum, and drives the router through `oneshot` (the playground's `call`), without a port:

```
use axum::body::Body;
use axum::extract::{Path, State};
use axum::http::{HeaderMap, Request, StatusCode, header};
use axum::response::{IntoResponse, Response};
use axum::routing::get;
use axum::{Json, Router};
use serde::{Deserialize, Serialize};
use serde_json::json;
use shelf::{FromRow, Patch, Table};
use std::sync::Arc;

#[derive(Debug, Serialize, FromRow, Table)]
#[shelf(table = "products")]
struct Product {
    #[shelf(primary_key)]
```

```
    id: i64,
    name: String,
    price_cents: i64,
    #[shelf(version)]
    version: i32,
}

#[derive(Debug, Deserialize, Patch)]
#[serde(deny_unknown_fields)]
#[shelf(model = Product)]
struct ProductPatch {
    name: Option<String>,
    price_cents: Option<i64>,
    /// Not in the body: the handler sets it from If-Match.
    #[serde(skip)]
    #[shelf(version)]
    version: i32,
}

type Db = Arc<tokio_postgres::Client>;

/// A product, with its version as the ETag.
fn product(product: Product) -> Response {
    let etag = format!("\"{}\"", product.version);
    ([(header::ETAG, etag)], Json(product)).into_response()
}

/// shelf's errors as responses: what the client can act on, and nothing else.
fn failure(error: shelf::Error) -> Response {
    let (status, body) = match error {
        shelf::Error::NotFound { .. } => (StatusCode::NOT_FOUND, json!({"error": "not
found"})),
        shelf::Error::Stale {
            current_version, ..
        } => (
            StatusCode::PRECONDITION_FAILED,
            json!({"error": "changed since you read it", "currentVersion":
current_version}),
        ),
        _ => (
            StatusCode::INTERNAL_SERVER_ERROR,
            json!({"error": "internal error"}),
        ),
    };
    (status, Json(body)).into_response()
}

async fn show(State(db): State<Db>, Path(id): Path<i64>) -> Response {
    match Product::find(&*db, &id).await {
        Ok(found) => product(found),
        Err(error) => failure(error),
    }
}

async fn edit(
    State(db): State<Db>,
    Path(id): Path<i64>,
    headers: HeaderMap,
    Json(mut patch): Json<ProductPatch>,
) -> Response {
    // If-Match: "3" is the version the client read.
    let read = headers
        .get(header::IF_MATCH)
        .and_then(|value| value.to_str().ok())
        .and_then(|tag| tag.trim_matches('"').parse().ok());
    let Some(version) = read else {
        let body = json!({"error": "send If-Match with the ETag you read"});
        return (StatusCode::PRECONDITION_REQUIRED, Json(body)).into_response();
    };
```

```
    patch.version = version;
    match patch.apply(&*db, &id).await {
        Ok(saved) => product(saved),
        Err(error) => failure(error),
    }
}

fn patch(uri: &str, if_match: Option<&str>, body: &str) -> Request<Body> {
    let mut request = Request::patch(uri).header(header::CONTENT_TYPE, "application/json");
    if let Some(tag) = if_match {
        request = request.header(header::IF_MATCH, tag);
    }
    request.body(Body::from(body.to_owned())).unwrap()
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (
                 id bigint PRIMARY KEY, name text NOT NULL, price_cents bigint NOT NULL,
                 version integer NOT NULL DEFAULT 1
             );
             INSERT INTO products (id, name, price_cents) VALUES (1, 'Mug', 1299);",
        )
        .await?;
    let app = Router::new()
        .route("/products/{id}", get(show).patch(edit))
        .with_state(Arc::new(client));

    let requests = [
        Request::get("/products/1").body(Body::empty()).unwrap(),
        patch("/products/1", Some("\"1\""), r#"{"price_cents": 990}"#),
        patch("/products/1", Some("\"1\""), r#"{"name": "Blue mug"}"#),
        patch("/products/1", None, r#"{"name": "Blue mug"}"#),
        patch("/products/7", Some("\"1\""), r#"{"name": "Vase"}"#),
    ];
    let mut statuses = Vec::new();
    for request in requests {
        let line = format!("{} {}", request.method(), request.uri());
        let (status, headers, body) = shelf_playground::call(&app, request).await;
        let etag = headers
            .get(header::ETAG)
            .map(|v| v.to_str().unwrap().to_owned());
        println!(
            "{line:<18} {} {:<5} {body}",
            status.as_u16(),
            etag.unwrap_or_default()
        );
        statuses.push(status.as_u16());
    }
    assert_eq!(statuses, [200, 200, 412, 428, 404]);
    Ok(())
}
```

```
GET /products/1    200 "1"   {"id":1,"name":"Mug","price_cents":1299,"version":1}
PATCH /products/1  200 "2"   {"id":1,"name":"Mug","price_cents":990,"version":2}
PATCH /products/1  412       {"currentVersion":2,"error":"changed since you read it"}
PATCH /products/1  428       {"error":"send If-Match with the ETag you read"}
PATCH /products/7  404       {"error":"not found"}
```

The handler parses `If-Match` itself and requires it: a `PATCH` without one is answered 428 Precondition Required, which tells the client that this API does not accept blind writes. stockroom is more lenient: without `If-Match` it applies the patch at whatever version the row has (`version = coalesce($2, version)`). A shelf patch always checks a version, so the capstone gets stockroom's behaviour by reading the row first and retrying on `Stale`:

```rust
// orm/stockroom-shelf/src/repo/products.rs (excerpt)
pub async fn update(
    db: &impl Executor,
    sku: &Sku,
    expected_version: Option<i32>,
    mut patch: ProductPatch,
) -> Result<Product, AppError> {
    loop {
        let product = by_sku(db, sku).await?;
        patch.version = expected_version.unwrap_or(product.version);
        match patch.apply(db, &product.id).await {
            Err(shelf::Error::Stale { .. }) if expected_version.is_none() => continue,
            result => return result.map_err(app_error),
        }
    }
}
```

orm/stockroom-shelf/src/repo/products.rs

Each retry means another write succeeded in between, so the loop always makes progress; the capstone's test sends 30 patches without `If-Match` at once, and the row ends at version 31. The price is a read before every patch: two round trips where stockroom's `coalesce` needs one (PLAN.md §12, findings 51 and 54). The route names a product by SKU and `apply` takes the key, so the capstone reads first anyway.

Side by side — *an edit from two admin tabs*

The task: two tabs hold version 1 of a product; one saves a new price, then the other saves a new name against version 1, and must be told the product changed.

```rust
// SeaORM (illustration)
// Only the `Set` fields are written: a partial update by construction.
let changes = product::ActiveModel {
    name: Set("Blue mug".to_owned()),
    ..Default::default() // every other field NotSet
};
// No version attribute in SeaORM 2.0.4: the check is a filter, and the count says the rest.
let result = product::Entity::update_many()
    .set(changes)
    .col_expr(product::COLUMN.version, product::COLUMN.version.add(1))
    .filter(product::COLUMN.id.eq(1))
    .filter(product::COLUMN.version.eq(1))
    .exec(&db)
    .await?;
if result.rows_affected == 0 {
    // stale or gone: a second query to find out which, and no row returned either way
}
```

By hand, as stockroom's `products::update` writes it (a `coalesce` per field and for the version, then a second query when nothing was updated), and with shelf, in one program that runs the same two tabs through both:

```rust
use shelf::{FromRow, Patch, Table};
use tokio_postgres::Client;

#[derive(Debug, FromRow, Table)]
#[shelf(table = "products")]
struct Product {
    #[shelf(primary_key)]
    id: i64,
    name: String,
    price_cents: i64,
    #[shelf(version)]
    version: i32,
}

#[derive(Debug, Default, Patch)]
```

```
#[shelf(model = Product)]
struct ProductPatch {
    name: Option<String>,
    price_cents: Option<i64>,
    #[shelf(version)]
    version: i32,
}

/// By hand: stockroom's shape. One statement when it works, two when it does not.
async fn update_by_hand(
    db: &Client,
    id: i64,
    expected_version: i32,
    name: Option<&str>,
    price_cents: Option<i64>,
) -> Result<Product, shelf::Error> {
    let updated = db
        .query_opt(
            "UPDATE products SET
                 name = coalesce($3, name),
                 price_cents = coalesce($4, price_cents),
                 version = version + 1
             WHERE id = $1 AND version = $2
             RETURNING id, name, price_cents, version",
            &[&id, &expected_version, &name, &price_cents],
        )
        .await?;
    if let Some(row) = updated {
        return Product::from_row(&row);
    }
    // Nothing updated: gone, or changed by someone else first.
    let current = db
        .query_opt("SELECT version FROM products WHERE id = $1", &[&id])
        .await?;
    Err(match current {
        None => shelf::Error::NotFound { model: "Product" },
        Some(row) => shelf::Error::Stale {
            model: "Product",
            current_version: row.try_get::<_, i32>(0)?.into(),
        },
    })
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (
                 id bigint PRIMARY KEY, name text NOT NULL, price_cents bigint NOT NULL,
                 version integer NOT NULL DEFAULT 1
             );
             INSERT INTO products (id, name, price_cents) VALUES (1, 'Mug', 1299), (2,
'Mug', 1299);",
        )
        .await?;

    // By hand, on product 1.
    update_by_hand(&client, 1, 1, None, Some(990)).await?;
    let by_hand = update_by_hand(&client, 1, 1, Some("Blue mug"), None)
        .await
        .unwrap_err();

    // shelf, on product 2.
    ProductPatch {
        price_cents: Some(990),
        version: 1,
        ..ProductPatch::default()
    }
```

```
    .apply(&client, &2)
    .await?;
    let with_shelf = ProductPatch {
        name: Some("Blue mug".into()),
        version: 1,
        ..ProductPatch::default()
    }
    .apply(&client, &2)
    .await
    .unwrap_err();

    for error in [&by_hand, &with_shelf] {
        assert!(matches!(
            error,
            shelf::Error::Stale {
                current_version: 2,
                ..
            }
        ));
    }
    let rows = Product::all(&client).await?;
    assert!(
        rows.iter()
            .all(|p| (p.name.as_str(), p.price_cents) == ("Mug", 990))
    );
    Ok(())
}
```

	by hand	shelf	SeaORM
Same	One `UPDATE` that writes only what was sent and checks the version in its `WHERE`; the second tab's save changes nothing	The same	The same
Different	`coalesce` per field, so a field can never be set to NULL; telling stale from gone takes a second query, and the version to report comes from it	A flag per field, so NULL can be written; one statement for every patch of the type, which also returns the current version or says the row is gone; `Stale` and `NotFound` are errors with what a 412 or 404 needs	Partial by construction (`Set` fields only), but each combination of fields renders a different statement; no version support, so the check is a filter you add, and zero rows affected tells you nothing more

In shelf

The patch's parameters borrow from the patch, and the flags borrow from nowhere:

```
// orm/shelf/src/patch.rs (excerpt)
/// Whether a patch field is set, as the flag beside its value: `&true` or `&false`, which
live
/// for ever, so the parameters can still borrow from the patch alone.
#[doc(hidden)]
pub fn flag<T>(value: &Option<T>) -> &'static (dyn ToSql + Sync) {
    if value.is_some() { &true } else { &false }
}
```

orm/shelf/src/patch.rs

`&true` is a reference to a constant, which Rust promotes to a `'static` value, so `flag` can return a reference without anything to borrow it from. The value beside each flag is the field itself, `param(&self.price_cents)`,

an `&Option<i64>`: tokio-postgres's `ToSql` for `Option<T>` sends `None` as a NULL of `T`'s type, and for `Option<Option<T>>`, both `None` and `Some(None)` are NULLs, the flag deciding which of them is written.

When not to use `Patch`

- **A business operation.** An order's status transition (stockroom's `POST /orders/{id}/transition`) has rules about which changes are allowed from which state; a patch that can set `status` to anything bypasses them. Write a function with its own statement.
- **Counters and expressions.** `stock = stock - 1` depends on the row's current value; a patch writes a value from the client.
- **A field whose change must always be checked.** A patch writes whatever fields are present. If changing the price needs a permission that changing the name does not, check which fields are `Some` before applying it (Exercise 2's `changed` is the list).

Summary

- A partial update needs three states per nullable field: absent, `null` and a value, which is `Option<Option<T>>`. serde needs a one-line `deserialize_with` (or serde_with's `double_option`) and `#[serde(default)]` to read `null` as `Some(None)`. `deny_unknown_fields` keeps a misspelt field from becoming an empty patch.
- Three designs: (a) a `SET` list per shape, up to $2^n - 1$ statements per connection; (b) `coalesce`, one statement that cannot write NULL; (c) a flag per field, one statement with NULLs. Measured through lagproxy, (c) costs no more per patch than (b), and (a) a prepare per new shape. shelf uses (c).
- `#[derive(Patch)]` on a struct of `Option` fields with `#[shelf(model = …)]` generates the columns, a flag and a value per field, and the statement, built once in a `LazyLock` by `patch_sql`, because its placeholder numbers depend on the model's key.
- `#[shelf(version)]` on the model adds `version = version + 1` and `AND version = $n` to every update and patch; the patch carries the version it was made against, which a `const` assertion enforces at compile time.
- One statement tells success, `Stale { current_version }` and `NotFound` apart: a data-modifying `WITH`, a `LEFT JOIN` to one row, and a subquery that sees the version from before the update.
- `#[shelf(updated_at)]` sets `now()`, the transaction's start time: writes in one transaction share it.
- In HTTP, the version is the `ETag` and `If-Match` is the patch's version; `Stale` is a 412 with the current version. Patching "at any version" takes a read and a retry on `Stale`.

Exercises

1. **An empty patch.** Apply a patch with every field `None` to a versioned model. What happens to the row? Write `apply_if_any`, which sends nothing for an empty patch. *Hint:* look at the `SET` list `patch_sql` builds when no flag is true. Decide what to return when nothing is sent.
2. **What changed.** Give `ProductPatch` a method `changed(&self) -> Vec<&'static str>` that lists the columns a patch sets, for an audit log or a permission check. *Hint:* `Patch::COLUMNS` is in field order; pair it with each field's `is_some()`.
3. **A `bigint` version.** Declare the version column `bigint` and the model's field `i64`. Does everything still work, `Stale` included? *Hint:* look at the type of `shelf_version` in the statement, and of `current_version` in `Error::Stale`.

Solutions

1. The empty patch still writes: every `CASE` keeps its column, but `version = version + 1` (and `updated_at = now()`, if the model has one) is in the `SET` list regardless, so the row moves to the next version, and every other client's copy becomes stale. Checking before the round trip avoids that:

```rust
use shelf::{Executor, FromRow, Patch, Table};

#[derive(Debug, FromRow, Table)]
#[shelf(table = "products")]
struct Product {
    #[shelf(primary_key)]
    id: i64,
    name: String,
    price_cents: i64,
    #[shelf(version)]
    version: i32,
}

#[derive(Debug, Default, Patch)]
#[shelf(model = Product)]
struct ProductPatch {
    name: Option<String>,
    price_cents: Option<i64>,
    #[shelf(version)]
    version: i32,
}

impl ProductPatch {
    fn is_empty(&self) -> bool {
        self.name.is_none() && self.price_cents.is_none()
    }

    /// Applies the patch if it sets anything; `None`, and nothing sent, if it does not.
    async fn apply_if_any(
        &self,
        db: &impl Executor,
        id: i64,
    ) -> Result<Option<Product>, shelf::Error> {
        if self.is_empty() {
            return Ok(None);
        }
        self.apply(db, &id).await.map(Some)
    }
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (
                 id bigint PRIMARY KEY, name text NOT NULL, price_cents bigint NOT NULL,
                 version integer NOT NULL DEFAULT 1
             );
             INSERT INTO products (id, name, price_cents) VALUES (1, 'Mug', 1299);",
        )
        .await?;
    let empty = ProductPatch {
        version: 1,
        ..ProductPatch::default()
    };

    // Through `apply`: nothing set, and still a new version.
    let after = empty.apply(&client, &1).await?;
    assert_eq!((after.name.as_str(), after.version), ("Mug", 2));

    // Through `apply_if_any`: nothing sent, the version unchanged.
    let empty = ProductPatch {
        version: 2,
        ..ProductPatch::default()
    };
    assert!(empty.apply_if_any(&client, 1).await?.is_none());
    assert_eq!(Product::find(&client, &1).await?.version, 2);
```

```
    Ok(())
}
```

`apply_if_any` returns `None` rather than the row because it never read the row, and it does not check the version either: an empty patch against a stale version is not an error here. A handler that must answer an empty `PATCH` with the resource would `find` it, and compare the version itself. The derive could generate `is_empty` from the same fields as `params`; it is written by hand here.

2. `COLUMNS` and the fields are in the same order, so a zip pairs them:

```
use shelf::{FromRow, Patch, Table};

#[derive(Debug, FromRow, Table)]
#[shelf(table = "products")]
#[allow(dead_code)]
struct Product {
    #[shelf(primary_key)]
    id: i64,
    name: String,
    price_cents: i64,
    note: Option<String>,
}

#[derive(Debug, Default, Patch)]
#[shelf(model = Product)]
struct ProductPatch {
    name: Option<String>,
    price_cents: Option<i64>,
    note: Option<Option<String>>,
}

impl ProductPatch {
    /// The columns this patch sets, quoted, in field order.
    fn changed(&self) -> Vec<&'static str> {
        let set = [
            self.name.is_some(),
            self.price_cents.is_some(),
            self.note.is_some(),
        ];
        Self::COLUMNS
            .iter()
            .zip(set)
            .filter_map(|(&column, set)| set.then_some(column))
            .collect()
    }
}

fn main() {
    let patch = ProductPatch {
        price_cents: Some(990),
        note: Some(None),
        ..ProductPatch::default()
    };
    println!("{:?}", patch.changed());
    assert_eq!(patch.changed(), [r#""price_cents""#, r#""note""#]);
}
```

```
["\"price_cents\"", "\"note\""]
```

The names are quoted, as `COLUMNS` holds them; an audit log would strip the quotes, or the derive could generate a second list of plain names. `Some(None)` counts as a change: the patch sets `note` to NULL.

3. It works. The statement casts the version it reports, `"version"::bigint AS shelf_version`, and `Error::Stale` carries an `i64`, so an `int4` and an `int8` version are read the same way; `version + 1` is computed in the column's own type, and the patch's `version` field is typed by the server as the column, here `int8`:

```
use shelf::{FromRow, Patch, Table};

#[derive(Debug, FromRow, Table)]
#[shelf(table = "products")]
struct Product {
    #[shelf(primary_key)]
    id: i64,
    price_cents: i64,
    #[shelf(version)]
    version: i64,
}

#[derive(Debug, Patch)]
#[shelf(model = Product)]
struct ProductPatch {
    price_cents: Option<i64>,
    #[shelf(version)]
    version: i64,
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (
                 id bigint PRIMARY KEY, price_cents bigint NOT NULL,
                 version bigint NOT NULL DEFAULT 5000000000
             );
             INSERT INTO products (id, price_cents) VALUES (1, 1299);",
        )
        .await?;
    let patch = |version| ProductPatch {
        price_cents: Some(990),
        version,
    };
    let saved = patch(5_000_000_000).apply(&client, &1).await?;
    assert_eq!(saved.version, 5_000_000_001);
    let error = patch(5_000_000_000).apply(&client, &1).await.unwrap_err();
    assert!(matches!(
        error,
        shelf::Error::Stale {
            current_version: 5_000_000_001,
            ..
        }
    ));
    Ok(())
}
```

A version past `i32::MAX` is reported intact. The capstone's `app_error` is the place where an `i64` would not fit: stockroom's `StaleVersion` holds an `i32`, so it converts with `try_from` and reports an internal error for a version it cannot represent (Chapter 7, §7.5).

CHAPTER 10

A typed query builder

> **Sketch, 9 November 2026 (revised after S1c).** A plan of this chapter, not its text; it will be replaced by the chapter. Part III, first chapter. Builds on Chapters 6 and 7, and on main book Chapter 2 (parameters, injection), Chapter 3 (`= ANY($1)` with arrays) and Chapter 9 (keyset pagination). shelf: the column constants generated by `Table` (`shelf-codegen/src/columns.rs`) and the query module `shelf/src/query/` (`column.rs`, `expr.rs`, `render.rs`, `mod.rs`) (built, S1c); tests `tests/query.rs` and `tests/ui/query.rs`. Crates: tokio-postgres 0.7.18. Findings: §12 #34–#36, #41. Plan: orm/PLAN.md §4.0c, §7, Chapter 10; §10 (the builder's decisions).

`find` and `all` cover two queries. The third most common is "these rows, filtered, sorted, a page at a time", and its filter changes with the request: a tag if the client sent one, a price range if it sent that, both or neither. Writing that SQL by hand means assembling a string and a parameter list in step, numbering each `$n` correctly, which is exactly the code where injection bugs and off-by-one parameter errors live.

A query builder assembles it for you, from typed pieces. The derive already knows every column and its Rust type, so it can generate a constant per column, `Product::PRICE_CENTS`, that knows both. A filter built from it, `Product::PRICE_CENTS.gt(1_000)`, can only take an `i64`, and a mistake is a compile error at the call. The builder numbers the parameters, keeps the values out of the SQL, and stays small enough to read in one sitting.

In this chapter you will learn:

- what a query builder should type (columns and values) and what it should leave to SQL
- how `Table` generates a `Column<M, T>` constant per field, and what `PhantomData` does there
- how expressions become SQL with numbered parameters and an owned parameter list
- ordering, limits and keyset pages
- which mistakes fail to compile, shown with UI tests
- the escape hatch: raw SQL, inside a filter or instead of the builder

10.1 What to type

Plan:

- Typed: the column (it belongs to this model) and the value (it has the column's type).
- Left to SQL: functions, joins, subqueries; the builder accepts a raw fragment with its parameters instead of modelling them.

10.2 Column constants

Plan:

- `Product::PRICE_CENTS: Column<Product, i64>`, an inherent constant generated from each field, as visible as the field; only the quoted name is stored. `T` is what the column holds: the `try_from` source type, `serde_json::Value` for a `json` field (the expansion test `columns.expanded.rs`, printed).
- A **Rust refresher** on `PhantomData<fn() -> (M, T)>`: a zero-sized marker that makes the type carry the model and the value type, and why `fn() -> …` (always `Send`, `Sync`, `Copy`).
- An inherent constant shadows a trait constant of the same name: `Product::VERSION` is the column, `<Product as Table>::VERSION` the metadata (§12 #35).
- `Value<T>`: what a condition accepts. Measured on the compiler, not guessed: one widening impl (`i32` for `i64`) makes integer literals fall back to `i32` and `3_000_000_000` fail to compile, so shelf converts no numbers (§12 #34).

10.3 Expressions

Plan:

- `eq`, `ne`, `lt`, `le`, `gt`, `ge`, `between` and `any` (`= ANY($n)`, one array) on every column; `like` only on text columns, `is_null` only on `Option` columns, `contains` (`@> $n`, a one-element array) only on array columns: impl blocks on `Column<M, String>`, `Column<M, Option<T>>`, `Column<M, Vec<E>>`. `and`, `or`, and `!` through `std::ops::Not` (a method called `not` trips clippy's `should_implement_trait`).
- An expression tree rendered at each call into SQL text and a parameter list by one function that writes both (`render.rs`), so the `$n` and the list cannot drift; parentheses where an `OR` sits inside an `AND`. The rendered SQL printed (`query.sql()`, `query.params()`).
- How PostgreSQL types each parameter (§12 #36): against a domain column, `=`, `= ANY` and `LIKE` take the base type, so `&str` works there.

10.4 Owned parameters

Plan:

- The query owns its values (`Box<dyn ToSql + Sync + Send>`), so it can be built in one function and run in another, across an await and inside `tokio::spawn` (a test does).
- The cost: one allocation per parameter; nothing compared with a round trip.
- Its methods are inherent `async fn`s: unlike `async fn` in a trait, their futures leak their auto traits, so they are `Send` for a generic executor (a compile-only check in the tests).

10.5 Order, limit and pages

Plan:

- `order_by(Product::SKU.asc())`, `limit`, `offset` (parameters, so a page size is not a new statement); key order when nothing is ordered; `first` (`LIMIT 1`); `count` and `exists` on the same filter, ignoring order and paging.
- Keyset pagination with `after(Product::SKU, last)`: `"sku" > $n`, ordered by the column first (main book Chapter 9's recommendation), and why `offset` gets slower page by page.

10.6 What fails to compile

Plan:

- The captured errors of `tests/ui/query.stderr` (§12 #41): `&str` and `i32` for a `bigint` column, in `Value`'s words; another model's column, first as rustc reported it through `impl Into<Expr<M>>` ("… but trait `From<Raw<Product>>` is implemented") and then through the `Condition<M>` trait ("`Expr<Order>` is not a

condition on `Product`"), the change that reading the error made; `like` on an `i64`, `is_null` on a `String`, `contains` on a text.
- What still fails at run time: a raw fragment, and a column whose type changed in the database (Chapter 13).

10.7 The escape hatch

Plan:

- `fetch_all::<T>` with hand-written SQL, as always.
- `Expr::raw(r#"lower("name") = lower($1)"#).bind("tee")` inside a filter: its `$1…` renumbered among the builder's by a scanner that skips quoted text, `$tag$` strings and comments (`render.rs`, unit tested), and a panic where the fragment is used if it refers to a value it was not given (it would silently take another condition's).

The programs

- `run` (proposed): a filtered, sorted page against a temporary table.
- `run`: the rendered SQL and its parameters printed for three filters.
- `run`: keyset pages until the end.
- UI tests excerpted: the wrong value type, the wrong model, the missing methods.
- `run`: a raw fragment inside a filter.
- `should-panic`: a raw fragment with a placeholder it has no value for.

Side by side

A filtered, sorted page: by hand (stockroom's keyset `page`) · shelf (`Product::query().filter(…).order_by(…).limit(20).all(&db)`) · diesel (`products.filter(price_cents.gt(1000)).order(sku.asc()).limit(20)`, check).

In stockroom

`page` in stockroom/src/repo/products.rs: the optional tag filter written as `($2::text IS NULL OR tags @> ARRAY[$2])`, which a builder replaces with a filter that is simply absent.

Claims

- Opinion: a query builder earns its place for optional filters; for fixed queries, plain SQL is clearer.
- Docs: PostgreSQL's array operators (`@>`, `= ANY`).

When not to use the builder

- Reports, aggregates, CTEs, window functions: SQL with a `FromRow` struct.
- A fixed query run often: a hand-written statement caches as one statement; each filter shape is a different statement, cached once per shape on a deadpool connection.

Exercises

Plan:

1. Add `ilike`. *Hint:* one more operator in the renderer.
2. Add `in_range` for `Option` bounds, skipping a missing bound. *Hint:* build the expression conditionally.
3. Render a query and run it through `fetch_all` yourself. *Hint:* the builder's output is SQL and parameters.

> **Open questions.** Settled in S1c (PLAN.md §10): the value trait (§12 #34); one statement per filter shape, with `LIMIT` and `OFFSET` as parameters; `count` on both `Table` and `Query`. Still open: whether the chapter builds the `$n` scanner step by step or shows it finished.

CHAPTER 11

Relations without N+1

> **Sketch, 9 November 2026 (revised after S1c).** A plan of this chapter, not its text; it will be replaced by the chapter. Part III, second chapter. Builds on Chapters 6, 7 and 10, and on main book Chapter 3 (arrays as parameters), Chapter 7 (pipelining) and lagproxy (apps/lagproxy). shelf: the `has_many` and `belongs_to` keys in `attrs.rs`, the methods in `shelf-codegen/src/relations.rs`, the loaders in `shelf/src/relations.rs` (built, S1c); tests `tests/relations.rs`, `tests/ui/relations.rs`; the benchmark `examples/bench_relations.rs`. Crates: tokio-postgres 0.7.18, futures-util. Findings: §12 #41, #42; root PLAN.md §12 #21. Plan: orm/PLAN.md §4.0c, §7, Chapter 11; §10 (relations).

An order has lines. Showing a page of fifty orders with their lines is the textbook case where an ORM is either a great help or a disaster. The disaster is the obvious loop: fetch the orders, then, for each one, fetch its lines. That is fifty-one queries, and over a network each one costs a round trip, so the page that took a millisecond on localhost takes a second in production. It is called the N+1 problem, and the cure is older than ORMs: fetch all the lines for all fifty orders in one query, and sort them into their orders in Rust.

shelf makes the cure the easy path. A `has_many` attribute on `Order` generates a loader that takes a slice of orders and returns their lines grouped by order, in one query with `order_id = ANY($1)`. The chapter measures the loop and the loader through lagproxy, then looks at the two other answers, a join and pipelining, and explains why shelf has no lazy loading at all.

In this chapter you will learn:

- how `belongs_to` and `has_many` declare a relation, and the methods they generate
- what the N+1 problem costs, measured through lagproxy
- how batched loading with `= ANY($1)` fetches every child in one round trip
- how to pipeline independent loads on one connection
- when a join is the better query, and what it costs in duplicated rows
- why shelf has no lazy loading

11.1 Declaring relations

Plan:

- `#[shelf(has_many(OrderLine, key = order_id, as = lines))]` on `Order`; `#[shelf(belongs_to(Order, key = order_id))]` on `OrderLine`. `has_many` needs `as`: no pluralizing; `belongs_to` takes the model's name in snake case (`order_line`) unless `as` says otherwise. Parsing `as`, a keyword, works because syn's nested-meta paths accept keywords (`Ident::parse_any`). A **Macro lens** on `key = …`: it names a field of the *other* struct for `has_many`, which the derive cannot see, so the generated code names `OrderLine::ORDER_ID` and `line.order_id` and lets the compiler check them, spanned on the attribute's tokens (the captured errors of `tests/ui/relations.stderr`, §12 #41).

- Generated: `order.lines(&db) -> Vec<OrderLine>`, `Order::load_lines(&db, &orders) -> Vec<Vec<OrderLine>>` (aligned with `orders`), `line.order(&db) -> Order` (an `Option` when the field is written `Option<…>`), `OrderLine::load_order(&db, &lines) -> HashMap<i64, Order>`; the expansion of `relations.rs` printed.

11.2 N+1, measured

Draft prose:

Through lagproxy at 5 ms each way, one round trip cost 13.3–13.8 ms on this machine (§12 #42). Loading the lines of a hundred orders one order at a time took 2.7 seconds on a plain `Client`, which prepares every statement and so pays two round trips per order, and 1.35 seconds on a pooled connection, whose statement cache brings it down to one. The loop is not slow because PostgreSQL is slow; on localhost the same loop took 5 to 12 milliseconds. It is slow because it waits for the network a hundred times.

Plan:

- The loop timed through lagproxy, and counted with an executor of our own that wraps a client and counts its statements (`Counting` in `tests/relations.rs`: `Executor` is not sealed).

11.3 Batched loading

Plan:

- One query through the builder, `WHERE "order_id" = ANY($1)` with the orders' keys as one array, then a `HashMap` from key to children; the loader returns them in the parents' order. A nullable foreign key (`ForeignKey`, implemented for `K` and `Option<K>`) skips NULLs.
- Measured through lagproxy (§12 #42): 26.7–28.5 ms on a `Client` (two round trips, prepare and query) and 14.0–14.6 ms pooled (one), against 2.7 s and 1.35 s for the loop.
- Keys need `Hash + Eq + Clone`; relations follow single-column keys (`has_many` on a model with a composite key is refused at the attribute).

11.4 Pipelining independent loads

Plan:

- Lines and the store for a page of orders: two loaders, independent, joined with `try_join!` on one connection (a test does), costing about one round trip (main book Chapter 7; root §12 #21: one pipelined client beat a pool over latency). Every executor allows it: the loaders take `&db`, and a `Transaction` is shared by reference too.
- Pipelining the N+1 loop itself (`try_join_all` of `order.lines(&db)`) took 32–35 ms through lagproxy, nearly the batched time, but on localhost 3.3–3.5 ms against 0.2 ms batched: it hides the latency of a hundred queries, not their work (§12 #42).

11.5 Joins

Plan:

- A join returns each order once per line: duplicated parent columns, and a `FromRow` with `flatten` to read them.
- `json_agg` or `array_agg` of a composite to return one row per order; when that wins (a small number of children, one round trip).

11.6 Why no lazy loading

Plan:

- Lazy loading needs a hidden connection in the model and a hidden round trip in a field access; in async Rust it would also need `.await` on a getter. shelf makes every round trip visible instead (opinion, argued).

The programs

- `needs=lagproxy` (proposed): the N+1 loop timed.
- `needs=lagproxy`: the batched loader timed; asserts it is at least ten times faster than the loop for fifty orders (relative; for a hundred orders, 95–100× in release, §12 #42; to check in a debug build).
- `run`: the counting executor: the loop runs 101 statements, the loader 2.
- `needs=lagproxy`: two loaders pipelined with `try_join!`.
- `run`: a join with `flatten`, and `json_agg` into `Json<Vec<…>>`.

Side by side

Orders with their lines: by hand (stockroom's `orders::get`, two pipelined queries) · shelf (`Order::load_lines`) · SeaORM (`find_with_related`, check).

The data

Temporary `orders` and `order_lines` tables with fifty orders and a few lines each; stockroom's schema in the capstone.

Claims

- Measured: §12 #42 (N+1, pipelined and batched, through lagproxy and on localhost); root PLAN.md §12 #21 (pipelining against a pool).
- Opinion: no lazy loading; every round trip should be visible at the call site.

When not to use relation loaders

- A report across several tables: one SQL query.
- A deep tree (categories of categories): a recursive CTE.

Exercises

Plan:

1. Load lines and products for a page of orders, two levels. *Hint:* the second loader takes the first one's children.
2. Return lines ordered by a column, inside each group. *Hint:* order in SQL, group in Rust.
3. Write the `json_agg` version as a `FromRow` struct. *Hint:* `#[shelf(json)]`.

> **Open questions.** Settled in S1c (PLAN.md §10): the names and signatures; a `Vec` per parent for `has_many`, a `HashMap` for `belongs_to`; single-column keys only; pipelining works through any executor. Still open: whether composite foreign keys become an exercise.

CHAPTER 12

Bulk writes and streams

> **Sketch, 9 November 2026 (revised after S1c).** A plan of this chapter, not its text; it will be replaced by the chapter. Part III, third chapter. Builds on Chapters 5, 7 and 8, and on main book Chapter 5 (domains and arrays), Chapter 9 (`RowStream`, backpressure, `pin!`) and Chapter 10 (COPY, binary COPY, pgload). shelf: `Insert::insert_many` and `Insert::copy_in` in `shelf/src/bulk.rs`, with `COLUMNS`, `COPY` and `arrays` from the derive; `fetch_stream`, `Query::stream` and `ModelStream` in `shelf/src/stream.rs`; `Executor::query_raw` and `copy_in` (built, S1c); tests `tests/bulk.rs`, `tests/stream.rs`, `tests/ui/stream.rs`; the benchmark `examples/bench_bulk.rs`. Crates: tokio-postgres 0.7.18, futures-util. Findings: §12 #37–#39, #44–#46; root PLAN.md §12 #1, #4, #13, #22, #34–#37. Plan: orm/PLAN.md §4.0c, §7, Chapter 12; §10 (bulk writes, streams).

Inserting ten thousand rows one `insert` at a time is the N+1 problem again, in the other direction: ten thousand round trips. The main book measured it with pgload: one `INSERT` per row was about a hundred times slower than binary COPY on localhost, and pipelining or `unnest` batches recovered most of the difference (root PLAN.md §12 #22). A model layer that offers only `insert` pushes every bulk load into that slow path.

shelf adds two bulk writes, both generated from the `New…` struct the reader already has: `insert_many`, one `INSERT … SELECT * FROM unnest(…)` with an array per column, and a binary COPY writer for the largest loads. Then it turns to the other large direction, reading, where a `Vec` of a million models is a million allocations held at once, and gives queries a stream of models that resolves its columns once and respects backpressure.

In this chapter you will learn:

- how `insert_many` sends a batch as one array per column through `unnest`
- why arrays of domain-typed columns need casts, and how shelf adds them
- how the derive writes a binary COPY row, and which types it must declare
- what each bulk strategy costs, measured against pgload's table
- how to stream models from a `RowStream`, resolving columns once
- how a COPY rejected at its start can kill a connection in 0.7.18, and how shelf avoids it

12.1 `insert_many`

Plan:

- One statement, `INSERT INTO "t" ("a", "b") SELECT * FROM unnest($1::"pg_catalog"."text"[], …) RETURNING …`, with one `Vec<&T>` per column built by the derive's `arrays` from the slice of `New…` values (`__private::array`).
- In the select list, `unnest` keeps a composite whole; in `FROM` it expands it into fields (root PLAN.md §12 #4), which matters for composite-typed columns.

- The casts: `unnest` of an untyped parameter cannot be resolved, so each array needs its type. shelf learns it from the struct's own `INSERT`, prepared: `Statement::params()` holds each column's type as PostgreSQL types the parameter, a domain as the domain, an enum by name, so a batch accepts exactly what `insert` accepts (§12 #37). A prepared `SELECT … LIMIT 0`, the first plan, would describe a domain column as its base type, which a domain newtype refuses.
- An array column cannot go through `unnest`: tokio-postgres cannot send `Vec<Vec<String>>`, and `unnest` flattens a 2-D array (§12 #37). `Error::Unsupported`, before anything is sent, pointing to `copy_in`.
- `RETURNING` gives the rows back in the order PostgreSQL inserted them, which in practice is the slice's; SQL does not promise it (a **SQL lens**).

12.2 Binary COPY from the derive

Plan:

- `NewProduct::copy_in(&db, &rows)`: the derive's `COPY "t" ("a", "b") FROM STDIN (FORMAT binary)`, joined at compile time, and a `BinaryCopyInWriter` given the prepared `INSERT`'s types, fed each row's `params()`: the same values `insert` sends.
- The main book's warning: binary COPY trusts the declared types, and a wrong type of the same size loads wrong data silently (root §12 #37); shelf's types are the columns' own, from the server.
- Pinned writer: `pin!` inside the function, which drops it on every return; `drop` on a pinned reference drops nothing (root §12 #34). A row refused on the client leaves the connection usable once the writer is dropped (§12 #38).

12.3 Measured

Plan:

- `bench_bulk`, pgload's rows and table, five ways (§12 #44): 20,000 rows in 0.77 s by a loop of `insert`, 0.15 s pipelined, 0.016–0.018 s by `insert_many`, 0.006 s by `copy_in`, beside pgload's 0.756, 0.120, 0.014 and 0.008 s; and 2,000 rows through lagproxy at 1 ms, where the loop takes 9.4–9.8 s and every batched form 9–14 ms.
- Why the two tables differ at 1 ms: pgload's timer also covers five round trips of setup and commit (§12 #45). A benchmark says what it times.
- A **Pitfall**: outside a transaction, every insert commits and waits for its WAL flush: 20,000 pipelined inserts took 0.91–0.98 s that way and 0.13–0.14 s in one transaction (§12 #46).
- A keyed table: WAL and an index add to every strategy, and make the batched ones noisy.

12.4 Streams of models

Plan:

- `Product::query().….stream(&db)` and `fetch_stream::<T>(&db, sql, params)` over `Executor::query_raw`: the columns resolved once from the prepared statement, so a wrong mapping fails before the first row, and each row decoded as it arrives. `ModelStream` boxes the `RowStream`, so it is `Unpin` and needs no `pin!` (a **Rust refresher** on why).
- Backpressure: each request's response channel holds one message, so an unread stream stalls every later request on the connection; a test shows a `SELECT 1` waiting until the stream is dropped (`tests/stream.rs`; main book Chapter 9).
- Who owns the connection while the stream lives (root PLAN.md §12 #13): `ModelStream<'a, T>` borrows the executor, so returning a pooled connection mid-stream is a compile error (E0505, `tests/ui/stream.stderr`). An HTTP body that must own its connection is Chapter 14's.

12.5 A COPY rejected at its start

Plan:

- In 0.7.18 a `COPY … FROM STDIN` that the server rejects before it starts (a missing table or column) leaves the connection unusable: rust-postgres issue #789 (root PLAN.md §12 #36).
- Preparing the `COPY` checks nothing, not even the table (§12 #38). shelf's guard is the prepared `INSERT` of 12.1, which fails cleanly for a missing table or column; a test drops a column and shows the error and the connection still working (`tests/bulk.rs`).

The programs

- `run` (proposed): `insert_many` of a thousand rows, counted back.
- `run`: binary COPY of the same rows, with the types learnt from the `INSERT`.
- `run`: the strategies timed on ten thousand rows in a transaction; asserts COPY is faster than the loop by at least ten times (relative; about 125× in release, §12 #44; to check in a debug build).
- `run`: an array column refused by `insert_many`, then loaded by `copy_in`.
- `run`: a stream of a hundred thousand models, counted without holding them.
- `run`: a missing column caught by the guard, and the connection still usable.

Side by side

Loading ten thousand rows: by hand (pgload's `unnest` batches) · shelf (`insert_many`) · sqlx (`QueryBuilder::push_values`, or `copy_in_raw`; check).

In stockroom

`stockroom seed` writes products and the ledger with binary COPY by hand (stockroom/src/cli/seed.rs).

Claims

- Measured: root PLAN.md §12 #22 (pgload); shelf's strategies, §12 #44–#46.
- Docs: rust-postgres issue #789; PostgreSQL's `COPY`.
- Opinion: offer `insert_many` by default, COPY when the batch is large and trusted.

When not to use shelf's bulk writes

- A client's CSV: COPY text through a staging table (main book Chapter 16), with errors reported by line.
- A load that needs `ON CONFLICT` per row: `insert_many` with a conflict clause, or `MERGE`.

Exercises

Plan:

1. Split `insert_many` into batches of a given size. *Hint:* chunks of the slice.
2. Make the stream report its row count at the end. *Hint:* `rows_affected()` once exhausted.
3. Load through COPY into a temporary table, then upsert. *Hint:* main book §10.6.

> **Open questions.** Settled in S1c (PLAN.md §10): the types come from the prepared `INSERT`; the stream borrows its connection. Still open: the debug-build ratio for a relative timing assertion.

CHAPTER 13

Trusting the mapping: schema checks and tests

> **Sketch, 9 November 2026 (revised after S1c).** A plan of this chapter, not its text; it will be replaced by the chapter. Part III, fourth chapter. Builds on Chapters 4 and 5, and on main book Chapter 4 (`Statement::columns`) and Chapter 17 (testing against a real database). shelf: `shelf::verify`, `Error::Schema` and `Mismatch` (`shelf/src/verify.rs`, `error.rs`), `Table::FIELDS` and `Executor::prepare_uncached` (built, S1c); the test suites (built, S1a): tests/expand.rs, tests/ui.rs, tests/from_row.rs, tests/verify.rs and the harness tests/common/mod.rs. Crates: tokio-postgres 0.7.18, trybuild 1.0.122. Findings: PLAN.md §12 #6, #11, #12, #40, #43. Plan: orm/PLAN.md §4.0c, §6, §7, Chapter 13; §10 (`verify`).

A struct and a table are two descriptions of the same data, kept in different places by different people. A migration renames a column; a column becomes nullable; a `bigint` becomes `numeric`. Chapter 5's check catches each of those, but only when the query runs, which may be the first request after a deploy, or a nightly job that runs at three in the morning.

This chapter moves the check earlier. `shelf::verify` prepares each model's own `SELECT` at startup, compares its columns with the struct's fields, asks the catalog which columns may be NULL, and reports every mismatch at once. Then it turns to the other side of trust, the macros themselves, and shows the three kinds of test shelf has used since phase S1a: expansion tests, UI tests and tests against the database, with what each one catches and what it cannot.

In this chapter you will learn:

- what can drift between a struct and its table, and when each kind of drift is caught
- how `verify` uses `Statement::columns()` and the catalog to check a model before it is used
- where to run it: at startup, and in a test per model
- how to test a derive: expansion tests, UI tests, integration tests
- what compile-time checking (sqlx's `query!`, Cornucopia) adds, and what it costs

13.1 What can drift

Plan:

- A renamed or dropped column, a changed type, a column made nullable, a new `NOT NULL` column a `New…` struct lacks: each with when it is caught today (Chapter 5: at the query; a NULL: at the row).

13.2 `verify`

Plan:

- Prepare `SELECT * FROM` the table (no rows read). Not the model's own `SELECT`: a statement that names a missing column fails at the first one, and `verify` wants them all. The derive lists each field's column and the type it reads in `Table::FIELDS` (`Field::new::<T>`, `fn` pointers to `column::<T>` and a NULL test), and each is checked against the statement.
- Prepared without the statement cache (`Executor::prepare_uncached`): a cached statement describes the table as it was, and a test shows `verify` missing a dropped column through deadpool's cache (§12 #43).
- Nullability is not in a row description. `Column::table_oid()` and `column_id()` (§12 #6) identify the table column, and `pg_attribute.attnotnull`, one query for every column, says whether it can be NULL; a nullable column read into a field that refuses NULL is `Mapping::Nullable`. A generated column is nullable unless declared `NOT NULL`, which PG 18 allows on a virtual one (§12 #40).
- A view's columns report the view's own OID, where every column looks nullable: `verify` checks their names and types and leaves NULLs alone (`pg_class.relkind`).
- Every mismatch in `Error::Schema { model, mismatches }`, in field order; its message, captured from the test, printed.

13.3 Where to run it

Plan:

- At startup, before the server accepts requests, so a bad deploy fails its readiness check (main book Chapter 16).
- In a test per model against a migrated database (main book Chapter 17's template).

13.4 Testing a derive

Draft prose:

shelf tests its derives three ways, because each catches what the others cannot. An expansion test runs the generator on a struct and compares the pretty-printed code with a file: it catches any change to what users get, and its diff is the review. A UI test compiles a broken struct and compares the compiler's errors with a file: it pins down the message and where it points. An integration test derives for real and runs queries against PostgreSQL: it is the only one that proves the generated code does the right thing.

Plan:

- Excerpts: `tests/expand.rs` and one `.expanded.rs` file; `tests/ui.rs` and one `.stderr` file; one integration test with the harness.
- Blessing: `SHELF_BLESS=1` and `TRYBUILD=overwrite`, and why the diff must be read.

13.5 Compile-time checking

Plan:

- sqlx's `query!` checks SQL against a live database (or saved metadata in offline mode) at compile time (docs; check the current details); Cornucopia generates Rust from SQL (main book Chapter 14).
- What they buy (no drift reaches a build) and cost (a database at build time, or metadata to keep in sync); where `verify` sits between.

The programs

- `run` (proposed): `verify` on a model that matches its table.
- `run`: a renamed column, the report printed.
- `run`: a nullable column read into `String`, reported from the catalog.
- Excerpts of the test files named above, checked by `book.py excerpts`.

Side by side

Catching a renamed column: by hand (at the first query) · shelf (`verify` at startup) · sqlx (`query_as!` at compile time, check).

Claims

- Docs: PostgreSQL's `pg_attribute` catalog; sqlx's offline mode (check).
- Opinion: check the mapping at startup; a deploy that cannot run should fail before it takes traffic.

When not to use `verify`

- Queries written by hand: `verify` checks models' own SQL; a `FromRow` struct read from a custom query is checked when that query is prepared (Chapter 5).

Exercises

Plan:

1. Run `verify` for every model in one call. *Hint:* a list of functions, one per model.
2. Report extra columns the struct does not read. *Hint:* the statement has more columns than `resolve` used.
3. Write an expansion test for a struct of your own. *Hint:* add a file to `tests/expand/`.

> **Open questions.** Settled in S1c (PLAN.md §10): a view's columns are checked for names and types, not NULLs; `Error::Schema { model, mismatches }`, in field order; `New…` structs are not checked (an exercise). Still open: whether exercise 1 needs a trait object over models (`verify` is generic) or a list of function pointers.

CHAPTER 14

Capstone: stockroom on shelf

> **Sketch, 9 November 2026.** First sketched on 7 November, and revised to the capstone as built in phase S2. A plan of this chapter, not its text; it will be replaced by the chapter. Part III, fifth chapter. Builds on every chapter of this book, and on main book Chapter 18 (stockroom end to end) and stockroom's repository layer (stockroom/src/repo/). shelf: `orm/stockroom-shelf` (built, S2), a library and binary on the `stockroom` crate. Crates: axum 0.8.9, deadpool-postgres 0.14.2, async-stream 0.3.6. Plan: orm/PLAN.md §4.0d, §7, Chapter 14; §10 (the S2 rows); §12 #47–#56.

stockroom's data layer is written by hand: about a dozen repository functions, each with its SQL, its parameters and its `TryFrom<&Row>`. It is the code this book set out to generate. This chapter rewrites part of it with shelf, behind the same axum routes, against the same schema, with the same domain types, and compares the two line for line.

The comparison is honest in both directions. shelf removes the mapping code and almost all of the everyday SQL, and the ported routes answer exactly as stockroom's do: one test suite runs against both routers and compares every reply. But an order read back costs a round trip more, because shelf has no joins, and the orders module ends up longer than stockroom's. And some code shelf should not touch at all: the guarded `UPDATE` that keeps orders from overselling, the state machine of an order, the `MERGE` import, the COPY export. Knowing where an ORM stops is the last thing this book teaches.

In this chapter you will learn:

- how a port can reuse an application's schema, types, errors and routes, and replace only its data layer
- how shelf's models fit a real schema: a domain, a composite, an enum, a `jsonb` document, a virtual column, a composite key
- what each repository function looks like rewritten with shelf, and what it costs in lines, statements and round trips, measured
- how to prove that a rewrite behaves as the original: one suite, two routers
- where shelf steps aside, and how raw SQL fills the gap

14.1 The shape

Plan:

- orm/stockroom-shelf: reused from stockroom unchanged are the migrations, the domain types, `AppError`, `in_transaction`, `CancelOnDrop`, the `Db` extractor and the router for every route not ported (§12 #47). New: the models and the repository (`src/repo/`), the handlers (stockroom's, copied: its handler modules are private), the error conversion, `serve` with `verify` first.

- The router: the ported routes, then stockroom's static paths that `/products/{sku}` would capture, routed to stockroom's router by name (`into_service`: `route_service` refuses a `Router`), then everything else by fallback (§12 #50, measured).
- The errors: `impl From<shelf::Error> for AppError` is refused by the orphan rule (E0117, captured; §12 #49), so the repository converts with `map_err(app_error)`. The same rule as Chapter 1's tuples, met from the other side.
- `stockroom-shelf verify` and `serve`: every model checked at startup, all mismatches at once.

14.2 Models for a real schema

Plan:

- `Store` (the composite `Address`, through postgres-types' derive), `Product` (the domain `Sku` with its hand-written `accepts`, `Cents`, `text[]`, `jsonb`, `version`, `updated_at`), each deriving serde beside shelf: the model is the JSON.
- `Stock`, keyed by `(store_id, product_id)`, with the virtual column `available`, read only (`generated`). **`verify` refused it as an `i32`** (measured, §12 #48): a generated column is nullable in the catalog unless declared `NOT NULL`, though `on_hand - reserved` of two `NOT NULL` columns never is. The model says `Option<i32>`; the schema is stockroom's.
- Orders: the API's order names its store by code and each line's product by SKU, which no row holds. So stockroom's request and response types are reused, the tables get models of their own (`OrderRow` with `belongs_to(Store)` and `has_many(LineRow)`, `LineRow` with `belongs_to(Product)`), and the response is assembled from them. `NewLineRow` sends the domain `Quantity` through `insert_many` (Chapter 12's cast from the prepared `INSERT`).

14.3 Line for line

Plan:

- Side by side: `stores::create` (13 lines, then 4), `products::update` (45, then 16, with the retry below), `orders::get` (46, then 10 plus the assembly).
- Measured, statements and round trips (§12 #54: a counter of Parse and Execute messages, then lagproxy at 5 ms each way; a round trip 14.4 ms):

Function	stockroom	shelf
stores, product reads and creates	1 statement, 1 round trip	the same
`products::update`	1, 1	2, 2: the read by SKU, then the patch
`orders::place`, 3 lines	9 run, 2 re-prepared per call, 9 round trips	7 run, 6 round trips
`orders::get`, 3 lines	2, 1 (two joins, pipelined)	4, 2 (no joins)

- Measured, lines of Rust (§12 #55): stores 57 against 34, products' ported items 150 against 87, orders' `place` and `get` 148 against 175. Hand-written SQL: 15 statements against 1.
- A `PATCH` without `If-Match`: shelf's `Patch` always checks a version, stockroom's does not. The port patches at the version it read, and tries again when it is stale (§12 #51; 30 concurrent patches, all applied).
- The NDJSON stream: shelf's `ModelStream` borrows its connection, and the body must own it, so the stream lives inside a generator that owns both (§12 #52).

14.4 Where shelf steps aside

Plan:

- The reservation: `UPDATE stock … WHERE available >= $3`, pipelined in product-id order, keeps its SQL inside a `place` whose reads and inserts are shelf's. stockroom's oversell and deadlock tests pass on the port.
- Unported, on purpose: an order's transitions (one `UPDATE` checks the state machine and the version, `RETURNING old.status`), stock adjustments (`on_hand = stock.on_hand + EXCLUDED.on_hand`, which an upsert that sets columns cannot say), serializable transfers, the `MERGE` import, the COPY export, `LISTEN`, the reports.
- The escape hatch for `orders::get`: stockroom's two joins as raw SQL, read with `FromRow` structs through `fetch_one` and `fetch_all`: one round trip again, at the price of the SQL.
- The pattern: shelf for the everyday reads and writes, SQL for the statements whose wording is a rule.

14.5 What a production ORM would add

Plan:

- Joins (the round trip `get` lost), migrations generated from models, lazy relations, change tracking, other databases: what each would cost shelf in simplicity, and which crate already does it (diesel, SeaORM, toasty).

The programs

- `run` (proposed): the ported router on a pool of one, driven with `oneshot`: create a store and a product, patch it with `If-Match`, place an order, read it back. It needs the shelf playground to depend on stockroom-shelf.
- The tests the chapter points to: `tests/routes.rs` (11 scenarios against both routers, 63 replies compared, and which handler answered, from the statement cache; §12 #53), `tests/concurrency.rs`, `tests/models.rs`.
- `examples/line_for_line.rs`, the measurement (§12 #54).

Side by side

This chapter is one long side by side: stockroom's hand-written repository functions against the shelf versions, with the same routes on top. The third column is the comparison of costs.

The data

stockroom's schema, migrated by stockroom's own migrations; the tests clone stockroom's test template, through stockroom's own harness.

Claims

- Measured: the statements and round trips per ported function (§12 #54), the lines (§12 #55), the routes' replies (§12 #53), `verify` on stockroom's schema (§12 #48).
- Opinion: an ORM should cover the boring 80% and get out of the way of the rest.

When not to use shelf

- Every statement in 14.4, and any statement whose `WHERE` clause carries a business rule.
- A response that joins several tables, when its round trip matters.

Exercises

Plan:

1. Port `GET /stores/{code}/stock`. *Hint:* a store by code, its `Stock` rows by filter, their products by loader; `available` is read only.
2. Give shelf's `Patch` an optional version (`version = coalesce($n, version)`), and remove the retry from `products::update`. *Hint:* `patch_sql`, and what `None` must mean.
3. Write `orders::get` with joins through `FromRow`, and measure it with `line_for_line`. *Hint:* two structs, two pipelined `fetch_*` calls.

> **Open questions.** (1) Whether the chapter's runnable examples drive stockroom-shelf from the shelf playground (a dependency of the playground on the capstone), or show it through excerpts and its tests only. (2) How much of `place` to print: it is 86 lines, most of them stockroom's validation.

Appendix — attribute reference, generated code and phrasebook

> **Sketch, 9 November 2026.** A plan of this appendix, not its text; it will be replaced by the appendix. It collects reference material from every chapter: the `#[shelf(…)]` attributes (Chapters 4, 6, 8–11), what each derive generates (from the expansion tests in `orm/shelf-codegen/tests/expand/`), a phrasebook from shelf to diesel, SeaORM and sqlx (Chapter 0's comparison and each chapter's side by side), a glossary, and further reading. Entries already known are drafted; the rest is planned. Plan: orm/PLAN.md §4.3, §7, Appendix.

Look things up here once the chapters have taught them: which attribute does what and where it may go, what a derive writes for a given struct, and how to say a shelf call in the crate you use at work.

A Attribute reference

Drafted for the attributes that exist (phase S1a); the rest are added as their phases build them.

On the struct, read by `FromRow`:

Attribute	Meaning	Chapter
`rename_all = "…"`	Column names follow a convention: `"snake_case"` (the default), `"camelCase"`, `"PascalCase"` or `"SCREAMING_SNAKE_CASE"`	4

On a field, read by `FromRow`:

Attribute	Meaning	Chapter
`rename = "col"`	Read the column `col`; overrides `rename_all`	4
`skip`	Do not read; use `Default::default()`	4
`flatten`	A nested `FromRow` struct, read from the same row	4
`json`	Read a `json` or `jsonb` column through serde; an SQL `NULL` reads as JSON `null`	4
`try_from = Type`	Read a `Type`, then convert with `TryFrom`; the type may be quoted	4

Rules, as the derive enforces them: keys are refused if unknown or repeated; `skip` goes with nothing else; `flatten` goes with nothing but itself; `json` and `try_from` exclude each other; every error is reported at its token, all in one build.

Plan for the attributes still to come: `table`, `schema`, `primary_key`, `generated` (Chapter 6); `model` (Chapter 8); `version`, `updated_at` (Chapter 9); `belongs_to`, `has_many` (Chapter 11).

B What each derive generates

For this struct:

```
// orm/shelf-codegen/tests/expand/basic.rs (excerpt)
#[derive(FromRow)]
struct Product {
    id: i64,
    sku: String,
    r#type: String,
    tags: Option<Vec<String>>,
}
```

`#[derive(FromRow)]` generates, as the expansion test pins it down:

```
// orm/shelf-codegen/tests/expand/basic.expanded.rs (excerpt)
#[automatically_derived]
impl ::shelf::FromRow for Product {
    type Columns = (usize, usize, usize, usize);
    fn resolve(
        columns: &[::shelf::__private::tokio_postgres::Column],
    ) -> ::core::result::Result<Self::Columns, ::shelf::Error> {
        ::core::result::Result::Ok((
            ::shelf::__private::column::<i64>(columns, "Product", "id", "id")?,
            ::shelf::__private::column::<String>(columns, "Product", "sku", "sku")?,
            ::shelf::__private::column::<String>(columns, "Product", "type", "type")?,
            ::shelf::__private::column::<
                Option<Vec<String>>,
            >(columns, "Product", "tags", "tags")?,
        ))
    }
    fn from_row_at(
        row: &::shelf::__private::tokio_postgres::Row,
        columns: &Self::Columns,
    ) -> ::core::result::Result<Self, ::shelf::Error> {
        ::core::result::Result::Ok(Self {
            id: ::shelf::__private::get::<i64>(row, columns.0, "Product", "id")?,
            sku: ::shelf::__private::get::<String>(row, columns.1, "Product", "sku")?,
            r#type: ::shelf::__private::get::<
                String,
            >(row, columns.2, "Product", "type")?,
            tags: ::shelf::__private::get::<
                Option<Vec<String>>,
            >(row, columns.3, "Product", "tags")?,
        })
    }
}
```

Plan:

- The other expansion tests (`rename`, `skip_flatten`, `json_try_from`, `generics`) in the same form, each one line per attribute in the text and the full expansion as an excerpt.
- `Table`, `Insert` and `Patch` as their phases add them.

C Phrasebook

Plan: one table per task (map a row, find by key, insert, partial update, filter a page, load children, bulk insert), with the shelf call and the diesel, SeaORM and sqlx equivalents, each checked against the crate's version at the time of writing (check). A first row, to show the shape:

Task	shelf	sqlx	SeaORM
Read rows into a struct	`#[derive(FromRow)]`, `fetch_all`	`#[derive(FromRow)]`, `query_as`	`#[derive(DeriveEntityModel)]` (check)

D Glossary

Draft entries:

- **Derive macro**: a procedural macro attached with `#[derive(…)]`; it receives the item and adds code next to it (Chapter 3).
- **Expansion**: the code a macro produces in place of its invocation, or beside it for a derive; shelf prints it with `pretty` (Chapter 3).
- **Helper attribute**: an attribute a derive declares (`attributes(shelf)`) so the compiler passes it through to the derive (Chapter 4).
- **Hygiene**: the rules that keep a macro's names from clashing with the caller's; paths are not covered, hence the absolute paths in generated code (Chapters 2 and 3).
- **Resolve**: shelf's step that finds and checks each field's column once per statement (Chapter 5).
- **Span**: where a token came from; errors are reported at spans (Chapter 4).
- **TokenStream**: the sequence of tokens a procedural macro receives and returns; `proc_macro2::TokenStream` is the same thing usable outside the compiler (Chapter 3).
- **N+1**: one query for a list, then one per item (Chapter 11).

Plan: the terms of Part II and III as their chapters are written.

E Further reading

Plan:

- *The Little Book of Rust Macros*; the Rust Reference on macros by example and procedural macros.
- syn's and quote's documentation; dtolnay's proc-macro workshop.
- diesel's, SeaORM's and toasty's guides; the main book's Appendix F.

> **Open questions.** (1) Whether the phrasebook belongs here or in Chapter 0, with a short version in the other. (2) Whether B should include every expansion test or one per derive.